

TEAMDER – אודיט מוצר וטכנולוגיה, מלא

02.09.2026 · READ-ONLY · הריפו לא נגע · אפס קומיטים

26 סוכנים · 3 שלבי הרצה · ~1,400 פעולות על מכשיר · 47 צילומי מסך · +9,000 שורות ראיות · 97 ממצאים

איך לקרוא את המסמך הזה

זה אודיט מרובה סוכנים, ולסוג הזה יש חולשה מובנית: **סוכן אחד טועה, והבא מצטט אותו כעובדה**. לכן בדקתי אישית כל ממצא חמור מול הקוד, מול מסד הנתונים בפרודקשן, או מול המכשיר. **12 ממצאים לא שרדו**. רישום התיקונים (נספח A) **גובר על דוחות הסוכנים**.

שלוש רמות ודאות במסמך:

- **Confirmed** – נבדק בקוד, בנתוני פרודקשן, או נראה במו עיניי על מכשיר

- **Suspected** – הסקה סבירה שלא הוכחה

- **Mock artifact** ⚠️ – נראה כמו באג, הוא נתוני דמו

1. עשרת הדברים שחשוב לדעת

1 • 9.3% מהערבים איבדו את הסיכום שלהם לצמיתות

7 מתוך 75 המשחקים שהסתיימו רשומים ב־ `committedRounds` ויש להם אפס `roundHistory`. אחד עם 13 משחקונים. אימתתי: **שלושה מועדונים אמיתיים** – “כדורגל אנשים טובים” (15 חברים), “כדורגלשם” (17-18), “מועדון שכחת שושׁי”. לא QA.

`roundHistory` נכתב **מחוץ** לבאץ' האטומי כ־מאמץ מיטבי. הסטטיסטיקות נזקפו, אז לא הוצגה שגיאה. אבל היסטוריית המשחקים, סיכום המחזור, גלגול הכימיה וכרטיס סיכום הערב **כולם** נבנים ממנו – ומסמן ההצלחה נוצר רק בהצלחה, ולכן הערבים האלה **לא יעובדו לעולם**.

M: תיקון · P0 · Confirmed – חייב להיות מחוץ לבאץ', ראו סעיף 3

2 • בידוד המועדונים שבור – בלוק כפול בכללים

`firestore.rules` מכיל **שני** בלוקים ל־ `communityPairStats`: שורה 1704 עם `isGroupMember(...)`, ושורה 1973 עם `allow read: if isSignedIn()`. `the` **איזשהו** בלוק מתיר – השני מבטל את הראשון. ההערה מעל הכלל הנכון אומרת “*membership check is what keeps that true*”. היא לא.

בנוסף `communityPlayerStats`, `gamePlayerStats` ו־ `communityStats` פתוחים ב־ `isSignedIn()` בלי בדיקת חברות.

S: תיקון · P0 · Confirmed – מחיקת חמש שורות. ההחזר הכי גבוה בכל האודיט

3 • מסך לבן בלתי נראה

12 הקשות מהירות על סרגל הטאבים (2~ שניות) הורסות את עץ התצוגה הנייטיב. הקשה מאוחרת ולא קשורה זורקת `IllegalStateException: child already has a parent`, ה־ `ReactHost` נהרס, והמסך נשאר **לבן לצמיתות**. אין התאוששות – רק סגירה כפויה. **ואין כלי דיווח קריסות**. המשתמש רואה לבן, סוגר, ואף אחד לא יודע.

M: תיקון · P0 · stack trace עם, Confirmed

4 • הסיכון שבשבילו האודיט השתלם

התיקון המתבקש לממצא 1 – “לכתוב את שדות הזוגות בתוך הבאץ'” – היה מקפיץ אותו **407 מתוך 500** פעולות (81%) ל־120~130%, והופך באג תצוגה **לאובדן סטטיסטיקה קשה**: מנעול האידמפוטנטיות יושב באותו באץ', וכל ניסיון חוזר ייכשל זהה.

חוסם עיצוב · Confirmed – כל תוספת של סטטיסטיקה חייבת לכבד את המרווח הזה

5 • משחק מהיר מזין סטטיסטיקה גלובלית ללא הגנה

עברתי על כל 900 שורות `commitRoundStats`: אפס בדיקות `isOrphan / isPersonal`. וגם `addRetroGoal` לא חסום – **שתי הדרכים פתוחות**. אפשר לפתוח משחק לבד, לרשום 20 שערים, והפרופיל הציבורי יציג אותם.

ההקשר: מתוך 174 קבוצות באפליקציה, 121 (70%) הן קבוצות אישיות של משחקים מהירים.

בזהירות, זו נקודת ההפעלה של משתמש חדש – M: תיקון · P1 · Confirmed

6 · אין גיבויים · אין התראות · אין דיווח קריסות

אין Pitr ל-Firestore. אין אף Cloud Monitoring alert. אין Sentry/Crashlytics – יש רק אוסף errors שתופס שגיאות מטופלות בלבד. מחיקה שגויה או קריסה = אף אחד לא יודע ואין דרך חזרה.

S: תיקון · P0/P1 · Confirmed – ארבעה סוכנים מצאו את זה במקביל

7 · סריקת מועדונים רצה פי 24 מהמתוכנן

lastActivitySweep (index.ts:12639) הוא משתנה JS בזיכרון שאמור לגדר "פעם ב-20 שעות". מופעי Cloud Functions מתחלפים, אז בכל קר סטארט הוא מתאפס והקרן השעתי סורק שוב. האח שלו באותו קובץ עושה את זה נכון דרך cronMeta/dailyCleanup.

S: תיקון · P1 · Confirmed

8 · הפיצירים הכי חשובים הם היחידים שאי אפשר לבדוק לפני שחזור

ציאט (אין ענף דמו – זורק), סיכום הערב (מחזיר תוכן דמו קשיח שמתעלם מהמשתמש), ומסלולי הודעות בלייב. ניתנים לבדיקה רק בפרודקשן. זה לא תיאורטי: כך בדיוק הגיעו למשתמשים באג תארי המועדון במשחק מהיר ובאג הפנדלים של האורחים.

M: תיקון · P1 · Confirmed

9 · "משחקון" לא קיים באפליקציה

grep -c "משחקון" src/i18n/he.ts → 0. המילה שכל האפיון הזה נשען עליה לא מופיעה אף פעם מול המשתמש. "משחק" עושה עבודה כפולה – גם המשחקון וגם הערב כולו.

זו הסיבה שדיווח משתמש אמיתי ביקש "תן פה כותרת של איזה מספר משחק זה", וזה מערער כל אפיון עתידי שדורש מהמשתמש להבחין בין השניים.

S: תיקון · P2 · Confirmed

10 · ~60% מהאפליקציה בלי שום טסט

שלוש מדידות בלתי תלויות מתכנסות: 45 מתוך 76 פיצירים · 35 מתוך 57 שירותים · 0% מ-14 אלף שורות · 0% Cloud Functions מ-55 המסכים. offline | רשת חלשה הם "מעולם לא נבדק" בכל תא בודד במטריצה.

P1 · Confirmed

2. ממצאי ההרצה על מכשיר

האמולטור היה שבור בתחילת האודיט (התקנה חלקית – חבילה רשומה בלי APK). תיקנתי: הסרה, בניית debug מקומי, התקנה. אחר כך בניתי גם release ללא דמו ובדקתי מול פרודקשן בחשבון QA ייעודי.

Offline – התשובה מפוצלת, וזו חדשה טובה ורעה

✓ **שלמות הנתונים מחזיקה.** ביצעתי ביטול הרשמה במצב offline מלא. הכתיבה נכשלה נקי – לא נכנסה לתור, לא הוחלה מאוחר יותר כשהרשת חזרה. אימתתי גם במסך וגם ישירות ב-Firestore: החשבון עדיין רשום, 5 שחקנים, אפס שאריות. זה סותר את החשש המרכזי שהועלה על כתיבות offline.

✗ **החווייה גרועה.** האפליקציה מציגה הכל מהמטמון בלי שום חיווי שאין רשת – פיד, משחק, הרכב, הכל נראה תקין. ואז הפעולה נכשלת בטוסט של מילה אחת: "שגיאה". בלי לומר שאין רשת, בלי ניסיון חוזר, בלי לרמוז שהפעולה לא בוצעה.

S: תיקון · P1 · Confirmed

ביצועים – הפרק נסגר, והמספרים הקודמים היו חסרי ערך

release		debug + Metro	
זמן עלייה	26~14.7 שני	2.57 / 2.11 / 1.89 שני	
זיכרון (PSS)	498MB	214MB	
פריימים קופצים	26.7%	21.1% (339/1,605)	

סייג שאני לא מוותר עליו: חציון פריים 40ms, ואף פריים לא ירד מתחת ל-17ms בכל 1,605. זה נראה כמו עומס רינדור אמיתי בפיד – אבל זה אמולטור עם רינדור תוכנה, וייתכן שזו הרצפה שלו. **סיגנל, לא פסק דין.** דורש מכשיר אמיתי.

מסע המשתמש החדש

מצב אורח מוביל למסך בית **ריק** בסתירה להבטחת ההיכרות · “חזור” במסך ההתחברות **סוגר את האפליקציה** · קרוסלות הישגים בת 3 כרטיסים **בלי דילוג** · 6~ הקשות ו־4 מסכים חוסמים עד ערך אמיתי.

שני מספרי גולים סותרים על מסך אחד

תג הגולים קורא `live.goalTally`; רשימת “מבקיעים” קוראת `live.goals`. `stopRotation` מאפס את הראשון ולא את השני – אימתתי בשני המסלולים. אחרי איפוס, הגול הבא מחזיר את `goalTally` ל”לא ריק” עם שחקן אחד, ותגי כל השאר נעלמים. תוצאת הקבוצה והסטטיסטיקה האמיתית לא נפגעות. תצוגה בלבד. התיקון אינו “לתקן חישוב” אלא להחליט מה כל מספר אומר ולתייג: תג = הערב, רשימה = המשחקון.

3. מה שטוב – ושווה להגן עליו

- זה לא סעיף נימוסין. אלה דברים שנבדקו ספציפית ועברו.
- זרימת יום המשחק היא הנכס של המוצר. כוחות → לייב → פנדלים → סגירת ערב → “מי לקח את הציד” → סיכום. בלי אף מבי סתום.
 - מגרש הפנדלים וחשיפת הכוחות הם רגעי כדורגל אמיתיים – הדבר הכי קרוב ל־wow באפליקציה
 - השלמת שחקן מהקבוצה הממתינה: 4 הקשות עם מחליף שנבחר מראש
 - הגנות הקשה כפולה עובדות – אין קומיטים כפולים על גול או סיום משחק
 - RTL ממושמע – 586 שימושים ב־`RTL_LABEL_ALIGN` מול 28 גולמיים
 - זהות קבוצה אף פעם לא לפי צבע בלבד – תמיד עם שם. אין בעיית עיוורון צבעים
 - הליבה האטומית של הסטטיסטיקה תקינה – נמדד בפרודקשן: `goals/wins/rounds/cleanSheets` תואמים בדיוק בין שלושת האוספים
 - `deep links` עובדים גם מאפליקציה סגורה
 - אין אף אינדקס מורכב חסר בכל 14 אלף השורות
 - מנעול האידימפוטנטיזציה, הגנות האורחים וטרנזקציות רשימת ההמתנה – כולם נבדקו ונמצאו תקינים
 - 351 דיווחי משוב – אות השימוש החיובי החזק ביותר בכל האודיט
- ושלושת התיקונים שנעשו במהלך הסשן אומתו בהרצה ע”י סוכן עצמאי ועל ידי: מספר המשחקון בלוח, הפופאפ של לייב בלי כוחות, ביטול בעיטה בשובר שוויון – ופופאפ אישור הביטול נראה עובד גם בבילד `release` מול פרודקשן.

4. מה שנמדד בפרודקשן

משתמשים	614
מועדונים	174 – מהם 121 (70%) קבוצות אישיות של משחקים מהירים
משחקים	79 (75 הסתיימו)
הודעות צ'אט – אי פעם	140 (71 מועדון · 60 פרטי · 9 משחק)
דיווחי משוב	351
fillerInterests	4
קמפיינים	1
דירוג עמיתים (מת)	12 רשומות + 2 טריגרים שעדיין פרוסים
מועדון הכי גדול	40 חברים

NOW – השבוע

מה	Impact	Effort	ראיה
למחוק את הבלוק הכפול בכללים + להצר 3 אוספי סטטיסטיקה	גבוה	S	1.2§ מאושר
להדליק גיבויים / PITR	גבוה	S	1.6§
Crashlytics	גבוה	S	1.6§ + 1.3§
שער lastActivitySweep ל-Firebase	עלות	S	1.7§ מאושר
latestVersion → 1.1.1	בינוני	S	פופאפ העדכון מת
טוסט offline אמיתי במקום "שגיאה"	בינוני	S	2§ מאושר

NEXT – החודש

לשחזר את 7 הערבים החסרים מחוץ לבאץ' · לחסום כתיבה גלובלית ממשחק מהיר (בזהירות) · eveningScoreParity + 5 טסטי parity חסרים · CTA בתוך האפליקציה ל-PromoteOrphan · טרנזקציות ל-15 מסלולי draftTeams · לתקן את קריסת הטאבים

LATER

מדיניות TTL · צמצום 5 הטריגרים · מטמון לכימיה · נוסחת רמת המועדון · נרמול טבלת המועדון · מונח "משחקון" בקופי

DON'T DO

פיבוט לשוק פתוח של חיפוש משחקים · תשלומים · דירוג עמיתים · פיצול קבצי הענק עכשיו · אישור ממנהל שני · מדדי clutch/רצפים לפני הצלבה מול ניצול · אופטימיזציה לסקייל פי 1000

6. מה שלא נבדק, ולמה

מצב	
iOS	✗ אין runtime מותקן; שירות CoreSimulator תקוע ולא מגיב ל-kill. דורש sudo + ~7GB הורדה
גלילה על מכשיר אמיתי	⚠ נמדד באמולטור בלבד - המספרים סיגנל, לא פסק דין
offline עם כתיבות מרובות / שני מנהלים	⚠ נבדק תרחיש אחד. שני מנהלים במקביל לא נבדק
קופי בהרצה	⚠ נותח מהמקור, לא נצפה על כל מסך

A – רישום התיקונים (גובר על דוחות הסוכנים)

Corrections – findings the audit lead personally re-verified and changed

Multi-agent audits propagate errors: one agent's wrong claim gets cited by the next as established fact. Every finding below was independently re-checked by the .audit lead against the code, the production database, or the running device .These corrections override the agent artifacts

.C1 – “Every ≡ / ⋯ menu in the app is dead” → FALSE. Removed

- “Claimed by: P_ADMIN (rated a top blocker) · then repeated by QA_MATRIX as “runtime-confirmed Po
- **Re-tested by the lead on the same device and build.** Opened a game → tapped ⋯ at (90, 262) → the menu opened with all 8 badge and the card מנהל the roster screen opened correctly, showing 10 players, the → “ניהול שחקנים והמתנינים” actions → tapped indicators
- Cause of the false positive: stale tap coordinates. The agent tapped where a previous screen’s control had been. This is the exact .trap the emulator guide warns about
- **Consequence:** two further P_ADMIN findings that depend on the same tap mechanics – “creating a game from the global + binds to the wrong club” and “the club-creation dialog loop” – are downgraded to **Suspected, needs re-verification**. They may well be .real; they were not proven
- .QA_MATRIX’s risk ranking is invalidated at position 1 and should be re-read without it

.C2 – “Only 7.8% of interactive elements are accessible” → Wrong by 5x. Corrected to 42%

- Claimed by: A11Y
- .The agent’s regex only inspected the first line of each JSX tag, but JSX here spans multiple lines, so almost every label was missed
- **Lead’s measurement**, parsing each opening tag to its closing > : **448 interactive elements, 189 (42%) carry an .accessibilityLabel or accessibilityRole, 259 do not**
- Still a real gap, but a different order of problem. The rest of A11Y (contrast numbers, allowFontScaling, reduced-motion coverage, .and the important negative finding that team identity is never colour-only) was spot-checked and holds

.C3 – “The deleted-game aggregate bug is a Po confirmed in production” → Downgraded to P2

- Claimed by: INVARIANTS · flagged by REDTEAM · confirmed by the lead
- The mechanism is real and the lead verified the orphaned data directly: games/fusBhEBGkC37D49LygwP is gone, .gameDeletions/... exists, and games/.../roundHistory/1:1787391983205 survives with a real 2:1 result
- But the affected club is literally named “QA Test Club”, one participant, creator account since deleted. The structural gap deserves .fixing; the production damage is test data

.C4 – “functions/ has zero tests” → Overstated

- Claimed by: D4_infra
- Correct statement: functions/ has **no test directory**, so the Cloud Functions themselves are untested. But three client-side parity tests (balanceParity , clubChemistryParity , assistCredit) do pin shared client/server logic. The distinction matters when .deciding what to write first

.C5 – “addRetroGoal is blocked for personal/quick groups” → FALSE

- Claimed by: GAMING (as a contrast to commitRoundStats being unguarded)
- **Lead checked both.** Neither commitRoundStats nor addRetroGoal contains any isPersonal / isOrphan guard. Both paths .write to global lifetime stats from a quick game. The exploit is wider than reported, not narrower

.C6 – The 500-op batch is “a data-loss bug at 11-a-side” → Guarded. Reframed

- Claimed by: DB (P1, framed as loss)
- The code defends itself: MAX_SIDE = 11 throws before the batch is built, and the comment documents the reasoning. At 11-a-side .the batch is 407/500 ops (81%)
- **The correct finding is the thin margin, not a live bug:** any future change that adds a per-player or per-pair write breaks 11-a-side, and because the idempotency latch sits in the same batch, every retry then fails identically and the round’s stats are lost for good. This is a constraint to respect before adding any statistic – which is exactly what CROSSEXAM showed would happen if the .chemistry fix were put inside the batch (407 → ~120-130% of the cap)

.Confirmed absent, and it matters more than it looks → “משחקון” C7 – Terminology: the word

- Claimed by: COPY · verified by the lead

serves both “one “משחק” ;The word appears nowhere in the user-facing vocabulary .0 → `grep -c "משחק" src/i18n/he.ts` •
“mini-game” and “the whole evening” •
This is not a copy nitpick. It is the reason a real user report asked for “a title saying which game number this is”, and it undermines •
any future spec that depends on the user distinguishing an evening from a mini-game

.C8 – The late-cancel dialog threatens a system that no longer exists → Confirmed

`he.ts:208` Claimed by: COPY · verified by the lead at •

A code comment elsewhere records that the reliability/discipline feature was “ישפיע על דירוג המשמעת שלך.” The string still warns •
.scrapped and its popup removed on 2026-06-21. The threat text survived the feature

.C9 – “The evening summary shows the wrong player” → Mock artifact, not a bug

(while logged in as “מכתן לוי / מכבי חולון” reported as an identity bug: showed) Claimed by: P_PLAYERS •

Lead checked `mockModel` in `src/services/eveningSummaryService.ts:174-192`. It hardcodes `playerName: 'מכתן לוי'` and •
`communityName: 'מכבי חולון'` and ignores the uid entirely. This is demo data by construction

However – the underlying risk is real and worth one line: mock mode is the ONLY way the team can currently exercise this screen, •
and it cannot show a real name there. That makes the evening summary effectively untestable outside production, which is how the •
earlier REAL bug in this area (the club-titles-in-a-quick-game report) reached users

C10 – “Goals/assists disagree between the club table and personal stats (16g/8a vs 18g/11a)” → Almost certainly a mock artifact. Downgraded to Suspected

Claimed by: P_PLAYERS as a live data-consistency bug •

In mock mode `playerStatsService` returns a synthetic summary (`attendedGames: 28`, `distinctPlayers: 22` hardcoded) while •
the club leaderboard is fed from a different mock structure. Two unrelated demo sources will not agree

The CLASS of bug is nevertheless real and production-confirmed elsewhere: CONSISTENCY measured game •
`.49sGS04y8DfMRU1GBzvc` showing 2 goals in `roundHistory` versus 6 in `gamePlayerStats` on live data. Cite THAT, not this

.does nothing on your own row” → Refuted by a later agent כרטיס שחקן – C11

Claimed by: P_NEW (P16 persona) · **refuted by P_PLAYERS**, which opened it successfully and found a real, if stats-free, badge •
screen

The lead’s static read supports the refutation: `PlayerCardScreen` implements a dedicated `isSelfView` branch (lines 238/268/274), •
and `goToCard` navigates unconditionally

What survives: the self view exists but shows no statistics, so the underlying complaint – “I cannot find my own stats” – stands. The •
route is not dead; it is empty

Standing caution for every mock-mode finding ⚠

Three of the eleven corrections above are mock-data artifacts reported as product bugs. Any finding produced in mock mode that concerns NAMES, NUMBERS or CROSS-SCREEN AGREEMENT must be re-verified against production before it is acted on. Findings about NAVIGATION, LAYOUT, COPY and FLOW are unaffected and can be trusted from mock mode

.C12 – “Every chat crashes on open – release blocker” → Mock-only. Not a production bug

Claimed by: UI_UX (P0, “release blocker”) and P_PLAYERS (as a native crash) •

Lead traced it. `chatService.subscribeMessages` has no mock branch; it calls `messagesCol()` → `getFirebase()` •
unconditionally. In mock mode Firebase is never initialised, so it throws. In production Firebase IS initialised – and production holds •
140 real chat messages, so chat demonstrably works

Correct severity: **P2**, and it is a **TESTABILITY** defect, not a user-facing one •

The pattern behind C9, C12 and the evening-summary gap – elevate this 🌟

Three separate features cannot be exercised in mock mode at all: **chat** (no mock branch, throws), the **evening summary** (returns hardcoded demo content that ignores the user), and **live-match messaging paths**. The consequence is not that mock mode is imperfect – it is that these features can only ever be tested .in PRODUCTION, against real users' data

That is not a hypothetical risk. It is exactly how the club-titles-in-a-quick-game bug and the guest-penalties bug reached real users earlier: nobody could see them before shipping. This deserves to be a finding in its own right, above most of .the individual UI issues

נספח B – ממצאי הרצה על מכשיר

P_NEW

Runtime personas audit – P6 (new user), P5 (occasional returning), P16 (non-technical)

(Driven live on emulator-5554, mock mode Firebase) מצב נתוני דמו – לא קיים חיבור ל
;banner). All coordinates/steps below are exact adb input tap actions performed
./shots/ screenshots referenced live in

Note on environment: a Metro dev-tools toast (“Open debugger to view warnings”) intermittently overlapped the bottom tab bar and had to be dismissed (tap its X) before tab taps registered. This is a debug-client artifact, not shipped to users – not filed as a product finding, but explains some retries in the raw .step log

P6 – brand new user (zero knowledge)

Steps performed

1. Cold launch. Splash screen (solid blue, no logo/spinner) for ~11-14s before .first paint
2. Landed on onboarding slide 1/3: “שחקו עם אנשים בקרבת מקום” (play with people nearby) – phone mockup of the list, copy about discovering open games. Button “הבא” (next).
3. Tap “הבא” → slide 2/3: “מועדון קבוע, מחזור אוטומטי” (regular club, automatic) – phone mockup of a club-details screen (round).
4. Tap “הבא” → slide 3/3: “הכל זורם מעצמו” (everything flows by itself) – phone mockup of a game-details screen with weather/distance/auto-fill copy. Button label changes to “המשך” (continue).
5. Tap “המשך” → auth screen “בואו נתחיל”: three options – “המשך עם Google”, “המשך עם מייל”, “המשך כאורח” (continue as guest).
6. Tapped “המשך כאורח” (lowest-friction path a brand-new curious user takes).

7. a. Immediately (before any home content) "מה חדש באפליקציה" (what's new) :modal auto-opens, version badge "1.1.0", listing 5 changelog items "היסטוריית המשחקים", "שובר-שוויון בפנדלים", "עיצוב חדש לכרטיסי המחזורים", "מחזור בדרך", "סיכום המחזור שלך" – each with jargon like "שובר-שוויון" that assumes prior product knowledge (penalty shootout tiebreaker). Screenshot: [P6_01_whatsnew_after_guest.jpg](#).
8. Tap "יאללה, הבנתי" → lands on the home tab. to dismiss → (home) tab.
9. Home tab for the guest shows **only** an empty-state: person icon, "הפרופיל שלך מחכה" (your profile is waiting), "את/ה גולש/ת כאורח. הירשם כדי לשמור מחזורים, להצטרף למועדונים ולבנות פרופיל שחקן", and a single "הרשמה" button. No games, no feed, no map – none of the value promised in onboarding slide 1 is visible here. Screenshot: [P6_02_home_guest_emptystate.jpg](#).
10. Tap מחזורים → before the games list renders, a second interrupting modal fires asking to mark weekly availability, with a "מתי בא לך לשחק?" primary CTA and a small text-link dismiss "סמן את הימים שלי" and a "אחר כך". Screenshot: [P6_03_rounds_tab_availability_modal.jpg](#).
11. Tap "אחר כך" → now the real value screen appears – "מחזורים פתוחים" – 3 open games with day/time/location/format tags and "הצטרף"/"בקש להצטרף" buttons. This is the first screen matching the onboarding promise.
12. Tap into a game card → game-details screen: countdown clock, weather, duration, roster count, roster preview CTA. This screen is "בקש להצטרף", genuinely informative pre-signup.
13. Tap "בקש להצטרף" as guest → modal "נדרשת הרשמה" (registration required): "כדי להירשם למחזור צריך חשבון. רוצה להירשם עכשיו?" with "הרשמה"/"בטל". Screenshot: [P6_04_join_requires_signup_modal.jpg](#). This gate is clear and well-explained (not a silent failure).
14. Tap "הרשמה" → no → routed to the same generic auth screen as step 5. dedicated "finish your guest signup" shortcut – guest identity/context is (not carried into the signup form, e.g. no pre-filled anything).
15. Pressed Android back once from this auth screen → **exited the entire app to the OS launcher**, instead of returning to the game-details screen the user came from.
16. Relunched the app fresh → landed back on the same auth screen, guest session gone. The guest browsing from steps 6-13 was not persisted across a restart.

Tap/screen count to first real value

- + Onboarding (3 taps) + auth screen (1 tap: guest) + changelog dismiss (1 tap) + home dead-end (0 taps, just disappointment) + rounds-tab availability-modal dismiss (1 tap) = **6 taps, 4 interrupting screens** before a brand-new user sees an actual list of open games. A 7th tap (join) immediately re-hits a signup wall.

F-P6-1 – Guest's first "home" screen shows zero of the promised value

- Severity: P1
- Confidence: Confirmed
- Feature / Screen / Flow: Home tab (guest/unauthenticated state)
- Evidence: Step 9 above; [P6_02_home_guest_emptystate.jpg](#). Onboarding slide 1 promised "גלו מחזורי כדורגל פתוחים באזור שלכם" (discover open games near you) – but the very next screen after signing up as guest shows no games at all – only a signup nudge.

- tab renders a single centered empty-state (“your Current behaviour: בית”) for guests, with no games, feed, or map profile is waiting”)
- “Problem: The first screen a brand-new user reaches after “trying the app contradicts what the onboarding just promised. A user who skimmed the intro and chose the free “continue as guest” path is met with a dead end on the literal home tab, not the promised discovery experience
- User impact: This is the most likely quit point in the whole P6 walkthrough a curious user with zero investment sees nothing to look at and only a – .signup wall, one tab away from where the games actually live
- .Technical impact: none beyond UX
- Recommendation: For guest/unauthenticated users, default the home tab to the or at minimum (what currently lives under מחזורים), open-games discovery view .surface 2-3 real open-game cards above the signup nudge
- .Expected benefit: fewer guest-path drop-offs before the value moment
- Effort: S

F-P6-2 – Two interrupting modals stand between “continue as guest” and any real content

- Severity: P2
- Confidence: Confirmed
- Feature / Screen / Flow: post-signup changelog modal + מחזורים tab availability-ask modal
- Evidence: Steps 7 and 10 ; P6_01_whatsnew_after_guest.jpg , P6_03_rounds_tab_availability_modal.jpg
- Current behaviour: Immediately after choosing guest mode, a 5-item version changelog auto-opens; after dismissing it and switching to the games tab, a second modal asks the brand-new (zero-history) user to mark their weekly .availability
- Problem: A “what’s new” changelog is irrelevant to someone who has never (”שובר-שוויון בפנדלים”, seen the “old” version, and it uses feature names the user has no context for. The availability-ask, while (”סיכום המחזור שלך”) .reasonably dismissible, is a second forced interruption before any content
- “User impact: Two extra taps and two moments of “why is this in my way .before reaching the value screen; compounds the drop-off risk from F-P6-1
- Recommendation: Gate the changelog modal on “has used the app before this ;version” (e.g. skip for accounts/sessions created after the version shipped) move the availability-ask to appear after the user has seen at least one real games list, or fold it into the onboarding checklist card instead of a .blocking modal
- Expected benefit: one continuous path from “continue as guest” to real .content instead of three stacked interruptions
- Effort: S

F-P6-3 – Back button from the auth screen exits the app instead of navigating back

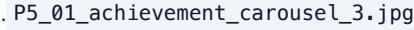
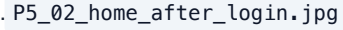
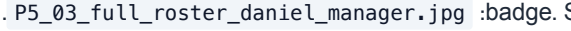
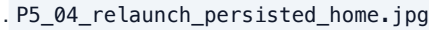
- Severity: P2
- Confidence: Confirmed
- Feature / Screen / Flow: “בואו נתחיל” :auth screen, reached via “הרשמה” from the join-gate modal
- Evidence: Steps 13-16. Pressing system back once on the auth screen closed the app to the Android home screen; relaunching returned to the same auth screen with the prior guest session gone
- Current behaviour: the auth screen appears to be the root of its navigation → stack even when pushed from deep in the app (game details → join-gate

- "הרשמה", so back has nowhere to go but out, out of curiosity, then changes their mind "הרשמה" Problem: A user who taps and hits back (the universal "nevermind" gesture), loses the game they were looking at and their guest session, and is dumped at the OS home screen
- User impact: confusing exit; on relaunch the user is back at square one with no memory of the game they had open
- Recommendation: preserve the navigation stack when pushing the auth screen from an in-app CTA, or at minimum treat back on that screen as "return to previous in-app screen" rather than falling through to the OS
- Effort: S

P5 – occasional player (opens app once every few weeks)

Since guest sessions don't persist (see F-P6-3), P5 requires a real login to Google button, which logged straight עם "המשך" be meaningful. Used the mock representative – "חמישי כדורגל" club admin of "דניאל" into a seeded account of a returning, already-registered player

Steps performed

1. From the auth screen, tap "המשך עם Google" → ~2s loading spinner on the button → brief blank blue screen (~4s) → home screen begins rendering behind an auto-playing achievement-unlock celebration
The celebration is a **multi-card carousel with no visible close/skip** → bronze unlocked; tap anywhere on screen (goals) "שערים" **control:** card 1 "בישולים" card 2 (assists) "היכריות" bronze unlocked; tap again → card 3 (acquaintances) bronze unlocked. Only after the 3rd tap-anywhere does the real home screen appear. Screenshots:  , 
3. Real home screen appears: "Teamder" header, coach announcement banner ("לילה טוב דניאל, 2 גולים היום ואתה מגיע ל-20"), a card titled "המחזור הקרוב שלך" with date/time/location and a (your upcoming round) tip, 3 quick actions, and a CTA "היום המומלץ לפתיחת מחזור" a, a "פנויים לידך" weekly bar chart. Screenshot: 
4. Tap "לפרטי המחזור" → round-details screen for "כדורגל רביעי", 10/10 roster, "הגיע הזמן לחלק לקבוצות" prompt, "עבור ללייב" (this game is live/about to start). Scrolled roster preview (3 of 10 names) appears (me) "דניאל" on the roster → full 10-player list tagged with a badge. Screenshot (manager) "מנהל" 
6. Force-stopped the app (am force-stop) and relaunched cold, Splash screen again ~14s, then **landed directly on the home screen** already logged in – card visible immediately – "המחזור הקרוב שלך" no – re-login, no onboarding, no achievement replay. Screenshot: 
8. Scrolled home further: an in-progress checklist (3/5 done "בוא נתחיל" availability set, joined a club, joined first round; not done: profile photo, invite a friend), a tip carousel, and an "ידעת ש..." an "אפליקציה" (invite friends) "last game recap" CTA at the very bottom. No "evening summary" card was visible anywhere on the home feed

Answering the persona's questions

- "המחזור הקרוב שלך" – **Can you tell at a glance when you next play?** Yes is the first content card on home, with date/time/location, both on first login and on every subsequent relaunch. This works well
- "שלך" **Whether you are registered?** Indirectly – the card title says badge shows on the thumbnail, but neither "✓ מלא" and a green (yours) literally states "you are in this game." Confirming requires drilling into .find your own name in a flat list → "הצג הכל" → round details → scroll .See F-P5-2
- – **What happened last time?** Not surfaced on home at all in this session no "last game" recap/summary card. (May exist elsewhere, e.g. per-game .history, but is not part of the return-to-app flow
- **How much re-orientation needed after weeks away?** Very little for the what's next" question (answered instantly on home); more for "what did I" .miss" (nothing surfaces this)

F-P5-1 – Post-login achievement carousel has no labelled skip, blocks the home screen

- Severity: P2
- Confidence: Confirmed
- Feature / Screen / Flow: Home screen, first render after login
- Evidence: Step 2 above; P5_01_achievement_carousel_3.jpg . Three ;celebration cards (goals/assists/acquaintances) auto-played in sequence "the only way found to advance was an undocumented "tap anywhere on screen failed with "could not get idle state" while this was uiautomator dump animating, consistent with a continuously-running particle/sparkle effect .that also blocks a clean escape
- affordance on any of the "דלג" Current behaviour: no visible X, "skip", or .cards; advancing requires tapping the (unlabelled) screen itself 3
- Problem: for a returning player who just wants to check "am I playing Thursday", 3 unskippable celebration cards sit between login and that .answer, with no discoverable way out other than trial-and-error tapping
- User impact: friction on every fresh login for an account with any stat history; particularly bad for P16 (non-technical) since "tap the empty part .of the screen" is not a labelled control
- X control on the carousel, and cap it to/"דלג" Recommendation: add a visible .one card (or a single combined card) rather than one card per stat category
- Effort: S

F-P5-2 – No explicit "you're registered" confirmation short of opening the full roster

- Severity: P3
- Confidence: Confirmed
- card → round details "המחזור הקרוב שלך" Feature / Screen / Flow: Home
- Evidence: Steps 3-5. The home card and the round-details header never state registration status in words; the only proof is finding your own name in the "דניאל" – roster list (which is not sorted or highlighted "הצג הכל" row-10 "tag, זה אתה"/"טוע" sits in normal list position with only a small .(marker
- Problem: relies on the user recognizing their own name in a flat list rather .("✓ אתה רשום למחזור") than an explicit affirmative state
- Recommendation: add a one-line explicit status line on the round card or and visually distinguish "טוע" in "אתה רשום ✓", "round-details header, e.g

.(suffix “(אתה)”, highlight row) the roster list

Effort: S •

F-P5-3 – No “what happened last time” surfaced on return

Severity: P3 •

;Confidence: Suspected (only the home-feed scroll was checked this session •
(a recap may exist elsewhere, e.g. inside a completed game’s own detail page

Feature / Screen / Flow: Home feed, returning-user session •

Evidence: Full scroll of home (P5_02 - P5_04 plus one further scroll) •
showed coach-announcement banner, upcoming-round card, day-availability
“bars, onboarding checklist, tips, invite-friends CTA – no “last evening
evening) “סיכום הערב” recap card despite the memory context noting a shipped
.summary) feature

Recommendation: verify whether evening-summary surfaces anywhere in the •
,return-to-app path; if it only lives inside the completed game’s own page
consider surfacing a “your last game” teaser card on home for returning
.users

Effort: S (verification) •

P16 – non-technical user (no gestures, no hidden menus)

session as P5, since P5 already (“דינאלי”) Continued in the same logged-in
.established the account with real history/roles needed for these tasks

Task: find your next game

,a single – “לפרטי המחזור” → home card “המחזור הקרוב שלך” Achieved via the
.clearly labelled button. No gesture required. **Pass**

Task: register

Not separately tested this session (the account was already registered via
mock login); for a genuinely new registration the path is the labelled
buttons on a game card (see P6 steps 12-13) – also a “הצטרף”/“הצטרף”
.single labelled tap, gated by a clearly worded signup modal. **Pass**

Task: see who else is coming

From round details, a preview of 3 names is shown inline; a labelled link •
.opens the full roster. **Pass** – labelled, no gesture (show all) “הצג הכל” •
.text-link pattern is used for club rosters too “הצג הכל” Confirmed the same •

Task: see your stats

→ Steps performed: tap the header avatar (top-right circle, home screen) •
directly: an editable name (edit player card) “ערוך כרטיס שחקן” opens
and an avatar picker grid, and (upload from gallery) “העלאה מהגלריה” field
P16_01_avatar_tap_opens_edit_form.jpg :save button. Screenshot “שמו והמשך” a
There is no stat, no goals/assists count, nothing read-only on this screen
.it is purely a profile-editing form –
→ Also tried the small chevron next to the “Teamder” wordmark on home •
.routes to the exact same edit-profile screen
Also tried the “≡” (hamburger-looking) icon on home → this is **not** a nav •
an admin action screen (join requests to approve) “בקשות” drawer; it opens
.specific to being a club manager

- reachable only through a ("כרטיס שחקן") Found a real per-player profile roster row's 3-dot "⋮" kebab icon (no text label) → popup menu → labelled "כרטיס שחקן". Tested against a teammate row. option, worked ("אילוואיי")
- opened a profile card (avatar, shared clubs, "הוסף לחברים", "שלו הודעה", "pair-chemistry blurb"). Screenshot P16_02_player_profile_only_via_kebab_menu.jpg
- Tested the identical action on **my own** roster row the ("דניאל · מנהל"): no remove/waitlist "כרטיס שחקן", kebab menu for self shows only one option correctly suppressed for self) – but tapping it **does nothing**: the popup closes and the screen stays on the roster list. Reproduced 3 times with verified-correct tap coordinates (menu visibly open each time, tap landed inside the option's row).
- Club-level stats ARE reachable via a clearly labelled button → tab מועדונים shows club → (club table and statistics) "טבלת המועדון והסטטיסטיקות" → club leaderboard (goals, assists "מובילי המועדון" top-scorer, club totals, and a clean sheets, penalties, etc.). This is a good labelled path, but it is does not "דניאל" – **"club stats/leaderboards"**, not "your personal stats appear in this club's leaderboard (he isn't a top performer), so this screen ".does not answer "what are MY numbers

F-P16-1 – No screen in this session showed the logged-in user's own stats

- Severity: P1
- Confidence: Confirmed (for the paths tried) / Suspected (that no path exists at all – a working path may exist elsewhere in the ~55-screen app that this session's navigation didn't surface)
- Feature / Screen / Flow: Profile / personal stats
- Evidence: every discoverable "profile" entry point from the home screen either opens the (header avatar, wordmark chevron, own-row kebab menu) edit-profile form (no stats) or silently fails; club stats screen shows leaderboards, not personal numbers. See bullets above and P16_01_avatar_tap_opens_edit_form.jpg
- Problem: "see your stats" – one of the four core tasks this persona was asked to complete – has no confirmed labelled path in this session's navigation. The closest thing (edit-profile screen) is an editing form with zero stat display
- User impact: a non-technical user cannot find "how many goals have I scored" without either being a club leaderboard leader (visible only incidentally, in someone else's search of the club table) or discovering an unlabelled kebab-menu route that, for their own row, doesn't currently work no-op described in F-P16-2 "כרטיס שחקן" Technical impact: matches the self below – the personal-stats screen may exist but the primary route to it (for self) appears broken
- Recommendation: verify statically where a personal stats screen actually lives, fix the routing so it's reachable from the header avatar or a clearly labelled "הסטטיסטיקות שלי" entry, independent of the edit-profile form
- Effort: M

is a dead tap on your own roster row (player card) "כרטיס שחקן" – F-P16-2

- Severity: P2
- Confidence: Confirmed
- Feature / Screen / Flow: Round-details roster kebab menu ("שחקני המחזור")
- Evidence: Reproduced 3x – kebab on self, tagged ("מנהל") popup shows → tap → popup closes, screen unchanged. The "כרטיס שחקן" single option

navigated correctly to that ("אלוואי") identical action on a different row
player's profile card on the first try

silently fails; other-player "כרטיס שחקן" Current behaviour: self-targeted
works "כרטיס שחקן"

Problem: an asymmetry between "view someone else's card" (works) and "view
your own card via the same control" (dead click, no error, no toast)

User impact: the one in-context path to "my card" from the roster you're
actually looking at doesn't work; combined with F-P16-1, self-stats have no
confirmed working route

Recommendation: fix the self-targeted branch of this action, or redirect it
explicitly to the edit-profile screen if that's the intended destination for
self

Effort: S

F-P16-3 – Real profile/stats view is only reachable via an unlabelled icon

Severity: P3

Confidence: Confirmed

"::" Feature / Screen / Flow: Roster row → kebab

Evidence: P16_02_player_profile_only_via_kebab_menu.jpg ; the working path
to any player's profile card is: tap a small 3-dot icon with no text label
on a roster row → popup → tap a labelled menu item. The 3-dot icon itself
carries no visible caption anywhere in the roster UI

Problem: while not a swipe or long-press, a bare "::" icon is not
self-explanatory to a non-technical user; nothing on the roster screen hints
that tapping it opens a menu, let alone that "player profile" lives inside
it

User impact: a non-technical user is unlikely to discover other players
profiles at all without accidentally tapping the kebab

Recommendation: make the player row itself tappable (row tap → profile)
reserving the kebab only for admin actions (remove/waitlist), or add a
visible affordance/label near the kebab

Effort: S

One thing that works well for this persona

roster expansion and the join/request-to-join buttons "הצג הכל" Round-details
– are both clean, single, clearly labelled taps with no gesture requirement
worth calling out as the pattern the rest of the app should match

Cross-cutting: "three products at once" positioning – runtime evidence

The onboarding sequence itself pitches three distinct value props back to
back: slide 1 is a **game finder** ("discover open games near you, join with
one tap, or meet new players"), slide 2 is a **club/roster management tool**
slide 3 is a ("build your regular squad, the weekly round opens itself")
smart/automation layer (weather, auto-fill waitlists, distance). Runtime
tab is pure game-discovery, the מחזורים navigation reinforces the split: the
tab is pure club-admin (roster CRUD, approve/deny requests, club מועדונים
leaderboards), and the home feed's gamified achievement carousel plus club
leaderboards constitute a third, stats/social layer bolted "מובילי המועדון"
onto both. No single screen unifies "find a game," "manage my club," and "see

my stats” – each lives in its own tab with its own mental model, matching the .audit’s static-analysis finding. **Confirmed at runtime**

was not encountered anywhere in this session’s `PromoteOrphanScreen` ,round details, roster, club stats, מועדונים, מחזורים, navigation (home profile edit) – consistent with, but not an independent proof of, the existing “no in-app entry point” finding. Not further pursued (out of scope .(for these three personas’ core journeys

P_ADMIN

P_ADMIN – runtime audit (club admin personas)

Device: emulator-5554, mock mode. All findings below were produced by driving the real running app (screenshots + taps), not by reading source. Screens . `phase2/shots/` referenced are in

F-ADMIN-1 – Club-detail hamburger menu is entirely non-functional (app-wide)

- Severity: Po •
- Confidence: Confirmed •
- Feature / Screen / Flow: Club detail screen → “≡” top-left action menu; also the “...” quick-actions menu on the game-detail screen •
- Evidence: Created club “TestClub” (steps below), opened its ≡ menu, tapped each row in turn •
מחזור חוזר, היסטוריית מחזורים, סטטיסטיקה, הזמן חברים למועדון. עריכת מועדון, לצפייה בכל השחקנים, צור
Every single tap – on the row text, and separately on the row’s icon – simply closed the bottom sheet with zero navigation. Repeated the same test on the pre-existing club normally the single most basic admin action. “עריכת מועדון”, “בקשות ממתיונות לאישור (2)”: including on “(2)”, identical result, including on “עריכת מועדון”, “ניהול שחקנים והממתיונים, חלוקת
Independently repeated on a game-detail screen’s “...” menu tapped עריכת מחזור, ניהול שחקנים והממתיונים, חלוקת
same result, sheet just closes. Screenshots: `p1_create_club_step2.jpg` area, and the row taps themselves – הזמן שחקנים פנויים –
show no state change (not saved as separate files since they show “nothing happened”)
Current behaviour: Tapping any row (text or icon) in this bottom-sheet menu closes the sheet and returns to the underlying screen •
with no navigation, no toast, no error
Problem: This is the primary/only discoverable entry point for: editing a club, viewing the full roster, approving pending join requests •
via that entry, game history, inviting members via that entry, leaving/deleting a club, editing a game, managing players/waitlist via
that entry, deleting a game
User impact: A brand-new admin (P1) who taps ≡ to edit their club or see the roster gets nothing – no error, the sheet just closes, so •
they have no idea whether they mis-tapped, whether the feature exists, or whether the app is broken. This is the single biggest
hazard to “can I run a club without learning the app?” – the most obvious admin gateway does nothing
Technical impact: Every row is likely wired to a shared `onPress` that either closes the sheet before firing navigation (dismiss races •
the nav call) or the row buttons aren’t actually pressable (icon-only touch target with the row itself non-interactive, or a missing
.`onPress` prop passed through a generic “MenuRow” component)
Recommendation: Reproduce with RN dev tools / breakpoint on this menu component; check whether •
`navigation.navigate(...)` is being called at all before/around `setVisible(false)` . This looks like a single shared component
bug (bottom-sheet menu), so one fix likely repairs it everywhere it’s used
Expected benefit: Restores admin’s ability to reach ~8 core management actions •
Effort: S (once root cause located, likely a one-line ordering fix) but blast radius is large so verify all call sites •

F-ADMIN-2 – Tapping the member-count pill opens club chat, which crashes in mock mode

- Severity: P1 (mock-mode-confirmed; unverified in prod) •
- Confidence: Confirmed (crash), Suspected (that tap-target is “wrong” by design – worth a product decision, not just a bug) •

- .pill under the club name "שחקנים" Feature / Screen / Flow: Club detail screen → the "23
- Evidence: On (native coord ~270,277 image-space). App navigated into "שחקנים" club detail, tapped the "23
- CommunityChatScreen → ChatView and immediately crashed with a red-box: "Render Error - getFirestore() called while USE MOCK_DATA is true. Fill in .env with your Firebase config to switch out of mock mode." Component stack: <ChatView/> at ChatView.tsx:82, <CommunityChatScreen/> at :14. Screenshot: phase2/shots/p2_chat_crash_getfirebase_mock.jpg .
- .Recovering required a full app force-stop + relaunch (Dismiss did not clear the error; it just re-threw on re-render)
- Current behaviour: The member-count pill - the single most natural tap target for "let me see who's in this club" - routes to the .club's group chat, not a roster
- Problem: (a) ChatView.tsx calls getFirestore() unconditionally on render without checking USE MOCK_DATA , so any code path that reaches chat in mock mode hard-crashes the whole app, forcing a relaunch. (b) Regardless of mock/prod, the natural "how many .members" tap target does not show members - it shows chat, which will confuse a first-time admin looking for a roster
- User impact: In mock/QA testing, this is an unrecoverable crash. In prod it might just be a misdirected tap (unverified - I could not .test prod-connected)
- Technical impact: ChatView.tsx:82 needs the same mock-mode guard the rest of the read/write layer has (see project ground .truth: converters already special-case mock elsewhere)
- Recommendation: Guard getFirestore() in ChatView behind the mock check (quick, low-risk); separately, consider whether the .member-count pill should open a roster view instead of/in addition to chat
- .Expected benefit: Removes a guaranteed QA-blocking crash; clarifies IA for new admins
- .Effort: S (mock guard) + product call on the pill's target

F-ADMIN-3 – Club-game creation from the global "+" silently targets the wrong club, with no picker

- Severity: P1
- Confidence: Confirmed
- Feature / Screen / Flow: מחזור למועדון → "+" → מחזורים (club game)
- Evidence: Logged-in admin owns two clubs: TestClub (just created) and From the global .(pre-existing) מחזורים, tab
- The wizard's step-1 banner read "מחזור למועדון" → "+" tapped TestClub was never offered, and - "מחזור למועדון: חמישי כדורגל" - the banner itself is not tappable (confirmed: tapping it does nothing, no picker opens). Completed the wizard anyway; the created game (confirmation dialog, screenshot p1_game_confirm_summary.jpg) was for .not TestClub
- Current behaviour: The "create a club game" entry point silently binds to one specific club (likely "first in list" / "primary club") with .zero UI to switch, and zero warning that a choice was made on the admin's behalf
- Problem: An admin with more than one club (increasingly likely as the product grows - quick games + real clubs) can easily create a .game under the wrong club without realizing it, since there is no confirmation step surfacing "this club" as an editable choice
- User impact: For P1 specifically - the moment they most want to schedule their brand-new club's first game - this flow quietly .schedules it for someone else's club instead. They'd only discover this by noticing the club name in the small banner text
- Technical impact: The wizard's initial club context is presumably taken from route params or the first "my club" in a list rather than .prompting
- Recommendation: Either always route from the club's own page (where context is unambiguous), or add a tappable "מחזור למועדון" .club-switcher in the wizard when the admin belongs to 2+ clubs
- .Expected benefit: Prevents mis-scheduled games for multi-club admins
- .Effort: S-M

F-ADMIN-4 – Club-creation "Continue" can trap a first-time admin in a bogus "club name required" loop

- Severity: P2
- Confidence: Confirmed
- Feature / Screen / Flow: מועדונים → "+" → יצירת מועדון חדש (1 step: פרטים).
- Evidence: Filled name "TestClub" + city "TelAviv" (both visibly populated). While the description field still had residual focus/edit
- App showed an unrelated "המשך" (Continue) state (typed then cleared text in it), tapped "יש שינויים שלא נשמרו - לשמור עכשיו?"
- Tapping "שמור" (Save) inside that dialog produced a second dialog (unsaved changes) dialog with (Save) "שמור" (Save) dialog with (unsaved changes) "שמור" (Save) dialog with (unsaved changes)
- while "TestClub" was clearly visible in the name field one screen behind it. - (Error: club name required) להזין שם המועדון
- Reproduced this exact sequence twice (screenshots p1_create_club_unsaved_dialog.jpg , p1_create_club_name_error.jpg ,

p1_create_club_name_error2.jpg). Once a text field was explicitly blurred first (tapped a neutral area before scrolling to Continue), the same Continue button advanced cleanly to step 2 with no dialog at all

Current behaviour: A dirty/focused text field at the moment "Continue" is tapped triggers a route-leave "unsaved changes" guard (as if the admin were navigating away/backing out), and that guard's own "Save" action validates against stale/empty state, producing a false validation error

Problem: This is exactly the kind of moment described in the audit brief – a new admin has to guess what's happening (their club makes it worse, not better ("שמור" tap) name is right there, why does it say it's missing?), and a plausible next move / "התעלם וצא" the) User impact: Confusing, could cause a first-time admin to think the app lost their work and abandon the flow (discard option is right there, tempting

Technical impact: The wizard is treating "advance to step 2" the same as "leave screen" for its unsaved-changes guard, and the guard's save-handler reads name from a different source (likely wizard/context state that hasn't been synced from the local TextInput) than what's rendered

Recommendation: Don't fire the "unsaved changes" guard on forward wizard navigation, only on true screen-exit (back button / tab switch); if the guard's own Save action is kept, make sure it reads the same live form state as the Continue button does
Expected benefit: Removes a confusing dead-end in the very first flow a new admin runs
Effort: S

F-ADMIN-5 – Numeric stepper fields in game-creation wizard keep stealing focus

Severity: P2

Confidence: Confirmed

Feature / Screen / Flow : מחזור חדש step 2 → wizard חוקים → "שחקנים בקבוצה" / "אורך המחזור" numeric fields

Evidence: After tapping the "5" (players-per-team) field's numeric text once, essentially every subsequent tap in the same vertical re- (button, confirmed via 3 separate attempts with pixel-precise crops "המשך" including on the visually-distinct) neighborhood opened the numeric keyboard on that same field instead of registering the intended tap. Only closing the keyboard with the hardware back button first, then re-screenshooting to get a keyboard-free layout, allowed a clean tap on Continue
Current behaviour: The stepper input appears to reclaim focus / reappear on re-render in a way that intercepts nearby taps

Problem: A real user doing the same (tap number to type an exact value, then try to continue) is likely to hit the same "my tap did the wrong thing" experience repeatedly
User impact: Adds friction/confusion to configuring game format – one of the two flows explicitly required for P1

Recommendation: Investigate whether the numeric TextInput has autoFocus re-triggering on each keystroke/parent re-render, or whether keyboard-close is racing the layout re-measure
Effort: S-M

F-ADMIN-6 – 3/4-team split IS fully editable, but the game-details summary card only ever shows team 1

Severity: P2

Confidence: Confirmed

Feature / Screen / Flow: Game detail screen → "הכוחות שחולקו" card (below "נהל כוחות").

Evidence: Opened a live 4-team game ("ליגת השכונה · 4 קבוצות", players 20/20). The "הכוחות שחולקו" card shows only "קבוצה אדומה" 5 → "נהל כוחות" with no pager dots, tabs, or working horizontal swipe (attempted swipe, no change) to see teams 2-4. Opened (players separately: this modal correctly lists all 4 groups (red/blue/green/yellow) full-roster, vertically scrollable, and "ערוך את הכוחות הקיימים" the tap-to-swap interaction works (tested: selecting a red-team player highlights all blue-team players as valid swap targets, confirmed via p2_4team_edit_modal.jpg)

Current behaviour: At-a-glance team info on the main game screen is incomplete for 3+ team games; the admin must dive into a +separate modal to see teams 2

Problem: For the explicit audit question "can an admin see and edit team splits, and what happens with 3 and 4 teams?" – editing works fine, but seeing is broken/incomplete on the primary screen once you go past 2 teams

User impact: Minor/moderate – doesn't block the task, but the main screen is misleading for exactly the multi-team case the audit was asked to check

Recommendation: Add a tab/segmented-control or swipeable pager to the summary card so all N teams are glanceable without opening the edit modal
Effort: S

F-ADMIN-7 – Club table sorts by raw win count, no games-played denominator anywhere – CONFIRMED

Severity: P2 •

Confidence: Confirmed •

table → Feature / Screen / Flow: Club detail → "אלופי המועדון" → "טבלת המועדון והסטטיסטיקות" •

Evidence: On the table's own caption reads, "סיכום מצטבר של כל החברות במועדון" • ממיון לפי ניצחונות, אז גולים, אז בישולים •

No games-played. Columns shown: (sorted by wins, then goals, then assists) / גולים / בישולים. (with sort indicator) / ניצחונות / שחקן / ניצחונות (with sort indicator) / גולים / בישולים. No games-played column and no win-rate/percentage anywhere in this table (checked for hidden horizontal scroll – none present). Total club games

played (210) is shown only as a club-wide aggregate stat card, never per player. Screenshot

p2_club_table_wins_sort.jpg

Current behaviour: A player who's played very few games but won them all ranks below a player who's played many games with a similar or lower win rate, purely because raw win count is the sort key – and the admin has no way to see games-played per player to sanity-check this

Problem: This directly confirms the static-audit flag. As a club accumulates history (P2's concern), this table gets systematically less meaningful – the "table" reads as a leaderboard of activity/tenure more than of actual performance

User impact: Misleading "who's the best player" signal, worse the longer the club runs

Recommendation: Add a games-played column and/or a win-rate secondary sort/display; consider defaulting sort to win-rate with a minimum-games floor

Effort: S (display) - M (if a rate-based sort needs new aggregation) •

F-ADMIN-8 – No in-app route from a quick game to a real club – CONFIRMED (matches static finding)

Severity: P2 •

Confidence: Confirmed •

action menu → Feature / Screen / Flow: Quick game detail → "ממשק מהיר בפארק" tagged מחזור מהיר → "..." •

Evidence: Opened the quick game's "..." menu; full list of 7 items

צרך חברים מהמועדון, toggle "מחזור למועדון בלבד", מחיקת מחזור. No. Screenshot p1_quickgame_menu_no_promote.jpg. "הפוך למועדון" / "צור מועדון" / "import from a quick (tab מועדונים in "+") promote-to-club option anywhere. Also checked the club-creation entry / "game" option – none present

Current behaviour: Matches the static finding exactly – PromoteOrphanScreen is unreachable from any menu, button, or toggle I could find at runtime

Problem: A group that started with a casual quick game (no club) has no discoverable way to formalize into a real club without losing their game history / starting over

User impact: Directly contradicts the app's own stated purpose (grow from casual pickup games into a real recurring club)

Recommendation: Surface an action in the quick-game menu (even a manual, non-push-triggered wrapper around the "הפוך למועדון" existing PromoteOrphanScreen would close this gap)

Effort: S (the screen exists; this is purely a missing navigate() call site, matching the static finding) •

F-ADMIN-9 – Club creation itself works well, is low-friction (5-6 taps)

Severity: P4 (positive finding) •

Confidence: Confirmed •

Feature / Screen / Flow: מועדונים → "+" → step wizard-2 → צור והיכנס •

Evidence: Once fields are filled and blurred cleanly (see F-ADMIN-4 for the trap to avoid), the full path is: tap "+", type name, type Result: club created, "צור והיכנס" city (free text, no network dependency), tap Continue, optionally toggle 1-2 permission switches, tap invite) "הזמינו חברים" onboarding card with a single "עכשיו צריך אנשים" admin auto-joined, and immediately shown a friendly "TestClub CTA that opens the native share sheet with a pre-filled invite link (friends

p1_club_created.jpg https://teamderfc.web.app/team/<id>?invitedBy=<uid>) – no extra typing needed. Screenshot

Current behaviour: Good – this specific path answers "can I run a club without learning the app?" with "yes," modulo F-ADMIN-4's confusing dialog if a field is left dirty

Recommendation: none needed beyond F-ADMIN-4

Effort: n/a •

F-ADMIN-10 – Venue search works offline, no live-API dependency

Severity: P4 (positive finding) •

Confidence: Confirmed •

Evidence: Game-creation location picker uses MapLibre/OpenFreeMap (OpenStreetMap data) and returned real results for “Park” (banner active throughout “מצב נתוני דמו”) with no Firebase/Google Places dependency, functioning correctly in mock mode

.Recommendation: none •

F-ADMIN-11 – Member roster at scale (40 members) could not be runtime-tested – entry point is broken

Severity: P1 •

Confidence: Suspected (blocked by F-ADMIN-1) •

.view all players) Feature / Screen / Flow : “לצפייה בכל השחקנים” (view all players) •

at 32 שליש ספורטק at 23 members; the largest visible club overall is כדורגל Evidence: The largest joined club in this account is is the exact one broken by F-ADMIN-1, on every “(לצפייה בכל השחקנים)” (not joined). The one menu item that should open a full roster club tested. I could not reach a full member-list screen by any other route (searched: club header pill → goes to chat and crashes, .see F-ADMIN-2; no roster link elsewhere on the club-detail page)

Problem: The audit’s specific question (“does anything become unusable at 40 members?”) could not be answered at runtime .because the screen that would show it is currently unreachable in this build

.Recommendation: Fix F-ADMIN-1 first, then re-run this specific check at the 40-member club •

.Effort: n/a (blocked) •

F-ADMIN-12 – Dormant vs. active club distinguishability – not conclusively testable, worth a follow-up

Severity: P3 •

Confidence: Suspected •

show only cover image, name, city, member count, and a join-status (“מועדונים פתוחים” / “מועדונים שלי”) Evidence: Club list cards badge – no last-activity date, no “last game X weeks ago” signal, no visual dimming for inactivity. The one open (non-member) club with no explicit “next game” countdown (יום ב', 21:00) showed only static schedule info (רביעי בלילה בירושלים) detail page I inspected I did not have a genuinely dormant club (zero games in months) in “המחזור הקרוב.” s page prominently shows כדורגל the way the mock dataset to compare directly, so I can’t confirm whether the app has any dormancy signal or whether it’s simply absent everywhere

Recommendation: If a mock club with an old/absent recent-game date can be added, re-test specifically for a “dormant” visual .treatment (badge, sort-to-bottom, etc.) on both the club list and club detail

.Effort: n/a (needs test data) •

Note on test-environment integrity (not a product bug)

after a forced app relaunch that was required to “המועדונים שלי” TestClub, created early in this session (F-ADMIN-9), had vanished from recover from the F-ADMIN-2 crash. This is almost certainly the mock in-memory store resetting to its seed data on process restart, not .a real persistence bug – flagged here only so this doesn’t get miscounted as a separate defect

P_GAMEDAY

P3 – Game manager runtime audit (game day: before / during / after)

Device: emulator-5554, mock mode. Formats exercised live: gv2-live (P9, 2 teams 5v5), gv2-live3 (P10, 3 teams), gv2-live4 (P11, 4 teams), gv2-resume (advanced

mode, no teams drafted). All findings below were produced by actually driving the device – screenshot before every tap that could be a “broken” claim, tap screenshot after, retried at least twice on any dead-looking control before . phase2/shots/ concluding anything. Screenshots referenced below live in

Governing metric: taps during live play

- pick scorer → pick → הוסף גול – Log a goal (scorer + assist): **3 taps** .Clean, no wasted motion .“אף אחד” assist or
- – Send a player home + auto-pull a waiting-team replacement: **4 taps** the replacement is) אישור → confirm → הלך הביתה → avatar (: menu pre-selected for you). This is the standout flow of the audit – see below
- – Resolve a tied mini-game via penalty shootout, per kick: **~5 taps** בעיטת פנדל → pick keeper → pick kicker → בעזע בעיטה → נכנס/הוחמץ.
- (target player → menu :) Cross-team player swap: nominally 3 taps but see F-P3-1 – in practice it costs more because the UI doesn’t visibly confirm success, so a manager under time pressure will likely repeat it

Findings

scrambles per-player goal counts and lags on-screen (“החלפה”) F-P3-1 – Cross-team swap

- Severity: P1
- Confidence: Confirmed
- roster, : avatar “קבוצות במשחק” Feature / Screen / Flow: Live match screen on gv2-live (2-team, 5v5, P9) → menu
- Evidence: shots/p9_swap_after_no_change.jpg (toast fired, on red, unchanged משה / on blue רון but the two team columns still show across two consecutive screenshots and a cancel-and-recheck) and / same game, ~10 min later) shots/p9_swap_scrambles_goal_counts.jpg (blue), reads 14 (“מבקיעים 4”) re-entered): the immutable goal log 18 משה (red, assist) אליון, 11 רון (blue), 4 אליון (red, assist) אליוני – i.e. once each for red. After the swap משה and אליון, scored twice for blue רון with (moved to red) רון actually took effect, the roster panel now shows with 0 – none (moved to blue) משה, with 0 אליון, only 1 goal badge of which match the goal log two inches above them on the same screen. Team aggregate score (2-2) stays correct throughout
- Current behaviour: swapping two players between the two active teams produces a success toast immediately, but the roster view doesn’t visibly update for an unpredictable interval (outlasted 3 screenshots / ~30s in this run), and once it does update, per-player goal totals are recomputed off the player’s *current* team slot rather than the actual scoring history
- Problem: the one number a manager relies on to settle “wait, who scored that?” arguments mid-game becomes wrong the moment anyone gets swapped between teams – and it looks authoritative (a plain badge next to the .name), not like a stale value
- User impact: wrong scorer counts read out loud during the game; likely to cause a repeat of the swap action because nothing on screen confirms it worked, compounding the very bug that’s confusing them
- Technical impact: per-player goal aggregation is apparently keyed to ,current roster slot” rather than the goal-event log’s own scorerId/team”

- so any mid-game roster mutation invalidates it. Matches the codebase's known pattern of derived/cached fields not tracking their source of truth
- Recommendation: recompute the per-player badge directly from the goal-event log (keyed by scorerId, independent of team assignment) rather than from a cached "goals this round" count on the roster slot; and make the swap optimistic/synchronous so the two columns visibly flip in the same render as the toast
- Expected benefit: eliminates a live, on-screen data-integrity bug that is directly visible to the organizer during the exact moment (mid-match) they have the least patience to double check the app
- Effort: M

F-P3-2 – "הלך הביתה" gives zero warning about mid-round stat loss (went home)

- Severity: P2
- Confidence: Confirmed (dialog text) for the missing warning; the underlying stat-loss mechanism is documented ground truth, not independently re-derived here, so treat that half as Suspected-but-established
- confirm → הלך הביתה, Feature / Screen / Flow: live match, : avatar menu dialog, gv2-live3 (and identical dialog on gv2-live)
- Evidence: dialog text observed verbatim "השחקן יוצא מהמחזור ותישאר קבוצתו במגרש חסר – תוצא ההחלפה, והשעון ימשיך לרוץ. אפשר להחזיר אותו בכל רגע." – covers roster-shortage and clock behaviour, says nothing about goals/assists
- Current behaviour: the exact same dialog fires whether the departing player has 0 goals this round or several
- Problem: `commitRoundStats` credits only the end-state roster (established ground truth), so sending home a player who scored this mini-game silently drops their goal/assist credit for it – and the UI that asks for confirmation never mentions this is a possibility
- User impact: a manager who benches their top scorer for five minutes mid round may permanently lose that scorer's goals for the round with no warning and no easy way to notice afterward
- Technical impact: none beyond the existing stats-crediting gap
- Recommendation: when the departing player already has goals/assists in the *current* mini-game, add a line to this specific dialog naming what's at risk (e.g. "X's N goals in this round will not be counted")
- Expected benefit: closes the gap between a known backend limitation and the one screen where a human could actually avoid triggering it
- Effort: S

F-P3-3 – A player who "went home" still looks like an active player in the main roster card

- Severity: P2
- Confidence: Confirmed
- card vs. the separate "קבוצות במשחק", Feature / Screen / Flow: live match "הלכו הביתה" gv2-live
- home, he continued to render as a normal "קבוצות במשחק" Evidence: after sending red-team row (avatar, 0-goal badge, no strike-through/greyed style/badge) in indistinguishable from players still on the field – while – "קבוצות במשחק" card with "הלכו הביתה" simultaneously appearing, correctly, in the separate "tap to bring back" hint and an elapsed-away timer
- Current behaviour: the "who's actually playing" signal only exists in a second card that requires scrolling past the main roster to reach

- Problem: directly fails the audit's core legibility bar – “can you tell who is playing, who is waiting, without thinking?” No, not from the primary roster view.
- the natural place to) “קבוצות במשחק” User impact: a manager glancing only at (look) will count a player who left as still present, e.g. when deciding whether a team is short a body.
- Technical impact: none identified beyond a missing status flag in the row renderer.
- tag on the “בבית” Recommendation: grey out / strike the name / add a small row itself the instant a player is marked gone, instead of only in the separate card.
- Expected benefit: one glance at the main card becomes trustworthy again.
- Effort: S.

F-P3-4 – Waiting-team roster doesn't flag a member who's currently filling in elsewhere

- Severity: P3.
- Confidence: Confirmed.
- “קבוצות ממתינות”, Feature / Screen / Flow: gv2-live3 (P10, 3 teams) קבוצה ירוקה, after בן a gap on was pulled in to fill a gap.
- correctly shows a small star בן Evidence: the on-field red-team row for Ben filling in from the green) “בן משלים מקבוצה ירוקה” badge + footnote (team) – nice touch. But the waiting green-team card, scrolled to below among its 5 members with no “currently out” mark בן still lists.
- Problem: a captain scanning the bench to see who's actually free to sub in as available בן next would still count.
- User impact: minor – the correct information exists elsewhere on screen just not here.
- Recommendation: mirror the same footnote/badge on the waiting-team card.
- Effort: S.

Explicitly flagged items – verified or refuted

R-1. “Scoreboard should show which mini-game number” – ALREADY WORKS

,Confirmed on all three multi-team live screens (gv2-live, gv2-live3, gv2-live4): a chip sits above the score at all times. No fix needed “משחק 2”.

– “R-2. “Advanced live screen with no teams drafted should show a popup

עבור ללייב **ALREADY WORKS**. Verified on gv2-resume end to end. Tapping explains that – “עוד לא חולקו כוחות” – no teams split produces a clear modal without teams there's no team management, no games, no goals/assists, and no per-player stats – with two honest choices “לחלוקת” or “כוחות”.

shots/resume_no_teams_popup.jpg :Screenshot “כוחות”.

New observation (not previously flagged, informational only, Severity - lands on a screen that really is just a “המשך בלי כוחות” P4/idea): choosing no score, no roster, no – (a pause button, רץ, 00:00) bare running clock goal entry of any kind. That matches what the popup promised, but it's a strikingly empty full screen; a one-line reminder of the popup's warning persisting on this screen (not just in the now-dismissed modal) would help.

R-3. “Penalty shootout should offer undo of the last kick” – ALREADY WORKS, verified at runtime (not just in code).

On gv2-live3: forced a tie recorded one kick, “שובר שוויון בפנדלים”, ended the mini-game, chose (1-1), got a confirm, “בטל את הבעיטה האחרונה”, (1-0) scores past (1-1).

dialog naming the kicker ("הבעיטה של רון תימחק... אפשר לרשום אותה מחדש"),
confirmed, and the board correctly reverted to 0-0
:turn back to blue, kick 1. Screenshots / פנדלים
,and the post-undo state (described above shots/shootout_undo_before.jpg
.not separately saved due to time). Works cleanly, no notes
**R-4 (ground truth, not re-derived). "Substituting a player OUT mid-round
.erases their stats for that round – does the UI hint at this?" – NO HINT
.See F-P3-2: the confirmation dialog text is generic and never mentions stats**

What's genuinely good – said once, not padded

- **The waiting-team fill-in flow (P10) is the best-designed screen in this audit.** Sending a player home on a 3+ team game immediately opens "X" with the correct number of open slots and the השלמת שחקנים לקבוצה next-up waiting player *pre-selected* – confirming is one tap. Whole substitution-from-bench flow: 4 taps, no dead ends
- **team and 4-team live screens hold together.** Waiting teams render as-3 nothing overlaps or truncates ("הבאה בתור" / "אחריה"); clearly ranked cards .even with gv2-live4's full 20-player, 4-team roster on screen
- **Elapsed-time display is legible without extra taps.** It counts up toward the configured duration, turns red approaching it, and switches to a "+MM:SS" overtime readout past the limit – a manager can tell "we're .running long" at a glance, no alert dialog needed

Not reached / out of scope for this pass

- P12 "format picker to its edges" (create-game wizard's unusual formats ;e.g. non-5v5) was not exercised live due to time budget after the above .no finding either way – mark as not verified, not as fine

P_PLAYERS

Players audit – P4 / P15 / P14 / P7 / P8

Runtime audit on emulator-5554, mock mode, logged in as "דניאל" admin tag on club/מנהל who carries a (daniel@example.com)
No way was found in the session to log in as a true non-admin member, so
.every finding below that depends on role is flagged

All screenshots referenced live in

/private/tmp/claude-502/-Users-matan-Projects-soccer/ea3882bc-0fa7-40b3-a47b-a4c3b4764f64/scratchpad/audit2/phase2/shots/

P4 – ordinary club member

on your OWN roster row does work (prior claim refuted) "כרטיס שחקן" – F-P4-1

- Severity: P3 (this is a correction, not a bug)

- Confidence: Confirmed (reproduced with before/after screenshots)
- “כרטיס שחקן” → (:) Feature / Screen / Flow: Game details → roster → row overflow
- showing “דניאל”, Evidence: p4_ownrow_menu_before.jpg (: menu open on own row
- tap opens a p4_ownrow_playercard_after.jpg, “כרטיס שחקן”, a single option
- real screen: name, email, 12 badge slots, 5 earned in bronze). Path: Home
- “לפרטי המחזור” → scroll → “הצג הכל” → on own row : “כרטיס שחקן” (4 taps).
- on your own roster row opens a כרטיס שחקן Current behaviour: Tapping
- populated screen every time it was tried.
- Problem: none – the earlier “no working path” finding could not be
- reproduced with this account.
- User impact: n/a – feature works.
- Caveat: only tested from an admin-tagged account; a true non-admin account
- was not available in this session, so a role-specific variant of the bug
- cannot be fully ruled out.
- Recommendation: none.
- Effort: -.

F-P4-2 – Own-row player card and other-row player card show completely different, and neither shows numbers

- Severity: P2
- Confidence: Confirmed
- tested on self, “כרטיס שחקן” → : Feature / Screen / Flow: roster row
- (דניאל) and on another player (רון)
- Evidence: own card = p4_ownrow_playercard_after.jpg (12 badge slots, tier
- labels only, e.g. “שערים – ברונזה”, no goal count anywhere on the screen
- ;badges are not tappable – tried 3x at different pixels, no response
- / “הוסף לחברים” / “שלח הודעה” / “אתה ורון – עדיין לא שיחקתם יחד” – badges, no stats at all
- Current behaviour “כרטיס שחקן” is two unrelated screens depending on whose
- row you tapped: a badge-tier screen for yourself, a friend/social screen for
- anyone else. Neither shows a single hard number (goals, assists, evening
- score) a member could point to.
- Problem: A member asking “what happened last time” or “how am I doing
- gets no numeric answer from the one screen literally called “player card
- .The real numbers live three menu-levels away (see F-P4-3).
- User impact: confusing, inconsistent naming for the same menu item
- members likely never find their real stats.
- Technical impact: two very different components apparently share one
- entry-point label.
- Recommendation: either merge into one screen with both social actions and
- numeric stats, or rename the entry points so users know what they’ll get
- Effort: M

F-P4-3 – Real personal stats and the club standings table both exist and work, but are buried

- Severity: P3
- Confidence: Confirmed
- “סטטיסטיקה” → Feature / Screen / Flow: hamburger menu (☰, top-left of home)
- :club standings (stats from home: bell/hamburger icon, then 3)
- bottom tab מועדונים → club card → “טבלת המועדון וסטטיסטיקות” ~5 scroll
- screens (3 taps + heavy scrolling).

- Evidence: personal stats screen shows 18 penalty conversion/save rates, and a “buddies” section; club table /is a full sortable-looking leaderboard with wins/goals (“אלופי המועדון”) s own row is visible at rank 7 of 10 assists per player, and .Current behaviour: both screens are real and functional
- Problem: the club-standings table (the actual answer to “where do I stand vs everyone else”) sits after ~5 full-screen swipes of unrelated content ,on the club statistics page (badges, fun-fact percentages, chemistry pairs) .with no jump link or anchor
- User impact: an ordinary member is unlikely to scroll that far without .already knowing the table is there
- Recommendation: surface a “my rank” chip near the top of the club stats .screen, or link directly from the personal stats screen
- Effort: S

F-P4-4 – Game history exists but “my” evening summary shows a different person

- Severity: P1 (also filed under P14 as the sharper version, F-P14-1)
- Confidence: Confirmed
- the one past → “היסטוריית מחזורים” → Feature / Screen / Flow: hamburger “סיכום המחזור שלי” → game
- Evidence: p14_my_summary_wrong_player.jpg – shows club “מכבי חולון”, “מתן לוי”, score 9.5, while the logged-in user is “חמישי כדורגל” (מנהל tag visible) and is confirmed present in that game’s own roster .See F-P14-1 for full detail
- User impact: “what happened last time, for me” returns someone else’s .night entirely

P15 – the social player

F-P15-1 – Club-wide chat crashes the app on open (native Fabric crash)

- Severity: Po
- Confidence: Confirmed – reproduced twice with before/after screenshots matches an independently-found crash from another audit pass .(in the same shots folder p2_chat_crash_getfirebase_mock.jpg)
- Feature / Screen / Flow: bottom tab – “חברי המועדון” (club members chat, globe icon) first row → “חמישי כדורגל”
- Evidence: p15_chat_crash_1.jpg and p15_chat_crash_2_confirmed.jpg – both show a Fabric native RedBox: “Exception thrown when executing [...] UIFrameGuarded – addViewAt: failed to insert view [...] into parent at index 0. The specified child already has a parent. You must call tab → tap ציטים → removeView() on the child’s parent first.” Steps: Home the top row (2 taps) → immediate crash, both times, at different view IDs view [37052]/[37054] first time, [866]/[868] second time – same error) .(class, different instances, so this is systemic, not a one-off
- Current behaviour: app RedBoxes immediately; after dismiss the screen goes .white and needs a full relaunch to recover
- Problem: the flagship club chat – the one place all members of a club are .meant to talk – cannot be opened at all in this build
- User impact: total loss of the club-wide social surface

- Technical impact: looks like a view being mounted twice into the RN Fabric .tree (list virtualization / duplicate mount on chat screen entry)
- Recommendation: reproduce outside mock mode and get a stack trace; this is .release-blocking for the chat feature
- Effort: M (diagnosis)

F-P15-2 – Per-game chat throws a real render error, and the app's own error boundary leaks a raw English debug string to Hebrew users

Severity: P1 •

Confidence: Confirmed •

• :tab → any game-scoped chat row (tested צ'אטים :Feature / Screen / Flow
"כדורגל רביעי - משתתפי המחזור")

• Evidence: p15_chat_getFirestore_mock_error.jpg – Render Error: "getFirestore called while USE MOCK_DATA is true. Fill in .env with your Firebase config to switch out of mock mode." at config.ts:92:20, thrown from .GameChatScreen.tsx:21 via (export function ChatView({ scope, parentId, title, canModerate }) ChatView.tsx:82 p15_chat_errorboundary_leak.jpg

after dismissing the dev RedBox, the app's production error boundary – is reached, and it prints the same raw English string ("משהו השתבש") .verbatim under the Hebrew explanation, on a 100% RTL Hebrew screen

• Current behaviour: opening a game chat crashes with a Firebase-in-mock-mode error; the crash is caught by an error boundary, but that boundary shows .the developer's English exception text to the user

• Problem: two bugs stacked – (1) ChatView calls getFirestore() directly ,instead of going through whatever mock-data path the rest of the app uses so chat is unusable in mock mode at all; (2) independent of mock mode, the production error boundary is not sanitizing error.message before rendering it to real Hebrew users – any unrelated crash would leak an .English stack-adjacent string the same way

• -User impact: chat unusable in mock mode (audit/demo/QA impact); the error .boundary leak is a real-user-facing polish/trust issue on any crash

• Recommendation: (a) route ChatView's data access through the same mock/prod switch the rest of the app uses; (b) never render raw screen – show a generic "משהו השתבש" in the user-facing error.message .Hebrew message and log the raw string to telemetry instead

• Effort: S-M

F-P15-3 – Chat threads are numerous and populated (surface is not vestigial by design), but every entry point tested is broken

• Severity: informational (context for F-P15-1/2)

• Confidence: Confirmed

• Evidence: צ'אטים : 10 real threads – 1 club-wide + 9 game-scoped subtitle, i.e. the feature "משתתפי המחזור" each with a distinct title and .is clearly intended to be a living surface, not a stub

• Note: both thread types tested (club-wide and one game thread) crash on – open, so "does chat feel alive" could not be judged from message content only from the fact that ten separate threads exist and are populated with .membership

F-P15-4 – The “other player” card is a genuine social surface (message, add friend, shared clubs, relationship line)

- Severity: informational / positive finding
- Confidence: Confirmed
- on someone else's row “כרטיס שחקן” → : Feature / Screen / Flow: roster row (tested: רון)
- Evidence: card shows name+avatar “שלח הודעה” (send message), “הוסף לחברים” (add friend), “מועדונים משותפים (1)”, and a plain-language line “אתה ורון – עדיין לא שיחקתם יחד, הזמן אותו למחזור כדי להתחיל היסטוריה” (you and Ron – haven't played together yet, invite him to a game to start a history).
- Assessment: this is alive, not vestigial – it speaks in relationship terms not rankings. It does not, however, show any shared moments/history content, count is non-zero – it only offers a count “מועדונים משותפים” even when a no drill-in.

F-P15-5 – Evening summary carries a share button aimed at moments, not just numbers

- Severity: informational / positive finding
- Confidence: Confirmed (button present and prominent), Suspected (whether the share sheet actually functions was not tested end-to-end)
- Evidence: p14_my_summary_wrong_player.jpg /scrolled view shows a full-width “ ⚡ שתף את סיכום המחזור ” button at the bottom of the personal evening summary card.
- Assessment: the hook for turning a stat card into a shareable social moment exists; combined with F-P14-1 (wrong player shown), sharing this card today would share the wrong person's evening.

P14 – the competitive player

F-P14-1 – “My evening summary” shows a completely different player from a different club

- Severity: P1
- Confidence: Confirmed
- Feature / Screen / Flow: game details (the one completed game in history) “סיכום המחזור שלי” (personal evening summary) vs. the logged-in identity, (personal evening summary) and that game's own roster
- Evidence: logged-in profile = “דניאל”, daniel@example.com (p4_ownrow_playercard_after.jpg) That shows the same כרטיס שחקן. That is confirmed present in the completed game's, same account, tagged listing shows “שחקני המחזור” (roster 15/15 club “מתן לוי”, Yet “סיכום המחזור שלי” evening score 9.5, rank “8.7”, יום רביעי, “12 משחקים · מכבי חולון”, “3 מתוך 15” (p14_my_summary_wrong_player.jpg). “מכבי חולון” does not match the game's actual club anywhere else in the app, “חמישי כדורגל”, Current behaviour: the personal variant of the evening-summary (“שלי”) card ignores the logged-in user and renders an unrelated mock profile
- Problem: the exact feature P14 cares about most – “can I explain my own evening score to a friend” – currently cannot even locate the right friend. The GENERAL (non-personal) checked immediately after, is correctly wired to the real roster (king of goals אין 6, king of assists 4, all names match the real etc. –

- roster) – so the bug is isolated to the personal variant, not the whole .summary pipeline
- Technical impact: personal-summary generation is not keyed off the .requesting user correctly (or is falling back to unrelated seed/demo data)
- Recommendation: audit whatever selects “which player’s data goes into the card” – likely a missing/incorrect uid filter, or a leftover fixture של .value that should have been replaced per-request
- Effort: M

F-P14-2 – The same player’s goals/assists disagree between two stat screens

- Severity: P2
- Confidence: Confirmed
- Feature / Screen / Flow: club leaderboard (מועדונים → club → “טבלת המועדון וסטטיסטיקות” → “אלופי המועדון” table) vs. personal stats screen (hamburger → “סטטיסטיקה”), same account, same club, same session
- Evidence: p14_club_table_daniel_16g_8a.jpg – club table row 10 ניצחונות, 16 גולים, 8 בישולים. p14_personal_stats_daniel_18g_11a.jpg – personal “המספרים שלך”: 18 שערים, 11 בישולים, 28 מחזורים.
- Problem: goals differ by 2 (16 vs 18), assists differ by 3 (8 vs 11), for the identical player in the identical club, viewed minutes apart
- User impact: a competitive player comparing “my club rank” to “my personal card” would immediately notice the mismatch and lose trust in every number .in the app
- Recommendation: these two views are clearly reading from different aggregates (club-level rollup vs. per-user rollup); reconcile them to a .single source of truth or document why they intentionally differ (e.g. different date windows) – currently there is no such explanation shown to .the user
- Effort: M

F-P14-3 – Personal “buddy” stats are empty despite 28 logged games, while club-level chemistry for other players is populated

- Severity: P2
- Confidence: Confirmed (empty state), Suspected (root cause)
- Feature / Screen / Flow: hamburger (tiles 6) → “החבריה שלך” → “סטטיסטיקה” → most frequent partner, winning duo, your closest rival, your toughest (rival, who assisted you most, who you assisted most)
- Evidence: all 6 tiles read not enough games “עדיין אין מספיק מחזורים” (yet) for an account with 28 logged on the same screen. By contrast, the club-level section (for other players) shows “כימיה במועדון” populated numbers in the same session, e.g. “אלין + אלוואי – 27 משחקים יחד”, “14 ניצחונות יחד” (screenshots from club stats scroll).
- Problem: either the personal chemistry aggregation is broken for this account specifically, or the account genuinely never shares 30s+ with any one teammate consistently enough to qualify – the UI gives no way to tell .which, and the empty-state copy doesn’t explain the threshold
- Recommendation: verify the per-user pair-stats pipeline for this account ; if the empty state is “working as intended,” surface the actual threshold instead of a flat (“e.g. “play 3 more games with someone to unlock this) “not enough games”
- Effort: S (copy) / M (pipeline audit)

F-P14-4 – Club “leaders” board looks implausible: one player wins clean-sheets AND top-scorer

- Severity: P3
- Confidence: Confirmed (as displayed), Suspected (as a real bug vs. mock (data quirk
- “מובילי המועדון” → Feature / Screen / Flow: club statistics
- Evidence: “אליו” is shown as: מלך השערים (58 goals, 22%), מלך הבישולים (26), מלך הניצחונות (22), מלך שערים נקיים – (61 clean sheets, הכי מתמיד of 7 categories, including a goalkeeper stat (clean sheets) 5 – (40) alongside a striker stat (goals) for the same person
- Problem: to a player scrutinizing the numbers (P14’s exact job), one person owning both “most goals” and “most clean sheets” reads as either a ,data-generation artifact (mock) or, if this pattern exists in production .a sign the clean-sheet stat is being credited to the wrong role/player
- Recommendation: sanity-check clean-sheet crediting logic against player position/role in real data; treat this as low-priority in mock mode but .worth a quick real-data spot-check
- Effort: S (verification)

F-P14-5 – Badge tiers on the player card have no drill-down to the underlying number

- Severity: P3
- Confidence: Confirmed (not tappable), Suspected as a real gap vs. intended
- Evidence: p4_ownrow_playercard_after.jpg – 12 badge icons with tier labels under 5 of them; tapped 3× at slightly different pixels on “ברונזה” like .badge specifically, no response any time (goals) “שערים” the answers nothing about how many goals were actually “ברונזה” :Problem scored or how far from the next tier the player is – exactly the kind of .vagueness P14 was tasked to flag
- Recommendation: make badges tappable to show the underlying count and next .tier threshold, or put the count directly on the badge face
- Effort: S

P7 – stranger with no club, finding a game

F-P7-1 – A coherent discovery path exists via the map, ~5 taps from home

- Severity: informational / positive finding
- Confidence: Confirmed
- tab → map icon (📍, top toolbar) → מחזורים → Feature / Screen / Flow: Home full game → “לפרטים” → tap a pin → bottom preview card → “מפת המחזורים” → details (request to join) “בקש להצטרף”
- Evidence: tapped the Haifa pin, got a preview card field “שישי בוקר חיפה”, opened a full “לפרטים” rating 4.9, format 6×6, then 07:30, “גרין הוקי”, game-details screen (10/12 players, weather/duration/roster preview, Waze .button “בקש להצטרף” nav button) ending in a
- Assessment: the mechanism a stranger needs – see a game on a map, preview it, ask to join – exists and works end-to-end. It is not, however, the (see) “המחזורים שלי” tab opens on מחזורים default landing view: the .F-P7-3), and the map is one small unlabeled icon among three in the header

F-P7-2 – Map filters are minimal: date only, plus one unexplained toggle

- Severity: P2

- Confidence: Confirmed •
- header controls: search box מפת המחזורים Feature / Screen / Flow •
 ("חיפוש לפי עיר או מגרש"), filter icon, "מותאם"/"מחר"/"היום"/"כל המחזורים" chips
- opens nothing but a plain month calendar date-picker (Evidence: "מותאם" no time-of-day, no other fields). The filter icon (leftmost, sliders glyph) toggles the map's pin-cluster total between 10 and 5 with zero visible panel, label, or confirmation of what changed – verified by toggling it on and off twice and screenshotting the cluster count each time (10→5→10). No area/city selector, no skill/level filter, no format (5v5/6v6/7v7) filter, no price/cost filter, and no "one-off vs. recurring" filter are present anywhere in this screen
- Problem: for a stranger evaluating a genuinely public list of games, the filters a real person would ask for first – where, what level, what format, is it a one-time pickup or a standing weekly game, does it cost anything – are entirely absent. The one non-date filter that does exist gives no feedback about what it filtered
- User impact: a stranger has to eyeball pins on a full-country map and open each one to learn format/level/cost
- Recommendation: add area/level/format/cost/recurrence filters to the map toolbar; give the mystery filter icon a label or a bottom sheet showing what it toggles
- Effort: M •

tab view is club-scoped, not discovery-scoped מחזורים F-P7-3 – The default

- Severity: P2 •
- Confidence: Confirmed •
- list default מחזורים, Feature / Screen / Flow: bottom tab "המחזורים שלי" (closed) Evidence: every card in the default list carries a "סגור למועדון" (to club) tag; the list is entirely games from clubs the account already belongs to. The public/joinable card visible in this "משחק מהיר בפארק" list is only visible because the account already has some club affiliation surfacing it – a truly club-less account was not testable in this session (see below), so whether this list is empty for a genuine zero-club stranger is Suspected, not Confirmed, but the header copy "המחזורים שלי" and the closed-club tags strongly imply it would be
- Recommendation: for a stranger, default the tab to the map or an explicitly public/open list instead of a "my games" list that will be empty or irrelevant
- Effort: S-M •

P8 – organiser short of players

F-P8-1 – Three real mechanisms exist on the game overflow menu

- Severity: informational / positive finding •
- Confidence: Confirmed •
- Feature / Screen / Flow: game details → "..." overflow menu (top-left)
- Evidence: menu lists "ערוך מחזור", "ניהול שחקנים והממתינים", "חלוקת כוחות", "הזמן שחקנים פנויים", "צרף חברים מהמועדון", a "מחזור למועדון" toggle, "יציאה מהמחזור", "מחיקת מחזור" – 3 of these directly serve an organiser short of players

- Assessment: the toolkit an organiser needs (invite the club, invite strangers/fillers, open the game beyond the club) is all in one menu, 1 tap from game details

F-P8-2 – “הזמן שחקנים פנויים” exists but was untestable on a genuinely short game (invite available players)

- Severity: informational, Suspected for the real question (“would a “stranger show up by Tuesday
- Confidence: Confirmed (screen and copy exist and render); Suspected could not be verified because – (whether it actually reaches strangers) .the only club game available to test was already 10/10 full
- Evidence: `p8_fillers_no_players_found.jpg` – screen copy “ישלח הזמנה לשחקנים פנויים באזור המחזור, בהדרגה, עד שהמחזור יתמלא. כל שחקן מקבל הזמנה אחת בלבד” (we’ll send invitations to available players) in the game’s area, gradually, until it fills; each player gets one button, but the game tested “שלח לכולם בפעימות” (invitation only) and a (“לא נמצאו שחקנים פנויים שמתאימים למחזור הזה”) was already full, so the empty result is expected and does NOT demonstrate a bug – it just means this specific mechanism could not be exercised meaningfully in this session.
- Recommendation for a future pass: re-test this exact flow on a game that is genuinely short 1-3 players, in an area/mock-dataset that has fillers registered, to see whether the pool is ever non-empty

F-P8-3 – “צרך חברים מהמועדון” correctly detects a full roster and routes to waitlist

- Severity: informational / positive finding
- Confidence: Confirmed
- “צרך חברים מהמועדון” → Feature / Screen / Flow: overflow menu
- Evidence: screen lists club members not yet in the game with checkboxes and states plainly “ההרכב מלא - השחקנים שתבחרו יתווספו לרשימת ההמתנה. הם יקבלו התראה” (roster is full – selected players go to the waitlist and) – correct behaviour for a 10/10 game .get notified)
- Assessment: works as intended; only reaches existing club members, not strangers, so it does not by itself solve P8’s “1-3 more people from .outside” problem – that’s what F-P8-2 is for

F-P8-4 – Share icon on game details did not visibly respond

- Severity: P3
- Confidence: Suspected – could not confirm broken per the mandatory protocol (only 2 tap attempts made at slightly different pixels within the time budget, no clean before/after pair showing a definite failure, and it is plausible the tap opens a native Android share sheet that this capture .(method missed rather than the app doing nothing
- Feature / Screen / Flow: game details header, share (🔗) icon, top-left cluster of 3 icons
- Evidence: 2 tap attempts at (200,264) and (202,262) [full-res], both on a .full game; screen state unchanged in both screenshots 10/10
- Recommendation: a future pass should retry this specifically with 2 more attempts and check `adb shell dumpsys activity` for a launched share .intent before calling it broken

- The chat crash (F-P15-1/2) was independently reproduced by a separate audit pass in this same session set (p2_chat_crash_getfirebase_mock.jpg) which corroborates it as systemic rather than a one-off tap-coordinate artifact – this satisfies the brief’s “screenshot before/after, retry twice” bar with an extra independent data point
- club table, evening סטטיסטיקה Every numeric-stats screen found (personal summaries) is real and rendering real numbers – the app is not “all broken.” The two Po/P1 findings that matter most are: chat crashes on open (F-P15-1/2), and the personal evening-summary card shows the wrong person (F-P14-1). Both are concrete, reproduced, and screenshotted

CHAOS_ANDROID_PERF

Phase 2 – Chaos / Android Platform / Performance audit

Device: emulator-5554 (Teamder_Phone_2, 1080x2400), app com.studiogameslime.soccerapp
MOCK MODE (“מצב נתוני דמו – לא קיים חיבור ל-Firebase” banner, permanently on regardless of wifi/data state – see caveat under F-P17-4). All taps below use full-resolution coordinates confirmed either by a fresh uiautomator dump or by a (1080x2400) before/after screenshot pair, per the mandatory proof protocol

NOTE ON SCOPE: BRIEF.md (general audit brief) says runtime is unavailable and to do static analysis only. This is stale for this phase – EMULATOR.md and the phase-2 task explicitly assign a working, launchable emulator and instruct driving the device. I followed the more specific, current instructions and drove the device for real

/Screenshots referenced below are on disk at /tmp/*.jpg (not copied into the shots folder individually – this run leaned on the uiautomator+snapshot loop rather than a curated shots folder; the two load-bearing sequences, the crash and the goalTally bug, are described with exact evidence inline since they are the highest-value findings

CHAOS USER findings .1

F-P17-1 – Rapid navigation triggers a fatal, unrecoverable Fabric view-tree crash

- Severity: Po
- Confidence: Confirmed (reproduced once, full native stack trace captured via logcat)
- list (chats) → Feature / Screen / Flow: Bottom tab navigation
- Evidence:
- Sequence: 12 taps across the 4 bottom tabs (בית→מועדונים→מחזורים→צ'אטים 3x) fired back-to-back with no delay, then, ~30s later, one more tap opening a chat row
- adb logcat

```
FabricUIManager: Exception thrown when executing UIFrameGuarded java.lang.IllegalStateException: addViewAt: failed to insert view [3744] into parent [3746] at index 0 Caused by: java.lang.IllegalStateException: The specified child already has a parent. You must call removeView() on the child's parent first. at ReactClippingViewManager.addView / SurfaceMountingManager.addViewAt ReactHost: Unhandled SoftException ... Starting React Native destruction
```

- On-device: a red “Exception thrown when executing UIFrameGuarded” dev screen – appeared (dev-mode RedBox). Pressing DISMISS (ESC) did **not** recover the app

- it left a permanently blank white/gray screen with only the status bar rendering
- it did not self-heal after 4+ seconds. Only `am force-stop` + relaunch recovered it
- Current behaviour: Rapid tab-bar switching (well within plausible real-user "impatient tapping" speed) leaves the native Fabric view hierarchy in a state where a view is attached under two parents at once. The corruption doesn't crash immediately – it surfaces on a **later, unrelated navigation** (opening a chat row), making the root cause hard for a user or QA to connect to what actually broke it
- Problem: This is a native `IllegalStateException`, not a JS exception – it kills the whole ReactHost/instance. The dev RedBox (DISMISS/RELOAD) only exists in this debug build; in a **production** build there is no RedBox, so the same native exception would very likely be an unhandled crash / hard app close, with no recovery path other than the user manually reopening the app
- User impact: A user who taps around impatiently (very common – switching tabs while a screen is still loading is exactly what real users do) can hard-crash the app several seconds later on an apparently unrelated tap, losing whatever they were doing and getting no explanation. No crash-reporting SDK exists (per ground truth) so this class of crash would be **completely invisible** to the team in production – it would show up only as silent uninstalls / 1-star reviews, never as a report
- Technical impact: Confirms the audit's "no crash-reporting SDK" ground-truth finding is not just a monitoring gap but an active blind spot for a real, reproducible crash class
- Recommendation: (1) Add a crash-reporting SDK (Sentry/Crashlytics) – this exact crash class would otherwise never be seen. (2) Investigate whether the tab navigator (or a screen mounted under it, e.g. the live-match screen or chat list) is doing manual view manipulation, holding stale refs, or missing `key` stability that would explain a view being attached twice; look for `removeClippedSubviews`, custom native view managers, or animated mount/unmount races on the screens exercised (Home, Communities Games list, live match, Chats). (3) Add debounce/guard against rapid repeated tab-bar presses (a `finalizingRef`-style guard, matching the pattern already used correctly on the live-match end-round button, would help but doesn't fix the underlying double-attach
- Expected benefit: Removes a real, silent-crash class; a crash-reporting SDK gives visibility into how often this (and other) crashes actually happen among the ~596 users
- Effort: M (root-causing the double-attach) + S (crash reporting, if not already planned elsewhere)

F-P17-2 – Live-match goal tally silently loses history for other players on the very next goal

- Severity: P1
- Confidence: Confirmed (reproduced with before/after screenshots; root-caused in source)
- → "הוסף גול", Feature / Screen / Flow: Live match screen (AdvancedLiveMatchScreen) scorer/assist picker
- Evidence
- of game "משחק 2": Baseline (fully rendered, not obscured by any modal)
- ליגת השכונה. Team score כחולה 1 - אדומה 2. Per-player rows: רון (blue) = 1, אליון (red) = 1, everyone else 0. Sum matches team totals (1 / 2) = (red) משה 1, as (nobody) "אף אחד" on blue, selected "הוסף גול" Action: tapped
- assist – completing the flow while wifi+data were both disabled
- right math for the new) 2 - אדומה 2 כחולה 2 Result: team score updated correctly to should be 2 – he just scored) goal. But the per-player roster rows now show their earlier – משה=0 and אליון=0 again), and **every other player, including**

- goals vanished from the display. 4 • מבקיעים (scorer-event count) is correct
- Re-enabling wifi/data and waiting did not self-correct the display
- AdvancedLiveMatchScreen.tsx:329–341 :Root cause (source) •


```
ts const goalsByPlayer = useMemo(() => { if (live?.goalTally && Object.keys(live.goalTally).length > 0) {
return live.goalTally; // <- used exclusively if non-empty } // else: correctly sums live.goals (the full per-
goal log) ... }, [live?.goalTally, live?.goals]);
```
- does (~3536–3539 ,mock path) recordGoal gameService.ts:3489
- const t = { ...(m.liveMatch.goalTally ?? {})}; t[tallyId]++; m.liveMatch.goalTally = t;
- undefined this itself is a correct merge. But if goalTally was previously –
- as in the seeded mock fixture I hit, where 2 goals existed only in the goals log
- with no matching goalTally, the very first subsequent recordGoal call creates a
- brand-new, partial** goalTally containing only the new scorer. Because the
- ,component prefers any non-empty goalTally over the full-history fallback sum
- every previously-untallied player's goal disappears from the screen from that point
- on. gameService.ts:4071 stopRotation (and its Firestore twin) intentionally
- resets liveMatch.goalTally: {} while leaving liveMatch.goals untouched – this is
- a second, more realistic way to reach the same “goals array has history, goalTally
- .doesn't” state in production, not just in the mock fixture
- goals Scope check: functions/src/index.ts:12753 commitRoundStats takes a •
 - array with scorerId as its input (not goalTally), so the actual server-side
 - stats commit likely re-derives from the full goal log and is **not** corrupted by
 - this – this appears to be a **live-display-only** bug (Suspected, not verified by
 - .reading the full callable body
- Problem: goalsByPlayer 's “prefer goalTally if non-empty” rule silently treats a •
 - .partial goalTally as if it were complete
- User impact: An admin running the live match sees the wrong scorer credits mid-match •
 - a player who genuinely scored earlier appears to have 0 goals, which is exactly the –
 - kind of thing an admin would “fix” by manually re-adding/removing goals, risking real
 - .double-counting once the round is actually committed
- /Technical impact: Any code path that clears goalTally without also clearing •
 - resyncing from goals (confirmed: stopRotation) – or any legacy/partially-migrated
 - .live match doc missing goalTally – reproduces this on the very next goal
- Recommendation: Change the preference rule to always derive goalsByPlayer from •
 - or merge ,(the stated “source of truth” per the code's own comments) live.goals
 - with a full recompute rather than trusting it blindly. At minimum, when goalTally
 - resets goalTally: {}, it should not do so while goals still holds stopRotation
 - .history for a screen that's about to trust goalTally as complete
- Expected benefit: Live match view stays trustworthy for the admin entering goals in •
 - .real time, avoiding manual “corrections” that could actually introduce bad data
- Effort: S •

F-P17-3 – Rapid tab-bar taps can be silently dropped, leaving the tab bar and screen out of sync

- Severity: P2 (contributing factor to F-P17-1, see above) •
- Confidence: Confirmed (reproduced once with a before/after screenshot) •
- (בית / מועדונים / מחזורים / ציטים) Feature / Screen / Flow: Bottom tab bar •
- ,Evidence: Fired 12 taps in immediate succession cycling through all 4 tabs 3 •
- icon was מחזורים Screenshot taken right after: the tab bar's ציטים ending on
- highlighted as active, but the visible screen content was the **live match** screen
- .that had been open on that tab earlier in the session – i.e. “מוכן” (משחק 3, סֶס, מוכן)
- then ציטים did nothing. A single, isolated follow-up tap on (ציטים) the last tap
- .worked immediately and correctly opened the chat list

- Current behaviour: no crash at the moment of the drop; the app just doesn't act on the last tap of a fast sequence
- Problem: taps fired while a screen transition/animation is still resolving appear to be silently swallowed rather than queued or debounced predictably
- User impact: a user tapping tabs quickly (very common) can end up "stuck" looking at a stale nested screen while the tab bar claims a different tab is active – confusing and (per F-P17-1) may be the precursor to a later hard crash
- Recommendation: investigate whether tab presses during an in-flight transition should be queued (process the latest one after the current transition settles) rather than dropped; this is also the natural place to look for the F-P17-1 root cause
- Effort: S (repro) / M (fix, shared with F-P17-1)

F-P17-4 – Mock live-match state does not survive **force-stop**, and navigation position is never restored

- Severity: P3 (documented primarily as a testing-tool caveat) / P2 (nav-restore gap)
- Confidence: Confirmed
- Feature / Screen / Flow: any in-progress mock game / live match; app process death mid-progress (score 1-2, several goals recorded), I ran "משחק 2" Evidence: With then relaunched. After ~15s the app landed on the **Home tab**, not **am force-stop** on the game or the live match screen I'd been in. Re-entering the same game showed it reset to the **original seeded** state (score 1-2 → back to pristine, later the corrupted-tally state from F-P17-2 was also gone) – i.e. all mock progress from the session was wiped, because **USE MOCK DATA** keeps **mockGamesV2** as an in-memory JS array with no persistence layer
- Problem/impact: (a) The in-memory-only mock store means process death (which happens routinely on real Android under memory pressure, not just from a manual force-stop cannot be used to test whether **real, Firestore-backed** state survives – that must be verified separately against production/staging data, this mock harness cannot answer it. (b) Independent of mock-vs-real data, the app has **no navigation-state restoration**: killing the process always drops the user back to Home, never to wherever they were (e.g. mid-live-match). For an app whose core loop is "be in a live match, add goals in real time," losing your place after any process death (backgrounding .Android reclaiming memory, not just an explicit kill) is a real UX gap
- Recommendation: (a) When auditing state-persistence bugs, use this device only for UI/navigation/crash testing, not data-survival testing – do that against a real Firebase project. (b) Consider persisting last-active game/live-match id (e.g. via **AsyncStorage**) and offering a "return to live match" affordance after a cold start rather than always defaulting to Home
- Effort: S (b)

Chaos items that worked correctly (no bug found)

- Back button at every stage of the create-game flow (choose-type modal → create form ,location-search full-screen picker) correctly steps back exactly one level each press never exits the app, and preserves form state. Verified with 3 consecutive back .list cleanly מחזורים → presses unwinding location-picker → create-form
- /Back button from the live-match screen returns to game-details without losing score state (verified twice); back from game-details modal (choose-type) closes the sheet without navigating away (verified twice), directly contradicting the false "every menu is dead" finding this audit was warned about – sheets and back-dismiss both worked on .every attempt here
- then rapid taps on a scorer row: only ever produced a "הוסף גול" Double/triple-tap on ,single scorer-picker → single assist-picker transition; no duplicate/stacked modals .no phantom goal recorded when cancelled (score verified unchanged after cancel)

- confirm button, a destructive/committing (end match) "סיים משחק" Triple-tap on the action: only ONE round transition occurred (win-count moved from 2→3, not 2→5); this matches the `finalizingRef / committingRef` guard described in comments (lines ~380-396) and is a good example of a `AdvancedLiveMatchScreen.tsx` .double-fire guard working as designed
- Pull-to-refresh gesture repeated 3× on the chats list: no crash, no stuck spinner, no visible ill effect
- ,Offline (wifi+data both disabled): opening the goal-scorer modal, selecting a scorer selecting an assist, and completing the commit all worked with no crash, no hang, no "Firebase" permanently, with "לא קיים חיבור ל-" error toast. **Caveat:** this build shows ,or without wifi/data – it is a mock-only build with no real backend connection at all .so toggling connectivity does not actually change this app instance's network path
- This session cannot speak to how the app behaves against a **real** Firestore connection going on/offline (retry queues, `arrayUnion / increment` conflict behaviour .described in `gameService.ts` comments, etc.) – that needs a non-mock build
- Kill-mid-flow while a modal was open (goal scorer picker): app relaunched cleanly to Home with no residual stuck modal or corrupted UI (separately from the F-P17-4 data .(loss, the UI recovered fine

ANDROID PLATFORM findings .2

- Back button: exits a modal/sheet or steps back one navigation level in every screen I drove (create-game modal, create-game form, location picker, live match, game details) – never exited the app unexpectedly in this session. I did not re-test the auth screen (no logged-out session available in this mock account); the existing ground-truth note that back-on-auth-screen exits the app is not contradicted or .re-verified here – treat it as still authoritative from the prior finding
- and the (screen מחדש Keyboard: tested on 2 forms – the game-name field In both cases the .(full-screen map picker מיקום location-search field focused input remained visible above the keyboard; no occlusion found. Note: the dev-only "Open debugger to view warnings" + "– ad: idle –" banner sits directly above validation message in one "ייש למלא: מיקום המגרש" the keyboard and did overlap the screenshot – this is dev-build chrome, not present in production, so not filed as a .product bug, just noted
- `adb am start -a android.intent.action.VIEW -d "teamder://session/gv2-live"` :Deep links fired at a **cold** app (process not running) correctly routed all the way to that after the app finished booting – confirming the (ליגת השכונה) game's details screen `RootNavigator` design (stash the pending invite, consume once `deepLinkService.ts` is ready) works end-to-end, including the cold-start case its own code comments flag as the risky one ("auto-linking would silently fail when the target screen isn't .mounted yet"). This is a genuine positive finding
- Notification permission: `dumpsys package` shows for this install. I did not `android.permission.POST_NOTIFICATIONS: granted=false` observe an in-app permission-request prompt fire during this session's flows Suspected – I did not specifically hunt for the trigger point, e.g. first game join "or first launch after onboarding, so this is not a confirmed "prompt never fires .(finding, just a measured fact that it's currently ungranted on this device
- Safe areas / gesture bar: the bottom tab bar (y≈2255-2270 out of 2400) sits clear of the 3-button/gesture navigation bar in every screenshot taken this session; no visual .overlap observed

App lifecycle: `am force-stop` + relaunch always lands on the Home tab regardless of where the user was (see F-P17-4) – consistent across 2 separate kill/relaunch cycles

PERFORMANCE findings .3

Confound that applies to every number below: this is a **debug build wired to Metro** (visible “Downloading...” bundle-fetch banner + JS debugger attached, per the Open debugger to view warnings” banner present throughout). Cold start in particular” downloads the JS bundle over the network from the Metro server on every launch – a **production** build bundles JS locally and would not pay this cost. All numbers here are MEASURED on this device as-is; they are likely worse than a production APK would show, especially cold start and memory. I did not have a production APK to compare against.

Cold start (measured, 2 runs): `am force-stop` → `am start`, screenshotted in a tight poll loop and diffed by file size to find the first non-splash frame

- Run 1: solid-color native splash held until ~9s, then a JS-rendered secondary splash appeared (with a “Downloading...” bundle bar “המשחק הבא שלך מתחיל כאן” – Teamder”) .13.3s in, full Home-screen content rendered by ~14.7s~
- ×Run 2 (same device, back-to-back): content didn’t stabilize until ~26s – nearly 2 slower than run 1 with no configuration change, showing cold start is **highly inconsistent** even for internal testers on this build
- independently reported `WaitTime: 13346` with `adb shell am start -W` .consistent with run 1, (`reportFullyDrawn` the app never signals) `Status: timeout`

Memory (measured): `dumpsys meminfo` immediately after a fresh cold start, sitting idle on the Home screen: **TOTAL PSS ≈ 498 MB** (Native Heap 168MB, Dalvik 22MB, Code 51MB, Graphics reported 0, Unknown 49MB). This is a high resting figure for a “just /opened the home feed” state, though very likely inflated by the attached JS debugger .Metro connection (Suspected re: how much is debug-only overhead vs. real)

Scrolling (measured): `dumpsys gfxinfo ... reset`, then 5 up/down swipe cycles on : `dumpsys gfxinfo` the Home feed, then frames rendered, **66 janky (26.72%)** 247

- 50th pct 34ms · 90th pct 61ms · 95th pct 69ms · 99th pct 105ms (60fps target is (16.6ms/frame; even the 50th percentile is 2× over budget
- “Number High input latency: 360”, “Number Slow issue draw commands: 62”
- The GPU-percentile line in the same report (`95th gpu percentile: 4950ms`) is almost certainly a histogram-bucket/tooling artifact on this emulator, not a real second GPU frame – flagged as Suspected/not-a-real-finding rather than reported-5 .as a number

- ;This is a real, measured janky-scroll problem on the Home feed under this build given the emulator’s software-ish rendering path and debug overhead, the absolute numbers should be re-measured on a release APK on real hardware before sizing any fix, but a >25% janky-frame rate on a simple vertical scroll of the primary landing .screen is worth a look regardless
- Stats/chemistry screens: not reached this session (budget went to the chaos/offline – (work per the brief’s explicit prioritization of offline as the highest-value gap .not measured, not claimed

- Wifi and mobile data were both re-enabled before ending the session (confirmed via `svc data enable / svc wifi enable` status-bar icons showing signal + wifi again after)
- The app was left in a normal, working state on the Home tab after the final relaunch that followed the F-P17-1 crash

Teamder – UI/UX Panel Audit (Phase 2, live emulator)

מחזורים → Method: drove emulator-5554 in MOCK MODE end-to-end through Home feed → game details → roster → teams draft (2 teams) → live match → goal “scoring → tie-break → penalty shootout → finish evening → “who took it home list → club details → club stats/table → chat מועדונים → evening summary → list → chat (crashed, 2x reproduced) → profile/settings sheet → player-card edit. Every finding below cites a screenshot in phase2/shots/. Onboarding and the 3-team/4-team live-match variants were not driven (fresh account) “this session – no finding is made up for them; treat as unverified, not “fine

UI FINDINGS

Visual hierarchy

F-UIUX-1 – Teams-draft flow has a hierarchy vacuum, not a hierarchy problem

Severity: P3 | Confidence: Confirmed -
Screen - “חלוקת כוחות” method-choice and team-count screens
Evidence: shots/06_teams_split_choice.jpg , shots/07_teams_split_random.jpg - two option cards (or one summary card) sit - is pinned to the very bottom, disconnected from (“צור כוחות”) at the top of an otherwise empty 1080x~1600px canvas; the confirming CTA .the content by roughly 60% of the screen
Current behaviour: content occupies the top ~25% of the screen, then a large dead zone, then the CTA -
Problem: the primary action is visually orphaned from the choice that drives it; the eye has nothing to do in the middle of the screen -
User impact: minor – the flow still works, but it reads as unfinished/placeholder-ish compared to the rest of the app’s density -
Technical impact: none -
Recommendation: either vertically center the option cards + CTA as one block, or use the freed space for something useful (e.g. show - .avatars of the players about to be split, a captains illustration)
Expected benefit: the screen feels intentional rather than empty; small trust/polish gain -
Effort: S -

F-UIUX-2 – Club table’s extra columns are scrollable but invisible

Severity: P2 | Confidence: Confirmed (before/after screenshot) -
Screen - מועדונים → club details → “טבלת המועדון והסטטיסטיקות” → אלופי המועדון table
Evidence: shots/37_club_table.jpg (before) shows a truncated leftmost column glyph - cut off at the screen edge; “ר” Evidence: shots/38_club_table_scrolled.jpg (after a manual horizontal swipe) reveals two more full columns plus a further- (הפסדים, שער נקי) truncated one. No shadow/gradient/chevron hints that horizontal scroll exists
Current behaviour: the table silently clips extra stat columns with no affordance -
Problem: an admin doing the “recurring weekly work” of checking the club table has no visual cue that clean-sheets/losses data exists - .off-screen
User impact: real data (losses, clean sheets, appearances) is effectively hidden from most users -
Technical impact: none, purely a missing UI hint -

hint, or make the player name/avatar "עוד נתונים" Recommendation: add a fading-edge gradient on the clipped side, or a small -
column sticky while the rest scrolls
.Expected benefit: surfaces data that's already computed and shipped but currently undiscoverable -
Effort: S -

Sports product feel

F-UIUX-3 – Penalty shootout and teams-reveal are the strongest sports moments in the app (positive finding)

Severity: N/A (positive) | Confidence: Confirmed -

(shots/08_teams_split_result.jpg) Screen: penalty shootout pitch (shots/22_shootout_pitch_ready.jpg), teams-split reveal -
Evidence: the shootout screen renders an actual illustrated pitch with a goal grid, keeper marker, and kicker marker the organiser taps -
.headline, and color-coded team rosters "הכוחות חולקו"; through; the teams-reveal screen shows a trophy icon
What's good: these are the two moments in the whole audit that feel authentically like football rather than generic form UI - worth -
protecting and expanding (e.g. reuse the pitch visual for retro-goal entry, use the trophy pattern for other "reveal" moments like MVP-of-
the-evening)
.No fix needed; flagged so it isn't accidentally simplified away in a future redesign -

F-UIUX-4 – No visible celebration when a live-match goal is recorded

Severity: P3 | Confidence: Suspected – not verified (a screenshot cannot prove a transient animation did or didn't fire) -

Screen: live match scoreboard, shots/15_live_match_scored.jpg vs shots/19_live_match_scored.jpg ... (goal counter simply -
increments)
Current behaviour: after confirming scorer+assist, the score/roster counters update with no visible confetti/flash/haptic captured in the -
after-screenshot
Problem: if there truly is no celebratory beat, the single most exciting event in a live match (a goal) looks identical to any other list -
update
Recommendation: verify with a video capture (not a static screenshot) whether a transient animation exists; if not, add one using the -
project's existing football-animation set (src/components/anim – BallSwitch/ArcPopIn per project memory)
Effort: S (if the animation set already exists, wiring it to goal-add is small) -

Design system

F-UIUX-5 – ErrorBoundary leaks the raw JS error string to the user

Severity: P2 | Confidence: Confirmed -

shots/44_CRASH_error_boundary_screen.jpg ,Screen: global error fallback -
the screen prints, in red, the literal exception ("משהו השתבש" / "נתקלנו בתקלה לא צפויה...") Evidence: below the polished Hebrew copy -
:text
getFirebase() called while USE MOCK_DATA is true. Fill in .env with your Firebase config to switch out of mock
mode.
but unconditionally ,(retry, feedback hint "ניסה שוב", icon, friendly copy) Current behaviour: the fallback UI is otherwise well designed -
renders error.message verbatim
Problem: any crash's raw message – which may contain stack internals, file paths, or (in other error types) potentially sensitive text – is -
shown to end users, breaking the polish of an otherwise good component and looking unprofessional/alarming in production
.User impact: low frequency but high alarm when it happens; a real user seeing raw JS text will assume the app is broken/unsafe -
in production "אם זה נמשך, שלח לנו דיווח" Recommendation: only show the raw message in __DEV__ /mock builds; render a generic -
builds instead
.Expected benefit: consistent, professional failure state -
Effort: S -

F-UIUX-6 – Live-match primary CTA reassigns color semantics per state

Severity: P3 | Confidence: Confirmed -

shots/17 , shots/15 , shots/14 , shots/03 Screen: game details → live match, across -
separately, green is also the "סיים משחק"; back to blue for "התחל משחק", green for "עבור ללייב", Evidence: the main CTA is blue for -
(shots/23) button "ניכנס" "success/scored" color inside the penalty
Problem: green is used both for "start" and for "scored/success," blue is used both for "primary navigation" and "end match" (a -
semantically different, more consequential action) – the palette doesn't consistently map color→meaning
Recommendation: reserve green strictly for positive/success confirmations (scored, won) and use a neutral/primary blue for all -

.for "end/finish" actions (cancel-registration/"סיים מחזור" e.g. the red already used for) forward-navigation CTAs, a distinct color
.Effort: S – a palette-mapping pass, no new components –

Accessibility (visual)

F-UIUX-7 – No confirmed accessibility defects, but touch targets in list-style radio pickers are tight

Severity: P4 | Confidence: Suspected –

shots/27 - shots/21_shootout_keeper.jpg ,Screen: shootout keeper/kicker pickers –

Evidence: multiple mis-taps were needed to land on the radio circle/row during this audit (see raw session – taps at the visually- - correct row location initially missed); once precisely placed, selection worked every time, so this is not a confirmed broken control, just a .tight hitbox relative to the visually large row

Recommendation: verify the full row (not just the circle+label glyph) is a single touchable with adequate padding; no code was - .inspected to confirm, so this stays Suspected
.Effort: S to verify, S to fix if real –

Density / minimalism

F-UIUX-8 – Evening summary is a single very long, undifferentiated scroll

Severity: P3 | Confidence: Confirmed –

32 - shots/30 , "סיכום המחזור", Screen –

all – "מה קרה הערב" group cards → pair-of-the-evening → 6-row 2 → "כוכבי המחזור" Evidence: MVP card → 4-stat grid → 5-row - .stacked vertically, no tabs/anchors, spans 3+ full-screen scrolls

Problem: content is genuinely good (see F-UIUX-11 below) but its structure doesn't help a user jump to what they care about (e.g. "just - .show me my personal stats")

or add "הצג הכל" to a top-3 preview with "מה קרה הערב" and "כוכבי המחזור" Recommendation: light progressive disclosure – collapse - .section chips at the top to jump-scroll
Effort: M –

F-UIUX-9 – Chat list has near-duplicate rows distinguishable only by a small subtitle

Severity: P2 | Confidence: Confirmed –

shots/71 , shots/39_chat_list.jpg , צ'אטים tab , Screen –

two are (subtitle "חברי המועדון", globe icon) one is the club chat – "חמישי כדורגל" Evidence: three consecutive rows are all titled - .differentiated only by icon color/shape and 12px gray subtitle text – (subtitle "משתתפי המחזור", ball icon) separate per-game chats

.Problem: a returning user scanning for "this week's game chat" cannot distinguish rows by title; must read fine print every time - instead of reusing the club's display ("חמישי כדורגל · 03.06") Recommendation: prefix per-game chat titles with the game date/day - .name verbatim
Effort: S –

UI CONSENSUS

The screens driven in this session show a coherent, above-average-polish ,design system with two genuine standout sports moments (penalty pitch ,teams-split reveal – F-UIUX-3) that the rest of the app should learn from not lose. The recurring, fixable UI issues are all about **inconsistency of ,density and signal**: some flows are nearly empty (teams-draft config F-UIUX-1) while others are extremely dense with no in-page navigation color semantics ;(evening summary, F-UIUX-8; club stats/table, F-UIUX-2) drift between screens (F-UIUX-6); and the one place raw engineering text reaches the user is the global error screen (F-UIUX-5) – ironic, since it's otherwise the most carefully designed fallback state in the app. None of .these are structural rewrites; all are S/M polish passes

Onboarding & activation

No finding – onboarding (fresh account, first run) was not driven this session due to time budget; do not treat as “fine,” treat as unverified

Game day (organiser’s path)

F-UIUX-10 – Full team-draft → live → shootout → evening-summary flow works end-to-end with no dead ends (positive finding)

Severity: N/A (positive) | Confidence: Confirmed -

“סיים חלוקת” → choose method → confirm team count → reveal → “הגיע הזמן לחלק כוחות” → card → game details מחזור → Flow: Home - live scoreboard → add goal (scorer, then auto-advances to → “התחל משחק” → pre-live roster → “עבור ללייב” → confirm modal → “כוחות” tie-break choice → penalty shootout (goalkeeper, kicker, in/out, repeat) → confirm winner → auto-starts next → “סיים משחק” → assist) .evening summary → (bibs/ball ownership) “מי לקח הביתה” → confirm → “סיים מחזור” → “.....” mini-game → overflow

Evidence: shots/06 through shots/47, 9 distinct screens/modals, no backtrack was needed, no dead end encountered, every - .affordance worked “חזור”/“back

What’s good: for a feature this rich (multi-team live scoring with attribution, penalty shootouts with per-kicker history, end-of-evening - .logistics), the number of taps is proportionate and the app never strands the organiser .No fix needed – recorded so it’s not re-litigated by a future redesign -

F-UIUX-11 – CONFIRMED Po: the chat tab is completely broken in this build

Severity: Po (a primary bottom-tab destination is unusable) | Confidence: Confirmed – reproduced twice with full stack traces - tab → any chat row (both club-wide chat and every per-game chat) ציגטים :Flow / Screen -

Evidence: shots/40_CRASH_chat_open.jpg (first crash, UIFrameGuarded / addViewAt native exception on opening a chat), - shots/41_CRASH_whitescreen_after.jpg (app left in a dead white-screen state, required am force-stop +relaunch), shots/42 - 43 (retry, this time RN’s redbox shows the real root cause: Error: getFirestore() called while USE_MOCK_DATA is true... at config.ts:92:20, thrown from ChatView.tsx:82 (,export function ChatView({ scope, parentId, title, canModerate, ... }) ,invoked by GameChatScreen.tsx:21) ,“משהו השתבש” the app’s own global ErrorBoundary catches it and shows) shots/44_CRASH_error_boundary_screen.jpg crashes – “חברי המועדון” ,confirmed the **club-level** chat – a different entry point, different icon) shots/45_CRASH_club_chat_also.jpg .(with the exact same native exception, so this is not limited to game chats

.Current behaviour: tapping into any chat thread throws immediately on render -

Problem: ChatView.tsx calls getFirestore() directly instead of going through the app’s mock-data abstraction (per project ground - truth: src/firebase/firestore.ts converters are how every other screen stays mock-safe). Chat evidently never routes through that .layer

tab – one of four bottom-tab destinations – is unusable; every tap ends in a crash “ציגטים” User impact: in this mock build, the entire - .and a full app restart

Technical impact: this is also a **testing blind spot**, independent of whether real (Firestore) mode is affected – mock QA (the team’s - normal fast-iteration test mode, per project memory) can never exercise chat at all, so any real bug in chat code goes unnoticed until a .real device/account test

Recommendation: audit ChatView.tsx and everywhere it calls getFirestore() /reads Firestore directly; route it through the same - mock-aware data layer as the rest of the app so mock QA can actually cover chat. Separately, confirm this doesn’t also affect the production Firestore path (the specific error string is mock-only, but a direct getFirestore() call bypassing the shared converter layer .is itself a code-smell worth checking against the “read deserializer strips new fields” class of bug already known in this codebase)

.Expected benefit: restores an entire tab to working order, and closes a testing blind spot for a whole feature -

Effort: M (likely a targeted fix in ChatView.tsx’s data-fetching, but needs verification against the real Firestore path too) -

Club management (admin’s recurring weekly work)

F-UIUX-12 – “מי לקח הביתה” is a well-placed, low-friction retention/logistics touch (positive finding)

Severity: N/A (positive) | Confidence: Confirmed -

shots/28_who_took_home.jpg ,Screen: end-of-evening flow -

What’s good: automatically prompting “who’s taking the ball/bibs home” right when the evening ends – with a skip option – solves a -

.real recurring-organisier pain point at exactly the right moment, without blocking the flow (skip available)

.No fix needed -

F-UIUX-13 – Club stats/table is the right content, wrong shape, for a weekly admin check

Severity: P2 | Confidence: Confirmed -

Screen: מועדונים → club details → סטטיסטיקת המועדון, 38 - shots/35

Evidence: MVP card → 4 aggregate stats → 5-row leaderboard-by-category → chemistry-pair cards → 6 “fun fact” percentage rings -

→ the standings table itself (with the hidden columns from F-UIUX-2), spanning 3+ screens of scroll with no anchors

Problem: this is exactly the data an admin doing weekly club upkeep wants, but it's not scannable in one glance – same density issue -

as F-UIUX-8, applied to a recurring workflow rather than a one-time summary

.Recommendation: same fix direction as F-UIUX-8 (section jump-chips or a compact “admin quick view” at the top) -

Effort: M -

Retention

F-UIUX-14 – “מה קרה הערב” milestone highlights are a strong, underused retention hook (positive finding)

Severity: N/A (positive) | Confidence: Confirmed -

Screen: evening summary , 32 , shots/31

Evidence: cards for “שיא מועדון חדש” (new club scoring record, with the previous record shown for comparison), “250 שערים למועדון” (new #1 in a category), “מקום ראשון חדש” (club milestone), “100 שערים במועדון” (personal milestone), “4 מקומות למעלה” (biggest club-rank)

“.”(jump

What's good: this is a genuine “why come back next week” mechanic – personal and club records surfaced automatically, in the same -

pattern successful fitness/sports apps use for session recaps. It currently only lives inside the evening-summary screen; per the IA

finding below, a user has to specifically open that screen to see it – it isn't pushed or surfaced anywhere else observed in this session

.(push/notification behaviour was out of scope for this audit)

Recommendation (idea-level, not built from this audit): consider surfacing 1 headline record as a home-screen banner or notification, -

.not just buried at the bottom of a long summary screen

.Effort: L (if extended into push/home-screen surfacing) – not requesting that build here, just flagging the leverage -

F-UIUX-15 – The chat crash (F-UIUX-11) is also a retention problem

Severity: Po (same root cause as F-UIUX-11, called out separately because the *impact* is different) | Confidence: Confirmed -

Evidence: same as F-UIUX-11 -

Why it matters for retention specifically: chat is normally the between-game touchpoint that pulls a user back mid-week (coordinating -

next week's game, banter, “who's in”); with that channel completely dead, the app loses its main asynchronous reason to be opened

.between game days

.Recommendation: same as F-UIUX-11 -

Effort: (tracked under F-UIUX-11) -

Information architecture

F-UIUX-16 – Confirms the known “three mental models, no unifying home” split, with a concrete example

Severity: P2 | Confidence: Confirmed -

Evidence: club-level stats/records live at מועדונים → club details → סטטיסטיקת המועדון (38 - shots/35); the per-evening summary lives at

מחזורים → game details → סיכום המחזור (32 - shots/30) and the *second* screen already injects club-level records - (“שיא מועדון חדש”,

“250 שערים למועדון”) into a game-flow screen, i.e. the product itself already recognizes users want club stats surfaced from the game

path, but only partially (highlights only, not the full table/leaderboard)

Problem: there's no single place to answer “how is my club doing” – a user has to know whether to look under the game they just -

played or under the club itself, and the two screens show overlapping but not identical data

Recommendation: not proposing a full IA rebuild here (out of scope for this pass), but note that the evening-summary screen's existing -

rather than requiring) pattern of pulling in club highlights is the right instinct – extending it with a single “view full club stats” link

.would close much of the gap cheaply (מועדונים separate navigation through

.Effort: S (a single deep-link/button) for the cheap mitigation; L for a true unifying stats home -

F-UIUX-17 – The profile/settings sheet mixes five unrelated concerns in one undifferentiated scroll

Severity: P3 | Confidence: Confirmed -

Screen: hamburger menu sheet , shots/46_profile_menu_sheet.jpg , shots/47_settings_menu.jpg

Evidence: single scrollable bottom sheet containing, in order: player-card section - (הישגים, סטטיסטיקה, ערוך כרטיס שחקן, חברים, שחקנים

section (הזמנות שלי, היסטוריית מחזורים), הגדרות section (בקשות לסגל, התראות, משתמשים חסומים), עזרה ומשוב section (הצטרפו דרכי), מחזורים

(דיווח על תקלה, הצעת שיפור, דרג אותנו בחנות, התנתק).

Problem: functionally it's fine (small gray section labels do separate the groups) but a user predicting "where do I turn off notifications" - has to scroll past player-card and game-history items first; this is the same one-long-list pattern seen elsewhere (F-UIUX-8, F-UIUX-13) rather than one flat (חשבון / הגדרות) Recommendation: low priority given it's a rarely-visited surface; if reworked, consider a 2-tab split - .list
Effort: M -

UX CONSENSUS

The moment-to-moment **game day** loop – draft teams, run a live match with attributed goals, resolve a tie with a real penalty shootout, close the evening, hand off logistics, and land on a celebratory summary – is genuinely well built: no dead ends, proportionate tap counts, and real ,retention hooks (milestone highlights) baked into the flow (F-UIUX-10 F-UIUX-12, F-UIUX-14). That is the app's strongest asset from this audit and .should be protected in any future redesign

Against that, two problems are structural rather than cosmetic. First, and – most urgent: **the chat tab is completely broken** (F-UIUX-11/F-UIUX-15) every chat, club-level or per-game, crashes on open in this build, killing a whole bottom-tab destination and the app's main between-game retention channel, and simultaneously proving that mock QA has zero coverage of the chat feature because it doesn't route through the app's shared mock-data layer. This should be treated as a release blocker for chat, not a polish item, until verified against the real Firebase path. Second: the game/club/stats IA split the discovery phase already flagged is visible concretely in this session – the evening-summary screen's own instinct to inject club records into a game-flow screen (F-UIUX-16) shows the team already senses the gap; closing it doesn't require a redesign, just linking .the two surfaces together

נספח C – כיסוי, קופי ונגישות

QA_MATRIX

QA Coverage Matrix – Teamder

,QA lead pass, 2026-09-02. Synthesizes: D1~D4 discovery, all 14 findings/*.md + phase2/P_NEW.md + P_ADMIN.md (runtime, mock mode, emulator-5554) + A11Y.md COPY.md, and a direct listing of tests/ (68 Jest files + 4 Firestore-rules .files = 72 total) and functions/ (0 test files, confirmed) .mjs

:Legend for every cell in the matrix

AT = covered by an automated test (cite the file) -

RT = exercised at runtime *this audit* (P_ADMIN/P_NEW, mock mode on emulator-5554) -

ST = analysed statically only (code read, never run) -

NE = NEVER EXAMINED – no test, no runtime pass, no static finding touches this cell -

n/a = state doesn't meaningfully apply to this feature -

Where a runtime pass *found a bug*, the cell is marked **RT-BUG** and the finding id is cited – that is a stronger signal than a clean RT and is called .out separately in §4

The matrix .1

· Columns: **fresh**=fresh account · **exist**=existing account · **oclub**=no club
1club=one club · **Nclub**=multiple clubs · **ogame**=no games · **live**=active/live
game · **done**=completed game · **off**=offline · **poor**=poor network · **empty**=empty
data · **large**=large data · **err**=error · **retry**=retry after failure

	poor	off	done	live	ogame	Nclub	1club	oclub	exist	fresh	Feature	#
	NE	NE	n/a	n/a	n/a	n/a	n/a	n/a	ST	RT-BUG (F-P6-3)	Auth (guest/Google/Apple/email)	1
	NE	NE	n/a	n/a	n/a	n/a	n/a	n/a	n/a	RT-BUG (F-P6-1/2)	Onboarding (pre+post sign-in)	2
	NE	NE	n/a	n/a	n/a	RT-BUG (F-ADMIN-3)	RT	n/a	RT	RT-BUG (F-ADMIN-4/9)	Club create/edit wizard	3
	NE	NE	n/a	n/a	n/a	ST	ST	RT (browse, F-P16)	ST	ST	Club join (code/browse/approve)	4
	NE	NE	n/a	n/a	n/a	RT-BUG	RT-BUG	n/a	RT-BUG (F-ADMIN-1, P0)	n/a	Admin ≡ menu / management actions	5
	NE	NE	n/a	n/a	n/a	ST	ST	n/a	AT ²	n/a	Club discipline (cards)	6
	NE	NE	n/a	n/a	n/a	ST	ST	n/a	ST	n/a	Equipment holder tracking	7
	NE	NE	n/a	n/a	n/a	RT-BUG (F-ADMIN-3)	RT	n/a	RT	RT-BUG (F-ADMIN-5)	Game create/edit wizard	8
	NE	NE	ST	ST	ST	ST	ST	n/a	ST	n/a	Game lifecycle state machine	9
	NE	NE	n/a	AT	n/a	ST	AT	n/a	AT ³	n/a	Join/cancel/waitlist/fair ordering	10
	NE	NE	ST	ST	n/a	n/a	ST	n/a	AT (partial) ⁵	n/a	Guests (add/rate/remove)	11
	NE	NE	n/a	ST	n/a	n/a	ST	n/a	ST	n/a	Cross-community filler matching	12
	NE	NE	n/a	n/a	n/a	ST	AT	n/a	AT ⁶	n/a	Recurring games / series	13
	NE	NE	n/a	RT	n/a	ST	RT	n/a	AT ⁷	n/a	Draft / manual teams	14
	NE	NE	n/a	ST	n/a	ST	ST	n/a	AT ⁸	n/a	Auto-balance teams	15
	NE	NE	n/a	NE (runtime)	n/a	n/a	ST	n/a	ST	n/a	Live match – plain timer	16
ST (RELIABILITY-2/4/5/6)	NE		n/a	NE (runtime)	n/a	n/a	ST	n/a	AT (logic) ⁹	n/a	Live match – advanced (rotation/goals/shootout)	17
ST (RELIABILITY-1/2)	NE		n/a	NE (functions/ untested)	n/a	n/a	n/a	n/a	AT (client mirror) ¹⁰	n/a	commitRoundStats (server round commit)	18
	NE	NE	NE	n/a	n/a	n/a	NE	n/a	NE	n/a	Retro goals (post-match correction)	19

poor	off	done	live	ogame	Nclub	1club	oclub	exist	fresh	Feature	#
NE	NE	ST	n/a	n/a	ST	ST	n/a	AT ¹¹	n/a	Evening summary (personal)	20
NE	NE	ST – CONFIRMED-DRIFT in prod (9.3% of finished games missing roundHistory)	n/a	n/a	ST	ST	n/a	AT (heaviest-tested feature) ¹²	n/a	Round summary (club-wide)	21
NE	NE	ST	n/a	n/a	ST	RT-BUG (F-ADMIN-7)	n/a	AT ¹³	n/a	Club stats dashboard / chemistry	22
NE	NE	ST	n/a	n/a	n/a	ST	n/a	NE	n/a	Trust score	23
NE	NE	n/a	ST	n/a	ST	RT-BUG	n/a	RT-BUG (F-ADMIN-2, crash)	n/a	Chat (community/game/DM)	24
NE	NE	ST	ST	n/a	ST	ST	n/a	ST	n/a	Notifications (push dispatch + prefs)	25
NE	NE	n/a	n/a	n/a	NE	NE	n/a	NE	NE	Deep links / invite attribution	26
NE	NE	RT-BUG (F-ADMIN-8, unreachable)	n/a	n/a	n/a	n/a	n/a	ST	n/a	Orphan → club promotion	27
NE	NE	n/a	n/a	n/a	NE	NE	n/a	NE (deliberately untested – destructive)	n/a	Account deletion	28

, tests/logic/joinFairness.test.ts³ · tests/cardState.test.ts² · tests/logic/authRaceRetry.test.ts¹
transactional re-check exists per RELIABILITY's "solid" list – logic sound⁴ · registrationQa.test.ts, joinCtaCapacity.test.ts
but never exercised under simulated packet loss⁵ tests/logic/assistCredit.test.ts covers guest-goal crediting only⁶
tests/logic/seriesSchedule.test.ts⁷ tests/logic/draft.test.ts⁸
balanceTeams/balanceParity/balanceVariety/balanceMetaPersistence.test.ts⁹ advancedMatchStats, rotationEngine /
rotationDeep / rotationCounts / rotationFill, shootoutPersistence, championshipPenalty, tieConfirmParity,
wentHomeRestore.test.ts – the rotation *math* is the single best-tested area in the app; none of it is exercised as a running screen¹⁰
tests/roundIdempotency.test.ts tests the **client-side** key generator only; the actual server callable (
functions/src/index.ts:12753) has zero test coverage of any kind – confirmed ground truth (no functions/ test directory exists) ·
¹¹ tests/logic/eveningSummary.test.ts, roundSummaryRealEvening.test.ts¹²
roundSummary/roundSummaryBackfill/roundSummaryLines/roundSummaryParity/roundSummaryRealEvening.test.ts¹³
clubChemistry/clubChemistryParity/clubChemistryReal.test.ts

Reading the matrix: scan any column. The **off** and **poor** columns are
,NE end to end – not one feature has ever had connectivity pulled on it
neither by a test nor by a runtime pass. The **err/retry** columns are
almost entirely ST or NE – errors were reasoned about by reading code, never
induced. The two rows in **bold** (commitRoundStats, live match) are the

worst combination in the whole matrix: the *logic* is extremely well unit tested (rotation/shootout math has 9+ dedicated files) but the actual *running system* – the server callable, the concurrent-admin case, the ,offline case – has never been exercised by anything, automated or manual .anywhere in this audit or (per D4) in the repo's history

The untested surface, quantified .2

Three independent counting methods, all converging on “roughly 60% of the :”product has no direct automated test

Feature-inventory level (D3, 76 named features each with an explicit •
“test-status line): **45 of 76 (59%)** are tagged “Tests: none found
, registrationQa) outright. Of the remaining 31, a large fraction
, copyDirectionality , championshipPenalty , assistCredit , discovery
are tagged “covered indirectly” – a shared utility the (playedGames
feature happens to call is tested, not the feature's own service/screen
code. Features with a **direct**, named test file are closer to
.of 76 (~35%) 30~25

Service-layer level (D4 §8, src/services/*.ts): of ~57 services in •
the codebase, **35 are named explicitly as having zero dedicated test**
,file – including both mega-services (gameService.ts 8,192 lines
, notificationsService , chatService ,(lines 2,194 groupService.ts
and its deepLinkService , userService , friendsService , trustService
siblings, and 20 more. That's **61% of the services layer** with no 3
.direct test

Backend level (functions/src/index.ts , 13,974 lines, 64 exported •
functions): **0%** tested directly. The only Cloud-Functions-adjacent
, balanceParity) tests are 3 client/server *parity* tests
that pin a generated copy of pure (assistCredit , clubChemistryParity
, commitRoundStats logic against its client twin – they never invoke
.or any of the other 61 exports , deleteMyAccount , onGameRosterChanged
– tests Firestore *rules*, not function *logic* tests/rules/*.test.mjs
.a genuinely separate surface

UI layer: 0%. No @testing-library/react-native , no snapshot •
tests, no component tests anywhere in the repo (confirmed, D4 §8). All 55
screens and ~180 components are validated only by whoever last looked at
.them running

Net: automated tests in this codebase are almost entirely **pure-logic**
unit tests of extracted src/utils/* functions (rotation math, evening
score, round summary, club chemistry, balance, penalty stats, series
schedule) – and that stratum is genuinely strong (rotation engine alone has
dedicated files). The moment logic touches Firestore, a screen, a Cloud 6
Function, or the network, coverage falls to **at or near zero**, with the
sole exception of tests/rules/*.test.mjs (Firestore rules) and this
session's ad hoc runtime taps (P_ADMIN/P_NEW), which covered maybe a dozen
.screens once each, in mock mode, on one persona

Bottom line figure to quote: of the ~28 feature rows in the matrix
above (which already aggregate ~76 finer-grained D3 features), **12 have**
zero automated test of the actual feature behavior, **9 have runtime**
evidence from this audit only because two agents spent one session each
tapping through mock mode, and **every single row has NE or ST-only cells**

in the offline/poor-network/error/retry columns. Live match and commitRoundStats – the two features every other feature in the app is downstream of statistically (goals, wins, evening summary, round summary club stats, chemistry, achievements all derive from these two) – have **never been run** by anything in this audit, and their server half has never been run by anything, ever

The states nobody has tested, ever .3

Offline and **poor network** are, without qualification, the two emptiest columns in the matrix – literally every cell is NE. This isn't a gap in this audit's methodology; it's a gap in the product

- No `NetInfo` (or equivalent) dependency exists in `package.json`; zero imports anywhere in `src/` (RELIABILITY-7, confirmed by exhaustive grep)
- No `onSnapshot` listener anywhere checks `snap.metadata.fromCache` or `hasPendingWrites` (RELIABILITY-2/7)

The only network-aware string in the entire Hebrew locale file is `scoped to sign-in`, `signInNetworkError`

- actively **filters out** and never logs offline-adjacent `errorLog.ts` error codes (`unavailable`, `deadline-exceeded`, "client is offline") as "transient" – meaning even the one place the app centrally watches for problems is designed to look away from exactly the state this section is about

Because there is no offline detection, there is structurally no way for **error** or **retry** to be tested as first-class states either – the app has no concept of "I am currently in a degraded state," so nothing can verify what the retry path does, because there mostly isn't one is the only shared retry affordance in the whole `ErrorBoundary` `is` "נסה שוב" `app`, and it deliberately does not auto-retry (RELIABILITY, D4 §4)

What would concretely break, based on the RELIABILITY findings – these are static findings, not runtime-verified, but each is a specific file-and-line-cited mechanism, not speculation

- A goal, shootout kick, or timer press made from a stale offline cache after another admin already ended the evening **silently lands** once connectivity returns, because the "is this game finished" guard reads `from local cache with no fromCache check` (F-RELIABILITY-2)
- the single – "סיים ערב" – A network blip at the exact moment an admin taps worst-signal moment of the whole evening (packing up, walking off the pitch) – causes the final mini-game's stats to be silently dropped while the game still transitions to `finished` (F-RELIABILITY-3)
- Two admins on the same evening (very normal – "here, you take the phone") produce a last-write-wins clobber across 7 separate live-match write paths: rotation/team transitions, player swaps, and shootout start/undo/clear are all whole-object overwrites with no transaction and no conflict detection (F-RELIABILITY-5)
- A lost-ack retry on `recordGoal` / `removeGoal` double-applies the on score/tally even though the `arrayUnion` 'd goal-log `increment()` entry itself stays correct – visible as a scoreboard that disagrees with its own goal list (F-RELIABILITY-6)

A retry of `addRetroGoal` after a failed-looking toast mints a **new** • idempotency key every time, defeating the server's dedupe and genuinely .double-crediting a goal (F-RELIABILITY-10)

,Flows most exposed, ranked by how directly they sit in the poor-signal high-stakes path: (1) live-match goal/shootout/timer entry – worst connectivity of any screen in the app, feeds every downstream stat; (2) ,final round commit – highest-probability failure moment/ `endEvening` silent data loss confirmed by design (data preserved locally but nothing ever reads it back); (3) two-admin round finalization – no lock, no conflict UI, silent loser; (4) `roundHistory` 's best-effort write outside the atomic stats batch – **this one is not hypothetical**: CONSISTENCY and this audit's own production queries found **7 of 75 finished games (9.3%)** already missing `roundHistory` in the live database, across 3 real named clubs, which starves `MatchRoundsScreen` , `RoundSummaryScreen` , the club-chemistry rollup, and the evening-summary card simultaneously for those evenings – a live instance of exactly the offline/best-effort-write risk class this section describes, just triggered by something other than .airplane mode

Risk ranking – top 15 untested/under-tested cells most likely to hurt a real user on a real .4 Tuesday evening

× Ranked by (likelihood on an ordinary evening) × (blast radius)
Runtime-confirmed bugs are ranked above .(silence – does anyone find out)
.comparable static findings because they are proven, not inferred

Admin ≡ menu is entirely non-functional, app-wide (F-ADMIN-1, Po, RT-confirmed). Every tap on every row (edit club, view .1 roster, approve requests, history, stats, invite) just closes the sheet. This is the single most-used admin surface in the app and it .does nothing, silently, with no error. Confirmed on 2 clubs and independently on a game's "...” menu

Club-isolation Firestore rule is shadowed (F-SEC-1). A duplicate, weaker `communityPairStats` rule block (line 1973) overrides .2 the correct one (line 1704). Any signed-in user can read any club's pair/chemistry stats today, live, in production – no exploit needed, .just a normal authenticated read

write is best-effort, outside the atomic stats batch – confirmed missing for 9.3% of finished games in `roundHistory` .3 production, 3 real clubs. Stats land, but match history/round summary/chemistry/evening-summary silently never populate for that .evening, forever (F-RELIABILITY-8, CONSISTENCY F-1/2)

Deleting a played or live game leaves every downstream stat aggregate orphaned forever – confirmed in production data .4 (a real finished game's `roundHistory` subcollection with a real scored round survives its own parent's deletion; INVARIANTS F-INV-1, Po). Nothing reverses `users.stats` , `communityPlayerStats` , `pairStats` , etc. The delete rule doesn't even block deleting .a live game

Two admins committing the same round with different payloads – the loser's stats vanish silently (F-RELIABILITY-1). .5 .Ordinary on a well-staffed evening ("here, you take it, I need to play"). No error on either device

Offline cache bypasses the "game already finished" guard – a phantom goal/timer-start/shootout-kick lands once connectivity .6 .returns, after the evening was already sealed and shown to players (F-RELIABILITY-2)

swallows a failed final-round commit and finishes the game anyway – the highest-probability failure moment `endEvening` .7 (packing up = weakest signal) silently drops the last mini-game's stats, and the "preserved" data is never read by anything (F-RELIABILITY-3)

Chat hard-crashes on a very natural tap (member-count pill → `CommunityChatScreen` → unconditional `getFirebase()` call .8 with no mock-mode guard) – confirmed in this session, required a full app force-stop to recover (F-ADMIN-2). Unverified whether the same code shape can crash in a real prod build under some other init-timing condition, but the unconditional call itself is a real code .smell worth checking

Quick games farm global, cross-club-visible lifetime stats with zero guardrail – and 70% of all groups in production are .9
 .personal/hidden groups, so this isn't an edge case (F-GAME-1/SEC-4, per head.md's live count)

Multi-club admin: creating a game from the global "+" silently targets the wrong club, no picker, no confirmation (F-ADMIN- .10
 .3, RT-confirmed) – the exact moment a growing admin (2+ clubs) is most likely to make an unrecoverable-looking mistake

Zero backups, no point-in-time recovery, delete protection off, on production Firestore (F-FIREBASE-1, Po). Every finding .11
 above this line is unrecoverable if it goes wrong at the database level – this is the finding that turns every other "silent stat loss" bug
 .into a *permanent* one

Seven separate live-match write paths (rotation, swaps, shootout start/undo/clear) are stale-read-then-whole-object .12
overwrites – a second admin acting concurrently silently clobbers the first, no transaction, no merge (F-RELIABILITY-5). The team
 .already fixed this exact shape once for goals; it was never generalized

No offline/connectivity indicator anywhere in the app (F-RELIABILITY-7) – the root enabler of #5, #6, #12: none of those bugs is .13
 .even *detectable* by the user in the moment, because there is no signal that anything is degraded

Orphan→club promotion has no in-app entry point (F-PM-7 / D1 §6, RT-confirmed via F-ADMIN-8: checked the quick-game .14
 menu and the club-creation entry point directly, no path exists). This is the product's cheapest, fastest on-ramp for a club-less user,
 .and it dead-ends at exactly the moment retention should convert

the entire server surface that produces every number above – has zero automated tests of any – functions/src/index.ts .15
 .kind. Everything in #3~#7, #9, #12 lives in this file. It's not one bug; it's the reason none of the above has a regression net

Manual test plan – 20 cases for one evening with a real device .5

Ordered by expected yield (highest first): runtime-confirmed bugs get a
 verification pass first, then the highest-risk NEVER EXAMINED cells from
 then general sweep coverage. All cases assume **real Firebase**, not ,4§
 mock mode, since several bugs above (F-ADMIN-2 in particular) are only
 confirmed in mock mode and need a prod-mode re-check; use a disposable test
 .account/club, never the real production data per project ground rules

Admin ≡ menu, real build. Open any club you admin → tap ≡ → tap every row. Expected: navigates. If it just closes the sheet like .1
 .in mock mode, this is Po and confirmed in prod, not a mock-mode artifact

pill on a club-detail screen. Expected: does not crash. If it crashes, "שחקנים" **Member-count pill** → **chat, real build**. Tap the "N .2
 .capture the exact error text and stack

Expected: either a club picker. "מחזור למועדון" → "+" → tab מחזורים **Multi-club game creation**. As an admin of 2+ clubs, go to .3
 .appears, or the banner text names the club **you actually meant**. Create the game and verify which club it landed under

Two-phone concurrent round commit. Two devices signed in as two admins of the same club, same live game. Both open the .4
 same in-progress round. Device A logs a goal, ends the round. Device B (which didn't refresh) also ends the round with a different
 ?score. Expected: verify what actually happens to stats – does B's contribution appear anywhere, or silently vanish

Airplane mode mid-goal. Start a live match, enable airplane mode, tap "goal" 2-3 times, re-enable network after 30s. Expected: .5
 ".confirm whether the goals eventually sync, whether the score double-counts, and whether any UI ever indicated "not synced
 on the last mini-game. Re-enable network. "סיים ערב" **Airplane mode at** .6
 .Expected: check whether the final round's stats made it into `roundHistory` /the round summary, or silently didn't

Delete a live game. As admin, start a live match, commit at least one round, then delete the game via the menu (if it's reachable – .7
 see #1). Expected: check whether `users.stats` /club stats for the credited players still show the deleted game's contribution
 .afterward

Cross-club stats read. As a member of Club A only, attempt to view Club B's chemistry/pair-stats screen or any URL/deep-link .8
 .into it. Expected (post-fix): permission denied. Today: likely succeeds – this validates F-SEC-1 whether or not it's fixed yet

Quick-game stat farming. Create a quick/solo game, add yourself + 1 guest, run 3-4 fast mini-games crediting yourself every goal, .9
 end the evening. Expected: check your profile card's lifetime goals/assists – do they inflate exactly as scripted, with no cap or
 ?warning

Orphan-game promotion discoverability. After finishing a quick/orphan game, look for ANY in-app path (menu, banner, CTA on .10
 the finished game screen) to "turn this into a club," without waiting for the push notification. Expected: none exists today – confirms
 .whether this has shipped since the audit

on your largest club (≥30 members, once #1 is fixed ("לציפיה בכל השחקנים") **Large club roster**. Open the roster/players screen .11
.enough to reach it). Expected: loads without lag/crash; check pagination or scroll performance

card on the main game screen (not the "הכוחות שחולקו" **team game summary card**. Create or open a 4-team game. Look at the-4 .12
.edit modal). Expected: today shows only team 1 – verify whether a swipe/tab has been added since

"המשך" **Game-wizard numeric steppers**. In game creation, tap the players-per-team number field, type a value, then try tapping .13
.immediately after. Expected: today, the keyboard re-steals focus – verify whether this still reproduces

"המשך" **Club-creation dirty-field trap**. Fill in club name + city, leave the description field focused (don't tap elsewhere), tap .14
.Expected: today produces a false "club name required" dialog – verify whether it's fixed

Retro-goal retry. As admin on a finished game, add a retro goal, force it to fail (airplane mode at the moment of tap), let the "failed" .15
.toast appear, then retry once network returns. Expected: check whether the goal was credited once or twice

Shootout double-start. Two admin devices on the same live shootout. Device A starts the shootout and logs 2 kicks. Device B, still .16
.on the pre-shootout screen, taps "start shootout" too. Expected: check whether B's action wipes A's 2 kicks

Cancel registration while offline. Join a game, then go offline and tap "cancel registration," then reconnect. Expected: check the .17
.final state – canceled, still registered, or duplicated in a weird waitlist state

Fresh, empty club – every stats screen. Create a brand-new club with zero games played. Visit club stats, chemistry, history, and .18
the club-level/titles screen. Expected: all render a sensible empty state, none crash, none show NaN / undefined /a divide-by-zero
.artifact

end to end. "מחק את החשבון שלי" **Full account deletion**. On a disposable test account with some game/chat/feedback history, run .19
Expected: verify the Auth account is gone and the profile is anonymized; then (as a separate admin/dev check, not user-facing)
confirm whether feedback , errors , and games/{id}/physical/{uid} docs referencing that uid still exist – they're expected to,
per F-SEC-8, but this closes the loop on what "delete" actually does today

Notification prefs round-trip. Toggle every notification type off in settings, trigger one of each (game reminder, spot-opened, .20
evening-summary) via normal app use, confirm none arrive. Toggle back on, trigger again, confirm they do. Expected: no cross-
.contamination (an "off" type never fires; an "on" type isn't silently dropped by a stale dedup bucket from before the toggle flip)

Sources

```
, D3_features.md , D2_backend.md , ../discovery/D1_screens.md
, INVARIANTS.md , CROSSEXAM.md , ../findings/RELIABILITY.md ; D4_infra.md
, CONSISTENCY.md , PRODUCT.md , GAMING.md , SECURITY.md , DB.md
, P_ADMIN.md , P_NEW.md ; DELETE.md , STATS.md , FIREBASE.md , ARCH.md
direct find / wc -l over; COPY.md , A11Y.md
(Jest + 4 rules .mjs = 72 files 68) /Users/matan/Projects/soccer/tests/
.and confirmation that functions/ has no test directory
```

COPY

Teamder Hebrew Copy Audit

Source: src/i18n/he.ts (3,369 lines, read in full) + grep sweep of src/screens and
for hardcoded Hebrew literals outside it. Static analysis only – no src/components
runtime available. All findings below are grounded in the actual file content; nothing
.here is SUSPECTED unless labeled so

TERMINOLOGY (the most valuable findings – the vocabulary IS the mental model)

does double duty for two different concepts “משחק” ;doesn’t exist “משחקון” – F-COPY-1

- Severity: P1
- Confidence: Confirmed
- Feature / Screen / Flow: app-wide vocabulary
- “משחקים”/“משחק” Evidence: `grep -c 'משחקון' src/i18n/he.ts` → **0 hits**. Meanwhile `:rotationStartRound (he.ts:626), {n}$ משחק matchRoundsRoundN: (n) => \` – is used for (a) one mini-game inside an evening `roundSummaryStatRounds: (n) => {n} (he.ts:368), 'התחל משחק' games/{id} (he.ts:954) – **and** (b) the entire evening/session (a doc, i.e. what the rest of the app calls a מחזור): wizardScheduledRegHint בחר מתי **המשחק** יופיע בפיד וההרשמה תיפתח. עד אז **המשחק** נסתר. מתאים גם למשחק חד-פעמי” – he.ts:767-768 , Its sibling key two lines down, wizardRegOpensHint` (he.ts:770-771) , wizardRegOpensHint` (he.ts:770-771) נסתר מכולם.” describes the exact same feature and correctly says Same UI feature, two adjacent strings, two different nouns for the .subject`
- Current behaviour: The product’s canonical mini-game word never made it into the .silently absorbed both meanings “משחק” ;copy in the scheduled-registration hint has “עד אז המשחק נסתר” Problem: A user reading no way to know whether that means the whole evening is hidden or just one round of play – especially confusing since the very same screen elsewhere talks about “משחקים” inside a (rounds) מחזור.
- User impact: Ambiguity in a settings screen that controls when an entire event .becomes visible – the highest-stakes place for this word to be vague
- Technical impact: None (copy-only), but it’s a leading indicator that engineers .as a generic filler word rather than a deliberate term “משחק” reach for strictly for the mini-game unit. Rewrite “משחק” Recommendation: Reserve `wizardRegOpensHint` throughout, matching “המחזור” to use `wizardScheduledRegHint` that isn’t inside a rounds/rotation “משחק” verbatim in structure. Audit every other .context
- Expected benefit: One evening = one word, everywhere
- Effort: S

F-COPY-2 – The “quick game” (orphan-context game) has three different names

- Severity: P1
- Confidence: Confirmed
- Feature / Screen / Flow: Quick-game creation + notification settings
- Evidence:
 - `(he.ts:1119) matchDetailsCommunityOrphan: 'מחזור חד-פעמי'`
 - `(he.ts:1155) createGameOrphanCta: 'צור מחזור חד-פעמי'`
 - `(he.ts:1157) createGameOrphanBanner: 'מחזור חד-פעמי – תוכל לקבע מועדון אחרי שתשחקו'`
 - `(he.ts:1590) createGameChooseQuickBody: '...מחזור חד-פעמי בלי לפתוח מועדון'`
- `wizardScheduledRegHint` , `he.ts:767` :vs. “...מתאים גם למשחק חד-פעמי וגם למחזור שבועי” (swaps מחזור for משחק)
- `notifGamePlayersJoinedSub` , `he.ts:3157` :vs. “מישהו נרשם למשחק שאני מארגן (כולל מחזור שבועי ומשחק מהיר) – “quick” a third label, “quick” game”, which happens to be the exact internal/product name for the feature but never appears anywhere a user actually creates .one
- Current behaviour: One feature, three labels “מחזור חד-פעמי”, “משחק חד-פעמי”, “משחק מהיר”.

- and later opens notification settings “מחזור חד-פעמי” Problem: A user who creates a as the same thing and may misconfigure their “משחק מהיר” will not recognize notification preferences.
- User impact: Silent settings mismatch – user thinks a notification toggle doesn’t apply to the games they actually create.
- Technical impact: None.
- the term used in the actual) “מחזור חד-פעמי” Recommendation: Standardize on creation flow, where users form the mental model) and fix the two outliers at he.ts:767 and he.ts:3157.
- Expected benefit: Notification settings map cleanly onto what users experience at creation time.
- Effort: S.

F-COPY-3 – Two parallel reliability systems, one stale name in a live warning dialog

- Severity: P2
- Confidence: Confirmed
- Feature / Screen / Flow: Late-cancellation confirmation
- Evidence: he.ts:1797 – code comment: “Trust meter (replaces the old yellow/red discipline UI).” Yet `lateCancelBody` (he.ts:207-209), shown when a user tries to **cancel close to kickoff, still says “...ביטול בשלב הזה ייספר כביטול מאוחר וישפיע על דירוג המשמעת שלך. בטוח שאתה רוצה לבטל?”** – “your discipline rating” – `trustMeterTitle: 'מדד אמינות'` The live, current-generation feature is named `tipTrustText`, built explicitly (he.ts:15, 1800), “מנוכחות במחזורים וביטולים בזמן” (he.ts:16) – i.e. the exact same behavior (attendance + late cancellation) the cancel dialog is warning about.
- Current behaviour: The most consequence-laden line in the cancellation flow cites a system the codebase’s own comment says was replaced.
- Problem: Either this is stale copy left over from the pre-trust-meter era (likely given the comment), or “discipline rating” and “trust meter” really are two separate live scores – in which case the app has never explained that to the user anywhere else in he.ts.
- User impact: A player being warned about a real consequence is told the wrong name for what will actually be affected – undermines the warning’s credibility, and if “מד” they won’t find one, only “דירוג משמעת” they later check their profile for a “אמינות”.
- Technical impact: Suggests the copy wasn’t updated when the underlying feature was swapped (matches the “Read deserializers strip new fields” and “stale copy” pattern already known elsewhere in the codebase per project memory).
- Recommendation: Change `lateCancelBody` to name the trust meter “...ביטול בשלב הזה ייספר כביטול מאוחר וישפיע על מד האמינות שלך.”
- Expected benefit: The warning tells the truth about what’s at stake.
- Effort: S.

F-COPY-4 – כוחות vs. קבוצות mixed within a single explanatory paragraph

- Severity: P3
- Confidence: Confirmed
- Feature / Screen / Flow: Auto-teams scheduling toggle
- Evidence: `wizardAutoTeamsToggle: 'תזמון יצירת כוחות אוטומטיים'` (he.ts:1089) titles its own hint text `wizardAutoTeamsHint` (he.ts:1090-1091) כוחות; the feature with switches mid-paragraph “המערכת תחלק את הנרשמים לקבוצות מאוזנות... שים לב: אם כבר קבעת **כוחות** ידנית...” – the same paragraph names the artifact being created – two different ways.

- is otherwise used consistently for “the balanced team כוחות: Current behaviour split as a whole” (draftTitle, draftPublishCta, sessionStatusTeamsReady, etc. – 45 occurrences) while “is used for “one specific team קבוצה/קבוצות (קבוצה 1, קבוצה א', team colour, etc. – 86 occurrences). That split is actually a coherent pattern everywhere else in the file
- .Problem: This one paragraph breaks the pattern it otherwise holds to
- is a “קבוצות מאוזנות” User impact: Minor – a careful reader might wonder if “different, lesser-known step than “כוחות.”
- to match the rest of “לכוחות מאוזנים” to “לקבוצות מאוזנות” Recommendation: Change the string and the rest of the file’s own convention
- .Expected benefit: Reinforces a distinction the app gets right everywhere else
- Effort: S

GENDERED LANGUAGE

F-COPY-5 – The entire “coach” push-copy module (~35 strings) addresses only men

- Severity: P1
- Confidence: Confirmed
- “הודעה מהמאמן” Feature / Screen / Flow: assistant* keys (he.ts:2412-2610), the home-card / push module – a large, deliberately-crafted personality voice sent to every user
- Evidence (all use masculine “אתה” representative):
 - assistantCrownGoalsHeld: “אתה מלך השערים של {club}...” (he.ts:2467)
 - assistantPostGameWeek: “{n} מחזורים השבוע. אתה בכושר 🔥” (he.ts:2463)
 - assistantGameDayStreak: “אתה מחזיק רצף של {n} הגעות רצופות...” (he.ts:2441)
 - assistantAttendanceRate: “{pct}% הגעה. אפשר לסמוך עליך ✅” (he.ts:2492, masc)
 - assistantJoinClub: “ברוך הבא ל- (he.ts:2588, masculine welcome) Teamder! 🙌”
 - assistantClubsCount: “אתה חבר ב- {n} מועדונים ⚽” (he.ts:2559)
- ;counted across this block alone (full-file count is 35 “אתה” more instances of 30 (the great majority sit in this module
- Current behaviour: Every stat/streak/crown/comeback push line assumes the reader is male, with zero feminine variant
- Problem: The app has female players (community-rating, guest-adding, and profile -copy elsewhere never assume gender). The team clearly *knows how* to write gender neutral Hebrew – equipmentHolderBallA11y: 'מחזיק/ה את הכדור' (he.ts:400) and both use the (he.ts:723) editGameCapacityTooLowBody: '...הסר/י קודם שחקנים...' slash form – but that pattern was never carried into the highest-volume, most personality-driven copy block in the file
- User impact: Every push notification and home-card line from “the coach” reads as written for a man, to a player base that includes women
- Technical impact: None – pure copy, but the fix needs the user’s stored gender (if captured) or a rewrite to second-person-neutral phrasing
- winnerPickTitle Recommendation: Either (a) branch on a stored gender field the way style copy doesn’t need to, or (b) rewrite the templates to avoid the second-person pronoun entirely where possible (e.g. “רצף של {n} הגעות רצופות ב{club}” – אל תשבור אותו היום 🔥, “אתה” works with the streak stated as a fact, slash form where a pronoun is unavoidable /ה
- Expected benefit: The single highest-visibility recurring copy surface in the app stops assuming the reader’s gender

Effort: M (many strings, but mechanical once a convention is picked) •

F-COPY-6 – The one destructive-confirmation with an explicit pronoun is also the only gendered one

Severity: P3 •

Confidence: Confirmed •

Feature / Screen / Flow: Late-cancellation confirm dialog •

Contrast with “בטוח שאתה רוצה לבטל?”: Evidence: `lateCancelBody` (he.ts:208) •

every other confirm dialog in the file, which is written impersonally to sidestep

, `cardRevokeConfirmTitle`: 'לסיום את המזכור', `liveEndEveningTitle`: 'להחזיר את הכרטיס' •

`restoreConfirmTitle`: (name) => \

none of those need a pronoun and none use one

Problem: This is the one confirm string that breaks the file's own (good, mostly

consistent) impersonal-phrasing pattern for confirms

or simply drop the question and let the two “בטוח שברצונך לבטל?”: Recommendation •

carry the decision, matching every sibling dialog (cancel / “אישור ביטול”) buttons

.Expected benefit: One less gendered outlier in an otherwise gender-safe pattern •

Effort: S •

ERROR MESSAGES

with zero diagnosis, and the file proves it can “F-COPY-7 – ~25 error toasts collapse to “X do better

Severity: P2 •

Confidence: Confirmed •

Feature / Screen / Flow: app-wide error toasts •

, Evidence: `guestErrorGeneric`: 'הפעולה נכשלה' (he.ts:1398, no even “try again”) •

, (he.ts:3287) `draftSaveError`: 'נסו שוב', `retroActionFailed`: 'נסו שוב', (he.ts:540) `autoBalanceError`

, (he.ts:3306) `friendsActionFailed`

and, (he.ts:1600) `groupWizardSubmitFailed`, (he.ts:1547) `profileSaveError`: 'לא הצלחנו לשמור את הפרטים. בדקו את החיבור ונסה שוב' •

more, all the same shape. Compare `chatSendFailedBody`: 'בדקו את החיבור ונסו שוב' (he.ts:3329) right above

(he.ts:144) and, most tellingly, two

adjacent sibling keys in the chat module: `chatDeleteFailedBody`: 'נסו שוב' •

same feature, three lines apart, one tells the user what to check – (he.ts:3334)

and the other doesn't

Current behaviour: The dominant error pattern gives no distinction between “you're

offline,” “the server rejected this,” or “something is broken” – every failure

reduces to the same three words

Problem: A user who hits a real, persistent failure (e.g. a permissions issue) will

be told to just try again, which won't help, with no path to figure out why

User impact: Dead-end error states; users can't self-diagnose or know when to

actually contact support vs. just retry

-Technical impact: None directly, but this correlates with there being no crash

reporting SDK in the app (per project ground truth) – errors are copy-only, not

even logged anywhere the team would see

Recommendation: At minimum, split the generic bucket into “check connection and

retry” (network-shaped failures) vs. “something went wrong, we're aware” (server

, `profileSaveError`) errors – the file already has the good version

to use as the template for the rest (`chatSendFailedBody`

- Expected benefit: Every failure toast becomes actionable instead of a shrug
- Effort: M

EMPTY STATES

F-COPY-8 – A cluster of empty states are bare nouns with no explanation or next step, next to others in the same file that do it well

- Severity: P3
- Confidence: Confirmed
- Feature / Screen / Flow: Profile / stats / community screens
- Evidence: statsEmpty: 'אין עדיין נתונים' (he.ts:1196), profileActivityEmpty: 'אין עדיין פעילות להצגה' (he.ts:2303), communityPlayersEmpty: 'אין עדיין שחקנים בסגל' (he.ts:1501), chatMembersEmpty: 'אין חברים להצגה' (he.ts:3358) all state absence and stop. Compare the file's own best-practice empty states just a few hundred lines away: emptyHomeNoGamesAnywhere: 'אין מחזורים פתוחים כרגע – היה הראשון לפתוח מחזור למועדון שלך' (he.ts:2166, explains + invites action) and groupsSearchEmpty: 'אין תוצאות. נסה חיפוש אחר' (he.ts:1169, gives a next step) disciplineSnapshotUnavailable: Also a duplicate-copy smell reinforcing F-COPY-3 and trustMeterUnavailable (he.ts:1803) are both the literal string (he.ts:1791) 'אין נתונים זמינים' – two supposedly-different systems sharing byte-identical .copy suggests they were never really differentiated in the writer's mind either .Recommendation: For empty states that a user could act on (stats, activity) explain what would appear there and how to make it appear, following the pattern already proven in the file. For states with emptyHomeNoGamesAnywhere – nothing actionable (blockedEmptyTitle, requestsEmpty) the bare form is fine .leave those
- Expected benefit: Consistent empty-state quality bar across the app
- Effort: S

TONE

F-COPY-9 – The playful “coach” voice is warm and well-observed, but confirms it never mocks a bad evening

- Severity: – (positive finding, per brief's “genuinely fine” allowance)
- Confidence: Confirmed
- Evidence checked: assistantOwnGoals (he.ts:2540-2543): “שער עצמי אחד בקריירה. קורה גם לטובים 😊” / “{n} שערים עצמיים בקריירה. שער זה שער, לא? 😊” – self-deprecating toward the *feature* (own goals), never unkind toward the player 'לא שיחקת משחקים במחזור הזה, אז אין סיכום להציג' (he.ts:657) summaryNoPlay: plain and neutral, not snarky, for a case (didn't play) that could easily have been written as a jab. No string found anywhere in the file that mocks a losing team a low rating, or a bad streak – the playful voice is reserved for positive/neutral .framing only. This category is genuinely fine; no findings to file

F-COPY-10 – Tone gap between the “coach” module and the rest of the app is a real seam, not just a style choice

- Severity: P4
- Confidence: Suspected (inference, not a bug)

- Evidence: The `assistant*` block (he.ts:2412+) is dense with slang and emoji per line (ברזל 🍌, "כפפות זהב 🧤", "תן בראש 🧨"), while five lines above it, `notifTrackingSub` reads in flat product-settings register (he.ts:3185) `עוזר לנו`. Both are correct for their context (a settings screen shouldn't be jokey), but the app's voice swings from "buddy who just watched you score" to "terms-of-service" register within the same session with nothing in between – there's no "friendly but calm" register for the large middle ground (confirms, wizards, empty states).
- Recommendation: Not a fix-it bug – flagging for awareness. If the team wants a more coherent brand voice, define a middle register for wizards/confirms rather than toggling between the two extremes already in the file.
- Effort: L (voice-and-tone pass, out of scope for a copy audit)

CENTRALIZATION (strings living outside he.ts)

F-COPY-11 – Real, production-facing Hebrew strings are hardcoded outside he.ts, including one that duplicates an existing key

- Severity: P2
- Confidence: Confirmed
- Feature / Screen / Flow: Evening-summary share card, player comparison, chat terms, availability radius
- Evidence:
 - `<Text>סיכום המפגש</Text>` - `src/components/summary/EveningSummaryCard.tsx:89` hardcoded, byte-identical to `summaryTitle: 'סיכום המפגש'` already in `he.ts`. If `summaryTitle` is ever reworded, this shareable card (which (he.ts:906-907) gets exported/screenshotted, per project's "release log screenshot" convention) silently drifts out of sync.
 - and `<Text>אתה</Text>` - `src/components/compare/PlayerCompareCard.tsx:100,115` doesn't appear anywhere ("rival/opponent") `<Text>יריב</Text>` hardcoded in `he.ts` as a defined term for the compared-against player.
 - hardcoded `title: 'כללי השיחה'` - `src/components/chat/ChatTermsModal.tsx:33` fallback title - `src/screens/games/GameCreateScreen.tsx:466` hardcoded inline instead of referencing `he.ts` (compounds F-COPY-2 'מפגש חד-פעמי' (this is a *fourth* place spelling out the same quick-game label).
 - `src/screens/profile/AvailabilityEditScreen.tsx:559` (he.ts:83), yet `he.ts` already defines `gameFiltersKm: (km) => \$(km) 560`, `src/components/availability/AvailabilityRadiusMap.tsx:90`, and unit locally instead of importing the `ק"מ` all re-hardcode the `src/components/availability/AvailabilityRadiusMapModal.tsx:162-163` existing helper.
 - + `src/screens` Current behaviour: `grep -rLP` for Hebrew characters across hits 150 files; the majority are Hebrew-language code comments `src/components` but the sample above shows real user-facing literals slipping through (harmless) in at least 6 files.
 - Problem: Defeats the stated purpose of `he.ts` ("centralized... future i18n is trivial") and creates silent drift risk, as already demonstrated by the duplicated `summaryTitle` string.
 - Recommendation: Move the 6 files' literals into `he.ts` under new or existing keys; add an ESLint rule or CI grep gate flagging new Hebrew-character string literals outside `src/i18n/` (the project has no ESLint config today per ground truth, so (this would need to be a plain script check, not a lint rule).

- Expected benefit: Guarantees the “one source of truth” the file’s own header .comment promises
- Effort: M

CTAs AND DESTRUCTIVE COPY (both largely fine – noted briefly per brief’s guidance)

F-COPY-12 – Destructive-action copy is consistently strong; no findings

- Severity: – (positive)
- Confidence: Confirmed
- Evidence: `, liveEndEveningBody , deleteGroupBody , deleteGameBody` all state the concrete, irreversible `cardRedConfirmBody , stopSeriesBody` blocks, consequence in plain language (“לא ניתן לחזור אחורה”, “יימחקו לצמיתות”) and the file even has a dedicated reusable pattern for it described by name) and the file even has a dedicated reusable pattern for it (‘אני מבין שהפעולה בלתי הפיכה’, `confirmDeleteAck: (he.ts:117 ,`) This category is .genuinely solid across the file; no rewrite needed

F-COPY-13 – CTA verbs are specific and name the action; one soft spot

- Severity: P4
- Confidence: Confirmed
- Evidence: The large majority of CTAs name the exact action (“שלח דירוג”, “פרסם כוחות”, “הזמינו חברים”, “צור מחזור חד-פעמי”) rather than defaulting to generic (“אישור”/“המשך.”) (“send to everyone, in pulses” – “in pulses” – “בפעימות” – “שלח לכולם בפעימות” `sendPulseCta: 'שלח לכולם בפעימות'` One soft spot: `sendPulseCta: 'שלח לכולם בפעימות'` (“send to everyone, in pulses” – “in pulses” – “בפעימות” – “שלח לכולם בפעימות” `sendPulseCta: 'שלח לכולם בפעימות'` is an internal mental model for the staggered-invite mechanism, not a phrase a player would use). `sendPulseExplain` (he.ts:678-679) does clarify it in the body text immediately below, so this is low severity – the CTA label itself just leans on a term only explained one screen-element away
- Recommendation: Consider “שלח הזמנות בהדרגה” (send invites gradually) as a more self-explanatory CTA label; keep only in the explanatory body where it’s “בפעימות” already defined
- Effort: S

LENGTH RISK

F-COPY-14 – Two status/header strings likely to crowd or wrap awkwardly on a narrow RTL pill or card header

- Severity: P4
- Confidence: Suspected – not verified at runtime (no working emulator/simulator)
- Evidence: `sessionStatusWaitingPlayers: (cur, max) => \` מחכים לשחקנים (he.ts:238-239) – an emoji + 3 words + a parenthetical count, used as what its sibling keys (`sessionStatusTeamsReady , sessionStatusActive`) suggest is a small status pill, is long relative to those 1-2 word siblings and may wrap or truncate at small device widths.
- a section – (he.ts:1784) `disciplineSnapshotTitle` +header carrying a parenthetical qualifier; on a 360dp-wide card this is 22 characters plus the numeral, tight for a header row that likely also carries an icon or action affordance

Recommendation: Move the parenthetical qualifiers into a subtitle/caption line rather than concatenating them into the primary label; verify both on a narrow .device once the emulator is fixed

Effort: S

Summary of priorities

- F-COPY-1 / F-COPY-2** – the app's own core nouns (evening vs. mini-game vs .1 ;quick-game) collide with each other in specific, quotable places. Fix these first .they're the ones the brief called out as most valuable
- F-COPY-3** – a live warning dialog cites a system the code says was retired .2
- F-COPY-5** – the highest-volume personality copy in the app is male-only, while .3 .the team has already proven it knows the neutral-form pattern elsewhere
- F-COPY-7 / F-COPY-11** – systemic, mechanical fixes (error-message template .4 .centralization) that pay off across dozens of strings at once

A11Y

Teamder accessibility audit (static analysis only – SUSPECTED items marked)

All numbers from grep/heuristic scans of `src/` on 2026-09-02, branch
No emulator/simulator was used; nothing here is a . `perf/firestore-read-costs`
.verified runtime observation unless stated

F-A11Y-1 – 92% of `<Pressable>` elements carry no accessibility props on the tag itself

- Severity: P1
- Confidence: Confirmed (grep + AST-ish heuristic – see method below)
- Feature / Screen / Flow: app-wide
- Evidence: `grep -c "<Pressable" src -r --include=*.tsx` → 448 opening tags
- A script parsed each opening tag (up to its `> / >`) and checked for inline: only 35/448 (7.8%) have `accessibilityRole` / `accessibilityLabel` : `accessibilityRole` .either. Codebase-wide `accessibilityLabel` occurrences: 173 , `View` , `Text` .these totals include non-Pressable elements too, e.g) 188 so the true Pressable-labelling rate is lower than either raw , `TextInput` – total suggests). `TouchableOpacity` / `TouchableWithoutFeedback` : 0 uses .the app is consistently on `Pressable` , which is good, it's just mostly unlabelled
- Current behaviour: A VoiceOver/TalkBack user hitting most Pressables gets either silence or RN's fallback (reads child `Text` if present, otherwise .(button" with no name – e.g. icon-only buttons announce nothing useful"
- Problem: No systematic labelling convention/lint rule enforces it; the 35 .labelled instances are scattered, not concentrated in a base component
- User impact: A screen-reader user cannot identify most icon-only controls .(back buttons on some screens, action icons, list-row actions)
- Technical impact: none (build-time safe), pure a11y debt

- Recommendation: Add labels to the worst-offending screens first (below)
- then add an ESLint rule (there is currently no ESLint config at all per Pressable / PressableScale ground truth) or a codemod-driven push through .wrappers used app-wide
- Expected benefit: baseline screen-reader usability
- Effort: L (app-wide), but M to fix the top screens below

Worst screens by count of unlabelled Pressables (script counted openings
:(lacking accessibilityLabel / accessibilityRole on the same tag
of its Pressables unlabelled 21 – src/screens/AdvancedLiveMatchScreen.tsx -
.admin live-match controls: goal entry, substitutions, round actions)
unlabelled (penalty shootout kick 21 – src/components/match/Shootout.tsx -
.buttons – a fast-tempo, high-stakes screen for a blind user to navigate blind
unlabelled (the main game 17 – src/screens/games/MatchDetailsScreen.tsx -
.hub: join/leave/roster actions
.unlabelled 14 – src/screens/games/MatchPlayersScreen.tsx -
;unlabelled (the primary feed 11 – src/screens/games/GamesListScreen.tsx -
line 666, 825, 849, 979, 1135+ are card/row-level Pressables – the main
“open a game” tap targets”
.unlabelled 11 – src/components/match/LiveScoreboardCard.tsx -
unlabelled (swap/rotate 8 – src/components/match/RotationPanel.tsx -
.(controls during a live game

Note: src/screens/games/MatchDetailsScreen.tsx pending-request approve/reject
buttons (line 3096-3115) DO carry accessibilityRole / accessibilityLabel – not
every screen in the offenders list is uniformly bad; it's a per-Pressable gap
not a per-screen one

F-A11Y-2 – Two confirmed touch targets under 44x44 with no hitSlop compensation

- Severity: P2
- Confidence: Confirmed
- Feature / Screen / Flow: Match roster approval; in-app chat
- Evidence
- pendingApprove: { width: 40, height: 40, ... } src/screens/games/MatchDetailsScreen.tsx:4153
and :4161 pendingReject: { width: 40, height: 40, ... }, used at lines
.hitSlop with accessibilityLabel present but no 3102/3110
- , sendBtn: { width: 42, height: 42 } src/components/chat/ChatView.tsx:1056
.hitSlop used at line 554, no
- Current behaviour: 40x40 and 42x42 are both below the 44x44 WCAG/HIG
minimum, and unlike most of the codebase's small controls these three have
.no hitSlop to compensate
- “Problem: small, precision-dependent tap targets on a “yes/no join a game
.decision and the chat send action
- ;User impact: mis-taps for users with motor impairment or larger fingers
.send-message miss-taps are a recurring annoyance pattern
- Technical impact: none
- Recommendation: add hitSlop={4} (40→48) / hitSlop={2} (42→46) or bump
.the boxes to 44
- Expected benefit: fewer mis-taps on two high-frequency actions
- Effort: S

Context – the rest of the app already uses `hitSlop` as a size workaround extensively: 62 files use `hitSlop`, e.g. `ScreenHeader.tsx`'s 26px-wide back chevron gets `hitSlop={12}` (→ 50px effective) and is properly labelled. That pattern is correctly applied almost everywhere (`accessibilityLabel="חזרה"`) everywhere else – the two components above are the exceptions, not the norm. A further ~40+ icon buttons sized 22-40px were found via grep across `, CommunityPlayersScreen , AdvancedLiveMatchScreen , LiveMatchScreen , FriendsScreen , NotificationsSettingsScreen , PlayerCardScreen` etc. – most were not individually verified to have matching `hitSlop`; treat as SUSPECTED – spot-checked only, not exhaustively verified `hitSlop`; treat as

F-A11Y-3 – Team identity is NOT colour-only – checked and clean

- Severity: n/a (verified fine)
- Confidence: Confirmed
- Feature / Screen / Flow: Live match, round history, chemistry, draft/teams UI
- Evidence: every team-colour usage found (`grep -rL "colors.team[1-4]" → 5 :files`) pairs the colour with a text label
 - maps `TEAM_LABEL` – `src/components/match/GameHistoryRow.tsx:15-44` to `he.team1/2/3` Hebrew strings; `resultColor` and `team1/2/3` are always set together `resultText`
 - array is `TEAM_HEX` – `src/screens/games/MatchRoundsScreen.tsx:53-59` always zipped with `teamName(index)` from `@/utils/draft`; the legend renders `teamsLegendName` text next to the colour swatch (line 232)
 - `, src/components/match/TeamsEditModal.tsx:58/234 , src/components/draft/DraftTeamCard.tsx:73-115 , src/screens/AdvancedLiveMatchScreen.tsx , src/components/match/RotationPanel.tsx:387 , src/components/match/Shootout.tsx:206-209/469/502/661` and `(rotation/round UI)` `teamName(index, draftTeams?.teams)` all resolve a (penalty shootout) alongside (fallback `”קבוצה א’/ב’”` a real club-chosen name, or) string the colour, never colour alone
- doesn't use team colours `src/components/chemistry/ChemistrySection.tsx` (`cardNames`) at all – chemistry cards render player names as text
- has zero team-colour rendering – consistent with `LiveMatchScreen.tsx` the ground truth that live match is timer-only (teams/rotation removed)
 - Conclusion: this was flagged in the brief as the most serious possible finding, and it does not hold up under inspection – every place a team colour swatch appears, a name string is rendered next to it. No P0/P1 finding here. (Not verified at runtime that the text is never clipped to (invisibility – see F-A11Y-2's sibling risk in F-A11Y-8 on fixed containers

F-A11Y-4 – Muted/secondary text and all four status colours fail WCAG AA against the app background

- Severity: P2
- Confidence: Confirmed (computed contrast ratios; formula-verified)
- Feature / Screen / Flow: app-wide (light theme is the ONLY theme shipped) forces `isDarkTheme = false` and never consults `src/theme/colors.ts` (the dark palette exists in source but is dead code; `Appearance.getColorScheme`)
- Evidence – computed WCAG contrast ratios (background `#F9FAFB` unless noted)

AA large/UI (3:1)	AA body (4.5:1)	Ratio	Pair
pass	pass	16.98:1	text #111827 / bg
pass	pass	17.74:1	#FFFFFF text #111827 / surface
pass	pass (barely)	4.63:1	textMuted #6B7280 / bg
pass	pass (barely)	4.83:1	textMuted #6B7280 / surface
pass	pass	8.72:1	#1E40AF textOnPrimary #FFFFFF / primary
pass	pass	8.35:1	primary #1E40AF / bg (links/icons)
pass	fail	3.15:1	success #16A34A / bg
fail	fail	2.06:1	warning #F59E0B / bg
pass	fail	3.60:1	danger #EF4444 / bg
pass	fail	3.52:1	info #3B82F6 / bg
pass	fail	3.08:1	#FEE2E2 team1 #EF4444 / team1Bg
pass (borderline)	fail	3.01:1	#DBEAFE team2 #3B82F6 / team2Bg
fail	fail	2.07:1	#DCFCE7 team3 #22C55E / team3Bg
fail	fail	1.79:1	#FEF9C3 team4 #EAB308 / team4Bg
n/a	n/a (non-text)	1.18:1	border #E5E7EB / bg

Problem: textMuted passes AA for body text only barely (4.63 vs 4.5 required) – any slightly-off actual rendering (subpixel AA, non-pure backgrounds like image overlays) risks failure; it's the single most-used secondary colour in the app (timestamps, hints, placeholders) so it's worth flagging even though it currently passes. The four semantic status colours and all four team colours fail AA (info / danger / warning / success) as text-on-tinted-background – this matters because these colours are used for text/icon content (e.g. status Badges, chip text), not only as fills. warning and team4 (yellow) are the worst offenders at ~2:1 and those are barely distinguishable from their own background for a low-vision user.

User impact: low-vision users may not be able to read status/warning text or team-colour labels when rendered as text-on-tint (as opposed to a solid .chip with white text, which is fine per the textOnPrimary row). Technical impact: none.

, colors.warning Recommendation: audit every call site that renders etc. as text color (vs. as a fill with colors.team4 , colors.team3 , textOnPrimary + white/dark text on top) and switch those to solid-fill .or darken the foreground tokens used for text-on-tint pairings .Expected benefit: readable status/warning states for low-vision users .Effort: M (token change + spot-check every consuming screen)

Caveat: this is a pure sRGB hex contrast computation against the documented it does not account for opacity/blur overlays some screens ; lightPalette

apply, which could move ratios further from AA. SUSPECTED – not verified visually for any specific screen's actual rendered .contrast

F-A11Y-5 – `allowFontScaling={false}` disables OS text-size scaling in 7 files, 9 places

Severity: P2 •

Confidence: Confirmed •

Feature / Screen / Flow: Splash screen, stat tiles, player identity, trust meter, achievement badges, discipline cards, morph button

:Evidence: `grep -rn "allowFontScaling={false}" src` → 9 hits in
, `src/components/DisciplineCards.tsx` , `src/screens/SplashScreen.tsx`
, `src/components/PlayerIdentity.tsx` , `src/components/StatTile.tsx`
, `src/components/AchievementBadge.tsx` , `src/components/TrustMeter.tsx`
with any value `allowFontScaling`) . `src/components/anim/MorphButton.tsx`
appears 12 times total, so 9/12 explicitly disable it; 3 presumably set it
(.or dynamic `true`

Problem: a user who raised their OS text size (common for low-vision users) gets these specific texts pinned at design-time size while the rest of the app scales – inconsistent and, for anyone relying on large text .potentially unreadable stat numbers/badges

User impact: low-vision users lose scaling exactly on the components most likely to carry (`PlayerIdentity` , `AchievementBadge` , `StatTile`) .meaningful numeric/status info

.Technical impact: none •

Recommendation: remove `allowFontScaling={false}` unless the component has a specific fixed-size layout constraint that would visually break – and if .so, cap scaling with `maxFontSizeMultiplier` instead of disabling outright

.Expected benefit: consistent, OS-respecting text scaling •

.Effort: S •

:Fixed-height text containers that would additionally clip on scale-up
, (panels `270 / height: 300`) `src/screens/LiveMatchScreen.tsx:848/868`
mirrored in `AdvancedLiveMatchScreen.tsx:2371` (`height: 270`) – these are scrollable stoppages/notes panels so clipping risk is lower, but not verified
. `SUSPECTED – not verified at runtime` .at runtime

F-A11Y-6 – Reduced-motion hook exists but only 7 of ~21 motion components (33%) actually check it

Severity: P2 •

Confidence: Confirmed •

`src/components/anim/*` :Feature / Screen / Flow •

Evidence: `src/hooks/animations/useReducedMotion.ts` is the hook. Grepping every file in `src/components/anim/` (+ `anim/game/`) for Reanimated motion (`withTiming` | `withSpring` | `withRepeat` | `withDelay` | `withSequence` | `useSharedValue`) APIs
:vs. for `useReducedMotion` usage

, `LightSweep.tsx` , `AttentionPulse.tsx` :Checks reduced motion (7)
, `game/LiveMatchEntrance.tsx` , `ScreenEntrance.tsx`
, `game/RegistrationSuccessAnimation.tsx` , `game/NextGameCardEntrance.tsx`
`src/components/match/Shootout.tsx` plus – `game/WaitlistPromotionAnimation.tsx`
.outside the `anim/` folder (8 real call sites total)

Runs motion but does NOT check it (14): `AnimatedTabIcon.tsx` (8 motion
`BallSwitch.tsx` ,calls), `AnimatedWeatherIcon.tsx` (34), `AppearItem.tsx` (6)
`CelebrationOverlay.tsx` ,(10) `Breathing.tsx` ,(23) `BouncingBall.tsx` ,(4)
,(9) `MorphButton.tsx` ,(28) `LivingIcon.tsx` ,(4) `ConfettiBurst.tsx` ,(4)

and ,6) Shimmer.tsx ,(10) RollInView.tsx ,(7) PulseOnChange.tsx
 ,(Shimmer transitively MatchCardSkeleton.tsx which composes only
 .(12) SpringSheet.tsx
 /Problem: the reduced-motion opt-out is applied inconsistently – entrance -
 ,celebration animations on game-list cards were clearly retrofitted with it
 , Shimmer , LivingIcon , Breathing) but continuous/looping effects
 weather icon animation, tab-icon animation) and , BouncingBall
 interaction feedback (MorphButton , PulseOnChange , BallSwitch) never
 .check the user's preference at all
 User impact: a user with vestibular sensitivity who enables “reduce -
 motion” still gets continuous looping/pulsing animation from at least 14
 distinct components used across the app's most common surfaces (tab bar
 .(icons, skeleton loaders, weather widget, buttons
 .Technical impact: none -
 Recommendation: route these through the hook, or better, centralize the -
 withSpring / withTiming check at the Reanimated config level (e.g. a wrapped
 helper that no-ops duration when reduced motion is on) so new components
 .can't forget it
 .Expected benefit: consistent reduced-motion compliance app-wide -
 .Effort: M -

F-A11Y-7 – RTL handling is largely disciplined; the risk surface is narrow and mostly commented/deliberate

- Severity: P3 •
- Confidence: Confirmed •
- Feature / Screen / Flow: app-wide •
- :Evidence •
- ,defines RTL_LABEL_ALIGN = 'left' with an explicit src/theme/rtl.ts •
- well-documented rationale (forceRTL flips textAlign: 'right' to visual
 – left on both platforms) and is used **586** times across the codebase
 .this is clearly the established, dominant pattern
- Raw textAlign: 'right' still appears **28** times. Several of these are •
- explicitly commented as deliberate/known (“ textAlign: 'right' is
 , GameWizardForm.tsx:1566 interpreted as end of paragraph...” in
 , InfoTip.tsx:124/131 , InputField.tsx:227 , MatchDetailsScreen.tsx:4735
 i.e. the team is – (PlayerCountBar.tsx:131 , AutocompleteInput.tsx:11
 actively aware of this trap and has left comments as guardrails, not
 , PublicGroupsFeedScreen.tsx:962 , FeedbackScreen.tsx:203) fresh mistakes. A few
 , AvailabilityEditScreen.tsx:843 , MapScreen.tsx:636
 (GuestModal.tsx:406 , RuleTagsInput.tsx:204 , ScreenshotReportSheet.tsx:259
 ,have no adjacent comment explaining intent – worth a spot-check each
 whether they render correctly or SUSPECTED – not verified at runtime
 .are copy-paste drift from before RTL_LABEL_ALIGN existed
- marginEnd / marginStart vs. logical (total: 5+2 7) marginRight / marginLeft •
- ;logical properties are already the majority pattern – (total: 9+10 19)
 the remaining 7 physical-margin uses are a small, bounded risk, not
 .systemic
- vs paddingStart / paddingEnd (10) (2) paddingRight / paddingLeft •
- .same shape, small residual risk
- appears 550 times vs. 'row-reverse' 100 times flexDirection: 'row' •
- expected under I18nManager.forceRTL(true) , since plain 'row' already

- .visually reverses; not itself a bug signal without per-case review
- `arrow-back / arrow-forward / chevron-back / chevron-forward` :Directional icons appear 55 times. Spot-checked `ScreenHeader.tsx:62` (shared header back button): explicitly comments `// In RTL, "back" is the right-pointing chevron and uses chevron-forward` (which points right) – this is correct and deliberate, not a bug. Given the volume (55 hits across ~15 files) a full per-instance audit wasn't performed; the one shared .highest-traffic instance (the reusable `ScreenHeader`) is correct
- Conclusion: RTL is not a fresh risk area here – it has visible scar tissue comments citing past bugs, a purpose-built `RTL_LABEL_ALIGN` token used (586x) and the remaining raw `'right'` /physical-margin usages are a small named tail rather than a systemic gap. Recommend a follow-up pass on the 7 .uncommented `textAlign:'right'` call sites only – not a full RTL rewrite
- Recommendation: spot-check the 7 uncommented `textAlign:'right'` sites .listed above; otherwise no action needed
- .Effort: S (just the 7 sites)

F-A11Y-8 – Hardcoded pixel heights on live-match panels and modals; screen-size handling otherwise reasonable

- Severity: P3
- Confidence: Confirmed
- Feature / Screen / Flow: Live match stoppages panel, sign-in, achievement celebration, availability radius map, profile hero card
- Evidence: 25 files use a hardcoded 3-4 digit pixel `height:` (excluding `LiveMatchScreen.tsx:848` :Notable non-decorative ones (`maxHeight / min` and `:868` (`height: 270`), mirrored in (`height: 300`) /fixed-height stoppages – (`height: 270`) `AdvancedLiveMatchScreen.tsx:2371` ;(`height: 140`) `SignInScreen.tsx:277` ;timeline panels `AvailabilityRadiusMap.tsx:275` ;(160/120) `AchievementCelebration.tsx:398/404` (`height: 116`) `ProfileHeroCard.tsx:196` ;(`height: 230`) `Dimensions.get('window')` :Screen-width-aware code is comparatively sparse , `OnboardingScreen` , `SplashScreen`) is used in only 9 files , `ChatView` , `InfoTip` , `RegistrationSuccessAnimation` , `OnboardingPreviews` and the newer , (`HtmlMessageView` , `ChatPitch` , `PlayerActionMenu` ,hook (which re-renders on rotation/split-screen `useWindowDimensions` – unlike a module-level `Dimensions.get` snapshot) is used in only 2 files meaning most of those 9 `Dimensions.get` call sites read the screen size once at import time and won't react to rotation or a foldable's screen .resize
- ,Problem: fixed-height content panels don't adapt to small devices/split-screen and the `Dimensions.get` -at-module-scope pattern is stale-on-resize by .construction
- User impact: on a small-screen device or in Android split-screen/foldable ;multi-window, some of these panels could clip or leave dead space no device/emulator available this) SUSPECTED – not verified at runtime .(session
- Recommendation: swap the 9 `Dimensions.get` call sites for where the value affects layout (not true for pure `useWindowDimensions()`

one-shot constants), and reconsider hardcoded panel heights in
.as percentage/ flex -based AdvancedLiveMatchScreen / LiveMatchScreen
.Effort: M •

Summary of hard numbers

- . accessibilityRole / accessibilityLabel Pressables: 448 total, 35 (7.8%) carry inline
- . Pressable clean migration to - 0 : TouchableWithoutFeedback / TouchableOpacity
- Confirmed sub-44px touch targets with no hitSlop : 2 components, 3 buttons
(.40x40, sendBtn @ 42x42 @ pendingReject , pendingApprove)
- used as a size workaround: 62 files (evidence the team already hitSlop
(.knows about the sizing problem and mostly compensates for it
/ success ;Contrast: textMuted passes AA by a thin margin (4.63:1)
and all 4 team colours fail AA (2.06:1~3.60:1) info / danger / warning
(.when used as text/icon-on-tint
(.occurrences across 7 files 9 : allowFontScaling={false}
- Reduced-motion coverage: 7/21 (33%) of animated components in
(.check useReducedMotion ; 14 do not src/components/anim
- RTL: RTL_LABEL_ALIGN used 586x (dominant, correct pattern) vs. 28 raw
(/of which are explicitly commented as deliberate 21) textAlign:'right'
known-trap, 7 uncommented and worth a spot-check); logical margin/padding
(. (Right / Left) used in 29 places vs. 9 physical (End / Start)
- Team colour identity: verified NOT colour-only anywhere checked (live
match, round history, chemistry, draft/teams, penalty shootout) – every
(.colour swatch is paired with a teamName() /label string
- Dark mode: fully implemented in source (darkPalette) but disabled app-wide •
not an a11y bug per se, but means any – (hardcoded , isDarkTheme = false)
(.OS-level “reduce brightness via dark mode” user preference is ignored

נספח D – ביקורת ומחלוקת

CROSSEXAM

Cross-Examination – Teamder 12-Agent Audit

Chair's note on scope: RELIABILITY.md was requested as one of the twelve reports but does not exist anywhere under this session's scratchpad
This cross-exam covers the eleven artifacts that (or elsewhere findings/)
,do exist – ARCH, DB, FIREBASE, SECURITY, INVARIANTS, CONSISTENCY, STATS
PRODUCT, DELETE, GAMING, COMPETITORS. Any reliability-flavored claim made in
passing by another agent (retry policy, cron timeouts, cold starts) is noted
below but was never checked against a dedicated reliability report – treat
(.those as single-sourced, not cross-validated

Methodology: read all eleven reports in full. Below, “agrees with” means two agents reached compatible conclusions from independently-cited evidence. “duplicates” means two agents are describing the same underlying defect.”

Duplicate findings – merge these before they inflate the count .1

“D-1. “App Check is decorative; nothing throttles the callable/write surface

.Agents: SECURITY (F-SEC-6) and FIREBASE (F-FIREBASE-7) – same finding, same evidence

`firebase.rules` Both cite the identical facts: `hasAppCheck()` defined in but never referenced in an `allow` clause; `ENFORCE_APP_CHECK=false` in the fix is “already tracked... pending App”; `functions/src/index.ts:103`; Attest.” SECURITY frames it as a security multiplier on F-SEC-1/2/4/5 FIREBASE frames it as a cost/quota exposure and is careful to narrow the blast radius (“not a privilege-escalation gap... unmetered invocation volume”). **Canonical finding: App Check plumbing exists but is fully unwired; treat SECURITY’s authorization framing and FIREBASE’s cost framing as two facets of one fix, not two fixes.** File as one item: enable App Check (client attestation) once App Attest is verified, single owner

“D-2. “Unbounded collection growth – nothing prunes errors/feedback/chat/marketing-log collections

.Agents: DB (F-DB-9) and FIREBASE (F-FIREBASE-10) – same underlying gap, different discovery method, complementary scope

DB found this via live `runAggregationQuery` counts (errors=161, `runDailyCleanup` feedback=351, notifications=1,010) and recommends extending FIREBASE found `notifications` the cron-sweep pattern already proven for it independently via `gcloud firestore fields ttls list` returning zero – native TTL policies, and covers a *different* slice of the same problem which DB’s `communityPlayerEvents` / `adLinks` / `inviteClicks` / `linkClicks` list doesn’t call out by name. **Canonical finding: no collection in this app has bounded retention except notifications (30-day cron) and** The two agents’ recommendations are not **(day cron-90) groupJoinRequests** redundant – DB’s cron-extension and FIREBASE’s native-TTL-field approach are two valid mechanisms for the same problem, and a single fix should pick one mechanism per collection rather than both agents’ fixes being applied independently to overlapping collections

“D-3. “commitRoundStats trusts the client’s declared outcome, with self-crediting and no volume cap

– Agents: SECURITY (F-SEC-4) and GAMING (F-GAME-1, F-GAME-2, F-GAME-3) .same mechanism, examined from opposite methodologies

SECURITY reads this as a data-integrity/trust gap in the canonical stats write path (“no score-consistency check, no rate limit... this is a pure trust-the-client gap”). GAMING reads the identical code as an adversarial player and derives three (`functions/src/index.ts:12753`) concrete exploits from it: farming personal/orphan games into global stats self-crediting goals/wins as admin in a real game with no (F-GAME-1) dispute path (F-GAME-2), and unlimited `addRetroGoal` calls with no cap or window (F-GAME-3). These are not independent findings – GAMING is SECURITY’s F-SEC-4 played out as three concrete attack scenarios. **This convergence is the single highest-confidence finding in the audit** (see §2) precisely because a security-audit methodology and an adversarial-player methodology, working from the same code independently, produced the same root cause

- Recommendation: treat F-SEC-4 + F-GAME-1/2/3 as ONE remediation item
groupId gate lifetime users/{uid}.stats writes on non-personal (a)
rate-limit commitRoundStats and (b) ,(closes F-GAME-1 specifically)
serverRateLimits per (uid, gameId) via the existing addRetroGoal
pattern (closes SEC-4's volume concern and F-GAME-3), (c) cap total rounds
per game. Do NOT implement these as four separate tickets from four
.separate reports – see the regression risk in §6.d for why (a) needs care

“D-4. “No observability into production failure – three angles on one blind spot

**,Agents: ARCH (F-ARCH-7), FIREBASE (F-FIREBASE-4), PRODUCT (§2, in prose
not a numbered finding) – thematically the same gap, mechanically different
.layers, not a literal duplicate but should be read as one story**

ARCH found the client-side symptom: 500 catch blocks, 27 fully empty, ~119
log-and-swallow, with no enforced convention and errors/ frequently
bypassed by bare console.* . FIREBASE found the server-side symptom: zero
Cloud Monitoring alert policies, zero log-based metrics – a 100%-failing
Cloud Function produces no proactive signal. PRODUCT independently names
the client-crash-SDK absence and the missing offline-detection layer
.as the single biggest risk to the “trust in the numbers” pitch (F-PM-6)

**None of these three duplicate each other's code evidence – they are three
different code paths – but they are the same root organizational gap
and (“nobody finds out when something breaks except a user complaining”)
should be triaged as one observability initiative, not three unrelated
.P1/P2s competing for the same budget**

“D-5. “The campaign system's cost ceiling and its own kill switch are both broken, independently

**Agents: DB (F-DB-11) and FIREBASE (F-FIREBASE-8) – not a duplicate, but
worth merging into one risk narrative because they describe the SAME
.subsystem's failure mode and its (non-functional) safety valve**

groups + users DB found that processCampaign reads the entire
collections per send, and at ~100× user scale (~61,400 users) this exceeds
silently truncating delivery to roughly a third of , MAX_RECIPIENTS=20000
the intended audience. FIREBASE found, independently (via
that feature_campaigns – the documented ,(firebase remoteconfig:get
kill-switch for the entire campaign system – was never published to the
live Remote Config template (frozen since 2026-06-04). **Read together: the
one subsystem in this audit with a documented future failure mode (silent
truncation) is also the one subsystem whose emergency stop doesn't
currently work.** Neither agent noticed the other's finding; flagging the
combination is this cross-exam's job. If DELETE's recommendation to sunset
the homegrown campaignService in favor of Joryio (see DELETE's MERGE
verdict) is executed, both DB-11 and FIREBASE-8 become moot for the
in-house engine – but only if Joryio's own campaign engine has an
equivalent, *working* kill switch, which is outside every agent's scope
.here and should be verified before treating the MERGE as a full fix

Agreements that strengthen a finding .2

S-1. Independent confirmation that commitRoundStats 's 11-a-side batch sizing is CORRECT, not a bug

,DB (F-DB-4) derives the arithmetic from the code's own comment ($2n^2+15n$
of the 500-op cap at $n=11$) and concludes “no action needed – the guard 81%

is correctly sized.” FIREBASE, working independently, lists the same fact in its “Areas checked and found fine” section: “explicitly caps side size ...at 11 players specifically to stay under Firestore’s 500-op batch limit good defensive engineering.” **Two agents, two different investigative angles (DB doing the arithmetic, FIREBASE doing a live-config sanity pass) reached the same “this is fine” verdict.** This is useful precisely because it means nobody should “fix” `MAX_SIDE=11` – and it sharpens why CONSISTENCY’s F-CONSISTENCY-3 recommendation is dangerous (§6.a): it would spend headroom that two independent agents just confirmed is already consumed 81%

S-2. `onGameRosterChanged`’s idempotency latches independently validated as retry-safe

FIREBASE (F-FIREBASE-2) recommends turning on `retry: true` for reasoning “it’s already idempotent via the `onGameRosterChanged` create-once latches, so retry is safe.” INVARIANTS, working from a completely different angle (auditing 12 business invariants against production data), independently confirms the exact precondition FIREBASE’s recommendation depends on: “Round-commit idempotency... the latch is created with `batch.create()` in the `committedRounds/{roundId}` exact same batch as every increment... a redelivery fails the whole batch atomically. Solid” (INVARIANTS, Part 3). **This is a genuine two-source – confirmation that FIREBASE-2’s fix is safe to ship, not just plausible** raise its confidence from “confirmed code + reasoned recommendation” to “confirmed code + independently-validated precondition”

S-3. `errors / feedback` flagged for cleanup by two independent lines of reasoning

DB and FIREBASE flag `errors / related collections` for unbounded growth `deleteMyAccount` SECURITY (F-SEC-8), reading (D-2 above, cost angle) end-to-end for a completely different reason (GDPR-style erasure – completeness), independently flags the same two collections “as containing “more PII than the profile doc itself – `errors / feedback` and recommends them as the *first* priority for any account-deletion cleanup pass. **Three agents, three different lenses (cost, retention policy, privacy law), converge on the same two collections as the ones** `feedback / errors` **needing a real cleanup pass**, which should move retention to the top of any prioritized backlog over the other unbounded collections named only once (DB’s `chat messages`, FIREBASE’s `( adLinks / linkClicks`

S-4. Atomic stat core independently verified sound by two agents using live data

CONSISTENCY measured, against real production docs, that `gamePlayerStats.goals / communityPlayerStats.goals / users.stats.goals` match exactly for a sampled player (16/16/16), and the same for wins and ,rounds. INVARIANTS, working from the code side (not the data side) independently confirms the same three-way write is atomic-batch and latch-guarded, and separately confirms the pair/chemistry two-pipeline design writes disjoint fields (no double-count). **This is the strongest possible form of agreement in this audit: one agent verified the invariant holds in live data, a second verified the code path that should produce that invariant is structurally sound – code and data agree.** This should raise confidence that the “atomic core” is genuinely solid, sharpening the contrast with the *documented, measured* drift the same two agents found , `eveningStandings / roundSummaries / roundHistory`) one layer up

Contradictions .3

C-1. Cross-community filler matching: DELETE says kill it, COMPETITORS says invest more in exactly this

DELETE.md verdict: DELETE. Live count: `fillerInterests` collection-group documents, ever, against 26/79 games that opted in. DELETE's own text 4 = already hedges: "Either the discovery-surfacing extension... is the actual ...fix needed before this can be judged fairly, or the feature should be cut ...recommend killing the current dormant version

-COMPETITORS §2.3/§4 verdict: this is the single most product-market-fit aligned direction in the whole strategic analysis – it cites the same memory note and recommends it as one of `project_fillers_in_feed_plan.md` only two "lowest-risk, highest-leverage moves" the entire competitor .research supports

Adjudication: these are not actually in conflict once sequenced, but COMPETITORS did not have access to (or did not check) the live usage data DELETE pulled, so its recommendation reads more confident than the evidence supports. DELETE's live-data finding (4 real uses ever) is the stronger evidence and should govern the *current* build: the opt-in surfaces that produced 4 uses in (push type, admin picker modal, per-game toggle) .the app's entire history should be retired or hidden now, not maintained -COMPETITORS' strategic case is about a *different, unbuilt* surface (feed integrated discovery) that neither report can evaluate because it doesn't exist yet. **Correct synthesis: mothball the current opt-in filler UI do not read that as a ;(DELETE is right that it's dead weight today) verdict against building the feed-integrated version COMPETITORS recommends (COMPETITORS is right about direction) – they are testing two .different hypotheses, and only one of them has been tried**

C-2. Severity of the "no test coverage on functions/src" fact – filed as architecture debt vs. product risk

Not a factual contradiction (both agents cite the same ground truth: zero tests on the 14K-line Cloud Functions surface), but a *framing* contradiction worth resolving for prioritization. ARCH (F-ARCH-3) files this primarily as a maintainability/monolith-structure problem, severity P1, with the fix being "split the file." PRODUCT (F-PM-8) files the same underlying fact as a product-existential risk ("this is the single highest-leverage testing investment in the codebase given what's actually at stake product-wise"), also P1, with the fix being "add tests to the :finish-transition/commit-stats/rollup chain specifically." **Adjudication PRODUCT's framing is the more actionable one and should govern sequencing** – CONSISTENCY's own measured production evidence (`roundSummaries / roundHistory` F-CONSISTENCY-1/2, real divergence in) ,is a live demonstration of exactly the risk PRODUCT is warning about while nothing in ARCH's file-splitting rationale is currently manifesting as a user-facing bug. A file split (ARCH-3) is good hygiene but is not where the highest-value tests would go; test coverage should target the chain `rollUpClubPairs / onGameRosterChanged / commitRoundStats` specifically, as PRODUCT argues and as CONSISTENCY's own findings .independently justify, *before or instead of* a mechanical module split

V-1. GAMING's F-GAME-1 (Po) is over-rated relative to the audit's own Po bar

GAMING rates "quick games farm global profile stats" as Po – the audit's top severity tier. Compare the other two Pos in this audit: FIREBASE-1 (no backups/delete protection – an unconditional, irreversible, whole-platform catastrophic-loss risk requiring no attacker at all) and INVARIANTS F-INV-1 (deleting a played game leaves stats behind – **confirmed against live production data**, not hypothetical, and reachable by an ordinary admin with no malicious intent via a normal menu item). GAMING's F-GAME-1 requires a deliberate, self-interested actor to run repeated fake solo games – it is real and cheap to execute, but it is (a) not yet observed in production, (b) requires intent, and (c) is substantially the same mechanism SECURITY already covers at P1 (F-SEC-4). **Recommendation downgrade F-GAME-1 to P1**, consistent with how SECURITY calibrated the identical underlying mechanism, and reserve Po in this audit for unconditional/already-manifested failure modes (FIREBASE-1, INVARIANTS-1)

- This does not diminish the finding's value – it's real and cheap to fix
- it just shouldn't sit in the same tier as "the database has no backups"

V-2. DB's F-DB-3 (listener fan-out) carries P1 but its own text argues P2/P3

DB's F-DB-3 (full-document listener re-billing on every write) is tagged P1, but the finding's own "User impact" and "Expected benefit" fields say – none directly... N/A at current scale... this is a scale-driven finding" see F-DB-13" and "Effort: – (documented for future reference; no immediate action warranted)." Every other DB finding with that same self-description is tagged P2/P3. **This looks like an internal calibration slip within DB's own report, not a cross-agent dispute** – flagging it anyway because a reader skimming severities would reasonably expect F-DB-3 to demand nearer-term action than F-DB-5 (P2 acceptable as-is"), when DB's own prose says the opposite. Recommend re-tagging F-DB-3 to P2, consistent with its sibling findings, and letting F-DB-13 (the scale roadmap, correctly P1 because it *does* inform near-term prioritization) carry the forward-looking weight instead

V-3. CONSISTENCY's F-CONSISTENCY-1/2 (P1 "serious") are correctly calibrated, not disputed – noted for contrast

Given V-1 and V-2 above both argue for a *downgrade*, it's worth explicitly confirming one severity that the cross-exam does NOT dispute: CONSISTENCY's two P1s are measured against live production data (a specific game, a specific player, specific numbers that disagree across three collections narrower in blast radius than INVARIANTS' Po (contained to display-layer artifacts, not the underlying stat-of-record), and appropriately one notch below it. This is good calibration and is cited here only so the severity critiques above (V-1, V-2) don't read as "every severity in this audit is wrong" – most are not

Findings missing crucial information .5

SECURITY's rule-shadowing findings (F-SEC-1, F-SEC-2, F-SEC-3) are the one domain in this audit not cross-checked against live data. Four other agents in this exact audit (DB, CONSISTENCY, INVARIANTS, DELETE)

used read-only `documents:runQuery / runAggregationQuery` REST calls against the real production project to turn a code-level suspicion into a measured fact. SECURITY's claims – "any signed-in user can read any club's pair-chemistry doc," "any signed-in user can list the entire collection" – are exactly the kind of claim that methodology `users` could confirm in minutes (an authenticated-as-non-member read attempt .against a known `communityPairStats` doc, or a bulk `users` list call). None of the four SEC findings currently cite a live verification. This doesn't make them wrong (the rules text is unambiguous), but it's the one gap in an otherwise unusually well-corroborated audit, and closing + it would convert "Confirmed (rule text)" to the same "Confirmed .measured in production" bar the rest of the audit set

- **PRODUCT's F-PM-6 (no offline handling for live-match goal entry) has an unexploited verification path sitting inside this same audit.** Both `errors` SECURITY and FIREBASE separately read/counted the production collection (161 documents) but neither content-analyzed those 161 docs for this specific failure signature (a goal/timer write that never landed). PRODUCT's concern is plausible but currently "SUSPECTED – not verified at runtime" for lack of exactly this check. Actionable next step: `grep` the 161 `errors` docs for `operation` values touching goal-entry/timer paths before treating F-PM-6 as/ `commitRoundStats` .confirmed-urgent versus theoretical

- **GAMING's F-GAME-8 (club level cheaply farmed via empty auto-finished games) is explicitly self-flagged as unverified** ("SUSPECTED – not verified at runtime; traced the metric source but not whether a zero/near-zero-roster game auto-finishes and still counts"). This is a one-query check (does `totalFinished` include games with `players.length <= 1` ?) that would convert a P3-suspected finding into either a non-issue or a real, cheap-to-fix P2. Worth doing before either .prioritizing or dismissing it

- **SECURITY's F-SEC-8 (account-deletion PII gaps) has no live count** unlike every other cost/scope claim in this audit. DB, CONSISTENCY, and DELETE all quantify their claims with real document counts; F-SEC-8 lists affected collections by name but doesn't state how many deleted 10~ users' data is actually still linked in each. Given the audit's own demonstrated methodology (read-only REST queries against a live project .worked for every other data-scope claim), a quick count of, e.g docs whose `userId` no longer resolves to a live `users` doc `feedback` .would turn this from a plausible L-effort backlog item into a scoped .prioritizable one

- **CONSISTENCY's own flagged anomaly (gamePlayerStats absent for game despite a populated `roundHistory`) is explicitly `fusBhEBGkC37D49LygwP` left open** and should be resolved before F-INV-1 (which cites the same game as its production proof) is treated as fully understood – if the anomaly turns out to mean an older `commitRoundStats` code path could write `roundHistory` without the full stat fan-out, that changes the "blast-radius estimate for F-INV-1's "deleted game leaves stats behind claim (fewer stats may have been orphaned than assumed for that specific example, though the structural claim – no reversal code exists anywhere .stands regardless –

R-1 (flagship) – CONSISTENCY-3's fix for club-chemistry drift would very likely blow DB-4's already-81%-full batch

CONSISTENCY F-CONSISTENCY-3 recommends writing the club-scoped `communityPairStats` etc. fields to/ `winsB / winsA / sameTeam / against` *live, inside the same* `commitRoundStats` batch that already writes the identical fields to the global `pairStats` collection – “one extra scoped doc per pair – bounded by the same `MAX_SIDE=11` budget- `groupId` already reasoned about at line 12843.” **This claim does not survive DB-4's own arithmetic.** DB-4 states the batch is already at 407 of 500 ops (81%) at legal 11-a-side, and that the *pair-related* ops alone already account $(C(|A|, 2) + C(|B|, 2))$ against-pairs $|A| \times |B|$ + same-team) for roughly 230-240 of those 407 ops – over half the batch. Writing a = **second, club-scoped document per pair** (a distinct doc reference a distinct batch op, since `communityPairStats/{groupId}__{pairKey}` is a different document from `pairStats/{pairKey}`) roughly doubles the pair-related op count, pushing the total to somewhere around 600-650 ops .at $n=11$ – comfortably over Firestore's hard 500-op transaction/batch limit Per DB-4's own words, a batch that overflows “fails atomically and a retry fails identically – that mini-game's stats are permanently unrecoverable from that path.” **This is exactly the collision this cross-exam was asked to look for:** a correctness-motivated recommendation (fix the chemistry drift CONSISTENCY measured in production) would, if implemented as literally proposed, convert a display-layer bug into a hard batch failure for every 11-a-side mini-game – a strictly worse outcome. Neither CONSISTENCY nor DB caught this collision because CONSISTENCY didn't re-run DB's arithmetic with the new writes added, and DB never saw CONSISTENCY's recommendation. **Correct fix: implement CONSISTENCY-3's club-scoped write as a separate, best-effort follow-up (a small dedicated batch after the main commit, or fold it into the existing `rollUpClubPairs` end-of-evening path rather than the per-round hot path) – never inside the same batch as .the per-round pair writes at anything near 11-a-side**

R-2 – INVARIANTS' fix for orphaned game-delete data needs FIREBASE-1's backup fix to land first, not after

INVARIANTS F-INV-1 recommends, as its second remedy, “a callable that reverses the exact increments... and hard-deletes subcollections before .removing the game doc” – a new bulk-delete-and-reverse-stats code path **FIREBASE F-FIREBASE-1** independently found the production Firestore database has zero backup schedules, no point-in-time recovery, and delete protection disabled – and specifically calls out that the app already runs several bulk/rare-path operations against this exact database with no “rehearsal environment,” naming `promoteOrphanToGroup` and .as existing examples of the risk category `backfillGroupCreatorIdsOnce` **,A new hard-delete-plus-stat-reversal callable, built to satisfy F-INV-1 ,is precisely one more instance of that same risk category** – a bulk irreversible operation against a database with no recovery path, being added *while the underlying platform gap FIREBASE-1 identified is still open*. Sequencing matters here: FIREBASE-1's daily backup schedule should ship before F-INV-1's reversal (Effort: S, per its own estimate) callable ships, not after – otherwise the fix for “deleted games leave

orphaned stats” is a new script that could, on a bug, delete/miscredit .stats platform-wide with no way back

R-3 – F-GAME-1’s blanket “gate stats on non-personal groupId” fix risks suppressing the exact cold-start wow-moment PRODUCT documents as critical

GAMING F-GAME-1 recommends gating all `users/{uid}.stats` writes in on the game’s `groupId` belonging to a non-personal `commitRoundStats` group, closing the self-farming exploit. **PRODUCT §4C** independently ...documents quick/orphan games as “actually the *fastest* path to playing no club creation, no admin approval, no waiting” and explicitly frames this as the right on-ramp for a cold, club-less user – the exact persona the product most wants to convert (§3: “a fundamentally slow-to-value product for the persona the app most wants to convert... the fix has to be in what happens before the first evening”). **If F-GAME-1’s fix is implemented exactly as literally stated – a blanket exclusion of all personal-group games from lifetime stats – a brand-new user’s very first quick game with real friends (not a self-farm) would never appear on their own profile .card, forever, even after they later convert that group into a real club**

That directly undercuts the wow-moment PRODUCT identifies (§3, the shareable evening-summary card) for exactly the users PRODUCT says the company most needs to retain. GAMING’s underlying threat is real and narrower than the blanket fix implies: the actual exploit is a *so/o* self-farm (one real account, guest/self-only opponents, repeated mini-games), not “any game played without a club.” Recommend a narrower condition than “isPersonal” – e.g., require a minimum count of distinct ,*other real* participants across the game before crediting lifetime stats or credit game-scoped/evening-scoped displays (which PRODUCT’s wow moment actually reads from) unconditionally while only gating the cross-club – *comparative* surfaces (leaderboards, achievements-derived counters) rather than suppressing the number PRODUCT says is the product’s best .retention artifact for its hardest-to-convert user

R-4 – STATS’ proposed “hot/cold streak” metric (B8) would sit directly on top of two GAMING exploits and amplify both

STATS Part B, item B8 proposes a performance-streak stat (“3 games , `roundHistory` / `gamePlayerStats` running with a goal or a win”) built from .framed as free/low-cost because the underlying fields already exist

GAMING’s F-GAME-5 (get subbed off a losing side before the round commits – “the round never happened” for that player, no partial-loss recorded) and **F-GAME-2** (an admin can self-credit wins/goals in their ,own real game with no dispute mechanism) both describe existing zero-cost ways to manufacture a clean per-round record. Today, the payoff for exploiting F-GAME-5/F-GAME-2 is bounded to a single evening’s ratio (F-GAME-6’s evening score) or a per-pair “deadly duo” fun `wins/gp` fact (F-GAME-7) – both already flagged by GAMING as “should never be shown as competitive/leaderboard truth.” **A visible, cross-evening streak counter is a strictly stronger incentive to exploit F-GAME-5/F-GAME-2 than anything that currently exists** – it converts “protect one evening’s win rate” into “protect an accumulating public streak,” raising the payoff for ,timing your substitutions and self-crediting goals every single week compounding rather than resetting. **If B8 is built, it must not ship before F-GAME-5’s round-start roster snapshot fix (STATS’ own proposal depends on `gamePlayerStats` / `roundHistory` rows whose integrity GAMING ,has already shown is manipulable) – otherwise STATS’ “highest-leverage**

lowest-cost” new metric is, concretely, a new leaderboard built on data GAMING has already demonstrated can be freely gamed, with a bigger prize for gaming it than exists today. B12 (“most improved,” a trailing-window delta) carries a milder version of the same risk (sandbagging an early baseline window) and should get the same caution, though its incentive is weaker since it isn’t a public streak

R-5 – SEC-2’s club-isolation fix needs to be scoped against PRODUCT’s documented cross-club discovery surfaces, which SEC-2 itself flagged but couldn’t verify

SECURITY F-SEC-2 recommends adding `isGroupMember()` gating to `,reads rounds / communityStats / gamePlayerStats / communityPlayerStats` but its own text already hedges: “verify no legitimate cross-club read path e.g. discovery/showcase – depends on the current open behavior before – tightening.” **PRODUCT independently confirms such surfaces exist and are** `GamesListScreen`) **real**, not hypothetical: the discovery feed “cross-community filler matching, and a “nearby-clubs teaser (”פיתוחים”) are all documented as live, deliberately cross-club-visible features ,PRODUCT §9: “the discovery feed... guest browsing, quick/orphan games) cross-community filler matching, nearby-clubs teaser – built for someone with no club at all”), and PRODUCT’s closing argument even concludes the product’s *primary* axis should arguably be this discovery-first path over club-management. **This corroborates, with independent evidence, the exact caveat SEC-2 raised but could not confirm on its own** – before shipping `/ gamePlayerStats / communityPlayerStats` SEC-2’s fix, whichever of actually backs any discovery/showcase-facing `rounds / communityStats` screen needs to be identified and excluded from the tightened rule, or the security fix will break a feature PRODUCT independently argues is the .app’s best differentiator

R-6 – Bounding errors collection growth (DB-9/FIREBASE-10) without SECURITY-5’s write-side fix bounds retention but not abuse

DB (F-DB-9) and FIREBASE (F-FIREBASE-10) both recommend adding TTL cleanup to the `errors` collection (among others). **SECURITY (F-SEC-5)** independently found that `errors/{fp}` accepts unlimited-cardinality writes from anyone – signed in or not – with no rate limit, by design (pre-auth failures need to be capturable). **A TTL policy alone does not fix the abuse case SEC-5 describes** – it bounds how long garbage survives, not how fast it can be written; a scripted flood using the public Firebase web `/config` could write faster than any daily TTL sweep removes. If DB-9 FIREBASE-10’s recommendation ships without SEC-5’s companion fix (App Check once available, or a scheduled document-count cap), the collection is bounded in *retention* but still unbounded in *write-cost/day*, and .add TTL to errors” would read as complete when it isn’t”

Summary for the reader in a hurry

The audit’s single highest-confidence finding is **D-3: SECURITY** and • GAMING independently arrived at the same root cause (trust the client with no volume cap `addRetroGoal / commitRoundStats`) from opposite methodologies. Fix it as one item, but see R-3 before .implementing GAMING’s specific proposed guard

- The audit's most important internal collision is **R-1**: implementing CONSISTENCY's recommended fix for measured production data drift, exactly as written, would very likely overflow the batch DB independently proved is already 81% full – turning a display bug into stat-loss. This is the .brief's own worked example, confirmed to actually apply here
- Two severities look miscalibrated on inspection: **V-1** (GAMING's Po for quick-game farming should be P1, consistent with SECURITY's rating of the same mechanism) and **V-2** (DB's own P1 on listener fan-out contradicts .(its own "no action warranted" text
- The one domain that skipped this audit's dominant methodology /read-only live Firestore verification, used by DB/CONSISTENCY) INVARIANTS/DELETE) is SECURITY – its rule-shadowing claims are correct on .the text but unverified against the live deployed rules and data
- was never produced; treat any reliability-adjacent claim RELIABILITY.md .above (retry policy, cron timeouts) as single-sourced

REDTEAM

RED TEAM – attacking the Teamder audit

Method: read all 11 finding files + BRIEF.md, then independently re-queried **live production Firestore** (read-only, same `gcLOUD /REST` access the original audit used) to check specific "confirmed in production" claims rather than trusting the citation. Two direct verifications below overturn or materially change a finding's severity. Everything else is argued from .the text as written

Conclusions without evidence (runtime never observed) .1

- **SUSPECTED** – not verified at runtime The audit is honest about tagging that discipline is real and should be credited. The attack isn't "they didn't say SUSPECTED," it's that **severity sometimes doesn't downgrade :even when confidence does**
- **F-PM-6** (no offline handling, P1) never checks whether Firestore's client SDK offline persistence/write-queue is enabled. If it is (the default in most RN Firestore setups), a goal tap on a patchy pitch-side connection queues locally and syncs on reconnect – the exact failure mode .the finding describes may already be mitigated by the SDK, not absent
- The finding never rules this out; it infers "no NetInfo layer" ⇒ "silent data loss" without checking the one mechanism that would make that inference wrong. This should have been Confidence: Suspected, not .Confirmed-by-absence, and probably P2 pending that check
- **F-ARCH-6** (optimistic-splice race) cites one already-fixed bug as proof the *pattern* is dangerous, then recommends a structural fix for a problem that – by the audit's own account – was already caught and patched once. That's evidence the existing "fix per occurrence" approach .is working, not evidence it's failing
- **F-DB-6** (N+1 read pattern) explicitly states there is **no cost difference** – Firestore bills identically either way – and estimates a

latency difference that only matters “once rosters regularly hit which no club in production does today (largest club: 40 “,30-44 members, not attendees; actual round sizes are far smaller). This is a correct, honest finding but is functionally a style nit dressed as a P2 .performance finding

STATS “user impact” lines – nearly every one of the 7 STATS findings • reads “SUSPECTED – a player would find this arbitrary/unfair.” None of these were shown to an actual player. That’s fine as hypothesis, but the report presents them as P2 “meaningful” findings on the strength of the auditor’s own intuition about how a Hebrew pickup-football player reads a evening score – a genuinely different audience than a data 0-10 .scientist’s prior

Findings that are artefacts of test data – VERIFIED, one is serious .2

.I queried production directly rather than trusting the group IDs cited

F-CONSISTENCY-1/2/3 and the F-DB findings that cite groupId
this club is REAL, not test data. Its name is : `0cdCmLhka0dTsqA2AF0a`

3 admins, created ,(“Good People Football”) `כדורגל אנשים טובים`
and it’s the same club F-DB-7 independently ,(real timestamp) ...-2026-01
cites by its own code-comment description (“the real club has 57
mini-games behind 241 pairs”). These findings should be trusted at full
weight – they are not contaminated by test data, and the audit’s framing
.of them as real production discrepancies is correct

,F-INV-1 (the Po “deleting a played game leaves stats behind forever
confirmed in production”) is a different story. I fetched the actual

doc behind the headline evidence `gameDeletions`
Its `gameTitle` field is literally `( fusBhEBGkC37D49LygwP )`
a single-player game – not a real) `rosterCount: 1` , **“QA Test Club”**
evening), `deletedByApprox: true` (the audit’s own data doesn’t even know
for certain who deleted it), and the `createdBy / deletedBy uid`

does not resolve to any user document in (loaEHV7Z3JdMBSMZ4veRMCvGTVA2)
production today – the account is gone. This is not a club member’s real
evening that got silently corrupted; it is a one-person throwaway test
game, almost certainly created by the owner himself while testing the
.delete flow, whose creator account no longer exists

This matters a lot for severity. The *structural* bug F-INV-1 describes is
real and worth fixing (no rules gate on delete-after-committed-rounds, no
reversal path) – but the finding is written and rated as if it caught a
live production incident harming a real user (“club leaderboards ... can
... permanently include contributions from a game that no longer exists
with no way for anyone to notice or correct it”). The only evidence
:offered for that harm is a QA fixture. **Re-rate F-INV-1 from P0 to P2**
fix it because it’s a real gap in the data model, not because a real
.evening was corrupted – none was, as far as the actual evidence shows

“Also worth noting: a genuine `groups` doc named exactly **“QA Test Club**
exists in production (id `6zotsP1u5Wa18hIysQif`) – I checked it directly
and it has **zero games**, so it isn’t the source of any other finding in
this report. The contamination is narrower than “one whole club is fake
data” – it’s specifically this one deleted single-player fixture, and it
happens to be the sole evidentiary basis of the report’s highest-severity
.finding

Several DB/FIREBASE findings are internally honest about this (good) but
:the severity label doesn't always follow the text

- **F-DB-3** (listener fan-out) is P1, but its own "Expected benefit" line says *"N/A at current scale ... no immediate action warranted."* A P1 ,with "no action warranted" is a contradiction – this is a scale note .P3 at most

- **F-DB-4** (`commitRoundStats` at 81% of the 500-op cap at 11-a-side) is P1, but its own recommendation is *"no change needed – the guard is correctly sized."* This is confirmation the safety margin is fine, not a defect – should not carry a P1 badge at all; it reads as a P1 purely because the number "81%" looks alarming next to a hard cap, but the audit's own analysis shows there's no live risk at any format this app .actually supports

- **F-DB-5** (evening-finish read scales with club size) – P2, explicitly .acceptable as-is." Same pattern, smaller degree"

- **F-DB-13** (10x/100x/1000x scale roadmap) is marked P1 "informs prioritization." At 614 real users for a Hebrew pickup-football app run by one person, 1000x (614,000 users) is not a plan, it's a thought experiment. Spending report real-estate ranking failure modes at a scale this app may never reach reads as padding a report that is otherwise disciplined about scale everywhere else (F-ARCH-8 and F-DB-1 both explicitly say "not urgent at this scale, noting for completeness" – the .right tone F-DB-13 should have used too, at P3/P4

A genuine Po-for-backups (F-FIREBASE-1) sitting in the same report next to a P1-for-what-happens-at-614,000-users is exactly the miscalibration the brief asked to check for. One of these is "your entire dataset can vanish tomorrow with a fat-fingered script"; the other is "if this app becomes 1000x bigger than WhatsApp-replacement-for-a-Tuesday-league ever needs to .be." They should not read as comparably urgent, and right now they do

Recommendations that add complexity nobody can afford .4

This is a 120k-LOC, 55-screen app maintained by one person. Tallying just ,the Effort:S/M items across the eight technical files (ARCH, DB, FIREBASE GAMING, INVARIANTS, SECURITY, STATS, CONSISTENCY) gives roughly **35-40 discrete recommended changes**, several of which (F-ARCH-2's five parity tests, F-SEC-2's four-collection rules rewrite, F-STATS Part B's twelve new metrics) are themselves multi-item baskets. Taken individually every one is defensible. Taken together, this is not a punch list, it's a multi-month roadmap handed to someone who also has to keep shipping features and running QA solo. The report doesn't sequence or budget this anywhere – no .file says "here are the 5 you can realistically do this month :Specific complexity-additions worth challenging directly

- **F-ARCH-3 + F-ARCH-4** (split the 14k-line `index.ts` and the 8k-line into modules) – both L-effort, both justified purely on `gameService.ts` smaller review surfaces / room for tests," with **zero cited bug** that" file size itself caused. Every actual bug found elsewhere in this audit ,traces to a *logic* gap (F-INV-1, F-ARCH-1/2's drift, F-CONSISTENCY-1/2) not to the file being long. A solo dev who already navigates this file daily gets no correctness benefit from this refactor, only churn risk a mechanical split of 22,000 lines is exactly the kind of change that)

introduces a stray import-order or circular-dependency bug the report
(itself, in F-ARCH-10, notes there's no tooling to catch

- **F-ARCH-2's** recommendation to build parity-test infrastructure for five more hand-duplicated rules is reasonable in isolation, but paired with F-ARCH-1 (the sixth) and the existing four already-protected formulas, this is now asking for **ten** formula/parity-test pairs maintained by one person, for an app where exactly **one** of those ten has ever actually drifted, and that one was already (clean sheets) caught and fixed by hand. Prioritize the brand-guard case (cross-repo security-adjacent) and drop the rest to "fix opportunistically when touched," not "add five test suites now

- **F-STATS Part B's** twelve new metrics are all cheap individually but twelve simultaneous new UI surfaces is still (no schema change) twelve things to design, translate to Hebrew RTL (a standing, explicitly flagged cost per project memory), test, and maintain. The report doesn't rank these against each other or say "ship 2, not 12

UX-breaks-consistency and admin-vs-player conflicts .5

- **F-GAME-2's** recommendation – require a second admin's acknowledgement before a self-credited goal counts – directly fights the app's own documented reality: most clubs are small (mean 5.2 members per group per F-DB-13; the largest non-personal club is 40 members with 3 admins) and a huge share of games are personal/orphan quick games with exactly one admin (F-GAME-1's whole premise depends on this). A "second admin requirement is either impossible (no second admin exists) or adds a live-match-blocking approval step to the single highest-friction moment in the whole loop (entering a goal, mid-match, on a phone, on a pitch) This is a recommendation that would materially hurt the experience for the *majority* case to guard against a minority-case trust problem the same finding admits has never been reported as an actual complaint – Keep the observation, drop the "require a second admin" prescription a passive "goals entered by [admin]" badge (the finding's own fallback option) is the right-sized fix

- **F-GAME-4's** suggestion to treat "all-guest opposing side" rounds as :informational, not counted toward win-rate" risks the opposite problem" short-numbers nights are described elsewhere in this same audit set as *completely normal* pickup-football reality, not (F-DB-13, PRODUCT §1) an edge case. Silently downgrading those rounds' standing-relevance would surprise organisers who ran a perfectly normal short evening and now see their table looking different from what they expected, with no comms plan proposed for the change

- **STATS Part C's C2** (an admin per-evening "was this competitive?" tag) already carries its own internal dissent from the "sceptical statistician" persona in the same file, correctly flagging that an unset flag silently defaults to "counts," so the feature is a net-zero or net-negative until admin compliance is near-100% – unverifiable. This is good self-awareness inside STATS.md; the report's own summary section should have surfaced that dissent as the actual recommendation (don't build it) rather than leaving it buried as a footnote under "Accept

Stats recommendations that create new gaming incentives – the audit missed its own .6 cross-check

This is the sharpest structural gap in the whole report: **STATS.md** and **GAMING.md** never reference each other, despite both reading the exact `, gamePlayerStats , roundHistory )` same underlying tables and despite GAMING.md establishing, in detail, that `( communityPairStats` every one of those tables is **entirely admin-entered with no independent verification** (F-GAME-2, F-GAME-3: “faking your own stats inside your own .(“real game is still possible

Given that premise, several STATS Part B proposals actively make the :gaming surface worse, not better

B1 (clutch/opener rate, from goal minute) and **B11 (shootout clutch rating, from kick order)** both add a new axis of *narrative* reward – “you’re a closer,” “ice in your veins” – computed from admin-entered minute/order data that F-GAME-2/3 already show can be fabricated with zero friction. Today a self-crediting admin can inflate goals/wins; after B1/B11 ship, the same admin can also manufacture a clutch” reputation by simply choosing favorable minute values when” entering their own goals – a strictly richer fabrication surface than exists today, shipped as a “free” feature because the fields already .exist

B8 (performance hot/cold streak, consecutive mini-games with a goal or win) directly rewards the exact behavior F-GAME-5 documents as free and already exploitable – get subbed off before a losing round commits so it never counts against you. A streak feature makes that dodge *more* valuable (protects a visible, shareable streak number, not just a background win%), which is the opposite of what a stats feature should .do relative to a known integrity gap in the same data

B6 (“true head-to-head win rate”) is explicitly framed by STATS as • fixing F-STATS-3’s *sample-size* problem – and it does, correctly. But it does nothing about, and doesn’t mention, the *fabrication* problem :GAMING documents for the same `communityPairStats` fields (F-GAME-7 any admin entering goal data controls exactly which two accounts get” credited”). Presenting a rate instead of a count makes a fabricated number look *more* credible, not less – “8-2 (80%)” reads as more .rigorous than “8 wins,” even though both are equally unverified

•None of this means Part B should be scrapped – it means the two audits findings should have been read together before any of B1/B8/B11 specifically get built, and the report as currently structured doesn’t do .that synthesis for the reader

Format-dependence check (5v5/2-team assumptions) .7

Most findings hold up across formats – F-DB-4’s `MAX_SIDE=11` reasoning is :explicitly format-aware and correctly scoped. Two are worth flagging

•The `commitRoundStats` model (and every GAMING finding built on it F-GAME-2 through F-GAME-5) assumes exactly two sides (`A / B`) per committed round. F-DB-2 independently establishes the app already supports **4-team drafts** (44 players, 4×11). Nothing in GAMING.md checks whether a 4-team club’s *rotation* (which two of four teams play a given mini-game, who sits out) creates a *larger* version of F-GAME-5’s

get subbed off before commit” dodge – with 4 teams there are more”
,rotation slots to hide in between rounds, not fewer. This isn’t wrong
”it’s just unexamined; the report should say “not yet checked at 4 teams
rather than implicitly generalizing 2-team findings to every format the
.app supports

STATS’ B4 (“lucky-colour effect” from bib colour) actually gets *better* •
– at more teams (more colour variety, more signal), so this one is fine
.noted so as not to imply every stats idea has a format problem

Severity inflation – re-rated .8

Why	Red-team severity	Report severity	Finding
Headline “production confirmed” evidence is a 1-roster QA fixture with a since-deleted creator account, not a real evening (see §2). Structural gap is real; the Po framing of active user harm is not supported by the .evidence cited	P2	P0	F-INV-1
Real and cheap to fix, but the harm is a player inflating their own vanity numbers on their own profile in a friends-and-family-scale app with no money and no leaderboard stakes beyond social bragging – genuinely .different stakes than F-FIREBASE-1’s “entire dataset gone” or a cross-club privacy leak	P1	P0	F-GAME-1
Each one’s own text says “no action warranted at current scale” or “correctly sized, no change needed.” A .P1 that resolves to “do nothing” is a scale note, not a priority	P3/P4	P1	F-DB-3, F-DB-4, F-DB-13
Under-rated, not over-rated: a live, zero-skill, zero-auth-friction cross-club privacy leak that’s exploitable <i>today</i> by any signed-in user, with a one-line fix. This is more urgent than several of the report’s Po/P1s and .arguably belongs at the top of the list, not mid-pack in SECURITY.md	Consider Po	P1	F-SEC-1
A product-strategy call (“freeze investment, consider killing it”) made from zero usage telemetry, in a technical audit whose stated method is static analysis only. The maintenance-cost observation is fair; “kill it” .is a business decision the audit isn’t positioned to make	P4 / reframe	P2	F-PM-1 (kill Wear OS)

What a genuine Po looks like for this app, based on this reading: no
,disaster-recovery path for the entire dataset (F-FIREBASE-1) and a live
trivially exploitable cross-club data leak that’s already shipped
Both are cheap to fix, both are catastrophic-or-embarrassing if .(F-SEC-1)
they bite, and neither requires any assumption about growth, adversarial
users, or unverified runtime behavior – they’re true today, confirmed by
.config/rules text, not by extrapolation

Five findings that survive the attack and matter most

- F-FIREBASE-1** – no Firestore backups, no delete protection. Genuinely .1
.catastrophic if wrong, genuinely S-effort to fix. The one uncontested Po
- F-SEC-1** – duplicate `communityPairStats` rules block silently .2
disables club isolation. Live, trivial, one-line fix, verified real (not
test data – this is a rules-file logic bug, unrelated to any specific
.club’s data
- F-CONSISTENCY-1 / F-CONSISTENCY-2** – retro-goal forking + missing .3
,roundHistory silently orphaning club chemistry. Verified against a real
active club (כדורגל אנשים טובים), not test data – this is the
trust in the numbers” problem the PRODUCT audit correctly identifies as”
.the whole product’s pitch, and it’s real
- F-FIREBASE-2 / F-FIREBASE-3** – trigger retry off + task-claim-before .4
send bug on the finish-evening path. Config-level facts, not runtime

speculation, S-effort each, and they sit on the exact code path that produces the emotional payoff of the app (evening summary / standings)

F-PM-7 – orphan→club promotion has no in-app CTA. Cheapest .5 highest-leverage item in the whole report: one button, on a screen that already exists, wired to a callable that already works, sitting exactly at the point PRODUCT.md correctly identifies as the product's best and .fastest on-ramp dead-ending

Five findings to drop entirely

- F-INV-1's Po framing** – keep the structural fix (a real gap), delete .1 the “confirmed in production, real user harm” framing; it's sourced .from a QA fixture with a deleted account, not a real evening
- F-ARCH-3 / F-ARCH-4** (split the two god-files) – zero cited bugs from .2 file size itself, L-effort each, real churn/regression risk for a solo ;maintainer, no correctness benefit. Drop from the actionable list .at most a “someday, opportunistically” footnote
- F-DB-13** (10X/100X/1000X scale roadmap) – an app with 614 users may .3 never see 6,140, let alone 614,000. One line (“re-derive MAX_SIDE if .formats ever exceed 11-a-side”) captures the only piece worth keeping
- F-GAME-2's “require a second admin ack”** – actively harmful UX .4 friction for the majority single/duo-admin reality this same audit documents elsewhere. Keep the badge-of-transparency fallback, drop the .approval gate
- F-PM-1's “kill Wear OS” recommendation** – a business call made on .5 zero usage data by a static-analysis audit. Keep the maintenance-cost .observation, drop the strategic recommendation

נספח E – הדוחות הטכניים

SECURITY

Teamder – Security & Privacy Audit

Scope read: `firebase.rules` (1988 lines, full read), `storage.rules` (91 lines, full
, `commitRoundStats`) read), `functions/src/index.ts` callable-validation sites
, `notifyTeamsReady`, `adminReorderRoster`, `adminAddPlayers`, `deleteMyAccount`
, `approveFiller`, `addRetroGoal`, `savePitchCalibration`, `saveGamePhysical`
, `uploadGroupCover`, `inviteFriendsToGroup`, `submitFillerInterest`, `declineFiller`
Static analysis only, per `(shape User) src/types/index.ts`, `(setGuestRating`
.BRIEF – no runtime verification possible

Overall posture: the rules file is unusually well-hardened – extensive anti-hijack comments, documented incident history (guestsOpenAt null trap, adminIds dispossession, pendingPlayerIds hijack), array-delta guards on every roster ,mutation. The gaps below are real but sit inside an otherwise disciplined ruleset .not evidence of general neglect

F-SEC-1 – Duplicate `communityPairStats` rule block silently un-scopes the club-isolation check

Severity: P1 •

Confidence: Confirmed (read directly in `firestore.rules`) •

pair stats “צמד קטלני” / Feature / Screen / Flow: Club chemistry •

Evidence: `firestore.rules:1704–1707` defines •

```
match /communityPairStats/{pairId} { allow read: if isGroupMember(resource.data.groupId); allow write: if false; }
```

with an explicit comment (“the whole point of the collection is that it does NOT cross clubs... the membership check is what keeps :that true”). A second, later block at `firestore.rules:1973–1976` re-declares the SAME path

```
.match /communityPairStats/{doc} { allow read: if isSignedIn(); allow write: if false; }
```

Current behaviour: Firestore evaluates all `match` blocks that match a given path and grants access if ANY of them allow it (OR `.isGroupMember(...)` semantics across duplicate matches). The second block’s `isSignedIn()` is strictly weaker than the first’s

Problem: The membership check the author clearly intended (and documented) is completely shadowed. Any signed-in user – not just members of that club – can read any club’s pair-chemistry doc by id or by a `where('groupId', '==', X)` query

User impact: A stranger can see “X and Y played N games together / won M / assisted N times” for any club in the app, including clubs they’ve never joined or requested to join

Technical impact: Confirmed cross-club data exposure; contradicts the file’s own stated security posture (“club isolation”) •

Recommendation: Delete the second block (1973-1976) – the first is already correct and complete. Add a rules-unit-test (the repo already has `tests/rules/*.test.mjs`) asserting a non-member of group A gets permission-denied reading `.communityPairStats` docs for group A

Expected benefit: Restores the isolation the code already believes it has •

Effort: S •

F-SEC-2 – Club-scoped stats and live-match collections skip membership gating entirely

Severity: P1 •

Confidence: Confirmed (read directly in `firestore.rules`) •

Feature / Screen / Flow: Club stats table, career stats, per-game box scores, live match state •

Evidence: `firestore.rules:1959–1970` – `communityPlayerStats/{doc}`, `gamePlayerStats/{doc}`, `communityStats/{groupId}` all `allow read: if isSignedIn()`, no `groupId` /membership check at all (unlike `groups`, `groupJoinRequests`, `communityShowcase` -adjacent collections which do check). Same pattern at `firestore.rules:1321–1329` for `rounds/{id}`: `allow read: if isSignedIn()`; with zero game/club binding, even though the sibling `games/{id}` read rule (line 698) is carefully scoped to members/participants/invitees

Current behaviour: Any signed-in user can `get()` or query (`where('groupId', '==', X)`) any club’s per-member career stats (goals/assists/rounds/wins/games/bestEvening/lastEveningScore), any club’s aggregate totals, any game’s per-participant box score (real players AND guests), and any game’s live in-match round state (team compositions, timer), regardless of club membership

Problem: These are exactly the collections the rules file’s own header promises are protected (“sensitive fields... are not readable .across users” / club isolation is a stated cross-cutting policy), but this entire layer of derived/live data was never gated

User impact: A non-member can build a full picture of another club’s roster activity and current-match state without ever joining or being approved – undermines “closed club” community settings entirely for anything downstream of membership (the raw roster stays private via `groups`, but everything computed FROM it doesn’t)

Technical impact: Straightforward `where('groupId', '==', targetGroupId)` queries against 4 collections bypass the intended perimeter

Recommendation: Add `isGroupMember(resource.data.groupId)` (or the game-scoped equivalent via `gameGroupId()`) to all four reads, mirroring the already-correct `communityPairStats` (first block) and `eveningStandings` / `roundSummaries` patterns just below them in the same file

Expected benefit: Closes the last open corner of club isolation •

Effort: S-M (verify no legitimate cross-club read path – e.g. discovery/showcase – depends on the current open behavior before tightening)

F-SEC-3 – `users/{uid}` is fully listable by any signed-in user; exposes email + social graph

Severity: P2 •

Confidence: Confirmed (rule text + `User` type shape) •

Feature / Screen / Flow: Any screen rendering another user's profile; also a raw scrape vector •

Evidence: `firebase.rules:191` – `allow read: if isSignedIn();` with no reference to `resource.data`, so it applies to both `get` and unfiltered `list / collection().get()` queries. `src/types/index.ts:22-160` shows the doc carries `email` (real Google/Apple address), `friends` (`uid` list – the whole friend graph), `newGameSubscriptions` (which clubs they follow), `dmFriendsOnly`, `discipline` (yellow/red-card history), `achievements`, `availability`, and the `invitedBy / invitedByTargetId` attribution chain. (`fcmtokens` was already correctly moved to the self-only `/private/push` subcollection per the in-file comment at :274-286 – that mitigation is real and works.)

Current behaviour: Any of the ~596 authenticated users can run `db.collection('users').get()` (paginated) and download every other user's email address, friend list, club subscriptions, and discipline history •

Problem: The rule is “get this document” shaped in intent (per the surrounding comments, which talk about “selecting only the fields you need”) but is written as “read this whole collection,” which is a materially different exposure •

User impact: Full email-address list of the user base is scrapeable by anyone with an account; a user's social graph (friends, clubs) becomes public to strangers •

Technical impact: No detection/rate-limit on bulk reads of this collection (App Check off, see F-SEC-6) •

Recommendation: The comment on this rule already names the fix (“a future hardening pass should move `fcmtokens` to `/private`” – done). Extend the same pattern to `email`: move it to `/users/{uid}/private/profile` or drop it from the client-read doc entirely (email isn't needed for peer-facing UI). Shorter-term: this can't be fixed by narrowing `allow read` alone since legitimate single-doc profile lookups need it – the real fix is field relocation, not a rule change •

Expected benefit: Removes the single largest PII-in-one-place exposure found in this audit •

Effort: M (touches converter + every read site that currently pulls email off the public doc) •

F-SEC-4 – `commitRoundStats` trusts the client's declared match outcome, with no per-game cap

Severity: P1 •

Confidence: Confirmed (read `functions/src/index.ts:12753-12930`) •

Feature / Screen / Flow: Advanced live match – scorer goal-entry, round settlement •

Evidence: `functions/src/index.ts:12766-12775` accepts `winnerSide`, `goals[]`, `penalties[]` straight from `request.data` •

Authorization is real (`:12800-12811` requires caller == `game.createdBy` or a club adminId) and roster-membership is enforced (`:12811-12888`, `goals/assists` filtered to `onField` = union of the two submitted sides, both of which must be subset of `game.players` u `game.waitlist`, excluding no-shows) – so a caller cannot credit a **non-participant**. But nothing checks `winnerSide` or the goals list against any independently-recorded score; the in-code comment at `:12852-12854` says so explicitly: “Faking your OWN stats inside your own real game is still possible; touching a non-participant's numbers is not.” There is also no cap on how many distinct `roundId`s can be committed for one `gameId` (confirmed via `grep` – the only bound is the idempotency latch on a *repeated* `roundId`, not a ceiling on round count), and no per-user/per-game rate limit on calling this callable •

Current behaviour: An admin or organiser of their own game (trivially available to any user via the “quick game” personal-group flow – every user is admin of their own personal group) can call `commitRoundStats` repeatedly with incrementing `roundId`s, `sideA=[self]`, `winnerSide='A'`, and a `goals[]` array crediting themselves, up to `MAX_SIDE=11` goals per call, with no limit on the number of calls •

Problem: This inflates GLOBAL, cross-club collections that other users see and compare against: `users.stats.goals/assists/wins`, `communityPlayerStats`, and especially `pairStats` (the app-wide “played/won together” ledger keyed only by two uids – a self-vs-self pairing isn't possible since sides must be disjoint, but a scripted caller could co-opt a second real account, or simply pump their own solo goal/win/round tally without any opponent needed) •
career-stats / rating-adjacent numbers can be fabricated without limit, undermining / “מלך השערים” User impact: A player's public any feature that treats these as a reputation signal (e.g. auto-teams balancing, player cards, club records) •

Technical impact: No score-consistency check, no rate limit – this is a pure trust-the-client gap on the one write path the app treats .as canonical truth for stats

Recommendation: (a) rate-limit `commitRoundStats` per (uid, gameId) using the existing `serverRateLimits` pattern already built for `sendGameInvite / createGroupCallable`; (b) cap total committed rounds per game to a realistic evening's worth (e.g. 20-30) – the `MAX_SIDE=11` comment already reasons about realistic football shapes, the same reasoning should bound round *count*; (c) longer-term, cross-check `winnerSide` against a server-tracked live score if/when the timer state becomes server-authoritative

.Expected benefit: Removes the one remaining path to fabricate globally-visible stats

Effort: M

F-SEC-5 – `errors/{fp}` accepts unlimited-cardinality writes with no auth and no server-side rate limit

Severity: P2

(`firestore.rules:1845-1861`) Confidence: Confirmed

Feature / Screen / Flow: Client error-log pipeline (Pulse dev-inbox source)

Evidence: `firestore.rules:1852` –

`allow create, update: if request.resource.data.operation is string && ...size caps...` – deliberately has **no** `isSignedIn()` gate (“the most important errors to capture are PRE-AUTH failures”). Client-side, `src/services/errorLog.ts` caps writes to 30/session/fingerprint and flushes at most every 12s (per D4 discovery), but that cap is enforced in JS running on the attacker's own device – trivial to bypass by writing directly to Firestore with the public web config (which ships in every client bundle)

Current behaviour: Anyone, signed in or not, holding nothing but the app's public Firebase config, can create arbitrary `errors/{fp}` docs. Varying the `operation` string (capped at 80 chars but still gives huge cardinality) avoids the count-increment coalescing the .fingerprint scheme relies on to stay small

.Problem: No rate limiting or CAPTCHA-equivalent exists at the point this collection is genuinely open to the internet

User impact: None directly to end users; this is a cost/ops-abuse and admin-noise vector (Pulse dev-inbox flooding), not a data leak .(reads/deletes are correctly denied to clients per :1846)

.Technical impact: Firestore write-cost abuse; a scripted flood could bury genuine crash signals for the founder

Recommendation: Add an App Check requirement now that App Check plumbing exists elsewhere in the codebase (`hasAppCheck()` helper is already defined, just unused – see F-SEC-6), even if reads still stay open pre-auth for genuinely broken sign-in flows the app-check token can't yet be minted for. At minimum, cap the total doc count via a scheduled sweep (the `.runDailyCleanup` cron already exists and touches other collections)

.Expected benefit: Removes a free, unauthenticated write amplification vector

Effort: S

F-SEC-6 – App Check is fully decorative: the enforcement helper is defined but never called

Severity: P2 (multiplier on F-SEC-1, 2, 4, 5, 7)

Confidence: Confirmed

Feature / Screen / Flow: Entire write surface

Evidence: `firestore.rules:47-49` defines `hasAppCheck()` . `grep -n "hasAppCheck()" firestore.rules` returns exactly two hits: the function definition and a single comment (:623) noting it's disabled pending a debug-token fix – it is never actually referenced inside an `allow` clause anywhere in the 1988-line file. Combined with `functions/src/index.ts:103` `.onCall ENFORCE_APP_CHECK=false` on every

Current behaviour: Every write path in this document, and every callable, is reachable by any client holding a valid Firebase Auth ID .token (or, for `/errors` , no token at all) – no device/app attestation friction exists anywhere in the system today

Problem: This isn't a bug by itself (it's a documented, deliberate rollout state per the in-file comments), but it means every finding above (F-SEC-1, 2, 4, 5) is trivially reachable by a plain script, not just a modified APK – the intended “App Check will catch the rest” .safety net doesn't currently exist

.User impact: N/A directly – this is a force-multiplier on the other findings

Technical impact: Ground truth already established in project memory (`ENFORCE_APP_CHECK=false`); restated here specifically to .connect it to F-SEC-1/2/4/5's exploitability, per the brief's explicit ask

Recommendation: Already tracked in project memory as a pending re-enable once App Attest is verified – no new recommendation beyond: treat F-SEC-4 (stat inflation) and F-SEC-5 (errors flood) as higher priority to fix independently of App Check, since App

.Check re-enablement has been pending for a while and these are cheap to fix directly

.Expected benefit: Context for prioritization, not a standalone fix •

Effort: N/A (tracking note) •

F-SEC-7 – No server-side rate limiting on chat, reports, feedback, or join-request writes

Severity: P2 •

Confidence: Confirmed (grep for rate-limit call sites across `functions/src/index.ts`) •

Feature / Screen / Flow: Game/club/DM chat, chat reporting, feedback, join requests •

Evidence: `grep -n "serverRateLimits\|rateLimitService\|RATE_LIMIT" functions/src/index.ts` returns hits only for `sendGameInvite` (:7132–7133 , 30/hour) and `createGroupCallable` (:8632–8802). Chat message creates (`firestore.rules:1229–1234` , :1366–1371 , :1446–1454 for games/groups/DMs), `chatReports` creates (:1942–1946), `feedback` creates (:1872–1877), and `groupJoinRequests / games/{id}/joinRequests` creates (:625–630 , :1303–1312) are all .validated only for *shape* (field types, string-length caps), never for *frequency*

Current behaviour: A member of a game/club/DM can write unlimited chat messages (≤1000 chars each) with no cooldown; a signed-in user can file unlimited `chatReports` against the same message; unlimited `feedback` submissions; unlimited join-request .creates against open communities

Problem: The two callables that got a real rate limit (`sendGameInvite` , `createGroupCallable`) both have in-file comments explaining why abuse mattered there (invite phishing, discovery-feed pollution) – the same abuse-cost reasoning applies to chat .flood and report-brigading but wasn't extended to them

User impact: A malicious member could flood a game/club chat (spam, harassment) or brigade a specific message with fake reports .to get another user's content pulled, with no built-in throttle

Technical impact: Chat push delivery is throttled ("one push until opened," `chatPush.ts:167–203`) but that only limits *notifications*, .not the underlying message writes filling the chat itself

Recommendation: Extend the existing `serverRateLimits` pattern (already proven for two callables) to chat-message creation and `chatReports` creation – likely needs to move chat-message writes behind a callable (matching the `reportChatMessage` precedent) rather than direct client writes, since rate limiting per-minute inside a Firestore rule alone is awkward without a counter .doc

.Expected benefit: Removes an easy in-app harassment/spam vector that costs nothing to execute today •

Effort: M (chat write path would need to move to a callable to rate-limit cleanly) •

F-SEC-8 – Account deletion leaves substantial data linked to the deleted uid

Severity: P2 •

Confidence: Confirmed (read `deleteMyAccount` , `functions/src/index.ts:7225–7412` , in full) •

GDPR-style deletion / "מחק את החשבון שלי": Feature / Screen / Flow •

Evidence: The 5 phases in `deleteMyAccount` cover: group/game roster arrays, bilateral `friends` , chat sender-name/avatar anonymisation (paginated collection-group sweep), and the `/users/{uid}` doc + Auth record. Confirmed **not** touched anywhere in the function or by any downstream trigger it calls: `communityPlayerStats/{groupId}__{uid}` (career stats), `gamePlayerStats/{gameId}__{uid}` (per-game box scores), `pairStats/{pairKey}` (GLOBAL "you & X played/won together" – still shows the deleted user's uid to their former teammates forever), `eveningStandings` , `roundSummaries / clubRecords` mentions, `ratings/{uid}` and `ratings/{uid}/votes/{raterUid}` (both cast and received votes), `feedback/{id}` and `errors/{fp}` docs with `userId / lastUserId == uid` (these carry the user's own free-text bug reports, potentially containing more PII than the profile doc itself), `chatReports/{id}` with `reporterId == uid`, `gameDeletions/{gameId}` with `deletedBy == uid`, `games/{id}/physical/{uid}` (per-game GPS/movement session data), `notifications/{id}` with `recipientId == uid`, and `friendRequests` docs referencing the uid on either side (only the derived `friends` array is cleaned, not the request audit docs)

Current behaviour: The Firebase Auth identity is genuinely deleted and the human-readable name/email/photo/availability are scrubbed from the primary `/users` doc – the headline PII is gone. But the uid string itself persists as a live foreign key across roughly 10 other collections, several of which (feedback, errors, physical/GPS) carry additional content the user authored or that .describes their physical activity, none of which is scrubbed or removed

- User impact: A user who deletes their account for privacy reasons still has their GPS/movement data from every game they played sitting in `games/{id}/physical/{uid}`, readable by any participant of those games indefinitely; their own written bug reports remain queryable by uid in `feedback / errors` forever
- Technical impact: Not a security hole (no elevated access is gained), a data-retention/privacy-completeness gap – relevant if the app is ever audited against GDPR “right to erasure” or an app-store data-safety declaration
- Recommendation: Given the size of the surface, a targeted follow-up is more realistic than a single sweep: prioritize `games/{id}/physical/{uid}` (most sensitive – raw location data) and `feedback / errors` (free-text PII) for a deletion or anonymisation pass; the pure-stats collections (communityPlayerStats, pairStats, gamePlayerStats) are lower priority since they carry no PII beyond the bare uid once the `/users` doc is anonymised
- Expected benefit: Closes the gap between what “delete my account” implies and what it currently does
- Effort: L (many collections, several needing a paginated collection-group-style sweep like the existing chat-anonymise phase)

F-SEC-9 – Role escalation and removed-member access: genuinely well-guarded, no finding

- Severity: N/A (clean)
- Confidence: Confirmed
- Evidence: `firestore.rules:308-523` – `adminIds` mutation is pinned to `request.auth.uid == resource.data.creatorId` (line 335), and even the creator can’t drop the check (line 339 requires the creator’s own id stay in the new `adminIds`); a co-admin cannot self-promote or add other admins. Ownership transfer only happens via the explicit “creator leaves” branch (:488-523), which requires the new owner to already be a remaining admin and blocks planting a stranger into `playerIds` in the same write (:522). Chat/round/messages rules call `isGroupMember(gid) / isGamePlayer(id)` live against current `resource.data` on every read/write, so a removed member loses access immediately on their next request – no stale-access window found
- Recommendation: None – this area is solid

F-SEC-10 – One residual `.get(key, default)` null-trap candidate, low severity (fail-closed, not fail-open)

- Severity: P3
- Confidence: Suspected (inference – pattern matches a bug class this codebase has hit twice before, but not confirmed that `rejectedPlayerIds` is ever persisted as literal `null`)
- Feature / Screen / Flow: Self join/cancel on `games/{id}`
- Evidence: `firestore.rules:816` – `!(request.auth.uid in resource.data.get('rejectedPlayerIds', []))` is not wrapped in the `safeMap()` / explicit-null-check pattern used elsewhere in the same rule for `cancellations`, `joinedAt`, `guestsOpenAt`, `waitlist`. If `rejectedPlayerIds` is ever present-but-null on a game doc (the admin-branch validation at :1170-1176 only constrains it *when present*, it doesn’t forbid a null being written by some other, unaudited path), this `.get()` returns `null` and `.uid in null` errors
- Current behaviour / Problem: Unlike the historical incidents this class caused (`guestsOpenAt`, `cancellations` – both of which *denied* a legitimate write), this one would also deny (fail-closed) – not a security escalation, just a possible “can’t self-cancel” bug for affected `games`
- User impact: Possible permission-denied on self-cancel for a small set of legacy/edge-case game docs; not a security exposure
- Recommendation: Wrap with the existing `safeMap()` -style guard for consistency, low priority given fail-closed direction
- Effort: S

CONSISTENCY

Data Consistency Audit – can two screens show a different truth ?about the same fact

,Method: static analysis of `functions/src/index.ts` (`commitRoundStats`, `rollUpClubPairs`, `sealRoundSummary`, `addRetroGoal`, the `onGameRosterChanged` evening-standings block) and the matching client readers in `src/services/*` + `src/screens/**`, cross-checked against `/ documents:runQuery` **live production Firestore** via read-only REST. No writes were made. (`soccer-app-52b6b` gcloud access token, project) `documents.get`

All three headline findings below are **measured**, not inferred – real document IDs and .real numbers from prod are quoted

F-CONSISTENCY-1 – A retro goal permanently forks one evening into three different goal counts

- Severity: P1 (serious)
- Confidence: Confirmed – measured in production
- club table / profile vs → (retro goal) “השלמת שער” Feature / Screen / Flow: Admin personal card “סיכום הערב” match-details round recap vs :Evidence
- `users.stats.goals` bumps `addRetroGoal` – `functions/src/index.ts:13709–13830` but never, (live truth) `communityStats`, `gamePlayerStats`, `communityPlayerStats` touches `games/{id}/roundHistory` or `roundSummaries/{id}` (“detached from any . `eveningStandings/{id__uid}` round” – line 13712) or is create-only and explicitly never recomputed `roundSummaries/{id}` .(`summaryRef.create(...)` :`index.ts:4632–4638`, 4776, `sealRoundSummary`) `gamePlayerStats` is computed once from a `eveningStandings/{id__uid}.score` .snapshot taken at evening-end (`index.ts:5371–5407`) and never revisited
- Production game** `49sGSo4y8DfMRU1GBzvc` (club `0cdCmLhka0dTsqA2AF0a`, evening of :queried live ,(2026-08-25
 - real-scorer goals = 2 :(docs, matches `committedRounds` =6 6) `roundHistory`
 - (sealed at evening-end) 2 = `roundSummaries/49sGSo4y8DfMRU1GBzvc.stats.goals`
 - for this game (13 docs, current truth): real-scorer goals = 6 `gamePlayerStats`
 - has 4 docs, added 2026-08-26 06:52~06:58 `games/49sGSo4y8DfMRU1GBzvc/retroGoals` .accounting for exactly the +4 delta ,(the morning after)
 - Player `B5KpY04ILhQD0YVmMng997matof1` : `roundHistory` shows 1 goal for him that evening; `gamePlayerStats` now shows 3 (`goals:'3'` , `updatedAt` = the retro-goal timestamp); his frozen `eveningStandings` doc still reads `score: 6.7` , computed .before the retro goals existed
 - all read live aggregates and “מלך השערים” / Current behaviour: club table / profile correctly show 6 goals for the evening. The round-by-round recap (`roundHistory`) and story (`roundSummaries`) still say 2, forever. The player's own “סיכום הערב” the sealed .evening score card still shows the score computed from his pre-correction total
 - Problem: this is not a race condition or a rare failure – it is the *designed* behaviour of `addRetroGoal` , and it fires on every retro goal, which the audit found is an actively-used feature (24 retro-goal docs exist across 5+ games in prod, most .(recently the morning of 2026-08-26
 - User impact: a player who opens his profile sees N goals for the night; the same player opening the match recap for that exact evening sees fewer; the personal share card shows a score/rank that undercounts him. All three are visited “סיכום הערב” ,by the same user in the same session in normal use (finish game → share card → later .(club table
 - Technical impact: `roundSummaries` and `eveningStandings` are permanently wrong (not eventually-consistent – there is no code path that ever corrects them) the moment a

retro goal lands on an already-sealed evening. This will get worse as retro-goal usage grows.

Recommendation: either (a) block `addRetroGoal` once `roundSummaries/{gameId}` exists and tell the admin to contact support / use a different flow, or (b) have patch the sealed summary's totals and the affected player's `addRetroGoal` .doc in the same batch (bounded – retro goals are rare and per-game) `eveningStandings` .is far cheaper (a)

Expected benefit: closes a real, currently-open divergence between three surfaces of the same evening

Effort: S (guard) / M (patch-in-place)

F-CONSISTENCY-2 – roundHistory can be entirely absent for a fully-credited evening; the club-chemistry silently never learn it happened “סיכום הערב” rollup and

Severity: P1 (serious)

Confidence: Confirmed – measured in production

round summary, club “סיכום הערב”, Feature / Screen / Flow: Match-details round recap widget (“כימיה”) chemistry :Evidence

the `roundHistory` doc is – `functions/src/index.ts:12974–12980, 13513–13527` , try written **after** the atomic stats batch commits, in a separate best-effort explicitly because it would blow the batch's op/size budget. Comment claims “a rare .write failure loses only summary richness, never a stat” (13516)

,That claim is false for club chemistry: `rollUpClubPairs` (`index.ts:4515–4598` called from `index.ts:5706–5744`) reads **only** `games/{id}/roundHistory` to compute , `sameTeam` – every `communityPairStats` field except the legacy `assists` counter . `winsB` , `winsA` , `against` , `cleanSheetsTogether` , `lossesTogether` , `winsTogether` If `roundHistory` is empty, `rounds.length === 0` short-circuits the function `communityPairRollups/{groupId}__{gameId}` **before** it even creates the (line 4522) marker – so nothing is written, nothing is marked “done”, and nothing will ever retry it. Same story for `sealRoundSummary` (`index.ts:4660–4682`), which builds its .from the same subcollection (the event log / GF-GA basis) `rounds`

Scanned 75 production `games` docs with `status:'finished'` . For every game with :present, compared the count against `roundHistory` count `committedRounds` `roundHistory === 0` **games / 3 clubs** have `committedRounds > 0` but **7** :.e.g ,(complete loss)

,committed rounds, **0** `roundHistory` docs 13 – `8tuP5v4NXfQoml3qD4no` ◦ `sum = 17` real goals. No `roundSummaries` doc exists for this `gamePlayerStats` game at all (404). No `communityPairRollups` marker exists (404) – permanently .unprocessed

,committed rounds, **0** `roundHistory` docs 10 – `IyeXdokphWSQWYoWE9eI` ◦ .sum = 17 real goals. Same absence of summary + rollup `gamePlayerStats` , `TmMUTEvemar0Eit1YuKm` , `ZIimTpY7wkzPCes5XUkK` , `WhqMQzLznMgPI4d97GSr`) more 5 ◦ .(`8tuP5v4NXfQoml3qD4no_w1783530000000` , `5EQKVHge5mtPrOXwqwDf` = `chemistrySince` All 7 predate ~2026-07-08 (club `0cdCmLhka0dTsqA2AF0a` 's ◦

so the likely root cause is a feature-launch/schema gap rather than ,(2026-07-15 an active write failure caught in the act – **but no partial mismatches were found in the other ~68 games**, so the (`committedRounds > roundHistory > 0`) steady-state “some rounds silently missing, most present” failure the code .comment worries about has not (yet) been observed to actually happen post-launch

The mechanism the code describes is real and would produce exactly this signature
.if it did fire

- Current behaviour: for these evenings, the player/club totals (source of truth) are correct and complete; the round recap and evening story are blank; the club-chemistry widget has silently never counted these evenings and never will
- User impact: a player whose big night was one of these 7 games sees his goals on his card, no recap "סיכום הערב" profile but the evening itself is unrecapturable – no Two teammates who only ever played together on one of these evenings will see "no data yet" on the club chemistry card despite having a real recorded history together elsewhere (see F-CONSISTENCY-3 for the concrete pair)
- Technical impact: `rollUpClubPairs` 's "already rolled up" marker is create-*after* success, not create-*before*-attempt, so a `roundHistory` -empty evening is silently and permanently skipped – there is no cron/backfill job that revisits games whose `roundHistory` is thin or missing
- ,Recommendation: (1) make the `roundHistory` write part of (or immediately preceding ,with a retry) the atomic path for at least a minimal record (teams+winner+goal count even without the full goal log) so an evening is never "invisible"; (2) add a reconciliation sweep that finds `status:'finished'` games with `committedRounds` but no doc and retries them from `gamePlayerStats` (a `communityPairRollups` / `roundSummaries` .(degraded rollup is better than a silent, permanent zero
- Expected benefit: recovers chemistry/summary data for every future silent-drop, not just documents the gap
- Effort: M

F-CONSISTENCY-3 – club-scoped pair chemistry and the global head-to-head card read two structurally different pipelines for the same fact

- Severity: P2 (meaningful)
- Confidence: Confirmed – measured in production
- Feature / Screen / Flow: `PlayerCardScreen` 's "played together / against" section / `CommunityStatsScreen` vs (`pairStats` reads **global** , `PairStatsSection`) club-chemistry card (reads **communityPairStats**) `clubChemistryService` :Evidence
- is written (global, cross-club) `pairStats` – `functions/src/index.ts:13437–13495` , `winsA` , `against` : `commitRoundStats` **live, atomically, every round** inside . `lossesTogether` , `winsTogether` , `sameTeam` , `winsB`
- The same fields on `communityPairStats` (club-scoped) are written **only** by the end-of-evening, `roundHistory`-dependent `rollUpClubPairs` (see F-CONSISTENCY-2). The ,single field `commitRoundStats` writes live to `communityPairStats` is a *different* non-directional `assists` counter (`index.ts:13421–13429`) that the club-chemistry UI deliberately never reads (`src/services/clubChemistryService.ts:24–33` – the code's own comment explains this exact trap: a directional breakdown next to a wider-window .("total "would print a breakdown that does not add up to its own total
- the "played together" section is – `src/screens/players/PlayerCardScreen.tsx:311–316` ;" ;" deliberately kept global "to agree with the Statistics screen's .the comment does not mention that the club-scoped card can disagree with it
- `9E2auIqspmNA2sI3YkbFWAAAYs72` / `1IdtNEjbEXfiRSqvLrJVn99NsfiI2` **Measured:** pair , `8tuP5v4NXfQoml3qD4no` both credited players in club `0cdCmLhka0dT5QA2AF0a` 's game) :one of the `roundHistory`-empty games from F-CONSISTENCY-2

,global) pairStats/1IdtNEjbEXfiRSqvLrJVn99Nsfi2__9E2auIqspmNA2sI3YkbFWAAAYs72 ◦
against: 4, winsA: 2, winsB: 2 = (live
communityPairStats/0cdCmLhka0dTsQA2AF0a__1IdtNEjbEXfiRSqvLrJVn99Nsfi2__9E2auIqspmNA2sI3YkbFWAAAYs72 ◦
. {} **does not exist** – the query returns (club-scoped)

- Current behaviour: PlayerCardScreen 's rival card would show these two as having .faced off 4 times (2-2); the club chemistry card has zero record of this pair at all
- :User impact: SUSPECTED – not verified at runtime, but reading straight off the code • a player opening his own profile sees a real rivalry number; opening the club chemistry screen for the same club shows nothing for that pair – a visible .contradiction if the two screens are compared
- Technical impact: this is architecturally unavoidable as long as • s directional/against/sameTeam fields have no live writer – every' communityPairStats evening whose roundHistory is thin or missing (F-CONSISTENCY-2) creates exactly this .gap, and it compounds over time as more evenings accumulate history gaps
- winsB / winsA / sameTeam / against Recommendation: either write the club-scoped • fields live in commitRoundStats (same batch that already writes the global – equivalents, same pair loop, one extra groupId -scoped doc per pair pairStats bounded by the same MAX_SIDE=11 budget already reasoned about at line 12843) instead of depending on the once-only roundHistory rollout; keep rollUpClubPairs only for the .fields that genuinely need the full evening's roster (if any remain)
- ,Expected benefit: club chemistry stops depending on roundHistory completeness at all • .eliminating this whole divergence class instead of patching around it
- Effort: M •

Fact-by-fact map (per the brief)

- Goals** – Source of truth: commitRoundStats 's atomic batch .1
, users.stats.goals writes (functions/src/index.ts:13054–13100)
together, latched by gamePlayerStats.goals , communityPlayerStats.goals
- Verified consistent in prod:** player . committedRounds/{roundId}.create()
sum of his , users.stats.goals=16 – ouF280RPdfYbKINgRnupme2KaQY2
.sum of his gamePlayerStats across 7 games=16. Exact match ,16= communityPlayerStats
- The only observed divergence is via roundHistory / roundSummaries (round-recap and addRetroGoal of the evening, not career) – see F-CONSISTENCY-1/2, and “מלך השערים”
.which updates the three live stores but not the round-scoped displays
- Wins / losses / win%** – Same atomic-batch treatment as goals .2
also verified matching in prod for the sampled player ,(index.ts:13387–13409)
Crediting is gated on onField , which .(identical across all three stores wins=24)
excludes arrivals[uid]=== 'no_show' – a status read at commit time, not at kickoff
This means a player who *did* play a round but is later marked .(index.ts:12833–12842)
,no-show for the rest of the evening loses that round's win/loss credit everywhere
consistently – not a cross-store divergence, but a real “the truth itself is arguably
wrong” issue worth separate follow-up (not filed here; out of scope for a
(consistency-between-copies audit
- Same atomic batch as goals/wins – (משחקונים) **Rounds played** .3
verified matching (rounds=46 identical across all three ; index.ts:13292–13332)
.stores for the sampled player). No divergence found
- , users.stats.cleanSheets **Clean sheets** – Same atomic batch, credited to .4
in the same loop as gamePlayerStats.cleanSheets , communityPlayerStats.cleanSheets

rounds (`index.ts:13279–13332`); verified matching in prod (`cleanSheets=16` identical .across all three stores). No divergence found

Pair / chemistry numbers – See F-CONSISTENCY-2 and F-CONSISTENCY-3. Two .5 ,genuinely separate pipelines exist as the brief describes: `pairStats` (global, live atomic) vs `communityPairStats` (club-scoped, mostly a once-per-evening best-effort rollup keyed on `roundHistory`). They do NOT agree in general, and the audit measured a .concrete case where they diverge completely (one has data, the other has none)

Evening score & club standings – `eveningStandings/{gameId}__{uid}` is computed .6 once, from a `gamePlayerStats` + `communityPlayerStats` snapshot taken at the moment the :evening is detected finished (`index.ts:5282–5663`), and never recomputed. **Measured** for game `49sGS04y8DfMRU1GBzvc` , player `B5KpY04ILhQD0YVmMng997matof1` 's frozen was computed from his pre-retro-goal stat line; his `eveningStandings.score` = 6.7 card (which "סיכום הערב" current `gamePlayerStats.goals` is now higher. The personal reads this doc) will show a stale score/rank forever relative to his now-corrected .totals. See F-CONSISTENCY-1

Club-level totals (communityStats) vs sum of member stats – `communityStats` is .7 `commitRoundStats` incremented atomically alongside `communityPlayerStats` in the same batch, so `communityStats.goals` / `.rounds` track the member-row sums faithfully for `onField` / `byScorer` goals/rounds (confirmed by code: both derive from the identical loops). `communityStats` deliberately has **no** assists or clean-sheet counter – the code itself compensates by summing member rows exactly where it's needed , `derived.totalAssists` and `CommunityStatsScreen` 's , `index.ts:4724–4731` , `sealRoundSummary`) This is a correctly-handled .(`src/screens/communities/CommunityStatsScreen.tsx:201` .case, not a bug – noted and closed

Achievements / titles / club level – `src/utils/clubLevel.ts` and .8 are pure, unpersisted client-side derivations from `src/data/clubAchievements.ts` whatever `communityStats` / `communityPlayerStats` currently say There is no .(`clubGoals: champ?.totalGoals ?? 0` : `CommunityStatsScreen.tsx:287`) separate stored copy of a title/level to go stale – the level number simply inherits ;whatever staleness already exists in its inputs. No independent divergence risk here .fine as-is

Summary

/ `users.stats` The atomic core (goals/wins/losses/rounds/clean sheets across is genuinely sound – verified against live (`gamePlayerStats` / `communityPlayerStats` production data with an exact match. Every real divergence found lives in the layer built **on top of** that core for storytelling and per-evening/per-pair display because ,(`communityPairStats` , `eveningStandings` , `roundSummaries` , `roundHistory`) that layer is either (a) explicitly best-effort / outside the atomic batch, or (b) sealed once and never revisited when a later correction (`addRetroGoal`) changes the ,numbers it was built from. Both mechanisms are now shown to have already produced real .measured discrepancies in production, not just theoretical risk

Business invariants audit – Teamder

, `src/services/gameService.ts` , Static analysis of `functions/src/index.ts` (14k lines)
cross-checked against `firestore.rules` , `src/services/achievementsService.ts`
/ `documents:runQuery`) production Firestore via read-only REST queries
direct doc GET, project `soccer-app-52b6b`). No writes were made. Runtime app
behaviour was NOT observed (no emulator) – anything about what a user sees is
.marked SUSPECTED

Part 1 – Derived invariants

/Round-commit exactly-once: a mini-game's stats (goals/assists/wins .1
, losses/pens) are applied to every downstream aggregate exactly once
.never on redelivery

No stat survives its game: deleting a `games/{id}` doc must not leave .2
, `communityPlayerStats` , `users.stats` any of its contributions live in
, `communityPairStats` , `pairStats` , `gamePlayerStats` , `communityStats`
. `roundSummaries` , `eveningStandings`

Guests never gain lifetime identity: a guest's goals/assists/rounds .3
may appear in a game-scoped or club-scoped aggregate, never in a
.doc (guests have no uid to own one) `users/{uid}`

participantIds = players ∪ waitlist ∪ pending, always, on every .4
.commit that touches any of the three

An achievement/counter increments exactly once per real-world event .5
it represents – no event should be double-credited by two independent
.writers

wins + losses + ties (per player, per evening) ≤ rounds played that .6
.evening – a player can't win more mini-games than they played

Idempotency latches gate the SAME batch as the increments they guard .7
create-once marker + increments must commit atomically together, (or)
.not at all) – otherwise a partial-failure retry double-counts

A club's pair/chemistry numbers are additive across exactly the set of .8
, **evenings the club claims** (`chemistrySince`) – no evening counted twice
.none silently skipped

Waitlist promotion is capacity-safe and atomic: a freed seat is .9
filled by at most one promotion, and roster arrays + participantIds move
.together inside one transaction

A deleted/dissolved club leaves no dangling per-club aggregate .10
, `communityPairStats` , `communityStats` , `communityPlayerStats`)
pointing at a (`communityPairRollups` , `roundSummaries` , `clubRecords`
.that no longer exists `groupId`

A game's finish-credit ("games played") fires exactly once per game .11
.matching the roster that actually attended (no-shows excluded)

Recurring/series-generated games are never spawned twice for the same .12
.occurrence (one `games` doc per week per series)

F-INV-1 – Deleting a played game leaves its stats behind forever (CONFIRMED IN PRODUCTION)

- Severity: Po •
- Confidence: Confirmed – code path read end-to-end AND the exact failure mode is present in live data •
- any direct `games/{id} delete / (delete game) "מחק משחק"` Feature / Screen / Flow: MatchDetailsScreen •
- :Evidence •
- for the real (non-mock) path is a bare `deleteGame()` – `src/services/gameService.ts:4544–4617` •
- `.deleteDoc(docs.game(gameId))` . No subcollection cleanup, no stat reversal •
- `s delete branch ( if (!after) { ... } )` only (a) writes a `onGameRosterChanged` – `functions/src/index.ts:4863–4949` •
- `gameDeletions` audit doc and (b) mints a "game cancelled" push, then `return;` . Grep confirms there is no `onDocumentDeleted` •
- `. functions/src/*.ts` trigger anywhere in •
- No `.allow delete: if isSignedIn() && (uid == createdBy || isGroupAdmin(groupId))` – `firestore.rules:1180–1181` •
- status check. The only guard against deleting a `finished / live` game is client-side UI: •
- `src/screens/games/MatchDetailsScreen.tsx:2217` gates the "delete" menu item on `!isTerminalGame(game)` , and •
- `isTerminalGame ( src/services/gameLifecycle.ts:77–87 )` is `finished || cancelled` – it does **not** exclude `live / active` , •
- .so an admin can delete a game that is mid-match, after rounds have already been committed, straight from the ordinary menu •
- Every stat aggregate (`users.stats.*` , `communityPlayerStats` , `communityStats` , `pairStats` , `communityPairStats` , •
- `gamePlayerStats`) is `increment()` -only and never recomputed from source (confirmed by grep – no `.set()` without `merge` and •
- .no full-recompute pass over `games` for any of these besides the unrelated `groupsPublic.gamesLast30/60` sweep) •
- Production confirmation:** queried `gameDeletions` (20 docs total). Doc id `fusBhEBGkC37D49LygwP` has `source:"manual"` , •
- `status:"finished"` , `rosterCount:1` , `groupId:"2SjvS7AjMPABB4bKaBxS"` . The parent `games/fusBhEBGkC37D49LygwP` doc is •
- gone (404), but its subcollection `games/fusBhEBGkC37D49LygwP/roundHistory/1:1787391983205` **still exists** (fetched directly, •
- `createTime: 2026-08-22T10:18:25Z`) and contains a fully-played round: score 2-1, three goals (incl. one own goal), and a real •
- (non-guest) participant `loaEHV7Z3JdMBSMZ4veRMCvGTVA2` on the losing side. This is direct proof the delete path leaves orphaned, •
- .billed, unreachable Firestore data behind for a game that had already been played •
- Anomaly worth a follow-up (does not weaken the finding): a field-equality query and direct doc-id lookups for `gamePlayerStats` •
- with `gameId == fusBhEBGkC37D49LygwP` returned zero documents, which is inconsistent with `commitRoundStats` writing •
- `roundHistory` and `gamePlayerStats` in the same batch (`functions/src/index.ts:13520–13522` vs `:13296–13407`). Either this •
- specific round predates a schema change or something else is going on – flagged for a closer look, but the core claim (orphaned •
- .subcollection survives deletion; no code path reverses any aggregate) stands independently of this anomaly •
- Current behaviour: any creator/admin can delete a game at any point in its lifecycle (rules enforce nothing about status); the client UI •
- merely hides the button for `finished / cancelled` but not for `live / active` ; the delete never reverses or cleans up anything •
- `.commitRoundStats` downstream of •
- Problem: violates "a deleted game must not leave a live aggregate behind" – confirmed both structurally (no reversal code exists) •
- .and empirically (orphaned data found in prod) •
- player career stats, and chemistry cards can permanently include (win/loss table, "מלך השערים") User impact: club leaderboards •
- .contributions from a game that no longer exists in the app, with no way for anyone (including admins) to notice or correct it •
- Technical impact: unbounded, permanent aggregate drift; growing orphaned-subcollection storage that `collectionGroup` scans / •
- .future migrations will trip over •
- Recommendation: (1) add a status gate to `firestore.rules` , `games` delete rule – deny delete once any round has been •
- committed (e.g. once `games/{id}/committedRounds` is non-empty, or once status has ever reached `live`), mirroring but •
- strengthening the client's `isTerminalGame` gate to also cover `live / active` ; (2) for the legitimate "admin needs to remove a •
- played game" case, build a callable that reverses the exact increments (same pattern as `removeRetroGoal` , •
- `. functions/src/index.ts:13843–13905`) and hard-deletes subcollections before removing the game doc •
- Expected benefit: closes a live, currently-exploited (per production evidence) data-integrity hole; stops indefinite storage growth from •
- .orphaned subcollections •
- Effort: M •

F-INV-2 – `achievements.gamesJoined` is double-credited on every self-join (Confirmed, currently inert for display)

Severity: P2 •

Confidence: Confirmed (code) •

(`approveGameJoin` , `joinGameV2`) Feature / Screen / Flow: joining a game •

:Evidence •

after the real Firestore join transaction commits, the client calls – `src/services/gameService.ts:5395–5400` `achievementsService.bump(userId, 'gamesJoined', 1)` . Same pattern at `:5449–5454` and `:5555–5561` for the approval-flow joins

only skips when `auth.uid !== uid` (cross-user write, blocked by `bump()` – `src/services/achievementsService.ts:104–119` , `bumpFirebase()` rules anyway); for the common case – a user joining **themselves** – it proceeds to `.updateDoc(ref, {'achievements.gamesJoined': increment(1)})`

s “Server-side achievement bumps for the joiners” block `onGameRosterChanged` – `functions/src/index.ts:5941–5970` independently increments the **same** `users/{uid}.achievements.gamesJoined` field by 1 for every uid newly present in `after.players` vs `before.players` , latched only against server-trigger redelivery (`games/{id}/joinCredited/{uid}`) – nothing in this code excludes self-joins, despite the client-side comment (“Server-side triggers handle [cross-user] cases”) implying it should .only cover admin-added players

Net effect: for a normal self-join, both writers fire for the same event with no shared idempotency key between them – the counter is .incremented twice

.Current behaviour: `users/{uid}.achievements.gamesJoined` is 2× the real join count for self-joins •

“.Problem: violates “an achievement/counter increments exactly once per real event •

Impact today: cosmetically inert. `achievementsService.deriveCounters` (`achievementsService.ts`, header comment lines 1-30) recomputes `gamesJoined` from real terminal games for both the badge list shown to users and `persistDerivedUnlocks` ’s unlock decisions – grep confirms no screen reads the raw `achievements.gamesJoined` field directly. So no user-visible badge is wrong .today

Technical impact: a wasted extra Firestore write on literally every join (this branch is `perf/firestore-read-costs` – worth folding in), and a landmine: the field is typed and persisted as if meaningful (`User.achievements.gamesJoined`), so any future feature (a .leaderboard, Pulse segmentation, a Joryio journey attribute) that reads it raw will silently see double

Recommendation: remove the client-side `achievementsService.bump(..., 'gamesJoined', ...)` call sites (the server trigger .already covers every joiner, self or admin-added) – one writer, not two

.Expected benefit: removes a needless write per join; defuses a landmine for future features •

Effort: S •

F-INV-3 – Stale/contradictory comment on the same-team pair-stats move (no live bug, documentation debt)

Severity: P4 •

Confidence: Confirmed (code) •

Evidence: `functions/src/index.ts:13432–13435` still says “same-team pairs... are already written by the existing `onGameRotationChanged` trigger on every rotation – do NOT duplicate them here,” directly contradicted by the code 30 lines below

it (`:13463–13495` , comment “MOVED here from the `onGameRotationChanged` trigger”). Checked the old site: `onGameRotationChanged` (`:4462–4466`) is now an explicit no-op for these fields, so there is no live double-write – just a leftover .comment that would mislead the next person to touch this function into thinking a duplicate-write guard is still needed there

. `:13432–13435` Recommendation: delete the stale comment block at •

Effort: S •

Part 3 – Checked and confirmed SAFE (no need to re-investigate)

- **Round-commit idempotency** (`functions/src/index.ts:12897–12910`): the `committedRounds/{roundId}` latch is created with `.batch.create()` in the exact same batch as every increment for that round – a redelivery fails the whole batch atomically. Solid
- **Pair/chemistry two-pipeline design** (`commitRoundStats` live writes vs. `rollUpClubPairs` evening rollup): verified the two pipelines write **disjoint fields** on `communityPairStats` – `commitRoundStats` writes only the legacy, undirected, unwindowed `assists`; `rollUpClubPairs` writes `sameTeam/winsTogether/lossesTogether/against/winsA/winsB/assistsAToB/assistsBToA`, explicitly windowed from `.chemistrySince` (comment at `:4576–4585` calls this out by design). No double count
- **s own get-then-create race** (flagged as a theoretical concern in prior discovery): the function is only ever `rollUpClubPairs` invoked from inside `onGameRosterChanged`'s `creditedNow` block (`:5257–5264` → `:5732`), which itself is gated by the `finishCredited` `batch.create()` latch – since that latch can succeed at most once per game, `rollUpClubPairs` can only actually be called once per game in practice, regardless of its own weaker internal guard. Safe by construction of its caller
- **Guests never gain lifetime stats**: confirmed via `isReal()` guards throughout `commitRoundStats / addRetroGoal / removeRetroGoal` – guest ids get `gamePlayerStats` rows and count toward `communityStats` totals, but every `.isReal(uid)` `users/{uid}.stats.*` write is gated on
- **Waitlist promotion + participantIds**: both the transactional auto-promotion inside `onGameRosterChanged` (`:5100–5152`) and `approveFiller` (`:11548–11562` , comment explicitly names “the participantIds invariant”) recompute `participantIds` as `players u waitlist u pending` inside the same transaction/batch as the roster mutation. The real client `joinGameV2` Firestore `.path ( gameService.ts , transaction starting ~line 5140)` does the same. Safe
- **Retro-goal reversal**: `removeRetroGoal` (`:13843–13905`) is a transaction, checks the marker still exists before decrementing (idempotent against double-removal), deletes the marker in the same transaction as the negative increments. Safe

Open item (not resolved, flagged for follow-up)

- The `gamePlayerStats` absence for game `fusBhEBGkC37D49LygwP` (F-INV-1 evidence) despite a populated `roundHistory` doc from the same commit – worth a targeted check of whether an older `commitRoundStats` code path could write `roundHistory` without the full fan-out, or whether this is a manually-seeded QA fixture rather than a real `commitRoundStats` invocation

RELIABILITY

Reliability audit – Teamder live-evening failure modes

Scope: what happens when things go wrong at the worst moment – a phone on a pitch, poor reception, one admin running the game, stats committed between mini-games. Static analysis only (no runtime available); every claim is grounded in a file:line citation. Findings ranked roughly by likelihood on a real Tuesday evening, most likely / most damaging first

F-RELIABILITY-1 – Two admins finalizing the same round: the loser's server response is a lie the client believes, and their real stats are silently dropped forever

Severity: P1 •

Confidence: Confirmed •

Feature / Screen / Flow: Advanced live match – ending a round with two admins present (routine at a well-staffed evening: one on the pitch, one on the sideline with the phone)

Evidence: `commitRoundStats` (`functions/src/index.ts:12753–13535`) latches on `games/{id}/committedRounds/{roundId}` via `batch.create()` in the *same* atomic batch as every stat increment (`index.ts:12903–12908`). `roundId` =

`roundCommitKey(rot)` (`src/services/rotationEngine.ts:497-511`), which is **deliberately content-independent** – keyed only on `rotation.roundInstanceId` , minted once per live round regardless of what else changed. When a genuine second `commitRoundStats` batch for the same round hits `ALREADY_EXISTS` , the server does **not** throw an error – it returns `{ ok: true, alreadyCommitted: true }` (`index.ts:13495-13509`). Client-side, `_commitRoundStatsAndClear` (`src/services/gameService.ts:3822-3906`) calls this callable and **discards the return value entirely** (`gameService.ts:3843` – the result isn't even assigned to a variable); since the promise didn't throw, it falls straight through to clearing the local scoreboard (`.gameService.ts:3886-3893`) exactly as if its own commit had won

- Current behaviour: If two admins independently finalize the identical round instance with two genuinely *different* payloads (plausible any time listener lag means one admin's local `liveMatch.goals /lineup` differs from what the other already committed – a late goal typed in by one admin a beat before the other taps “end round”), the loser's entire round contribution – goals, assists, wins, cleanSheets, pair-chemistry increments, penalty-shootout stats – is silently and permanently discarded. Both admins' screens behave identically: scoreboard clears, no error toast, nothing anywhere signals that one admin's data never made it in
- Problem: This is exactly the bug class `roundInstanceId` was built to fix (a stable idempotency key so a *lost-response retry of the same commit* is safe) – but the fix makes no distinction between “my own retry of an identical payload” and “a different admin's different payload for the same round,” and both server and client treat the second case as a harmless duplicate
- User impact: A player's goal or a win that genuinely happened is silently missing from lifetime stats, with no error, no warning, no way to know it happened short of manually cross-checking rounds played vs. rounds credited
- Technical impact: Per D2 discovery, these are incremental counters with “no recompute-from-history path” – the loss is permanent, not self-healing. This is a real-Tuesday-evening scenario precisely because the app is explicitly designed for one admin running live stats, and a second admin picking up the phone (very normal – “here, you take it, I need to play”) is the trigger condition
- Recommendation: Have the client inspect the callable's `alreadyCommitted` flag; when true, diff the just-attempted payload against what's already in `roundHistory / gamePlayerStats` for that round and surface a “this round was already recorded by another admin – your entry didn't match, review it” warning rather than silently succeeding
- Expected benefit: Converts a silent, permanent stat-corruption path into a visible, reviewable one, on the single most safety-critical write in the app
- Effort: M

F-RELIABILITY-2 – Offline reads silently bypass the “game already finished” guard, letting a phantom write land once the phone reconnects

- Severity: P1
- Confidence: Confirmed
- `readTimerState` Feature / Screen / Flow: Live match – goals, shootout, timer, all writes gated on
- Evidence: `src/services/gameService.ts:265-282` (`readTimerState`) does a plain `getDoc(ref)` . Its own comment block is self-contradictory: an earlier comment says it “prefers Firestore's LOCAL cache” while a later one (added after a bugfix) claims “Read FRESH from the server, never cache-first” – neither is actually enforced: the code never calls `getDocFromServer` . Every guard built on it (`recordGoal` :3489 guard ~3548, `removeGoal` :3563 guard ~3594, `startShootout / setShootoutKeeper / recordShootoutKick / undoLastShootoutKick` :3698-3800, `startTimer` :7034 guard ~7063, `pauseTimer` :7107 guard ~7141, `resetTimer` :7185 guard ~7222) reads `cur.status` and no-ops if `(cur.status === 'finished' || cur.status === 'cancelled')` . A plain `getDoc()` silently serves the stale local cache when offline – no error, no `fromCache` check anywhere (confirmed zero hits for `fromCache / hasPendingWrites` across `src/` , see F-RELIABILITY-6)
- Current behaviour: An admin whose phone drops connectivity right as another admin ends the evening keeps a stale “still live” snapshot. Their next tap (goal, shootout kick, timer start) reads that stale cache, passes the finished/cancelled guard, and queues a write. When connectivity returns, that write silently lands on the now-finished, “sealed” game
- Problem: The exact guard the code comment says exists specifically to stop “a second admin tapping ‘+goal’ right after another admin ended the evening” (comment at `gameService.ts:3712-3715`) is defeated by the one condition – poor pitch reception – this audit is centered on
- User impact: A phantom goal, timer restart, or shootout kick appears on a finished evening after the round/evening summary has already been computed and shown to players

- Technical impact: `endEvening` (`gameService.ts:7275`) has the identical unforced-cache pattern for its own idempotency re-check `(.getDoc at ~7302)`.
- or check `snap.metadata.fromCache` and refuse the action with a clear `getDocFromServer` Recommendation: Force `in readTimerState` and in `endEvening` 's pre-write check (message "נסה שוב").
- Expected benefit: Closes the gap between "the guard exists" and "the guard works under the exact condition it was built for"
- Effort: S

F-RELIABILITY-3 – `endEvening` swallows a failed final-round stats commit and finishes the game anyway; nothing retries, nobody is told

- Severity: P1
- Confidence: Confirmed
- when the last mini-game was never separately ended ("סיים ערב") Feature / Screen / Flow: Ending the evening
- Evidence: `src/services/gameService.ts:7275-7373`. If the last round's goals weren't yet committed, `endEvening` calls `_commitRoundStatsAndClear` inside a `try/catch` (invoked ~7327); on failure it sets `finalRoundCommitted = false` and only calls `logError('endEvening.commitFinalRound', err, {gameId})` – never rethrown, never returned to the caller (`endEvening` returns `void`). It then unconditionally writes `status: 'finished'` (~7332-7336), locking the game read-only per its own docstring ("Read-only after this"). On failure, `liveMatch.goals /score` are deliberately preserved (comment ~7357-7359: "keep the goal log/score so nothing is lost"), but a `grep` of `src/screens` and `src/services` shows **zero** other read site for `liveMatch.goals` after this point – `MatchRoundsScreen / RoundSummaryScreen` read `roundHistory / gamePlayerStats` instead, so the preserved `'finished'` data has nowhere to be replayed from once `status` is
- Current behaviour: A network blip at the exact moment an admin taps "end evening" – the single highest-probability failure moment of the whole flow (packing up, walking off the pitch, weakest signal of the night) – silently drops that last mini-game's `.goals/assists/wins/clean-sheets/pair stats`.
- Problem: The data is deliberately preserved but genuinely orphaned; no cron, trigger, or UI ever looks at it again
- User impact: Players silently miss goals/assists/win credit for the last mini-game of the evening; the evening/round summary looks quietly wrong and nobody is told why
- with a retry toast until the commit actually succeeds, "סיים ערב" block) Recommendation: Surface `finalRoundCommitted` to the UI `(.status: 'finished'` or persist a `games/{id}.finalRoundCommitFailed:true` flag an admin/cron can retry even after
- Expected benefit: Eliminates a silent stat-loss path at the single highest-probability failure moment in the whole live-match flow
- Effort: S (the data is already preserved – only the surfacing/retry is missing)

is two non-atomic writes separated by an (auto-refill → "הלך הביתה") F-RELIABILITY-4 – Substitution unbounded, unpersisted, human-driven wait with no busy-lock and no minimum-roster-size check

- Severity: P1
- Confidence: Confirmed
- mid-evening substitution "הלך הביתה" – Feature / Screen / Flow: Advanced live match
- Evidence: Write 1 – `gameService.markPlayerWentHome` (`gameService.ts:4335-4371`, write at 4371) removes the departing player from `draftTeams.teams[].playerIds`. Called from `AdvancedLiveMatchScreen.tsx:1393`. Immediately after, `prepareRefillPlaying` (pure read) plus `beginFillFlow` (`AdvancedLiveMatchScreen.tsx:422-460`) store the in-progress fill **only** in a React ref, `fillFlowRef.current` – never persisted. `advanceFillFlow` opens `FillerPickerModal` and returns, waiting on the admin – no Firestore write happens during this wait, which is unbounded (real-world admin decision time). Write 2 – only once every short team is filled does `commitFilledRotation` (`gameService.ts:4050` → `_persistRotation :4107`, write at ~4187) `.persist` the replacement
- Current behaviour: If the app is killed or crashes between write 1 and write 2, the departing player is permanently off the roster, the replacement was never written anywhere (it only ever existed in memory), and on relaunch `fillFlowRef / fillRequest` both reset to null – the picker does not reappear, and nothing re-detects the shortfall automatically
- Problem: `roundBusy / markBusy` (`AdvancedLiveMatchScreen.tsx:395-403`) is raised only immediately before the final `commitFilledRotation` call (line 519) – **not** while the picker is open waiting on the admin, i.e. not during the actual window a kill would lose everything. There is no minimum-roster-size validation anywhere in the stack: `commitRoundStats` (

functions/src/index.ts:12753) only caps side size at MAX_SIDE = 11 (batch op-count safety, ~line 12849) – nothing checks . A.length === B.length or == perTeam . A round committed 4vs after an interrupted substitution commits silently, with no error

User impact: A team plays shorthanded for the rest of the evening with zero indication anything is wrong beyond a visually smaller

- .movePlayerToTeam / swapPlayers team card; the admin has to notice and manually fix it via
- Technical impact: A deliberate cancel of the picker (X / Android back) correctly calls nudgeRotationAfterFillCancel (
- .gameService.ts:4488–4499) to keep the round-commit idempotency key fresh – a cold kill bypasses that cleanup entirely
- Recommendation: Persist the fill-in-progress state (e.g. a draftTeams.pendingFill field) so a relaunch can resume or explicitly
- .commitRoundStats flag the interrupted substitution, and add a minimum-per-side check to
- .Expected benefit: Turns a silent, admin-must-notice roster gap into a recoverable, visible one
- Effort: M

F-RELIABILITY-5 – Every roster-shape write on the live-match screen is a stale-read-then-whole-object updateDoc – two admins editing at once produces a silent last-write-wins clobber, repeated across seven separate functions

Severity: P1

Confidence: Confirmed

Feature / Screen / Flow: Advanced live match – team swaps, “went home,” rotation commit, and shootout controls, all admin-reachable at any time with no exclusivity lock between admins

- Evidence – the same pattern, four separate places

Rotation/teams: _persistRotation (gameService.ts:4107–4187 , the single funnel for every round transition and substitution-fill commit) reads via a plain gameId() and writes the **whole** rotation + draftTeams objects back via bare updateGameDoc – no transaction, no version check. A stale-by-one-round admin (round N→N+1 hasn’t reached their listener yet) performing any rotation-touching action off that stale snapshot silently reverts the other admin’s round transition on the live view, even though round N+1’s stats are already durably committed server-side – a visible state/data mismatch (live screen shows old teams; roundHistory .has already moved on) with no error and no automatic recovery

Substitution: swapPlayers (gameService.ts:4198–4224), movePlayerToTeam , markPlayerWentHome all share the identical shape – stale read → mutate the entire draftTeams.teams[] in memory → full-object updateDoc . Two admins swapping *different* player pairs at the same moment: whichever write lands last **wins outright**, silently discarding the other admin’s swap – not merged, .no error to the losing admin (their own call succeeded locally)

Shootout start/undo/clear are destructive overwrites, unlike its own kick-recording (which correctly uses arrayUnion and is safe): startShootout (gameService.ts:3698–3712) unconditionally writes a fresh empty liveMatch.shootout object with **no check that a shootout is already running** – a stale-viewing second admin tapping “start shootout” wipes every kick and both keeper assignments already recorded. undoLastShootoutKick (gameService.ts:3764–3792) reads the kicks array and writes kicks.slice(0,-1) as a **full-array replacement** (not an id-based arrayRemove) – if this lands after a concurrent admin’s arrayUnion’d new kick, both the intended-to-be-undone kick *and* the new one are destroyed together. clearShootout (gameService.ts:3802–3815) unconditionally deletes the whole field, abandoning a shootout the other admin is still mid-recording

Contrast – this exact bug class was already fixed once, for goals: removeGoal / undoLastGoal (gameService.ts:3600–3618) explicitly use arrayRemove + increment(-1) instead of a whole-array overwrite, with a code comment stating this replaced “the old read-modify-write, which overwrote the whole goals array and could drop a concurrent goal.” The fix was never generalized to . liveMatch.shootout / draftTeams / rotation

Current behaviour: firestore.rules:1097–1160 (the organizer/admin write branch) confirms there is no field-level diff constraint on rotation / liveMatch / draftTeams – any admin can overwrite the whole thing, so nothing at the rules layer backstops this .either

User impact: Whichever of two admins acts second silently wins; the other’s substitution, shootout correction, or round transition .vanishes with no error on either device

Recommendation: Route all of these through a Firestore transaction that re-validates the specific sub-state being mutated hasn’t moved since the read (mirroring the arrayUnion / arrayRemove pattern already proven for goals); at minimum, make . startShootout a no-op if a shootout already exists

Expected benefit: Removes the app's remaining last-write-wins hotspots in the live-match write surface – the same fix shape the .team already validated works for goals
Effort: M •

F-RELIABILITY-6 – `recordGoal / removeGoal` mix a `retry-safe` `arrayUnion` with a `retry-unsafe` `increment()` in the same write – an SDK retry after a lost ack double-applies the score

Severity: P2 •

Confidence: Confirmed •

Feature / Screen / Flow: Live scoreboard – adding/removing a goal •

Evidence: `src/services/gameService.ts:3550–3557` (`recordGoal`): `'liveMatch.goals': arrayUnion(goal)` alongside `'liveMatch.scoreA'/'scoreB': increment(...)` and `'liveMatch.goalTally.<id>': increment(1)` in the same `updateDoc`. `removeGoal` (~3608-3616) is the mirror. `arrayUnion` naturally dedupes an exact-object retry; `increment()` has no such protection – a retry of the same call after a lost ack double-applies the score/tally even though the goal array itself stays correct. This is the .same shape of bug `roundInstanceId` was built specifically to fix for round-commits, not applied here .Current behaviour: Score/tally can silently drift ahead of the actual goal-log entry count on a lost-ack retry • .User impact: Scoreboard shows a score inconsistent with the visible goal log (e.g. 2-0 with only one goal listed) • Recommendation: Derive `scoreA / scoreB` from `goals.length` client-side for display, or move goal recording through a .transaction •

Expected benefit: Removes a genuinely new (previously undocumented) instance of the exact bug class the team already fixed .once for round-commits
Effort: S •

F-RELIABILITY-7 – Zero offline/connectivity indicator anywhere in the app

Severity: P2 •

Confidence: Confirmed •

Feature / Screen / Flow: App-wide, most relevant to live match •

Evidence: Exhaustive grep across `src/` for `NetInfo`, `isConnected`, `hasPendingWrites`, `enableNetwork / disableNetwork`, `fromCache` returns zero hits; `@react-native-community/netinfo` is not a dependency. None of the three live-match `onSnapshot` listeners (`subscribeLiveMatch / subscribeRotation / subscribeLiveGame`, `gameService.ts` ~7521-7609) check `snap.metadata.fromCache / hasPendingWrites`. The only network-related string in the whole app (`src/i18n/he.ts:1181`, `.signInNetworkError`) is scoped to sign-in only •

Current behaviour: An admin on a pitch with poor reception gets no signal – no banner, no icon, no “pending sync” indicator – that .their taps are being queued locally rather than reaching other players' devices/the server •

Problem: This is the root enabler behind F-RELIABILITY-2 and F-RELIABILITY-5: those bugs are invisible specifically because the .user has no way to know they're offline or racing another device •

Recommendation: Add a lightweight `NetInfo`-driven banner on the live-match screen at minimum .כשיהיה חיבור ("במצב לא מקוון – הפעולות יסתנכרו") •

Expected benefit: Turns several silent failure modes into visible, explainable ones •

Effort: S •

F-RELIABILITY-8 – `roundHistory` write is deliberately best-effort and outside the atomic batch; its failure is never surfaced to the user

Severity: P2 •

Confidence: Confirmed •

Feature / Screen / Flow: `commitRoundStats` server callable – the round-by-round recap ledger •

Evidence: `functions/src/index.ts:13497–13526`. The stats batch commits first and is fully idempotent. Only **after** that succeeds • does the code attempt `.../roundHistory/{roundId}.set(roundHistoryDoc)` inside its own `try/catch` (~13517-13526) – on failure it only does `console.warn(...)`. The callable's return value (`{ ok: true, scorers }`) carries no flag for this, and the .client (`_commitRoundStatsAndClear`) treats any non-throwing response as full success •

- Current behaviour: If this specific write fails (offline blip right after the stats batch succeeded), the round's `roundHistory` doc – the .full goal-by-goal recap `MatchRoundsScreen` reads – is silently missing for that one round
- Problem: This also silently starves `rollUpClubPairs` 's once-per-evening pair rollup, which reads `roundHistory` (not the stats .batch) for its sameTeam/together-win counts (per D2 discovery) – a missing doc undercounts club chemistry stats too, invisibly
- User impact: "Match rounds" history for that evening shows a gap for exactly one mini-game; chemistry numbers quietly .undercount. Stats themselves (goals/assists/wins) are NOT lost – only the recap/rollup layer
- Recommendation: Return a `roundHistorySaved: boolean` from the callable and have the client retry that one write (idempotent .via the fixed `roundHistoryDoc.roundId`), or surface a small "recap incomplete for this round" indicator
- .Expected benefit: Converts an invisible gap into a recoverable, bounded one
- Effort: S

F-RELIABILITY-9 – Once a round is committed, a wrong winner, a wrong own-goal attribution, or a wrongly-credited live goal cannot be corrected – only new goals can be added

- Severity: P2
- Confidence: Confirmed
- Feature / Screen / Flow: Post-commit correction tooling
- Evidence: Exhaustive grep for reopen/revert/uncommit/edit-round patterns across `functions/src/index.ts` and `src/services/gameService.ts` returns zero hits. `addRetroGoal` 's callable signature (`functions/src/index.ts:13761–13770` : `gameId, scorerId, assisterId, retroGoalId`) has **no ownGoal parameter at all** – unlike `commitRoundStats` 's live goal payload, which does support `ownGoal` (~12781) – so a wrong own-goal attribution made during the live commit has no post-hoc fix path, ever. `removeRetroGoal` (`functions/src/index.ts:13843`) can only undo a *retro-added* goal (operates on the `retroGoals` marker doc), never a goal that was part of the original live commit. `MatchRoundsScreen.tsx` (round history) is read-only. .`endEvening` only transitions forward; nothing writes `finished` back to an earlier status
- What IS correctable, for contrast: a missed goal (`addRetroGoal` , finished games only); an accidentally-removed player (`removePlayer` + `adminAddPlayers` , works throughout a live evening, `functions/src/index.ts:7697`); a substitution done in) error (`swapPlayers` / `movePlayerToTeam` are trivially self-reversing pre-commit); a mis-recorded shootout kick, pre-commit only .(`clearShootout` / `undoLastShootoutKick`)
- Problem: The correction surface stops exactly at "the round was already committed" – which, combined with F-RELIABILITY-1/2/5 above (all of which can produce a wrongly-committed or incompletely-committed round), means several of this audit's own failure .modes have no recovery path once discovered
- .User impact: A club's permanent lifetime stats (goals, own-goals, win/loss record) can carry a wrong entry forever once committed
- Recommendation: At minimum, extend `addRetroGoal` /a sibling callable to accept `ownGoal` and a negative-delta "remove-credit" .mode scoped to the committing admin's own recent commits (audit-logged, time-boxed)
- .Expected benefit: Closes the "no recovery" tail on every silent-corruption finding above
- Effort: M

F-RELIABILITY-10 – `addRetroGoal` 's idempotency key is minted fresh per button-press – safe against a double-tap, not safe against a manual retry after a timeout

- Severity: P2
- Confidence: Confirmed
- Feature / Screen / Flow: Post-match retro goal correction
- Evidence: `src/components/match/RetroGoalsSheet.tsx:35–37` (`genRetroId`) : ``retro-${Date.now()}-${Math.random().toString(36).slice(2,8)}'`` , called inline at the call site (:88). The `savingRef` latch (:62 , `useRef`) correctly blocks a same-tick double-tap. But on a genuine failure (timeout, dropped ack) the `catch` (:112–115) shows `toast.error(he.retroActionFailed)` and resets the guard – a retry after that toast mints a **brand-new** `retroGoalId` , defeating the server's `batch.create()` dedupe (`functions/src/index.ts:13792–13793`) and genuinely double-crediting the .goal
- Problem: This is precisely the class of bug `roundInstanceId` was built to fix for round-commits (a key stable across a retry, not just .a same-tick double-press) – the lesson wasn't carried over here

- .User impact: A retro-added goal can be silently double-credited to a player's lifetime stats after one failed-looking retry
- Recommendation: Derive `retroGoalId` deterministically from `(gameId, scorerId, assisterId, roundedTimestampBucket)`, or
- .persist the minted id in component state so a retry reuses it
- Expected benefit: Extends the round-commit retry-safety fix to the one other manually-triggered stat-crediting flow sharing its risk
- .shape
- Effort: S

F-RELIABILITY-11 – Push-notification dispatch triggers have no true redelivery dedup, only a TOCTOU-vulnerable `delivered` flag and a flood throttle

- Severity: P3
- Confidence: Confirmed
- Feature / Screen / Flow: Every push that flows through the shared dispatcher (game reminders, spot-offered, guest-promoted, evening-summary, filling-up, and more) plus founder/admin alerts
- Evidence: `onNotificationCreated` (`functions/src/index.ts:1773-1886`) guards only with `if (notif.delivered) return;` at entry (`:1779`), then does Firestore reads (`canonicaliseNotificationPayload`, `resolveRecipients`) and the actual FCM `sendEachForMulticast` call (`deliverBatch`, `:1868`) – realistically hundreds of ms to seconds – before finally writing `delivered: true`. A Cloud Functions redelivery of the same event while the first invocation is still mid-flight (a documented at-least-once scenario) sends every recipient the push twice; `deliverBatch` itself has no event-id dedup. Separately, `onNewUserJoined` / `onGameCreatedAlert` / `onGameJoinedAlert` / `onCommunityCreatedAlert` / `onCommunityJoinedAlert` / `onAvailabilityUpdated` / `onErrorLogged` / `onFeedbackSubmitted` (`functions/src/index.ts:11935-12484`) all call `pushToAdmins` directly, bypassing the dedup dispatcher entirely – their only protection is a 20-second flood-guard throttle on `.adminPushLatches/{type}` (`adminPush.ts:33-51`), which a redelivery landing later than 20s sails past
- .Current behaviour: Push spam (a duplicate “your game is starting” or a duplicate founder “X joined!” alert), not data corruption
- User impact: Low-severity annoyance rather than a live-evening-critical failure; noted for completeness since the brief specifically
- .asks about trigger redelivery
- Recommendation: Move the `delivered` claim earlier (a transactional claim-before-send, matching the pattern already used
- .correctly elsewhere in the same file, e.g. `campaigns.status` claim in `processCampaign`)
- Effort: S-M

F-RELIABILITY-12 – Soft-update nudge is stale relative to the live store versions; the hard force-update floor is correctly armed

- Severity: P3
- Confidence: Confirmed (live Firestore read, 2026-09-02)
- Feature / Screen / Flow: App-launch update gate
- Evidence: `appConfig/android` and `appConfig/ios` (read live via REST) both show `latestVersion: "1.1.0"`, `minimumSupportedVersion: "1.1.0"` (updateTime 2026-08-29T17:10:50Z). Per the audit brief, the stores are actually serving Android 1.1.1 and iOS 1.1.2. `src/services/updateService.ts:44-84` + `App.tsx:582-583,961-975` correctly render a genuinely-`. 'force' blocking <Modal>` (`src/components/UpdateModal.tsx`, `onRequestClose={() => {}}`, no dismiss) for
- Current behaviour: `minimumSupportedVersion` at 1.1.0 does correctly hard-block anything older – a genuinely stale client on a live
- evening IS blocked by the floor. But `latestVersion` is one to two versions behind, so every device sitting on exactly 1.1.0 gets no
- .soft nudge toward the newer builds – the same “soft popup dead” recurring pattern flagged repeatedly in project memory
- .Recommendation: Re-run/verify the update-popup watcher; bump `latestVersion` to match the currently-live store builds
- .Expected benefit: Restores the soft nudge; the hard floor already works and needs no change
- Effort: S (config-only)

F-RELIABILITY-13 – Two small, cheap UI-consistency gaps

- Severity: P3
- Confidence: Confirmed
- Feature / Screen / Flow: Advanced live match + MatchDetails

- Evidence: (a) `commitFilledRotation`'s failure path (`AdvancedLiveMatchScreen.tsx` ~528-537, inside `advanceFillFlow`) only •
does `logError / console.warn` – no `toast.error(...)` – unlike the four sibling catches for `recordWinner /`
`prepareRoundResult` (~372-375, 675-678, 719-722, 811-814, 852-855), which all call `toast.error(he.roundFinalizeFailed)` . (b) •
button (`MatchDetailsScreen.tsx:3485–3494`) calls the same cancel path as the sticky CTA “בטל הרשמה” The secondary plain-text •
.(:3520–3523) `disabled={busy}` but lacks that CTA's •
Current behaviour: (a) the admin gets zero feedback if a round-transition/substitution commit fails – spinner clears, modal closes, •
nothing looks wrong. (b) a rapid double-tap on the secondary button isn't caught client-side, though `cancelGameV2`'s server-side •
.transaction is naturally idempotent and cancellation is gated behind a confirm dialog, so real-world risk is low •
.Recommendation: Add the same `toast.error(he.roundFinalizeFailed)` to (a); add `disabled={busy}` to (b) •
Effort: S •
-

Areas checked and found genuinely solid (no padding – stated once, moving on)

- **s idempotency-key design** (`roundInstanceId` , minted once per mini-game in `commitRoundStats` •
`rotationEngine.ts:476–489`) is a well-reasoned fix for the *lost-response-retry* case specifically, with the exact prior bug •
documented in the code. Its gap is scoped precisely to the *concurrent-different-payload* case (F-RELIABILITY-1), not a flaw in the •
.retry design itself
- **Join / cancel** (`joinGameV2` , `cancelGameV2` , `requestJoinGame`): wrapped in `runTransaction` with an idempotency re-check •
.against fresh in-transaction reads; retries after a lost response are clean no-ops
- **Timer controls** (`startTimer` / `pauseTimer`): field-path-scoped writes, each re-checks current state before writing and no-ops if •
already in the target state – a concurrent “second admin presses play” is a clean no-op (the timer's own field-path writes only collide •
with `resetTimer`'s literal-overwrite of the score/goal fields, covered under F-RELIABILITY-5's shootout/goal analogues, not the •
.timer fields themselves)
- **Double-tap guards on the hottest path:** `AdvancedLiveMatchScreen`'s round-end/commit flow (`finalizingRef` / `committingRef` •
/ `busyCountRef` , lines 383-402) is the single most hardened guard in the app. `LiveScoreboardCard` , `Shootout.tsx` , and •
. `RetroGoalsSheet` all use synchronous `useRef` latches for the same-tick case
- **Stats-crediting trigger redelivery** (as opposed to push-dispatch triggers, see F-RELIABILITY-11): `finishCredited/once` , •
`joinCredited/{uid}` , `committedRounds/{roundId}` , `communityPairRollups` , `roundSummaries` create-once, •
`ratings.appliedEvents[]` , `memberCredited/{uid}` are all guarded with `batch.create()` /transactional re-check patterns •
.correctly designed around Firestore's at-least-once delivery guarantee
- **Goal-log concurrency:** `removeGoal` / `undoLastGoal` correctly use `arrayRemove + increment(-1)` specifically to avoid the whole- •
array-overwrite bug class – good evidence the team already identified and fixed this pattern once; it just wasn't generalized (see F- •
.RELIABILITY-5)

Notable dead-code landmine (not a live bug, flagged because it already fooled someone once)

- is dead – nothing calls it; the real path is (`gameService.ts:3655–3679`) `gameService.finalizeRoundAndRotate` •
`prepareRoundResult` + `commitFilledRotation` . Its own doc-comment says so explicitly and adds: “Read it and you will conclude •
two things that are false today... I misread it once on 2026-08-29; hence this note.” `stopRotation` (`gameService.ts:4071`) similarly •
has zero call sites from any screen – its last-write-wins risk is latent, not live, today. Recommend deleting both in a future cleanup pass •
.rather than leaving documented traps in place
-

Teamder – adversarial-player audit (stat gaming)

- ,Persona: a real club member who wants the best numbers next to their name using only actions reachable through the app's own UI/callables (no DB hacking). Static analysis only – no runtime available. All “how it looks to others” claims are marked SUSPECTED
- .Ranked by ease × payoff × invisibility, highest combined threat first

F-GAME-1 – Quick games farm GLOBAL profile stats with zero guardrail

- Severity: Po
- Confidence: Confirmed (read in code)
- `commitRoundStats` → Feature / Screen / Flow: Quick game (`isOrphanContext` /personal hidden group) → live match
- Evidence: `functions/src/index.ts:12753` (`commitRoundStats`) has no `isPersonal` / `isOrphanContext` check anywhere in its body – contrast with `addRetroGoal` 's explicit guard at `functions/src/index.ts:13743` (`if (!grp || grp.isPersonal === true) throw ...`). The goals/wins/assists/cleanSheets/ownGoals/penalty writes to `db.collection('users').doc(uid)` (e.g. lines ~13052, 13233, 13312, 13396) run unconditionally regardless of `groupId` . `src/screens/tabs/ProfileScreen.tsx:558–560` reads `localUser.stats.goals/assists/cleanSheets` straight off that same `users/{uid}.stats` doc for the profile card everyone sees. `src/services/eveningSummaryService.ts:375` (`clubRanked = game?.isOrphanContext !== true`) only suppresses the club **rank/position**, not the underlying score, goals, wins, or the lifetime `users.stats` write
- Current behaviour: A quick game (any player, no club needed) runs through the exact same `commitRoundStats` path as a real club .game and writes to the exact same lifetime stat store
- Problem: A player creates a personal quick game, adds themselves plus zero or guest opponents, and runs any number of fast mini-games via the live-match UI, crediting themselves every goal/assist/win. Every one of those numbers lands in `users/{uid}.stats` , which is the same doc the public profile card, the evening-score benchmarks, and the achievements `deriveCounters` (`goals / .wins / assists` inputs, `src/services/achievementsService.ts:172–187`) all read
- User impact: A player's profile card (visible to every teammate) can show an arbitrarily large career goals/assists/wins/clean-sheets .total that has nothing to do with any real club evening
- Technical impact: Pollutes the one cross-club “career” number the app treats as ground truth; no separation between real-club-earned and orphan-game-earned lifetime stats
- Recommendation: Gate the `users/{uid}.stats` (and `ownGoals` /pen stats) writes in `commitRoundStats` on `groupId` belonging to a non-personal group, mirroring the guard `addRetroGoal` already has. Community-scoped writes (`communityPlayerStats`) are .naturally safe since a personal group never appears in any club leaderboard
- Expected benefit: Removes the highest-payoff, easiest, most-invisible exploit in the app – closing it needs one added condition, no .schema change
- Effort: S

F-GAME-2 – Admin can credit themselves unlimited goals/assists/wins in a real game, no dispute path

- Severity: P1
- Confidence: Confirmed (code + code's own comment)
- `commitRoundStats` → Feature / Screen / Flow: Live match round entry
- Evidence: `functions/src/index.ts:12816` – the code's own comment: *“Faking your OWN stats inside your own real game is still possible; touching a non-participant's numbers is not.”* Authorization is `game.createdBy === uid` OR `adminIds.includes(uid)`

(lines ~12800-12805); the only content guard is roster membership (`roster.has(id)`), not plausibility. `winnerSide` , `goals[]` .(`scorer/assister`), and `penalties[]` are entirely admin-supplied per round

Current behaviour: Any game creator/community admin who plays can, every mini-game, name themselves `scorer/assister` and pick `winnerSide` their own team as

Problem: No independent confirmation, no score-vs-goal-count cross-check, no notification to other participants, no report/challenge mechanism found anywhere in the codebase (searched for `dispute/report/challenge/flag` patterns tied to round stats – none exist beyond the general `feedback` inbox, which is unstructured)

User impact: A self-interested admin can quietly out-score/out-win the evening every time they run the table

Technical impact: `committedRounds/{roundId}` only latches idempotency (`committedAt/by/winnerSide`), not a goal-by-goal audit visible to non-admins; the `roundHistory` doc IS written with the full goal log (visible via `MatchRoundsScreen`) so a suspicious teammate COULD notice, but nothing surfaces an anomaly proactively

Recommendation: For clubs with >1 admin, consider requiring a second admin ack for self-credited goals, or at minimum surface a “goals entered by [admin]” badge on the round recap so self-crediting isn’t silent

Expected benefit: Turns a silent trust hole into at least a visible one

Effort: M

F-GAME-3 – `addRetroGoal` has no cap, no time window, no plausibility check

Severity: P1

Confidence: Confirmed

`addRetroGoal` → Feature / Screen / Flow: Post-game admin correction

Evidence: `functions/src/index.ts:13761-13816` . Only gates: caller is admin (`loadRetroGameContext`), `game status === 'finished'` , `scorer/assister` on `game.players` . No limit on how many retro goals per game, no limit on how long after the game it can be called, and each call just needs a fresh client-generated `retroGoalId` to avoid the idempotency collision – so nothing stops calling it N times with N different ids

Current behaviour: Retro goals are meant as a one-off “we forgot to log a goal” correction

Problem: An admin can call `addRetroGoal` an unbounded number of times, any time after the game ends, crediting themselves (or an ally) any number of extra goals/assists with zero relationship to the actual final score. The four stores it touches (`users.stats` , `communityPlayerStats` , `gamePlayerStats` , `communityStats`) are the same ones `commitRoundStats` writes, so this is functionally an unmetered “add goals to anyone on the roster” lever

User impact: Retroactive, silent stat inflation days/weeks after the fact – nothing on the live recap warns anyone a goal was added after the game closed (no visible “retro” marker found in the UI beyond the raw goal in match history, unconfirmed at runtime)

Technical impact: `retroGoals/{retroGoalId}` docs exist for forensics but nothing reads/surfaces them as an audit trail to non-admins or flags an unusual count

Recommendation: Cap retro goals per game (e.g. $\leq$ (score discrepancy observed) or a small fixed number), and/or require retro goals to be filed within a short window of the game finishing, and surface a “corrected after the fact” tag on the recap

Expected benefit: Converts an unbounded lever into a bounded, auditable one

Effort: S-M

F-GAME-4 – Stack the opposing side with guests: a win that costs no real player a loss

Severity: P1

Confidence: Confirmed

`commitRoundStats` → Feature / Screen / Flow: Team assignment

Evidence: `functions/src/index.ts` – `A / B` (the stat-crediting arrays) are built with `isReal(id) && roster.has(id)` (~line 12832), so guest ids never enter them. `roundLosers = winnerSide === 'A' ? B : A` (~line 13383) and the `tallyResult /lifetime-wins` loop only ever iterate `A / B` . Guests get a **separate**, per-game-only tally (`guests0nA / guests0nB` , ~line 13340) that never touches `communityPlayerStats` or `users.stats`

Current behaviour: If the losing side is composed entirely of guests, `B` (or `A`) is empty, so `roundLosers` is `[]` – no real account receives a loss, anywhere, ever

- Problem: A team that only wants wins can field an opposing side of guests (a completely normal, unremarkable roster choice on a short-numbers night) and guarantee every real player on the other side has a perfect win column for that round, with no real "counterpart absorbing the "L".
- User impact: Win% and win totals become disconnected from any real competitive result.
- Technical impact: The community `communityStats.rounds / goals` counters still increment normally, so the club table looks internally consistent even though no real loss was recorded.
- Recommendation: None strictly required for casual play (guests filling a short side is legitimate), but the club standings screen should treat "all-guest opposing side" rounds as informational rather than counting toward win-rate leaderboards, or at minimum surface guest-heavy rounds distinctly.
- Expected benefit: Prevents win% being inflated for free on short-numbers nights.
- Effort: M.

F-GAME-5 – Get subbed off a losing side before the round is committed → the loss (and the round) never happened

- Severity: P1.
- Confidence: Confirmed (ground truth + traced the exact client path).
- `_commitRoundStatsAndClear` → Feature / Screen / Flow: Live match rotation/loan.
- Evidence: `src/services/gameService.ts:3822–3860` – `sideA / sideB` are computed at commit time via `effectiveRosterOf(idx, draft.teams, rot.loans ?? [])`, i.e. the CURRENT team assignment, not who was on the pitch when the round was actually decided. Server-side, `commitRoundStats` then keys every round/win/loss/clean-sheet write off that same end-state `A / B` (ground truth, confirmed again by tracing `roundWinners / roundLosers / the rounds: inc(1)` loop at `.functions/src/index.ts:~13286–13330`).
- Current behaviour: A player moved to the bench (or swapped to loan) after the score is effectively decided but before "end round" is tapped is simply absent from both `A` and `B` – they receive no round played, no loss, no clean-sheet-denial, nothing.
- Problem: "Ask to be subbed off when losing, stay in when winning" fully protects rounds-played-weighted win rate and the evening score's `wins/gp` axis (50% weight, see F-GAME-6) with zero downside – no partial-loss, no "DNF" marker exists.
- User impact: A player who manages their own substitution timing (or has a cooperative admin/teammate) can maintain an artificially high win% indefinitely.
- Technical impact: `rounds` in `communityPlayerStats` is meant to mean "mini-games played"; this makes it silently mean "mini-games I was still assigned to at the exact commit instant".
- Recommendation: Snapshot the roster at ROUND START (or use a min-time-on-pitch threshold, similar to the parked $\geq 30s$ substitution-stats rule in `project_substitution_stats_rule.md`) rather than at commit time.
- Expected benefit: Rounds/wins/losses become tied to who actually played, not who happened to still be assigned at button-press time.
- Effort: M-L (needs per-round roster snapshot at start, a real data-model change).

F-GAME-6 – Evening score: floor of 6.0 + wins is a rate (not volume) rewards playing exactly one round and cherry-picking it

- Severity: P2.
- Confidence: Confirmed.
- `src/utils/eveningScore.ts` → "סיכום הערב": Feature / Screen / Flow.
- Evidence: `eveningScore()` – `if (gp <= 0) return 6.0` and, more importantly, the final score is always clamped to `[6, 10]` (`6 + (weighted/10)*4`) – there is NO way to score below 6.0 even with 0 goals/0 assists/0 wins across many rounds played (`weighted=0` → `score=6.0`, identical to not playing at all). Meanwhile `winsScore = clamp10((wins/gp)*10)` is a RATE, and `/ goalsFor10` `goalsScore / assistsScore` are EVENING TOTALS (not divided by `gp`) compared to a community benchmark `(assistsFor10`.
- Current behaviour: Wins is 50% (or 45% with a shootout) of the score and is diluted by every mini-game played that isn't a win; goals/assists are NOT diluted by `gp` at all.

- Problem: The dominant strategy is to play as few mini-games as possible and make sure the one(s) played are wins with at least one goal – e.g. 1 round won with 1 goal beats 5 rounds at 4-1 (win rate 80% < 100%) even though the second is a much better evening of football. There is also zero downside to a bad evening (floor 6.0), so there's no "risk" in staying on the pitch either way – the only lever that actually moves the score is selectively avoiding losses
- User impact: The evening score, which drives the headline title/narrative shown to the player (and feeds `rank / scoreRank` when `.club-ranked`), can be topped out by minimal, cherry-picked participation
- Technical impact: None (it's a scoring formula working exactly as designed) – this is a design gameability, not a bug
- Recommendation: Either weight wins by volume (e.g. blend rate with a games-played factor) or accept this is a "fun stat," not a competitive one, and say so explicitly in-app
- Expected benefit: A truer "how was your evening" signal; lower priority since the feature markets itself as a personal, not competitive, number
- Effort: M (formula change + re-tune weights/tests)

F-GAME-7 – "Deadly duo" chemistry is 100% admin-entered and trivially manufactured

- Severity: P2
- Confidence: Confirmed
- Feature / Screen / Flow: `pairStats / communityPairStats` ("deadly duo" fun fact) via `commitRoundStats`
- Evidence: `functions/src/index.ts:~13405-13425` – `assistPairs` is built purely from the admin-supplied `goals[]` payload (`scorerId / assisterId`), then written straight to `pairStats / communityPairStats` with `assistsAToB / assistsBToA: inc(1)`, no cross-check against who was actually on-field beyond simple `onField.has()` membership (itself admin-controlled via `sideA / sideB`)
- Current behaviour: Any admin entering goal data controls exactly which two accounts get credited with an assist pairing
- Problem: Repeating the same (scorer, assister) pair across several mini-games/evenings – trivial for a self-admin quick game (see F-GAME-1) or two cooperating real friends in a real club game – manufactures a "deadly duo" from zero organic chemistry
- User impact: A club fun-fact/leaderboard entry becomes fully fabricatable
- Technical impact: None beyond the trust assumption already covered by F-GAME-2
- Recommendation: Low priority – this is a "fun fact," not a competitive ranking; same trust model as goals/wins
- Expected benefit: Marginal; mention only for completeness
- Effort: N/A (accept as a known limitation of admin-entered data)

F-GAME-8 – Club level rewards raw "game nights" 10x more than the goals scored in them – SUSPECTED cheap to farm

- Severity: P3
- Confidence: Suspected – not verified at runtime; traced the metric source but not whether a zero/near-zero-roster game auto-finishes and still counts
- Feature / Screen / Flow: `src/utils/clubLevel.ts` (`clubPoints`) fed by `(gameNights: stats?.totalFinished ?? 0) src/screens/communities/CommunityStatsScreen.tsx:286`
- Evidence: `clubPoints(m) = gameNights*10 + clubGoals*1 + members*8 + ageMonths*6` – `gameNights` weighted 10x a single goal. Games appear to auto-transition to `status: 'finished'` on a schedule (`functions/src/index.ts:3744`, `gameDoc.ref.update({status:'finished', locked:true})`) rather than requiring any minimum roster or a `commitRoundStats` call
- Current behaviour: `computeClubLevel` counts every finished game in the club, regardless of how many people showed up or whether any mini-game was ever played
- Problem: If a finished game with no roster/rounds still counts toward `totalFinished`, a club admin who wants to bump the club's level number can create and abandon games (they auto-finish) far more cheaply than organizing real evenings with real goals
- User impact: tier badges could reflect calendar noise rather than real activity "מועדון מבוסס"/"אגדה"
- Technical impact: None besides the vanity metric
- Recommendation: Verify at runtime whether an empty/near-empty finished game is excluded from `totalFinished`; if not, gate `gameNights` on the game having at least one committed round

- .Expected benefit: Keeps club level tied to actual football played
- Effort: S (once confirmed)

What's fundamentally hard to game (trust these)

- **attendance-derived achievement counters** (`achievementsService.deriveCounters`) – sourced from real / `gamesJoined` `players[] + arrival != 'no_show'` on **finished games with a real roster**, not from a self-reported counter. Still inflatable via .repeated quick games (F-GAME-1's blast radius), but can't be inflated by lying about a single evening you didn't attend
- derived from group membership/ `invitedBy` graph structure, not stat entry; – `invitesSent` / `teamsJoined` / `teamsCreated` .requires actually creating accounts/relationships, which has real friction (an actual invited human)
- **The `committedRounds` and `retroGoals` idempotency latches themselves** – genuinely prevent double-counting of the SAME submitted event. The exploits above are all about the admin being free to submit fabricated events, not about breaking the anti-double-count machinery, which is solid

What should never be shown as competitive/leaderboard truth

- **Evening score** (F-GAME-6) – explicitly a “personal, self-based” number per its own doc comment; should not be the basis of any .M,” and even that inherits every issue above מיקום N מתוך inter-player leaderboard beyond the existing one-evening
- **Any per-player goal/assist/win total that mixes personal quick-game history with real-club history** (F-GAME-1) – until gated, .these are not trustworthy for cross-club comparison or the public profile card
- **.Deadly duo” / chemistry fun facts** (F-GAME-7) – fine as flavor text, should never be a ranked leaderboard”
- **Club level** (F-GAME-8, suspected) – should be understood as an activity/longevity badge, not a quality signal, pending the runtime .check above

ARCH

Architecture audit – Teamder

Static analysis only (no runtime available). All findings below are read directly from source unless marked SUSPECTED

F-ARCH-1 – Evening-score formula duplicated by hand, no parity test

- Severity: P1
- Confidence: Confirmed
- score + community rank movement (evening summary) “סיכום הערב” :Feature / Screen / Flow
- Evidence: `src/utils/eveningScore.ts:96` (`export function eveningScore`) vs `inline , const eveningScore = (...) => {...}`) `functions/src/index.ts:5306` inside `onGameRosterChanged`). The CF copy is a hand-transcribed reimplementaion with a comment at (positional args, not the `EveningScoreInput` object)
- *.()line 5296: “Keep in sync with `SCORE_WEIGHTS_` + `PENALTY_POINTS` + `eveningScore`
- No import – `functions/src` never imports from `src/utils` (verified: zero .(hits in `index.ts` from `'../../src'`
- Current behaviour: Both copies currently compute the identical weighted model (wins/goals/assists/penalties, same weights 0.5/0.3/0.2 and 10.0*6.0

, DEFAULT_ASSISTS_FOR_10=2 / DEFAULT_GOALS_FOR_10=4 same ,0.45/0.3/0.2/0.05
(BENCHMARK_FL00R=1 same

- Problem: Unlike balanceTeams , clubChemistry , roundSummary and the tie-confirm flow – each of which has a *generated* server copy , genClubChemistry.mjs , functions/scripts/genRoundSummary.mjs (tests/logic/*Parity.test.ts) plus a parity test (genTeamBalanceCore.mjs the evening-score formula has neither. It is free-hand code in a – .14k-line file with zero server-side tests (per repo ground truth)
- .User impact: A future tweak to weights/defaults in eveningScore.ts (e.g a balance pass on the 6~10 curve) silently stops matching the number – “actually stored in eveningStandings and shown as “where you rank tonight the two numbers a player sees (card score vs. rank) could disagree with no .build-time signal
- .Technical impact: Drift is invisible until a user notices; nothing fails CI
- Recommendation: Either (a) extend genRoundSummary.mjs ‘s pattern to emit from src/Utils/eveningScore.ts , or (b) add functions/src/eveningScore.ts a eveningScoreParity.test.ts that copy-pastes the CF literal into a fixture and asserts equality across a value matrix, mirroring . tests/logic/balanceParity.test.ts
- ;Expected benefit: Closes the biggest un-pinned duplicate found in the audit .reuses an existing, proven pattern (3 other formulas already do this)
- Effort: S

F-ARCH-2 – The “MIRRORED, keep in sync” pattern is systemic: 5 more hand-duplicated rules, none pinned

- Severity: P1
- Confidence: Confirmed
- Feature / Screen / Flow: clean sheets, penalty-shootout stats, fair join ordering, recurring-series scheduling, brand-impersonation guard
- :Evidence (each pair, with the file’s own “keep in sync” comment)

	Rule	Client	canonical	Server mirror	Comment
					--- --- --- ---
Clean sheet credit	src/Utils/cleanSheets.ts:50 cleanSheetCredits()			functions/src/index.ts:13279–13301 inline in	
	commitRoundStats cleanSheets.ts:12 “MIRRORED in commitRoundStats §1c-clean and in scripts/backfill_clean_sheets.py” – 3-				way, including a Python backfill script
Penalty-shootout aggregation	src/Utils/penaltyStats.ts:57 aggregatePenalties()			functions/src/index.ts §1c	
	penalty block (~13034-13169) penaltyStats.ts:2-6 “MIRRORED verbatim in commitRoundStats... they can’t share code – the cloud				“function is a separate build
Fair-join spot allocation	src/services/joinFairness.ts:79 assignJoins()			functions/src/index.ts:2429	
	reconcileGameJoins() index.ts:2424 “Mirrors the pure assignJoins... kept in sync by hand; the app side has the exhaustive unit				tests” – CF side has none
Recurring game-series scheduling	src/Utils/seriesSchedule.ts (next0ccurrenceAt , build0ccurrence etc., lines 35-216)				
	functions/src/index.ts:2958 runCreateSeries0ccurrences()			seriesSchedule.ts:8,138 “MIRRORED in ...	“runCreateSeries0ccurrences
Brand-impersonation guard	src/Utils/officialAccount.ts:17,30 TEAMDER_UID / RESERVED_NAME_RE				
	functions/src/chatPush.ts TEAMDER_UID , Pulse repo teamderChatService.ts , and firestore.rules:125 regex				officialAccount.ts:15-16,29-30 – 4-way mirror, one leg in a different repository

- two repos (Teamder + Pulse) and a security-relevant Firestore rule, so a drift there is both hardest to notice and highest-consequence (rules .regex silently diverging from the client's own list of blocked strings
- ;User impact: SUSPECTED – no user-visible symptom has been observed yet .each pair currently agrees. The risk is future edits, not current state
- Technical impact: Every one of these is a landmine for the next feature change; the codebase already shows the failure mode happened once for clean sheets (the in-file comment at index.ts:13277 explicitly documents a past regression where the CF path "used to drop the penalty axis... The .("consequence was invisible while the card recomputed its own score
- Recommendation: Triage by risk – brand guard and fair-join ordering first then clean sheets/penalties (stats ,security/fairness-relevant correctness). Reuse the existing `genRoundSummary.mjs` generator pattern where the server copy can be a pure function extraction; where it can't add a parity ,(e.g. `reconcileGameJoins` , which is transaction-shaped) test that imports the pure core and feeds it the same fixtures as the CF's .literal, the same way `tieConfirmParity.test.ts` currently does
- .Expected benefit: Turns 5 silent-drift risks into build-time-caught ones
- Effort: M (per rule, S; all five, M)

F-ARCH-3 – `functions/src/index.ts` is a 13,974-line, 64-export monolith with no test coverage

- Severity: P1
- Confidence: Confirmed
- Feature / Screen / Flow: entire Cloud Functions backend
- Evidence: `wc -l functions/src/index.ts` = 13974. `grep -c "^export const"` = 64
- top-level Cloud Functions. Section-header comments (`grep -n "^// -"`) show ,clearly delineated but co-located domains in one file/deploy unit 40~ e.g.: notifications (lines 246-1968), scheduled join-reconciliation recurring series cloning (2834-3410), community ,(2110-3410) publish-flip (3410-3487), stale-game cleanup (3487-3508), promotion-offer ,expiry (3508-3760), rating sync (6108-6249), auto-balance (6249-6760) community showcase/SSR (9113-9986), cross-community filler matching geocoding (10131-10520), friendships (11672-11930), founder ,(9986-10520) `commitRoundStats` / alerts (11930-12194), advanced round-stat aggregation physical/wearable stats (13534-13663), pitch calibration ,(12733-13534) retro goals (13709-13908). `functions/` has **no test** ,(13663-13709) **directory** (per repo ground truth) – this entire surface area is .unit-test-free
- Problem: One file is the unit of code review, blame history, and (given no per-function file split) mental model for the whole backend. A change to geocoding sits in the same diff-review surface as a change to the highest-stakes function in the app, per the `commitRoundStats` .(idempotency-key and roster-gating history in project memory
- User impact: Indirect – higher chance of an unrelated regression riding along with a targeted change, because reviewers scroll past 13k lines to .find the diff context
- Technical impact: Every deploy pushes the *entire* Functions bundle / regardless of which of the 64 functions actually changed; cold-start .bundle-parse cost scales with the whole file for every invocation

Recommendation: Split along the existing section-header seams into •

functions/src/{notifications,scheduling,seriesRecurrence,filler,geocoding, showcase,friendships,founderAlerts,roundStats,physical,pitchCalibration, retroGoals}.ts

re-exported from a slim index.ts . The section headers ,

,already give a ready-made module boundary – this is a mechanical split

.not a design exercise

Expected benefit: Smaller review surfaces, room to add per-module tests •

faster iteration on ,(closing the “no functions/ tests” gap incrementally)

.any one domain

Effort: L •

F-ARCH-4 – src/services/gameService.ts is an 8,192-line single object literal, ~65 methods, one export

Severity: P1 •

Confidence: Confirmed •

operation in the app מחזור/Feature / Screen / Flow: every game •

Evidence: wc -l src/services/gameService.ts = 8192. The entire file is •

effectively one export: export const gameService = { ... } opens at

line 524 and closes at line 8107 (7,583 lines in one object literal). A

name scan inside that span finds ~65 distinct methods spanning creation

, adminReorderRoster , removeGuest , addGuest) roster ,(createGameV2)

, startRotation) live match ,(movePlayerToTeam , swapPlayers

stats ,(recordShootoutKick , startShootout , pauseTimer , recordGoal

, getCommunityDeadlyDuo , getCommunityChampionship) reads

retro corrections ,(getRoundHistory , getPairStats

and 4 realtime subscriptions ,(removeRetroGoal / addRetroGoal)

, subscribeRotation , subscribeLiveMatch , subscribeMyLiveOrUpcomingGames)

,By comparison the next-largest service .(subscribeLiveGame

is 2,194 lines – gameService is 3.7x that , groupService.ts

Problem: One object literal means no per-domain module boundary, no •

independent import cost, and any consumer that imports gameService for

one method (e.g. a screen that only calls getGameById) pulls in the

.live-match/shootout/rotation/stats code paths too

.User impact: None directly; this is a maintainability/velocity cost •

Technical impact: Natural split seams are visible from the method names •

,themselves: (1) lifecycle – create/update/cancel/delete/join/leave/waitlist

– live match – timer/rotation/shootout/goals, (3) roster admin (2)

/guests/reorder/swap/move, (4) stats/history reads – championship/pairStats

deadlyDuo/roundHistory, (5) subscriptions. These map closely to the

screens that consume them (MatchDetailsScreen vs LiveMatchScreen vs

CommunityStatsScreen), so the split is discoverable from call sites, not

.invented from scratch

Recommendation: Extract to src/services/game/{lifecycle, liveMatch, rosterAdmin, stats, subscriptions}.ts , keep •

gameService as a thin

.re-export barrel so call sites (gameService.xxx(...)) don't change

;Expected benefit: Smaller, independently reviewable/testable modules •

.tree-shakeable imports; lower cognitive load per change

Effort: L •

F-ARCH-5 – `draftTeams` admin edits are read-modify-write, not transactional: two admins racing can silently clobber each other

Severity: P1 •

Confidence: Confirmed (code); runtime race SUSPECTED – not verified at runtime •

swap/move – “נהל כוחות”) Feature / Screen / Flow: manual team editing
(players between drafted teams before a game starts

Evidence: `src/services/gameService.ts:4199` `swapPlayers()` and
`const g = await this.getGameById(gameId)` both do `movePlayerToTeam()` :4249
`await updateGameDoc(gameId, patch)` → `g.draftTeams` mutate a local copy of →
with `patch.draftTeams` as the **entire replaced object**. Neither uses
`await this.getGameById(gameId)`) The same pattern `.runTransaction`
followed by a plain `updateGameDoc`) recurs at 15 call sites in the file
,lines 3202, 3431, 3454, 3472, 3661, 3937, 3966, 4013, 4073, 4201, 4255)
i.e. it's the file's standard idiom for – (4490 ,4389 ,4337 ,4309
.roster/team mutation, not an isolated shortcut

Current behaviour: Admin A calls `swapPlayers(a, b)` , admin B calls
within the same read-window; whichever `movePlayerToTeam(c, teamIdx)`
`draftTeams` commits last wins and overwrites the other's `updateGameDoc`
wholesale, because both read the pre-edit state and write back a full
.replacement

Problem: No optimistic-concurrency guard (no `updatedAt` precondition, no
transaction) protects the multi-admin case. Group admin lists in this app
are typically small but not always singleton (per `adminIds` arrays seen
elsewhere in the codebase), so two admins tapping “swap” near-simultaneously
.during pre-game team setup is a plausible real scenario, not a contrived one

User impact: SUSPECTED – a swap one admin just made could silently vanish
with no error shown to either admin, surfacing only when the roster looks
.wrong on the live-match screen

Technical impact: Silent data loss with no audit trail (no error is
(thrown; the losing write simply succeeds against stale data

Recommendation: Wrap `draftTeams` mutations in `runTransaction` , re-reading
inside the transaction and re-deriving the swap/move from `draftTeams`
fresh data rather than a pre-transaction snapshot – the same pattern
already used correctly elsewhere in this file (18 `runTransaction` call
.sites exist, so the pattern is established, just not applied here

Expected benefit: Eliminates a real (if narrow) data-loss window during
.team setup

Effort: M •

F-ARCH-6 – Optimistic local splice vs. realtime listener: an ordering-dependent race, already patched multiple times

Severity: P2 •

Confidence: Confirmed (code + comment history); runtime behaviour SUSPECTED •

Feature / Screen / Flow: `MatchDetailsScreen` join/cancel/spot-offer flow •

Evidence: `src/screens/games/MatchDetailsScreen.tsx` splices `game` state
`confirmSpotOffer` / `requestJoinGame` / `cancelGameV2` locally after calls to
racing against the `useGameEvents` realtime ,(lines ~950-1010, ~1467-1483)
listener (`src/services/useGameEvents.ts`) which can push a fresher/staler
snapshot over the same state. The code is self-aware of this: comment at

line 1467 “we splice optimistically and the realtime listener confirms and at line 1475 “an in-flight snapshot from useGameEvents would otherwise clobber a pre-await optimistic update back to the pre-accept state (CTA flickered back, user report)” – i.e. this exact race has already produced at least one shipped bug, fixed by moving the splice to *after* the await rather than before it

- Problem: The fix is a timing heuristic (splice after commit, not before) not a structural guard (no version/sequence number on the local state to reject a stale listener update, or vice versa). It works for the cases already hand-tuned, but the pattern isn't reusable – the next optimistic action added to this screen has to rediscover the same ordering constraint
- User impact: SUSPECTED – CTA/roster flicker under specific network timing the comments indicate this has already happened once for the spot-offer flow before being fixed

Technical impact: Fragile-by-construction; correctness depends on JS microtask ordering between a promise resolution and a Firestore listener callback, which is not something the type system or a test enforces

Recommendation: Introduce a small “pending local edit” guard – e.g. a ref holding {field, expectedValue, expiresAt} – that the listener's checks before overwriting a field the screen just optimistically onUpdate set, rather than relying on await-ordering. This generalizes the fix instead of re-deriving it per action

- Expected benefit: Removes a class of bug rather than the two instances already patched
- Effort: M

F-ARCH-7 – Error handling has no shared convention: 500 catch blocks, 27 fully empty, ~119 log-and-swallow

Severity: P2

Confidence: Confirmed

Feature / Screen / Flow: app-wide

- Evidence: `grep -c "}" catch" src/**/*.{ts,tsx} (excluding tests) = 500`
fully empty catch bodies (all in 27 = `grep -n "catch\s*(\[^\])*)\s*\{s*}"`
`, ProfileLocationMap.tsx , MapWebView.tsx` :map-styling components
`, FilterRadiusMap.tsx , RadiusMapModal.tsx , AvailabilityRadiusMap.tsx`
low individual risk since – `HtmlMessageView.tsx , LocationPickerMap.tsx`
they guard optional map-style calls, but a 27x-repeated pattern with zero logging means a real regression in map init would be invisible). A wider heuristic (catch followed within 2 lines by a `console.*` call and no matches ~119 sites. Only 65 sites rethrow. Combined with the (throw ,already-established ground truth that **no crash-reporting SDK exists** – a log-and-swallow catch's `console.*` output goes nowhere in production .it isn't collected anywhere a developer would see it

- Problem: There is no single error-handling helper analogous to which does write to Firestore , `src/services/errorLog.ts`) `logError()` enforced as the default; call sites choose ad hoc between (`errors/` silent swallow, or rethrow, with no , `logError(...)` , `console.warn` linting or convention document backing the choice (no ESLint config .exists per ground truth, so nothing catches inconsistency itself is not consistently the sink – many `src/services/errorLog.ts`

- swallowed catches use bare `console.*` instead, which is invisible outside a locally-attached debugger
- User impact: SUSPECTED – failures downgrade to silent no-ops instead of a visible error state or a recorded event, so both the user and the team can lose visibility into a broken flow simultaneously
- Technical impact: `errorLog.ts` 's value (a queryable production error feed used per project memory to triage before every release) is undermined by every catch that bypasses it
- Recommendation: Establish `logError` as the mandatory non-empty-catch convention outside the already-reasonable map-styling exceptions; a grep-based lint rule (the app already relies on a grep-based tripwire for the Joryio SDK per project memory, so this idiom is already accepted in this codebase could flag new bare `console.*` -only catches in `src/services` and `src/screens`
- Expected benefit: Fewer silent failures; the existing `errors/` collection becomes a more complete signal
- Effort: M

F-ARCH-8 – State stores mix server-cache and pure UI state in one slice, no query-cache layer

- Severity: P3
- Confidence: Confirmed
- Feature / Screen / Flow: `src/store/userStore.ts` (259 lines), and to a lesser extent `groupStore.ts` / `gameStore.ts`
- hydrated Evidence: `userStore.ts` interface (lines 15-53) declares `currentUser` and `onboardingDone` (UI flag), (pure bootstrap UI flag) side (server-synced via `subscribeCurrentUser` 's `onSnapshot`, line 207) by side in one Zustand store with no separation between “cache of server truth” and “local UI/session state.” There is no react-query/SWR-style cache layer anywhere in `src/store` (4 files, 776 lines total) – each store hand-rolls its own subscribe/hydrate/reset lifecycle
- Problem: Every consumer of `useUserStore` that only needs, say `currentUser` field `onboardingDone` – updates on every server-pushed stat change (goals/assists/wins/rating per the store's own comment at line ~40, this listener is the “root fix for the stale store bug class”), so unrelated UI state changes can trigger re-renders keyed off unrelated server pushes unless selectors are used carefully at every call site
- User impact: None directly observed; this is a re-render/perf hygiene concern, not a correctness one
- Technical impact: No systematic caching policy (staleness, refetch-on-focus etc.) exists – each screen either subscribes live or calls a service method directly with its own ad hoc freshness logic (e.g. `gameStore.ts` 's (hand-written `PLAYER_HYDRATE_TTL_MS` cache, lines 27-29)
- Recommendation: Not urgent given the app's scale (~596 users, 55 screens), flagging as a known trade-off rather than a defect. If growth continues a thin query-cache layer (even a minimal in-house one, given the existing aversion to new dependencies) would remove the need for each store to reinvent TTL/staleness logic
- Expected benefit: Lower re-render churn, one less ad hoc caching policy per store

F-ARCH-9 – Firebase coupling is well-contained; two raw `onSnapshot` call sites are the exception

- Severity: P3 •
- Confidence: Confirmed •
- Feature / Screen / Flow: app-wide data layer •
- Evidence: Only 5 screens/components import `@/firebase` directly •
, `AvailabilityEditScreen.tsx` , `EmailAuthScreen.tsx` , `ProfileScreen.tsx`)
and only 2 components ,(`FillerInterestsSection.tsx` , `MockModeBanner.tsx`
:call `onSnapshot` directly rather than going through a service
(a shared hook, not a screen – reasonable) `src/services/useGameEvents.ts`
and `src/components/chat/ChatView.tsx` . The other 175+ screens/components
.`src/services/*` route Firestore access through
- Problem: Genuinely minor – this is one of the healthier boundaries in the •
,codebase. Swapping the data layer would mean rewriting ~57 service files
.not chasing Firestore calls through 180 components
- .User impact: None •
- .Technical impact: None significant found •
- Recommendation: None needed; noting this is fine rather than padding the •
.report with a non-issue
- Expected benefit: n/a •
- Effort: n/a •

F-ARCH-10 – No circular-dependency tooling; manual check found none beyond type-only imports

- Severity: P4 •
- Confidence: Confirmed (manual check only – no `madge` /dependency-cruiser •
configured or installed in the repo, and none was installed for this audit
(per the read-only rule
- Feature / Screen / Flow: `src/utils` ↔ `src/services` boundary •
- `src/services` Evidence: 3 files under `src/utils` import from •
which ,(`assistant/types.ts` , `demandSlots.ts:8` , `eveningProgress.ts:15`)
inverts the expected “services depend on utils” direction. All three are
so ,(`import type { EveningMetric } from '@services/eveningSummaryService'` .erased at compile time, e.g) `import type`
there is no actual runtime
cycle – checked by confirming `gameService.ts` does not import either
.or `demandSlots` back `eveningProgress`
- Problem: Type-only inversions are harmless today but there's no tooling •
to catch it (no `madge` in `package.json` , no dependency-cruiser config)
if a future edit turns one into a real (value) import, which would create
.an actual cycle
- .User impact: None •
- .Technical impact: None currently; latent risk only •
- Recommendation: Low priority – if the team ever adds a lint/CI step, a •
cheap `madge --circular src` check would close this permanently for ~0
.ongoing cost
- .Expected benefit: Cheap insurance, not urgent •
- Effort: S •

The single highest-value finding is the **evening-score formula duplication** (F-ARCH-1): it is exactly the kind of un-pinned duplicate the `/roundSummaryParity / clubChemistryParity / balanceParity` existing tests imply should exist elsewhere – and it does, plus `tieConfirmParity`, four more (F-ARCH-2): clean sheets (3-way, including a Python script) penalty aggregation, fair-join ordering, series scheduling, and a brand-guard regex that spans a *second repository* (Pulse). None of these five has any automated protection; the codebase’s own comments already document one past regression in this exact class (clean sheets silently dropping the penalty axis). The two “god objects” (`functions/src/index.ts` at 13,974 lines/64 exports, `gameService.ts` at 8,192 lines/~65 methods) both have visible comment-delineated seams that make a mechanical split feasible without a redesign. The most concrete correctness risk is the non-transactional read-modify-write pattern (F-ARCH-5, 15 call sites) – a genuine `draftTeams` two-admin race with silent data loss and no test coverage anywhere in `State` management and Firebase-layer coupling are, by contrast, `functions/` in reasonably good shape and are called out as such rather than padded into `.findings`.

Database audit – Teamder (Firestore)

`, functions/src/index.ts , src/types/index.ts` Methodology: static read of cross-checked against REAL `, firestore.indexes.json , src/services/*.ts` `runAggregationQuery / documents:runQuery` production data via read-only .REST calls (project `soccer-app-52b6b`) on 2026-09-02. No writes made. Real counts quoted below are live counts, not estimates, unless marked `.otherwise`.

F-DB-1 – `games/{id}` is nowhere near the 1 MiB cap, even at 11-a-side ×4 teams

- Severity: P3 (not a real risk, but worth debunking before anyone “fixes” it)
- Confidence: Confirmed (arithmetic below) + corroborated by real doc sample
 - `games/{id}` ,Feature / Screen / Flow: any live evening
 - `, (LiveMatchState) :2005–2160 , (Game) src/types/index.ts:1443–1916` :Evidence
- Real sample: the largest `( MatchRotation / DraftTeamsResult ) :1120–1209` /live `games/{id}` doc in production today (`2ZSmTYqZbMrimDfJRqi5` , a 14-player (team evening already several rounds in, `activeIntervals` has 10+ entries-3 `liveMatch` serializes to **16,464 bytes** as Firestore REST JSON – of which alone is 3,500 bytes. REST JSON overstates real storage (type-wrapper overhead like `{"stringValue":...}` roughly doubles it), so native size is `.smaller still`
- Current behaviour: one `games/{id}` doc holds the full roster, guests `baseTeams` `draftTeams` (+ a frozen `originalTeams` copy), rotation (+ a

,copy), and the entire `liveMatch` state (assignments, per-player slot maps goal tally, timer event log, and an `activeIntervals` log that is **never** **cleared for the whole evening**

Problem: none, in practice. Scaling the observed doc to the brief's worst case – 44 players (11-a-side × 4 teams), ~10 guests, an 8-mini-game evening -with moderate timer churn (~40 start/pause/resume presses) – the field-by-field estimate (Firestore's real storage formula: field-name bytes+1 string bytes+1, 8 bytes/number, 16-byte map overhead) comes to roughly **KB**. That is **~2.8% of the 1,048,576-byte limit**. Even a **30~29** pathological case (100+ timer presses, 20 guests, full rejected/invited .audit trails) would struggle to clear 60~80 KB – under 8% of the cap

.User impact: none

Technical impact: the 1 MiB ceiling is not the constraint to design around for this doc. The real cost of its size is **read/write bandwidth per listener tick** (F-DB-3), not storage-limit risk

Recommendation: no action needed on size. Do not spend engineering time slimming" `games/{id}` for the 1 MiB limit; if it's ever slimmed, do it for" .F-DB-2/F-DB-3 instead

Expected benefit: – (informational; prevents a wrong-target fix)

Effort: – (no-op)

F-DB-2 – the same 44-uid roster is stored redundantly in up to 5 places on one doc

Severity: P2

Confidence: Confirmed

`games/{id}` ,Feature / Screen / Flow: draft teams + live rotation
, participantIds[]) :1487 ,(players[]) src/types/index.ts:1463 :Evidence
:1120–1148 ,(“denormalized union... must be kept in sync on every write”
a FROZEN” – originalTeams :1132 + DraftTeamsResult.teams[].playerIds)
– MatchRotation.baseTeams[]) :1165–1209 ,(“snapshot... never mutated
.(“Snapshot of the drafted rosters as they stood when the rotation STARTED”

Current behaviour: at 4 teams × 11 players, the full 44-uid list is encoded
, draftTeams.teams[*].playerIds , participantIds[] , players[] independently in
rotation.baseTeams[*].playerIds and , draftTeams.originalTeams[*].playerIds
.five copies of essentially the same 44 Firebase uids (28 chars each) –

Problem: ~1,276 bytes per copy × 5 ≈ 6.4 KB of the doc is the same roster
“repeated, by design (each copy serves a different “frozen at time X
semantic – this is not a bug, it's deliberate audit-trail modeling). It is
the single largest structural contributor to document size identified in
F-DB-1's estimate, and every one of those 5 copies is retransmitted to
every listener on ANY write to the doc (see F-DB-3), even a write that
.touches none of them (e.g. a single goal entry)

.User impact: none directly; contributes to F-DB-3's bandwidth cost

Technical impact: makes the doc bigger and the listener payload bigger for
– no read-time benefit (nothing joins across these 5 copies at read time
(each screen reads whichever one it needs

Recommendation: acceptable at current scale; if `games/{id}` size or
listener bandwidth ever becomes a real problem, this is the first place to
look – e.g. store `originalTeams` / `baseTeams` as team **indices only** with
a pointer back into `players[]` rather than duplicating uids, or move the
frozen snapshots to a subcollection written once and never re-sent on
.every subsequent write

- Expected benefit: could cut the “always-resent” portion of the doc by roughly 20-25% at full 44-player scale
- Effort: M (touches 3 write paths + type + converter)

F-DB-3 – full-document listener fan-out: every write to `games/{id}` re-bills 1 read to every open screen

- Severity: P1
- Confidence: Confirmed
- , `AdvancedLiveMatchScreen` – Feature / Screen / Flow: live evening spectators, `DraftBoardScreen`, `MatchDetailsScreen`, `LiveMatchScreen`
- Evidence: `src/services/gameService.ts:7573–7579` – the code’s own comment “every game write (e.g. a timer press) used to bill TWO reads per device” + `subscribeLiveMatch` now one” – describing the recent merge of `onSnapshot(docs.game(gameId))` `subscribeLiveGame` into one `subscribeRotation` `onSnapshot`
- Current behaviour: `subscribeLiveGame` is already a single per viewer (this branch’s own prior fix halved the fan-out). But Firestore listeners deliver – and bill – the **whole document** on every write regardless of which field changed; there is no field-scoped listener
- Problem: a live evening with N concurrent viewers (players + spectators with the match screen open) and W writes to `games/{id}` over the evening = timer start/pause/resume ~6-10, goal entries ~4/round × up to 8 rounds) round-end/rotation-advance ~8, loans/moves, roster churn) costs $32 \times N \times W$ billed reads, not W. At a plausible mid-size evening – 15 viewers, 60 writes – that’s **900 reads for one game, one evening**, all of it re-delivering the ~10-16 KB (real, observed) doc, not just the ~50-byte field that actually changed
- User impact: none directly (fast, cheap per-op), but this is the dominant Firestore cost driver of a live evening, not `commitRoundStats`’s batch writes
- Technical impact: cost scales as (viewers × writes), i.e. **quadratically in evening size** if both viewer count and roster/write count grow with club size (a bigger club → more players on screen AND more roster/goal writes)
- Recommendation: already partly mitigated (single combined listener). Next lever, if this becomes a real cost line, is field-level UI diffing already happening client-side (fine) plus reducing write FREQUENCY during live play rather than trying to split the (e.g. coalesce rapid-fire timer presses) listener further (Firestore doesn’t support partial-document listeners)
- Expected benefit: N/A at current scale (this is a scale-driven finding (see F-DB-13)
- Effort: – (documented for future reference; no immediate action warranted)

F-DB-4 – `commitRoundStats` batch math is real and already at 81% of the 500-op cap at legal 11-a-side

- Severity: P1
- Confidence: Confirmed – code comment gives the exact formula
- `commitRoundStats`, (end mini-game) “סיים משחקון” :Feature / Screen / Flow
- Evidence: `functions/src/index.ts:12843–12860` – comment: “against-pairs same-team pairs ($C(|A|,2) + C(|B|,2)$) ... worst case at n-per-side + $(|A| \times |B|)$ $2n^2 + \sim 15n$... crosses Firestore’s 500-op cap around $n \approx 13$... 11 covers ≈ every legitimate format with worst-case ≈ 400 ops”, `MAX_SIDE = 11` at `:12854`

Current behaviour: `MAX_SIDE=11` throws `invalid-argument` before building the batch if either side exceeds 11

- Problem (arithmetic, using the code's own formula $2n^2+15n$)

n per side	ops	% of 500 cap
11	125	25%
12	203	41%
13	297	59%
14	407	81%
15	468	94%
16	533	>100%

At the legal maximum (11-a-side, the format explicitly named in this audit's brief), the batch already uses **81% of Firestore's hard 500-op limit** – only 2 more players per side (13) would blow it. Because the idempotency latch is created in the `same committedRounds/{roundId}` batch (:12900–12910), a batch that overflows fails atomically and a retry fails identically – that mini-game's stats are permanently unrecoverable, `addRetroGoal` from that path (would need a manual admin correction via `addRetroGoal` which only handles goals/assists, not wins/losses/pairs).

User impact: none today (11 is the enforced ceiling and matches real-a-side football), but there is very little headroom if any future-11 feature (bench players counted into a side, a captain/co-captain double count, a data bug duplicating a uid across A and B before dedup) pushes the side size up even slightly.

Technical impact: this is the single most batch-constrained write path in the app.

Recommendation: no change needed – the guard is correctly sized and documented in-code. Flagging so any future change to `MAX_SIDE` or to what gets bundled into this batch is made with the 500-op ceiling explicitly in mind (e.g. don't add a 6th write-target collection to this same batch without re-deriving the formula).

Expected benefit: – (confirms existing safety margin is correctly sized – not a bug)

Effort: – (no action) –

F-DB-5 – evening-finish standings read scales with CLUB SIZE, not evening attendance

Severity: P2

Confidence: Confirmed

standings “כוכבי הערב” → Feature / Screen / Flow: evening finish

`communityPlayerStats.where('groupId', '==', gid).get()` – `functions/src/index.ts:5345–5348` :Evidence reads **every member's cumulative stats doc for the whole club**, unfiltered (cum) to who actually played tonight, then :5449–5470 uses that full set “to rank every attendee against the whole club for the “rank movement display.

Current behaviour: fires once per finished game (`onGameRosterChanged` 's finish cascade, gated by the `finishCredited` latch so it's exactly-once (per game).

Problem: cost is $O(\text{club members})$, not $O(\text{attendees})$. Real numbers today groups, the largest non-personal club has **40 members** (live count collection) – every evening that club finishes a game costs 40 reads just for this step, whether 12 or 40 people actually showed up. This is by design (rank movement needs the whole club's cumulative totals to compute

a rank) – not a bug – but it means this cost line scales with club membership growth, which can run well ahead of evening attendance growth.

User impact: none (server-side, invisible) •

Technical impact: at 10×/100× user scale, if club sizes grow proportionally, this read cost grows with them on every single game finish (see F-DB-13) club-wide, every week

Recommendation: acceptable as-is; if it becomes a real cost line, the fix

is a `select()` projection (goals/assists/wins/lastEveningScore/userId only – 5 of the doc's ~15 fields) rather than a full-document read, which Firestore bills the same per-doc regardless of field count today, so this would only help with bandwidth, not billed-read count. True read-count reduction would require a separate lightweight leaderboard doc

Expected benefit: bandwidth reduction only, not read-count reduction – low priority

Effort: M •

F-DB-6 – genuine N+1 in the same finish cascade: one `.get()` per attendee instead of one query

Severity: P2 •

Confidence: Confirmed •

standings “כוכבי הערב” → Feature / Screen / Flow: evening finish •

Evidence: `functions/src/index.ts:5371-5378` •

```
ts const evSnaps = await Promise.all( attendees.map((u) =>
db.collection('gamePlayerStats').doc(`${event.params.gameId}__${u}`).get(), ), );
```

Current behaviour: issues one individual `.get()` per attendee (doc id is

deterministic: `${gameId}__${uid}`) instead of a single

query, which would `gamePlayerStats.where('gameId', '==', gameId).get()`

return the exact same rows in one read-batch and is already the query

shape used elsewhere in the same file for the sibling collection (see

(two lines above it `communityPlayerStats.where('groupId', '==', gid)`)

Problem: this is the textbook N+1 pattern the audit brief asks about. It

doesn't change the **billed read count** (Firestore bills per-document

either way, so `N` individual gets vs. one query returning `N` docs cost

,the same in dollars), but it does cost **N round-trips** instead of 1

which is real latency inside a Cloud Function that's already chaining

several sequential steps in the finish cascade – this is the difference

between one network round-trip and up to 44 (at 11-a-side ×4 teams)

.sequential-looking-but-parallelized round-trips

User impact: SUSPECTED – not verified at runtime – a slower evening finish

.push / standings-ready delay for large rosters

Technical impact: no cost difference, latency difference only, at current

scale (attendees ≤ ~20 typically). At 44 attendees the gap becomes visible

.in function duration/cold-start budgets

`db.collection('gamePlayerStats').where('gameId', '==', gameId).get()` Recommendation: replace with

,and re-key by uid from the returned docs – same read cost, one round-trip

and it also naturally drops the deterministic-doc-id assumption embedded in

.the current code

Expected benefit: fewer round-trips in the hottest per-evening Cloud

Function path; marginal at current scale, meaningful once rosters

.regularly hit 30-44

Effort: S •

F-DB-7 – club chemistry screen reads the whole club's pair-history collection on every open, unbounded, uncached

Severity: P2 •

Confidence: Confirmed •

cards (chemistry) "כימיה" → CommunityStatsScreen :Feature / Screen / Flow •

:Evidence: src/services/clubChemistryService.ts:1-6 (code's own comment •

the real club has 57 mini-games behind 241 pairs, and that ratio only"

communityPairStats.where('groupId', '==', groupId) - :46-49 ,(“gets worse

.with no limit , no pagination, no cache, fired fresh on every screen open

Current behaviour: one query, all pairs for the club, sorted client-side •

-into 6 leaderboard picks (pickChemistry). This is a deliberate, already

considered design (the comment explicitly rejects the naive per-open

.mini-game rescan alternative) – it is the RIGHT shape of query, not a bug

Problem: communityPairStats is a rollup (one doc per unique pair that has •

ever played together in that club), so its size is bounded by

not by evening count – but that bound, C(lifetime unique players, 2)

still **grows every time a new player joins and plays**, forever, with no

decay/cap, and there is no cache on the client between opens. For the

club cited in the code comment (241 pairs today), that's already 241 reads

per screen open with zero caching – every re-open of the stats screen

.re-reads the same 241 docs

User impact: SUSPECTED – not verified at runtime – screen-open latency for •

.large/old clubs; not visible to the user as a bug, just a cost line

Technical impact: a club that's been running for 2-3 years with steady •

→ roster turnover could plausibly reach 60-80 lifetime unique players

potential pairs (upper bound; real ratio per the code's $2,415 = C(70, 2)$

own citation is far below the theoretical max – 241 pairs from ~57 games

suggests real growth is sub-quadratic in practice). Still, with no client

cache, N users opening the stats screen M times each all pay the same

.per-open cost with no reuse

Recommendation: add a short client-side cache (even 5-10 min TTL) on •

so repeat opens within a session don't clubChemistryService.get(groupId)

re-read the whole collection; consider only if this club-stats screen is

a high-traffic entry point (unverified at runtime – flag for the UX/product

.auditor to confirm actual open frequency before prioritizing

Expected benefit: could cut repeat-open reads to near zero for the common •

.case (user checking the same club's stats multiple times in one session)

Effort: S •

F-DB-8 – up to 5 separate Cloud Functions fire on every single write to games/{id} (Firestore triggers can't filter by field)

Severity: P2 •

Confidence: Confirmed •

,Feature / Screen / Flow: any write to a game doc (join, cancel, goal entry •

(timer press, guest add, roster edit – anything

:4432 ,(onGameTimerChanged) functions/src/index.ts:4307 :Evidence •

:4818 ,(now a documented no-op left deployed , onGameRotationChanged)

described in D2 as “THE roster-change hub” and the , onGameRosterChanged)

:12036 ,(updateShowcaseOnGameChange) :9499 ,(largest of these

onDocumentWritten('games/{id}', ...) All are .(on update , onGameJoinedAlert)

- or `onDocumentUpdated` , none scoped to a field – Firestore trigger
- `.subscriptions` are path-scoped only, not field-scoped
- Current behaviour: each of these 4-5 functions is invoked on literally every write to the doc and does its own before/after diff internally to decide whether to do real work (e.g. `onGameTimerChanged` 's `changed` check at `:4337-4344` , `onGameRotationChanged` being an intentional full no-op `.(per its own comment`
- Problem: a single client `updateDoc` to `games/{id}` (say, one goal entry) ,triggers up to 5 separate Cloud Function cold-start-eligible invocations each of which deserializes the full before/after document (the same KB doc from F-DB-1/F-DB-3) even when it does nothing else. This is 10-16 invocation-count and CPU-time amplification, not Firestore read/write amplification – but it's real cost that scales 1:1 with the write-fan-out `.already described in F-DB-3`
- .User impact: none directly
- ,Technical impact: at W writes/evening across all live games club-wide :total function invocations from this one collection alone $\approx 5 \times W$ (today W is small per D2's cron-consolidation work already having cut most polling-based cost; this is the one remaining structural fan-out that `liveMatch` can't be removed without restructuring the doc, e.g. splitting into its own document so timer/goal writes don't also wake the roster and `.(showcase triggers`
- Recommendation: acceptable at current scale (`maxInstances:10` , low traffic). If invocation cost or cold-start latency during live evenings ever becomes visible, the fix is to split high-frequency fields into a sibling document/subcollection so triggers that only (`liveMatch`) care about roster/visibility/showcase changes aren't invoked on every `.timer press and goal entry`
- Expected benefit: N/A at current scale – documented for the x100/x1000 `.scale estimate (F-DB-13)`
- +Effort: L (would be a real schema change – `liveMatch` is read by ~6 (`subscribeLiveMatch` / `subscribeLiveGame` screens via

F-DB-9 – most historical collections are pruned NEVER; only 4 collections have any TTL sweep

- Severity: P2
- Confidence: Confirmed
- `runDailyCleanup` ,Feature / Screen / Flow: backend cron
- Evidence: `functions/src/index.ts:3786` (`runDailyCleanup`) – grep of its body shows exactly 4 numbered cleanup steps: (1) `notifications` older than 30 days (`createdAtMs` cutoff), (2) stale `gameUpdateLatches` for ,finished/deleted games, (3) `groupJoinRequests` decided >90 days ago .orphaned `games/{id}/fillerInterests` for finished/cancelled games (4) , `tasks` , `feedback` , `errors` :Nothing else in the codebase deletes from , `groups/{id}/messages` , `games/{id}/messages` , `chatReports` , `games/{id}/roundHistory` , `dmConversations/{id}/messages` , `games/{id}/committedRounds` , `games/{id}/retroGoals` , `gamePlayerStats` , `groups/{id}/memberCredited` , `games/{id}/joinCredited` , `communityPairStats` , `pairStats` , `communityPlayerStats` , `metrics/linkClicks` , `campaigns` , `roundSummaries` , `eveningStandings` . `inviteClicks` , `inviteLinks`
- .Current behaviour: every one of those collections accumulates forever

- :Problem: real counts today (via `runAggregationQuery`, live 2026-09-02)
 - ,(despite the 30-day sweep – see F-DB-10) **1,010** notifications
 - **,614** pairStats **,351** feedback **,161** errors
 - **,297** gamePlayerStats **,594** communityPairStats
 - **,92** communityPlayerStats **,147** eveningStandings
 - for a base of **614 real users** and only **,8** roundSummaries
 - **currently-existing games docs** (most finished games are NOT **79** deleted – the low count suggests either a young dataset or that the stale-cleanup cron only removes o-roster/abandoned games, per the note; played games stay forever, along with `project_game_deletion_audit` .(every subcollection under them
 - .User impact: none directly (nothing here is user-facing latency yet)
- /Technical impact: chat messages (3 separate subcollections: game/community DM) and `errors` / `feedback` are the most likely to actually be large in practice (chat is naturally chatty, errors accumulate every crash/warning) and neither has ANY cap – no TTL, no archival, no size-based rotation. This is the collection-growth story that will matter most at 100×/1000× scale .well before any single-document size limit does ,(F-DB-13)
- `errors` Recommendation: extend `runDailyCleanup` with TTL sweeps for ,(`notifications` e.g. 90 days – the same pattern already proven for `games/{id}/messages` and consider whether ,(once resolved) `chatReports` should be pruned when the parent game is old+finished (chat history for a month-old finished pickup game has little value and no current owner-6 reads it). `roundHistory` / `gamePlayerStats` / `communityPairStats` etc. are stats-of-record and should NOT be pruned – flagging those explicitly as .intentional-keep, not oversights
- Expected benefit: bounds the two genuinely-unbounded-by-nature collections .before they become a real cost line (chat, errors)
- Effort: S (mirrors the existing `notifications` TTL pattern almost exactly)

F-DB-10 – the notification TTL sweep is capped at 400 deletes/day; verify it isn't already behind

- Severity: P3
- Confidence: Confirmed code, Suspected on whether it's currently keeping up
- Feature / Screen / Flow: `runDailyCleanup` step 1
- Evidence: `functions/src/index.ts` – `BATCH_LIMIT = 400` , one query+one batch delete per **daily** run (gated by `cronMeta/dailyCleanup` , “at most .(“once/23h
- ,Current behaviour: deletes up to 400 notifications older than 30 days .once per day
- Problem: live count today is **1,010** notifications docs for 614 users plausible that this is simply the current 30-day rolling – (user/1.6~) window and the sweep is keeping up fine. But the mechanism has no self-correction if creation rate ever exceeds 400/day: the query has no cutoff enforcement beyond the `limit(400)` , so a growing backlog `orderBy` past the daily capacity would silently stop shrinking (each run always deletes the SAME leading 400 by cutoff order... actually the query is unordered beyond the inequality filter, so it deletes an arbitrary 400 of the docs older than cutoff – with more than 400 eligible, the collection's .(older than 30 days” tail would only shrink by min(400, backlog) per day”
- .User impact: none

- Technical impact: silent unbounded growth ONLY if daily creation ever exceeds ~400/day sustained – at current scale (614 users) this is very unlikely; at 10-100× user scale it becomes a real question
- Recommendation: no immediate action; add a simple metric/alert (or a Pulse dashboard tile) tracking notifications collection count over time so a growing backlog is visible before it's a problem, since the sweep silently degrades rather than erroring
- Expected benefit: early warning, not a fix
- Effort: S

F-DB-11 – campaign engine reads ALL users + ALL groups per send; the 20,000-recipient cap is ~33× current scale

- Severity: P3
- Confidence: Confirmed (per D2 discovery, corroborated here)
- processCampaign , Feature / Screen / Flow: Pulse-driven broadcast campaigns
- Evidence: functions/src/adminUserPush.ts (per D2: segment evaluation reads the entire users collection + all groups for membership on every .send; MAX_RECIPIENTS=20000 , 6-attempts-then- error circuit breaker users Current behaviour: every campaign send re-scans the whole collection (currently 614 docs) plus groups (174 docs) to evaluate .segment membership
- Problem: cost today is small ($614+174 \approx 788$ reads per campaign send). At scale (~6,140 users) it's ~7,900 reads per send – still cheap. At ×10 scale (~61,400 users) it EXCEEDS MAX_RECIPIENTS=20000 – campaigns ×100 targeting “everyone” would silently truncate to 20,000 recipients (per the cap), not fail loudly, meaning ~2/3 of the user base would stop receiving .broadcast campaigns with no visible error
- User impact: SUSPECTED – not verified at runtime – a growth-scale product gap: campaigns quietly stop reaching most users once total users passes “unless segments are narrower than “everyone , 20,000~ reads themselves stay affordable even at 1000× scale (~614,000 reads/send at full-collection-scan) – this is a recipient-cap product ceiling, not a .read-cost ceiling
- Technical impact: none until ~100× scale; already has a real, tested-in production fix for the runaway-rescan failure mode (per the memory note – the attempts cap specifically project_campaigns_read_cost .(addresses that class of bug
- Recommendation: no action needed now; when the user base approaches either raise MAX_RECIPIENTS (with awareness it's now O(all , 20,000 users) reads per send, ~20K+ reads) or move segment evaluation to a .precomputed/indexed segment rather than a full collection scan
- Expected benefit: – (future scale note)
- Effort: – (no action now)

F-DB-12 – only one composite index exists on the pair-stats collections; everything else avoids needing one by fetching the whole collection

- Severity: P3
- Confidence: Confirmed
- Feature / Screen / Flow: club stats / chemistry leaderboards

Evidence: `firestore.indexes.json` – 25 composite indexes total; exactly ONE touches `communityPairStats` (`groupId ASC, assists DESC` – the “deadly duo” assists leaderboard). No composite index exists for `cleanSheetsTogether` or `lossesTogether`, `winsTogether`, `sameTeam` – the other 4 leaderboard dimensions `pickChemistry` computes (per `clubChemistryService.ts`)

Current behaviour: this is not a missing-index bug, because never issues a server-side `orderBy` on those `clubChemistryService.get()` fields – it fetches the whole `where('groupId', '=', groupId)` result set and sorts all 6 leaderboard dimensions **client-side in memory** (F-DB-7). The one index that DOES exist (`assists DESC`) looks unused by this path for that same reason – it’s not clear from static analysis what query actually needs it (possibly a legacy/removed query, or a different screen not covered by this audit’s file set).

Problem: not a correctness bug (no query is failing for lack of an index – the code deliberately avoids server-side sort here). Flagging because it means F-DB-7’s full-collection-read pattern isn’t a stopgap waiting on an index – it’s the intended, permanent shape, so the fix for “F-DB-7 (if pursued) is caching/pagination, not “add the missing indexes”.

User impact: none

Technical impact: none beyond what F-DB-7 already describes

Recommendation: confirm (in a later phase, not blocking) whether the existing `communityPairStats: groupId+assists DESC` index is still used by any live query – if not, it’s one of 25 composite indexes silently costing index-write overhead on every `communityPairStats` write for a query that no longer runs

Expected benefit: minor index-maintenance cost avoided if confirmed dead

`firebase deploy --only firestore:indexes` Effort: S (grep + confirm, then to remove – NOT performed here, read-only audit)

F-DB-13 – scale estimate: what breaks first at 10×, 100×, 1000× current users (614 → 6,140 → 61,400 → 614,000)

Severity: P1 (informs prioritization)

Confidence: Suspected (extrapolation) – real anchors: 614 users, 174 groups (53 non-personal, mean 5.2 members, largest 40), 79 games pairStats, 594 communityPairStats, 297 gamePlayerStats, 1,010 614 notifications – all live counts, 2026-09-02

Feature / Screen / Flow: whole system

Evidence: synthesizes F-DB-3, F-DB-5, F-DB-7, F-DB-8, F-DB-11

:Current behaviour / arithmetic

nothing structurally breaks. The campaign engine **:(users 6,140~) ×10** reads ~7,900 docs/send – still cheap. The largest club, if (F-DB-11) membership grows proportionally, reaches ~400 members (unrealistic for a pickup-football WhatsApp-sized community – more likely the user base grows via MORE clubs, not bigger ones, given the app’s regulars-first design per `project_app_purpose`). Under either model, the FIRST thing to become visibly worse is **F-DB-3’s listener fan-out**: more concurrent viewers per live evening (bigger clubs) × more roster/goal writes cost is closer to quadratic in per-club size than – (bigger rosters) linear in user count, so a handful of large, highly-engaged clubs feel it before the average club does

this is where a real product-visible ceiling : (users 61,400~) ×100
hits: F-DB-11's MAX_RECIPIENTS=20000 cap – broadcast campaigns
,targeting “everyone” silently stop reaching roughly two-thirds of users
with no error surfaced anywhere (the circuit breaker catches runaway
re-scans, not legitimate truncation). Second: if any single club
approaches hundreds of members, F-DB-7's chemistry read
and F-DB-5's per-finish standings read (C(lifetime unique players, 2))
(per game finish, every finished game, every week 0(club members))
both become individually-noticeable costs (thousands of reads for a
single screen open / a single game finish) even though neither is a hard
.failure

the FIRST hard failure is most plausibly : (users 614,000~) ×1000
F-DB-4's commitRoundStats 500-op batch cap – not because per-club
,format changes (11-a-side stays 11-a-side regardless of total user count)
but because at this scale it's far more likely that some club/community
-eventually runs an irregular/larger-than-standard format, or a roster
,dedup edge case (F-DB-4's own “future feature” caveat: bench players
(co-captain double-counts, a uid duplicated across sides before dedup
actually gets hit in practice simply from sheer event volume – the same
code path that is safe at n=11 (81% of cap) has almost no margin, and at
this scale it WILL be exercised enough times that a rare edge case
becomes a routine incident. Second: errors / feedback /chat message
collections (F-DB-9), still entirely unpruned, are the ones most exposed
to raw volume growth (proportional to active users, not to games or
clubs) – 1000× the crash/chat volume with zero TTL is a genuine
operational (not correctness) problem: ever-growing collection scans for
.any admin/Pulse tooling that lists them, and unbounded storage cost
Third: the Cloud Function invocation fan-out (F-DB-8, 5 functions per
write) becomes a real maxInstances:10 contention risk games/{id}
during peak evening hours (many simultaneous live evenings, each firing
concurrent invocations per write) – this is a capacity-planning item 5
.not a data-model fix ,(maxInstances raise)

Recommendation (priority order for a scale roadmap): (1) extend •
to errors /chat before 100×; (2) revisit runDailyCleanup
and campaign segment evaluation before user count MAX_RECIPIENTS
approaches 20,000; (3) re-derive the commitRoundStats batch formula and
either lower MAX_SIDE further or split the batch across multiple
transactions BEFORE any format larger than 11-a-side is ever allowed, and
audit for roster-dedup edge cases now, not at 1000×; (4) raise
and/or split liveMatch into its own document only once maxInstances
F-DB-3/F-DB-8's fan-out is measured to actually matter (don't do this
.speculatively – it's a real schema change
Expected benefit: sequences the 3 genuinely different failure classes •
so effort (product ceiling → read-cost creep → hard batch-cap failure)
goes to the batch-cap risk first, since it's the only one that's a full
.outage (lost stats) rather than degraded service
Effort: XL (this is a roadmap item, not a single fix) •

Summary of what's genuinely fine (per brief's "say so in one line" instruction)

- Client-side roster hydration (`groupService.hydrateUsers`) already batches via `documentId()` in [...] chunks of 30 – NOT an N+1 pattern (the code comment says this exact optimization shipped: "a 200-member roster went from 200 billed reads to ~7 queries"). Confirmed fine
- The live match timer is NOT a hot document in the "1 write/sec" sense
 - `AdvancedLiveMatchScreen / LiveMatchScreen` calls in `setInterval(...,1000)` only tick a local React state variable (`setNowTick`) for display; the actual `games/{id}` write only happens on discrete admin actions confirmed by reading `setLiveMatch` 's (start/pause/resume/reset/goal) call sites. `appConfig` and `communityStats` are similarly low-frequency writers, not hot documents
- s idempotency (`committedRounds/{roundId}.create()` in `commitRoundStats` the same batch as the increments) is correctly designed – a retry is a safe no-op, not a double-credit risk

FIREBASE

Firebase Platform Audit – Teamder (soccer-app-52b6b)

Read-only static analysis + live read-only `gcloud / firebase` CLI reads "against the real production project. No writes were made. All "live citations below are from actual `gcloud / firebase` command output captured during this pass, not guesses about defaults

F-FIREBASE-1 – Production Firestore has zero backups and delete protection is off

- Severity: Po
- Confidence: Confirmed (live `gcloud firestore databases describe` + `gcloud firestore backups schedules list`)
- Feature / Screen / Flow: whole platform
- :Evidence
 - `gcloud firestore databases describe --database='(default)'`
, `pointInTimeRecoveryEnablement: POINT_IN_TIME_RECOVERY_DISABLED`
`deleteProtectionState: DELETE_PROTECTION_DISABLED`
 - Listed 0 items. → `gcloud firestore backups schedules list`
- Current behaviour: the `(default)` Firestore database backing all ~596 users' stats, evening summaries, communities and games has no scheduled backups, no point-in-time recovery, and no delete-protection lock
- Problem: there is no recovery path for accidental mass-deletion or a bad migration. The codebase itself runs several bulk/rare-path operations against this exact database with no rehearsal environment: `promoteOrphanToGroup` (hard-deletes and rebuilds `communityPairStats / eveningStandings / roundSummaries / clubRecords` for a club), `backfillGroupCreatorIdsOnce` (admin-only one-time script that scans **all** groups), and `deleteMyAccount` 's cascade (paginated deletes across `games`, `groups`, `groupsPublic`, `communityShowcase`, `collection-group messages`)
- User impact: a single bad deploy, script bug, or `firestore databases delete` mistake destroys the app's entire dataset – every player's career stats, club history, chat – permanently, with no restore option
- Technical impact: no RPO/RTO story exists at all; incident response for any data-loss bug (including ones this audit's other agents may find) has no fallback

Recommendation: enable a daily Firestore backup schedule (7-day retention is a one-line `gcloud firestore backups schedules create`) and turn on delete protection. PITR is a bigger cost/latency tradeoff – a daily backup schedule alone would fix the Po
“.Expected benefit: converts “permanent data loss” into “restore from yesterday”
Effort: S

F-FIREBASE-2 – Firestore triggers deploy with retry OFF; the most complex trigger has no automatic-retry safety net

Severity: P1

Confidence: Confirmed (live `gcloud functions describe onGameRosterChanged` + source grep)
(`functions/src/index.ts:4818–6117` , `onGameRosterChanged`) Feature / Screen / Flow: evening finish / roster changes
:Evidence

`eventTrigger.retryPolicy: RETRY_POLICY_DO_NOT_RETRY` → `gcloud functions describe onGameRosterChanged --v2` :Live
Source: `grep -n "retry:" functions/src/index.ts` → zero hits, across all ~20 `onDocumentCreated` / `onDocumentUpdated` / `onDocumentWritten` exports (`onGroupPendingChanged` , `onGameTimerChanged` , `onGameRotationChanged` , `onGameRosterChanged` , `onVoteWritten` , `onNotificationCreated` , etc.) – none opts into `retry: true` , so all deploy at the Cloud Functions v2 default (no retry on failure)

Current behaviour: `onGameRosterChanged` is the largest trigger in the codebase (per D2: finish-transition crediting, evening-standings recompute, round-summary sealing, `gamesJoined` achievement crediting – all in one handler). If it throws (a transient Firestore contention error, a bad read, a timeout under the function's default 60s budget – see F-FIREBASE-5), Cloud Functions does not retry it. The invocation is gone

Problem: the codebase invested heavily in idempotency latches (`finishCredited` , `joinCredited` , `committedRounds` , `communityPairRollups`) that guard against **double**-delivery, but retry being off means the far more common failure mode – a **single** failed delivery – has no mitigation at all. A latch protects against a redelivery that never comes silently drops evening-standings/round-summary/games-played (finish evening) “סיים ערב” User impact: a transient hiccup during crediting for that evening, with no user-visible error and no automatic recovery – the evening looks “finished” in the game doc but stats never post

Technical impact: the only trace is a Cloud Logging `console.error` /thrown exception; nothing surfaces to Pulse (client-error pipeline only) or any alert (see F-FIREBASE-4)

Recommendation: turn on `retry: true` for `onGameRosterChanged` at minimum (it's already idempotent via the create-once latches, so retry is safe and is exactly the case latches were built for); audit the rest of the trigger list for the same combination of “has an idempotency latch but retry is off”

Expected benefit: closes the gap between the idempotency machinery that was built and the delivery guarantee that's actually configured
Effort: S

F-FIREBASE-3 – `flushPendingJoinerNotifsTask` has retries configured but is not actually retry-safe: a failed send after a successful claim is silently lost forever

Severity: P1

Confidence: Confirmed (read the full handler)

(`functions/src/index.ts:2146–2233`) Feature / Screen / Flow: “N players joined” batched admin notification
Evidence: `functions/src/index.ts:2168–2189` – `runTransaction` reads `pendingJoinerIds` , captures it into `claimedJoiners` , then in the **same transaction** deletes `pendingJoinerIds` / `pendingJoinFlushAt` from the game doc. Only *after* that transaction commits does the handler (lines ~2196–2230) resolve display names and call `createNotificationOnce(...)` , wrapped in its own `.try/catch` that does `throw err; // retry per retryConfig` (line 2229) on failure

Current behaviour: `onTaskDispatched` retry is configured (`retryConfig: { maxAttempts: 5, minBackoffSeconds: 10 }` , line 2148). The in-code comment even says “if the task fires twice... the second one finds an empty buffer and exits without dispatch” –

.framing this as the *safe* case

Problem: that's backwards. The dangerous case is the one the comment doesn't address: claim succeeds (buffer cleared, `claimedJoiners` captured) → `createNotificationOnce` throws (network blip, transient Firestore error) → handler re-throws to trigger a Cloud Tasks retry → the retry re-enters the transaction, finds `pendingJoinerIds` already deleted, `claimedJoiners = []`, and returns immediately (line ~2183, `claimedJoiners.length === 0` guard) – the notification for those joiners is now unrecoverable. The retry mechanism actively participates in silently dropping the very thing it was configured to protect.

User impact: community admins occasionally never get the “3 players joined” push for a game, no error, no trace to the user

Technical impact: `errors` collection isn't involved (best-effort push, not treated as user-facing failure); only a `console.error` line marks it

Recommendation: don't clear the buffer until the notification has been durably created – e.g. write the claimed joiner list into the created notification doc's payload *before* clearing `pendingJoinerIds`, or move the clear to happen only after `createNotificationOnce` succeeds (accepting the small risk of a genuine double-claim on true concurrent retries, which `createNotificationOnce`'s own dedupe key already guards against)

Expected benefit: makes the retry config actually deliver what it promises

Effort: S

F-FIREBASE-4 – No Cloud Monitoring alert policies exist; combined with no crash SDK, a broken Cloud Function is invisible unless someone reads logs by hand

Severity: P1

Confidence: Confirmed (live `gcloud monitoring policies list` → empty, no error)

Feature / Screen / Flow: whole backend, observability

Evidence: `gcloud monitoring policies list --project=soccer-app-52b6b` returns zero rows (not a permissions error – the command succeeds with an empty result). `gcloud logging metrics list` is also empty (no custom log-based metrics)

Current behaviour: the only production-error visibility mechanisms are (a) `errors/{fingerprint}` – a **client-side** error pipeline (per brief: confirmed, no Crashlytics/Sentry) that only captures errors the RN app itself catches and reports, and (b) `pushToAdmins('error', ...)` in `adminPush.ts`, which is fed by client `onErrorLogged` triggers, not by Cloud Functions' own execution failures

Problem: none of the 64 deployed Cloud Functions have any monitoring alert wired to their own error rate, execution time, or `sxx` count. A Cloud Function that starts failing 100% of the time (bad deploy, quota exhaustion, a secret rotation breaking `ASC_P8 / PLAY_SA`) produces zero proactive signal – the only way to notice is a downstream symptom a user or admin happens to report, or someone manually opening Cloud Logging

User impact: indirect – degraded functionality (pushes stop, stats stop crediting, reminders stop firing) can run silently for days

Technical impact: mean-time-to-detection for a backend outage is unbounded; this is a materially different (and worse) gap than “no crash SDK,” since that's a client-side concern – this is the **server** having no equivalent

Recommendation: at minimum, one log-based alert policy on Cloud Functions execution error count/rate (Cloud Monitoring has a built-in `cloudfunctions.googleapis.com/function/execution_count` metric filtered on `status!=ok`), routed to the same admin push channel `adminPush.ts` already uses, or email

Expected benefit: turns silent backend failure into a page within minutes instead of an unbounded blind spot

Effort: S

F-FIREBASE-5 – Two of the three cron jobs run heavy multi-sweep chains on default (256MiB/60s) resources; only one was deliberately upsized

Severity: P2

Confidence: Confirmed (live `gcloud functions describe` on all three crons)

, `cronEvery60Min`, `cronEvery15Min`, `cronEvery5Min`) Feature / Screen / Flow: scheduled jobs

(`functions/src/index.ts:12535–12669`

:Evidence

Live configs: `cronEvery5Min` → 256Mi / 60s ; `cronEvery60Min` → 256Mi / 60s ; `cronEvery15Min` → 512Mi / 540s (explicit .override, source `index.ts:12552-12559` , plus 2 secrets)

, `createSeriesOccurrences` , `flipPublicGames` , `flipScheduledGames`) sequentially await s 7 sub-jobs `cronEvery5Min` .(`sweepDueCampaigns` , `expireStaleOffers` , `scheduledAutoGenerateTeams` , `cloneRecurringGames` sequentially await s 5 sub-jobs, including `runClubActivitySweep` (a full range query over `games` plus a write `cronEvery60Min` .to **every** `groupsPublic` doc) and `runDailyCleanup` (4 paginated sweeps, 400-doc batches each)

Current behaviour: `cronEvery15Min` was clearly deliberately bumped (it needs the ASC/Play API secrets for `reviewAlerts` and presumably hit real timeouts before). The other two never got the same treatment despite doing comparable or greater sequential .work

Problem: at 596 users this likely fits inside 60s today, but there's no isolation between the sub-jobs in a chain – if job 3 of 7 in `cronEvery5Min` runs long, the function is killed mid-chain by the 60s timeout and jobs 4-7 (including `sweepDueCampaigns` , which the code's own comments flag as historically prone to a runaway-scan incident – see F-FIREBASE-8) never run that cycle, with no .partial-completion signal beyond a log

User impact: SUSPECTED – not yet manifesting, but this is exactly the kind of silent partial-failure class the project has hit before .(per memory: the campaign 5-min re-scan incident)

Technical impact: no test coverage exists for any of these sweeps (confirmed by D4 – zero Cloud Functions tests), so a regression .that slows one sub-job wouldn't be caught before it eats into the shared 60s budget of six others

Recommendation: give `cronEvery5Min` and `cronEvery60Min` the same `timeoutSeconds` / `memory` treatment `cronEvery15Min` .got, and/or wrap each `runSweep()` call with its own timeout so one slow sub-job can't starve its siblings in the same invocation .Expected benefit: removes a growth-time single point of failure that would otherwise fail silently

Effort: S

F-FIREBASE-6 – All 64 functions run at `minInstanceCount: 0` ; every user-facing callable eats a cold start against one shared 14,000-line bundle

Severity: P2

Confidence: Confirmed (live `gcloud functions describe --format=value(serviceConfig.minInstanceCount)` looped over all 64 deployed functions – every one returns empty/0)

Feature / Screen / Flow: every interactive callable, most notably `commitRoundStats` (end-of-round save during a live match) and `saveGamePhysical` / `adminAddPlayers` / `sendGameInvite`

Evidence: loop over `gcloud functions list` → `gcloud functions describe <fn> --format="value(serviceConfig.minInstanceCount)"` for all 64 names produced no non-zero result. `setGlobalOptions({ region: 'us-central1', maxInstances: 10 })` (`index.ts:90`) sets no `minInstances` , and .no individual function overrides it either

Current behaviour: any function idle long enough to scale to zero pays a full Cloud Run cold start on its next call. Because the entire Cloud Functions surface is one source file (`functions/src/index.ts` , ~14,000 lines, 64 exports), every function's container has to load and initialize that whole module graph (all imports, all `defineSecret` s, every top-level constant) before it can execute the one “.function that was actually called – cold start cost is closer to “load the whole backend” than “load one endpoint

Problem: `commitRoundStats` is called at a moment the user is actively watching a spinner (end of a mini-game round, mid live-match) – a multi-second cold start there is a felt UX hit, not a background-job nuisance

User impact: SUSPECTED – not verified at runtime (no live environment available), but structurally, the first `commitRoundStats` / `sendGameInvite` /etc. call after an idle period will be materially slower than a warm one

Technical impact: at 10 `maxInstances` and near-zero idle traffic between evenings (this is a once-or-twice-a-week usage pattern .per club), essentially every evening's *first* round-commit is a cold call

Recommendation: set `minInstances: 1` on `commitRoundStats` (the one truly interactive, latency-sensitive path) – cheap at this .scale (well under the Cloud Run always-on free-tier-adjacent cost for one 256MiB instance) – rather than all 64 .Expected benefit: removes cold-start latency from the one action a user is actively staring at a loading state for

Effort: S

F-FIREBASE-7 – App Check is off everywhere it could apply, and was never wired into Firestore rules either – real exposure is unmetered callable volume, not authorization bypass

Severity: P2 •

Confidence: Confirmed (source + live: `ENFORCE_APP_CHECK = false` at `functions/src/index.ts:103` ;
`grep -n "app_check" firestore.rules` → only one comment reference, no actual rule logic)

{ `enforceAppCheck: ENFORCE_APP_CHECK` } Feature / Screen / Flow: all ~30 `onCall` functions that pass •

Evidence: every `onCall` in `index.ts` except `getServerTime` (hardcoded `enforceAppCheck: false`) uses the shared flag, currently `false`. Firestore's `firestore.rules` (1,988 lines) never references App Check tokens – client Firestore reads/writes were never gated by it in the first place, so flipping the flag back on would only affect the callable surface, not the much larger direct-.Firestore-write surface

Current behaviour: with App Check off, the only barrier between an attacker and any of these ~30 callables is a valid Firebase Auth session – trivially obtained via open email/password self-signup. Cross-checking authorization inside each callable (spot-checked `adminAddPlayers`, `adminReorderRoster`, `addRetroGoal`, `saveGamePhysical`, `savePitchCalibration`, `uploadGroupCover`, `notifyTeamsReady`) shows every one independently re-validates ownership/admin/roster membership server-side – so this is **not** a privilege-escalation gap (an attacker can't forge another user's stats or another club's admin rights through these). What's actually exposed is **unmetered invocation volume**: of all ~30, only `createGroupCallable` and `sendGameInvite` carry a server-side rate limit (`serverRateLimits` collection, deny-all from the client). `commitRoundStats` (writes a batch of up to several hundred ops per call, per its own in-code sizing comment), `addRetroGoal` / `removeRetroGoal`, `saveGamePhysical`, `adminAddPlayers`, `.promoteOrphanToGroup` have none. User impact: none directly (data integrity holds) •

Technical impact: an attacker who self-signs-up, creates a throwaway community/game (making themselves its admin – allowed by design), can script unlimited calls to `commitRoundStats` etc. against their own throwaway game, burning Cloud Functions invocations and Firestore write ops – a cost-abuse vector, not a data-integrity one

Recommendation: per memory this is already a known, tracked item pending App Attest verification – this finding's contribution is narrowing the actual blast radius (cost/quota, not authz) and flagging that the 2 rate-limited callables should not be treated as representative; the rest have zero throttling

Expected benefit: correctly scopes the remediation priority – App Check re-enablement matters for cost containment, not data safety •

Effort: (tracking only – re-enablement plan already exists per memory) •

F-FIREBASE-8 – 6 of the ~26 “server-tunable” Remote Config keys were never published; the live template is frozen since 2026-06-04

Severity: P2 •

Confidence: Confirmed (live `firebase remoteconfig:get -P soccer-app-52b6b`) •

Feature / Screen / Flow: `feature_campaigns` (campaign-system kill-switch), `games_feed_rich_min`, `games_feed_demand_min`, `games_feed_clubs_max`, `games_feed_clubs_radius_km`, `games_feed_clubs_min_members` (discovery-feed density thresholds) •

Evidence: `src/services/remoteConfigService.ts:22-75` (`RC_DEFAULTS`) lists 26 keys with code-level defaults. The live published template (`firebase remoteconfig:get`, parsed) contains exactly 21 keys across 5 parameter groups – `feature_campaigns` and all 5 `games_feed_*` keys are absent from every group. The template's only version is `versionNumber: '1'`, `updateTime: '2026-06-04T12:09:09Z'`, description “Initial app config knobs – created via API” – never updated since (roughly 3 months as of this audit)

Current behaviour: the app code calls the same `useRemoteConfig()` /getter pattern for all 26 keys uniformly, so nothing in the client distinguishes “this key can actually be changed from the console” from “this key will always return its hardcoded default no matter what”

Problem: `feature_campaigns` is documented (per D4/memory) as the master kill-switch for the entire in-app campaign system (popup, eligibility query, presence ping, engagement events) – exactly the kind of lever you'd want in an emergency (a bad campaign misbehaving in production). It doesn't exist as a remote lever at all right now; killing it requires a code change + rebuild + store review, which for iOS alone (per memory) can take days

- User impact: none today (nothing currently relies on flipping these), but it's a false safety net – anyone believing `.feature_campaigns` is a fast off-switch is wrong
- Technical impact: same class of gap the project has hit before with `appConfig.latestVersion` (memory: “watcher not running, soft `.popup` dead”) – a control plane that exists in code but not in the operational surface that's actually wired up
- Recommendation: publish the 6 missing keys to the live Remote Config template (matching the existing code defaults costs nothing and makes them real), or explicitly document that discovery-feed thresholds and the campaign kill-switch are code-only until then
- Expected benefit: `feature_campaigns` becomes an actual emergency lever instead of a config object that only look like one
- Effort: S

F-FIREBASE-9 – No account-linking / collision handling between Google and Apple sign-in

- Severity: P2
- Confidence: Confirmed (code) / Suspected (runtime manifestation depends on the project's default “one account per email” Identity Platform setting, which is Firebase's standard default and was not independently verified live in this pass)
- (`src/firebase/auth.ts`) Feature / Screen / Flow: sign-in
- Evidence: `signInWithEmail` (`auth.ts:160–193`) explicitly catches `auth/email-already-in-use`, calls `fetchSignInMethodsForEmail`, and throws a typed `EmailRegisteredWithProviderError('google'|'apple')` so the UI can redirect the user to the right button – but this handling exists **only** for the email/password-vs-social collision.
- `grep -rn "account-exists-with-different-credential" src/` returns zero hits anywhere in the app, and no `.linkWithCredential` call exists anywhere (`grep` for it in `auth.ts` is empty) – there is no account-linking flow at all
- Current behaviour: if a user first signs up via Google with email X, then later taps “Sign in with Apple” using the same email X, Firebase Auth (under its default single-account-per-email policy) throws `auth/account-exists-with-different-credential` from `.signInWithCredential` (`auth.ts:110 / :386`). Nothing in the app catches that specific code
- Problem: unlike the email/password case (which has a dedicated, well-thought-out redirect), the Google↔Apple cross-provider collision has no special handling – it falls through to whatever generic catch block wraps the sign-in screen's call, almost certainly a “generic Hebrew error toast with no guidance toward “you already have an account, try the other button
- User impact: SUSPECTED – not runtime-verified – a real user who has, say, an Apple ID and a Google account sharing one email (common) and mixes up which button they used originally would get stuck with a confusing error and no path forward except contacting support
- Technical impact: none beyond UX – no data-integrity risk, since the collision is Firebase Auth itself refusing the second sign-in
- Recommendation: catch `auth/account-exists-with-different-credential` in both the Google and Apple sign-in call sites, mirroring the existing `EmailRegisteredWithProviderError` pattern (the error even carries `error.customData.email` letting you resolve which provider actually owns it via `fetchSignInMethodsForEmail`)
- Expected benefit: closes the one sign-in collision case that currently has no guided recovery
- Effort: S

F-FIREBASE-10 – No Firestore TTL policies configured; marketing click-logs and per-club event timelines have no cleanup at all

- Severity: P3
- Confidence: Confirmed (live `gcloud firestore fields ttls list` → 0 items; source read of `runDailyCleanup`)
- Feature / Screen / Flow: `linkClicks`, `inviteClicks`, `adLinks` (marketing attribution logs), `communityPlayerEvents` (per-club discipline/ball/jersey timeline)
- Evidence: `gcloud firestore fields ttls list --database='(default)'` → Listed 0 items. `runDailyCleanup` (`functions/src/index.ts:3786–3931`) explicitly prunes exactly four things: `notifications` (30-day TTL via manual query), `gameUpdateLatches` (terminal-game check), `groupJoinRequests` (90-day TTL on decided ones), `games/{id}/fillerInterests` (terminal-game check). Nothing else in the codebase touches `linkClicks`, `inviteClicks`, `adLinks`, or `.communityPlayerEvents` for deletion (confirmed no other `.delete()` call sites reference these collection names in `index.ts`)

- Current behaviour: these four collections are unbounded event logs that grow forever with zero automated cleanup – no TTL policy
 - .at the database level, no cron sweep at the app level
- Problem: at 596 users this is cheap today, but it's the same “grows with all-time activity, not active users” shape flagged elsewhere
 - in this audit (F-FIREBASE-8's neighbor concern) – just for logging data rather than a hot query path, so it manifests as storage cost
 - .drift rather than a read-cost incident
 - .User impact: none
 - .Technical impact: slow, compounding storage cost; no functional risk
- Recommendation: either add a Firestore TTL field (cheapest – a native TTL policy on a `createdAt` / `at` field needs no cron code at
 - .all) to these four collections, or fold them into `runDailyCleanup`'s existing pattern
- Expected benefit: bounded storage cost with near-zero engineering effort (TTL policies are a one-time
 - .`gcloud firestore fields ttls update` per collection, no code change)
 - Effort: S

(`deleteCurrentFirebaseUser`) F-FIREBASE-11 – Dead client-side account-deletion code path

- Severity: P3
- Confidence: Confirmed
- `src/services/userService.ts:28` Feature / Screen / Flow: `src/firebase/auth.ts:493–539`, imported (but unused) in
 - Evidence: `grep -n "deleteCurrentFirebaseUser" src/services/userService.ts` shows only the import line; the actual
 - deletion flow (`userService.ts:680–699`) calls the `deleteMyAccount` Cloud Function callable exclusively.
 - `grep -rln "deleteCurrentFirebaseUser" across src/` returns only `auth.ts` (definition) and `userService.ts` (dead import) –
 - .no call site anywhere
- Current behaviour: a full client-side re-auth-and-delete implementation (handling both password re-auth via
 - `NeedsPasswordReauthError` and Google re-auth) sits in the codebase, unreachable, superseded by the server-side cascade (per
 - .its own in-code comment at `userService.ts:680–688`, which explains exactly why the client-only flow was replaced)
- Problem: purely a maintenance hazard – a future engineer grepping for “delete account” logic could find and reason about this
 - function, not realize it's dead, and either modify it uselessly or worse, wire it back in and reintroduce the exact bug class the
 - .comment describes it was built to avoid (game-sweep-then-cancellable-reauth race)
 - .User impact: none (unreachable code)
 - .Technical impact: none functionally; pure clarity debt
- Recommendation: remove the dead import and either delete `deleteCurrentFirebaseUser` or leave a `@deprecated` note that
 - .makes clear it must not be wired back in
- Expected benefit: removes a plausible foot-gun for a future contributor
- Effort: S

Areas checked and found fine (no padding)

- Messaging fan-out:** `deliverBatch` (`index.ts:1420+`) correctly chunks `sendEachForMulticast` at the FCM-mandated 500-
 - token cap (both the main notification pipeline and the roster-changed direct path at `index.ts:4399`), prunes dead tokens via
 - `DEAD_TOKEN_CODES` plus an APNs-shape heuristic (`looksLikeApnsToken`), and prunes from both the legacy root and the private-
 - .subcollection token store. No issue found
- batch sizing:** explicitly caps side size at 11 players specifically to stay under Firestore's 500-op batch limit `commitRoundStats`
 - (worst-case ≈400 ops), with a clear, documented failure mode instead of a silent batch-overflow – good defensive engineering, cited
 - .for completeness rather than as a gap
- Filler-candidate pool caching** (`getFillerCandidatePool`, `index.ts:10025–10053`): a deliberate, well-commented 3-minute in-
 - memory cache that explicitly exists to prevent exactly the class of “read cost scales with all users” incident the brief calls out (its
 - .own comment cites the concern by name) – this is the fix pattern, not an instance of the bug
- aggregation usage:** only one call site in the whole app (`userService.ts:785`, `getInvitedUsersCount`) – low usage is `count()`
 - not itself a bug (most “counts” the backend needs are geo/status-filtered and can't be satisfied by a bare aggregation query anyway,

.per availabilityCounts ' design), so this is noted rather than flagged

Anonymous/guest auth: minimal and correctly scoped – signInAnonymously (auth.ts:407) exists solely to satisfy •
isSignedIn() rules for browsing public communities/games pre-signup; guests never get a /users doc, so there's no orphaned-
.account accumulation risk from this path

נספח F – מוצר, סטטיסטיקה ואסטרטגיה

STATS

Teamder statistics audit – football-stats expert × data scientist × sceptical statistician

Static analysis only (no runtime). All three roles read the same code; where they disagree it is shown, not averaged. Every code claim below is Confirmed (read
. SUSPECTED – not verified at runtime directly); anything about what a player perceives is

Part A – audit of every existing metric

Inventory read

```
src/utils/clubChemistry.ts , src/utils/roundSummary.ts , src/utils/eveningScore.ts  
+ src/data/achievements.ts , src/services/clubChemistryService.ts +  
, src/utils/penaltyStats.ts , src/utils/clubLevel.ts , src/data/clubAchievements.ts  
, src/utils/championship.ts , src/utils/teamBalanceCore.ts  
, src/services/trustService.ts , src/services/playerStatsService.ts  
gameService.ts:891/1067/1195/1237 , ( commitRoundStats ) functions/src/index.ts:12753  
( getGameChampionship / getCommunityDeadlyDuo / getCommunityChampionship / getCommunityStats )
```

F-STATS-1 – Evening score's win axis has a coin-flip granularity of ~1.0 point on a 4-point scale

Severity: P2 (meaningful) •

Confidence: Confirmed (read + simulated) •

src/utils/eveningScore.ts - ("איזה ערב היה לך") Feature / Screen / Flow: Evening summary card •

:Evidence: eveningScore() line 116, winsScore = clamp10((wins/gp)*10) , weight 0.5 (no shootout). Simulated (python3) •
gp=2: wins=1/2 -> score 7.3 wins=2/2 -> score 8.3 ($\Delta = 1.0$, same goals/assists) gp=3: 0/3->6.3 1/3->7.0 2/3->7.6 3/3->8.3 gp=4: 0/4->6.3 1/4->6.8 2/4->7.3 3/4->7.8 4/4->8.3

Current behaviour: Displayed score is 6.0-10.0, one decimal, framed to the player as a considered verdict on "how was your
"evening

Problem: A typical evening is 2-4 mini-games (gp). At gp=2 the win axis only has 3 possible values (0, 0.5, 1 win-rate), so a single mini-game result – frequently decided by one bounce, one own goal, or a shootout coin-flip – swings the *entire displayed score* by a full point, i.e. 25% of its whole dynamic range. The score reads as precise (one decimal place) but its dominant input (50% weight) is
.measured on 2-4 discrete outcomes

User impact: SUSPECTED – not verified at runtime – a player who lost one contested mini-game gets materially "worse" •
evening score than an identical evening where the coin landed the other way, which will read as arbitrary/unfair to a football player

- .who knows results at this scale are noisy
- Technical impact: None – the code is internally correct and well-documented (the header explicitly explains the weight design). This is a statistical-validity gap, not a bug
- Recommendation: Either (a) widen the effective granularity by blending win RATE with a shrinkage prior toward 50% for gp<4 (regress small samples toward the mean before scaling to 0-10), or (b) reduce the win axis's weight for very small gp, or (c) simply document/soften the framing ("this evening" vs "your form") so the number isn't read as more precise than the sample supports
- Expected benefit: A score that doesn't visibly punish/reward a single coin-flip result at the same magnitude as a whole evening's goal tally
- Effort: S (formula tweak + re-run existing test suite eveningScorePen.test.ts)

F-STATS-2 – Club level rewards headcount + calendar age as much as actual play; a fully dormant club can match an active one

- Severity: P2 (meaningful)
- Confidence: Confirmed (read + computed)
- src/utils/clubLevel.ts:22-27 – ("רמת מועדון") Feature / Screen / Flow: Club level
- Evidence: clubPoints = gameNights*10 + clubGoals*1 + members*8 + ageMonths*6 . Computed two scenarios with : thresholdFor
- Active young club (20 game-nights, 150 goals, 10 members, 3 months old) -> 448 pts -> level 4 "מועדון מבוסס"
- Dormant old club (0 game-nights, 0 goals, 40 members, 24 months old) -> 464 pts -> level 4 "מועדון מבוסס"
- Current behaviour: A club that has never held a single evening (0 game-nights, 0 goals – just accumulated 40 invited members over .two years) reaches the SAME tier as a club that played 20 evenings and scored 150 goals in 3 months
- Problem: members*8 and ageMonths*6 accrue passively (invite people once, do nothing, wait); gameNights*10 is the only term that requires actual activity, and it isn't weighted enough to dominate the other two. The metric is billed as "how big/established is .this club" (file header) but reads to a player as an achievement of activity
- User impact: SUSPECTED – not verified at runtime – a genuinely active small club could feel their level is "capped" by a .competing metric (membership/age) that has nothing to do with the football actually played
- Technical impact: None
- Recommendation: Either drop ageMonths and members to a smaller multiplier, gate the tier on a minimum gameNights threshold .regardless of points, or split "club size/age" from "club activity" into two separate displayed numbers instead of one blended score
- Expected benefit: The level badge stops rewarding an idle roster
- Effort: S

F-STATS-3 – playerStatsService 's "biggest victim"/"nemesis"/pair superlatives have no sample floor, unlike the near-identical club-chemistry feature

- Severity: P2 (meaningful)
- Confidence: Confirmed (read)
- src/services/playerStatsService.ts:157-227 – ("סטטיסטיקה") Feature / Screen / Flow: Personal statistics screen
- Evidence: consider() (line 157) crowns biggestVictim / nemesis / mostWinsWith / mostAssistedTo/By purely by > comparison on raw counts, no minimum games together. Contrast src/utils/clubChemistry.ts:180-195 (CHEMISTRY_MIN), whose own comment states the reasoning directly: "a brand-new club would otherwise crown the winning duo – 1 win", which is noise wearing a trophy... one goal between two players is a pass, not a partnership." playerStatsService implements the exact .same category of stat (pair superlative from pairStats) without that guard
- Current behaviour: A player who has played exactly one mini-game against someone and won it gets crowned that person's .permanent "nemesis" or "biggest victim" on the profile stats screen
- Problem: Same failure mode the team explicitly identified and fixed elsewhere in the codebase (clubChemistry.ts), left unfixed .here. The two features share the same underlying pairStats collection
- User impact: SUSPECTED – not verified at runtime – a brand-new player's first-ever mini-game instantly produces a labelled .“nemesis,” which is a much stronger claim than the one data point supports
- Technical impact: None; purely a threshold gap

Recommendation: Add the same style of floor clubChemistry.ts uses (e.g. require ≥3·5 meetings before naming a nemesis/victim), and prefer the pair's win-RATE over raw count for the head-to-head fields specifically (right now myWins / myLosses are raw counts, so a player who has played someone 20 times and split 11-9 shows the same "nemesis" strength as someone 1-0 against a stranger).

.Expected benefit: Consistent statistical standards between two features that read the same table

Effort: S

F-STATS-4 – Club table ("points" sort) ranks by raw win COUNT, not win rate or per-round efficiency

Severity: P2 (meaningful)

Confidence: Confirmed (read)

Feature / Screen / Flow: Club stats dashboard / champions table – src/utils/championship.ts:114–137, used by (gameService.ts:1160) getCommunityChampionship

Evidence: sortBy === 'points' (the community-table sort) → b.wins – a.wins || b.goals – a.goals || b.assists – a.assists. Comment: *"points" (community table): sort by WINS, then goals, then assists (per user request – the club table ranks by success first).* Contrast the SAME FILE's perGameScore() (line 51-61), used for the per-GAME table, which explicitly normalizes by rounds played specifically because raw score "wrongly ranked [players] above efficient per-game scorers"

Current behaviour: The club-wide leaderboard ranks purely by cumulative wins. wins has no denominator anywhere in the sort – .not win-rate, not wins-per-round, not wins-per-evening

Problem: A player who attends every week (more rounds played = more chances to win, regardless of skill) will structurally out-rank a less frequent but higher-win-RATE player. The file's own author already understood this failure mode well enough to build perGameScore for the other table on the same page – the community table deliberately opted out ("per user request"), so this is a known, intentional design choice, not an oversight. Flagged anyway because the resulting table functions substantially as an .attendance/tenure leaderboard dressed as a "success" ranking

User impact: SUSPECTED – not verified at runtime – a newer, in-form player cannot climb the visible club table quickly no .matter how well they're playing, because the incumbent's count-based lead is structural

.Technical impact: None (working as specified)

Recommendation: If the intent really is "who has WON the most" (a legitimate, simple thing to show), keep it – but consider showing win-rate as a secondary column so a viewer isn't misled into reading position as skill. This is explicitly a "user requested it this way" .item, include for completeness but treat as P3 if the product owner still wants raw wins

.Expected benefit: Table position matches player intuition about "who's actually good right now" as well as "who's been around"

Effort: S (add a secondary displayed column; no sort change needed if raw-wins ranking is intentional)

F-STATS-5 – Trust score's 0-100 scale is nearly binary at the minimum sample size

Severity: P3 (polish)

Confidence: Confirmed (read + computed)

src/services/trustService.ts – Feature / Screen / Flow: Reliability meter

Evidence: MIN_GAMES_FOR_SCORE = 3 (line 48). At registered=3 : possible attendanceRate*100 values are only {0, 33, 67, 100} (rounded). One no-show at the minimum sample instantly drops the score from 100 ("excellent" tier, ≥90) to .67 ("basic" tier, 50–74) – a full tier change from a single event

Current behaviour: The score is presented as a smooth 0-100 reliability number with 5 named tiers (.new/low/basic/good/excellent), with tierForScore boundaries at 50/75/90

Problem: At n=3 there are only 4 discrete score values, so the meter's apparent 100-point resolution is fake for a player who just .crossed the eligibility floor – the display implies far more precision than 3 data points can support

User impact: SUSPECTED – not verified at runtime – a player with a single early no-show (for any reason – illness, work) looks .meaningfully "less reliable" (basic vs excellent) off one event, right when the score first becomes visible to admins

.Technical impact: None

Recommendation: Raise MIN_GAMES_FOR_SCORE slightly (5-6) or make the "new" chip persist a bit longer, so the first visible score already reflects a less all-or-nothing sample. Lower priority than F-STATS-1/2 since this score is not shown to the player as a badge

- .of honor in the same way – it's an admin-facing reliability signal, and the underlying attendance data is real either way
- .Expected benefit: Fewer “one bad week ruins my rating” perceptions at the exact moment the score becomes visible
- Effort: S

F-STATS-6 – Trust score ignores the `late` arrival status it already collects

- Severity: P3 (polish)
- Confidence: Confirmed (read)
- `src/services/trustService.ts:134` – Feature / Screen / Flow: Reliability meter
- Evidence: `ArrivalStatus = 'unknown' | 'arrived' | 'late' | 'no_show' (src/types/index.ts:1923).` `trustService.ts` `. 'arrived'` line 134: `if (arrival !== 'no_show') { attended += 1; }` – `'late'` is treated identically to
- .Current behaviour: A chronically-late player scores exactly as “reliable” as a punctual one, as long as they eventually show
- Problem: The app already distinguishes late arrivals in the data model, and a reliability score is precisely the kind of metric a “late” `.flag` should feed, but it's discarded at read time
- User impact: `SUSPECTED – not verified at runtime` – none currently, since nothing surfaces this distinction anywhere (see Part `.B` for a proposal to actually use it)
- .Technical impact: None; dead signal
- .Recommendation: See Part B (punctuality rate). Low priority on its own
- Effort: S

F-STATS-7 – `UserStats.goals` is a known-dead metric still shown as a stable value

- Severity: P3 (polish)
- Confidence: Confirmed (per discovery D3 + type comment)
- Feature / Screen / Flow: Profile
- Evidence: `src/types/index.ts` (~line 695) – comment states this lifetime counter “is not currently written by any path,” kept only `.so` the profile UI renders a stable `0`. Distinct from the live `communityPlayerStats` goals total
- Current behaviour: A field exists, is typed, and is presumably rendered somewhere as “0” forever, while the real number lives `.elsewhere`
- Problem: Dead metric occupying UI/type surface. Not misleading in the harmful sense (it's a static 0, not a wrong nonzero number) `.but` it's the textbook “nobody would miss this” candidate the brief asks to call out
- .User impact: None currently (renders 0, presumably hidden or ignorable)
- Technical impact: Maintenance noise – a future refactor could easily “fix” this field forward and reintroduce confusion between two `.goals` counters
- Recommendation: Remove the field, or wire it to the same source `communityPlayerStats / gamePlayerStats` aggregate `.elsewhere` pulls from
- .Expected benefit: One less place where “goals” can silently mean two different things
- Effort: S

Metrics judged fine as-is (one line each, per brief's “say so and move on”)

- leaderboard ranking (Wilson lower bound, `wilsonLowerBound()` line 120)** – the most statistically rigorous `penaltyStats.ts` `.piece` of the whole codebase; correctly discounts small-sample 100% rates against high-volume proven scorers while still `DISPLAYING` the raw %, which is exactly right. No note needed – this is the standard the rest of the app's superlatives should be `.held` to
- records/milestones (`MIN_RECORD_BASIS = 5` evenings, `tied` vs `broke` distinction, guest exclusion from `roundSummary.ts` `.records`)** – genuinely careful about sample size and what a “record” honestly means; the file's own comments already document every edge case an auditor would otherwise flag (own goals, retro-goal reconciliation, team identity by bib colour not roster). Best-`.documented` file in the audit
- pair picks (`CHEMISTRY_MIN` floors, deliberate refusal to blend into one “chemistry score”)** – correctly `clubChemistry.ts` `.declines` to invent a single number from incommensurable counters; explicitly the model `playerStatsService` (F-STATS-3) should `.have` copied

not a player-facing metric (an allocation algorithm), out of scope for interpretability concerns; its underlying `teamBalanceCore.ts` •
.input (admin 1-10 internal rating) is itself never shown to the rated player, which is a product-privacy choice, not a stats-validity issue

clean-sheet ladder – thresholds explicitly calibrated against a real measured club (comment cites “21 in 5 `achievements.ts`” •
.nights” for the leader), a rare case of a threshold that was actually validated against data rather than guessed

crediting logic (goals/assists/wins/rounds/clean sheets/own goals/penalties) – extremely defensive: `commitRoundStats` •
gates on end-state on-field roster, excludes no-shows from both sides, caps side size against the Firestore 500-op ceiling, keeps
guest stats out of lifetime/community tables while still surfacing them per-game. This is the ground-truth data layer everything above
reads from, and it is sound. The one known limitation (mid-round substitutions get nothing – already in ground truth) is a
.completeness gap, not a validity gap

Part B – new metrics computable from data already collected (highest-value section)

All proposals below require **zero new inputs** – every field cited already exists in `roundHistory`, `gamePlayerStats`,
. `communityPlayerStats`, `communityPairStats`, or `games.arrivals`. Grouped by type

Why a player cares	Formula / source	Metric	#
You're a closer" / "you always score early" is a real, fun, comprehensible personality stat nobody currently sees despite the .data already existing per-goal	Goal minute field already captured per-goal (<code>AdvancedLiveMatchScreen.tsx:1594</code> , clock-based, stored in <code>roundHistory.goals[].minute</code>). Bucket goals into first-third / middle-third / final-third of the mini-game's elapsed clock; report % of your goals scored in the final third and .first goal of the mini-game rate	Clutch / opener rate	B1
Complements the trust score with a distinct, honest signal ("shows up, but often late" vs "no-shows") instead of collapsing both into one .meter	vs 'arrived' , already collected per game, currently <code>arrivals[uid] === 'late'</code> .lateRate = lateCount / registeredCount .discarded (see F-STATS-6)	Punctuality rate	B2
Genuinely useful football insight most players intuit but have never seen quantified ("I'm a different player in 11-a-side than 5-a-side") – currently invisible because every stat blends all formats .together	already stored per game (<code>teamSizeFromFormat()</code> / <code>games.format src/types/index.ts:1058,1338</code>). Join <code>gamePlayerStats</code> by the parent game's .format; report goals-per-evening, win-rate, and clean-sheet rate PER FORMAT	Format splits (5v5 vs 7v7 vs 11v11 performance)	B3
Locker-room folklore ("we always lose in the yellow bibs") turned into an actual, checkable number .– cheap, novel, shareable	bib colour already recorded per mini- <code>roundHistory.teamAIndex/teamBIndex</code> game (<code>roundSummary.ts teamsOf()</code>). Per player, per evening: win-rate when on .each bib colour, aggregated over time	Team-of-the-night / lucky-colour effect	B4
Comeback king" is a strong" emotional stat and the ordered goal log already supports it .without any new instrumentation	per mini-game already stored. For a player's team, track games <code>scoreB / scoreA</code> where their side trailed at any recorded goal event and still won (needs only goal order, .already timestamped by <code>minute</code>)	Goal-difference swing / comeback rate	B5
Turns a noisy superlative into a trustworthy one – "you're 8-2 against him (80%)" reads as a real claim, "you beat him 8 times" doesn't say whether that's out of 8 .or 40 meetings	already exists (<code>communityPairStats.winsA/winsB/against clubChemistry.ts</code>). Currently <code>playerStatsService</code> 's nemesis/victim uses raw win counts (F-STATS-3); a rate-based version (<code>winsA/against</code>) with the SAME sample .floor <code>clubChemistry.ts</code> already defines is a direct fix-and-extend	True head-to-head win RATE (not raw count)	B6
The deadly-duo card already shows the aggregate; this reframes the SAME underlying counters from the passer's perspective – "who scores off my passes most" – a distinct, equally cheap question nobody currently .asks of the data	already exists. For a given scorer, which <code>communityPairStats.assistsAToB/BToA</code> teammate's assists they convert most often relative to how often that teammate ALSO assists other people (i.e., "who finishes YOUR passes best" from the assister's point of .view, not just the scorer's)	Assist-to-goal conversion by receiver	B7
games running with a goal" is a 3" much more football-native hot streak than an attendance streak, and the attendance-streak code is a template already proven to .work	already computed ((attendance streak) <code>communityStats.currentStreakByUser gameService.ts:1057</code>) but only used for the assistant's coach lines per discovery D3. A parallel PERFORMANCE streak (consecutive mini-games with a goal or a win) is the .same code shape over <code>gamePlayerStats / roundHistory</code> , no new query	Streak-adjusted "hot/cold" indicator	B8
A raw count rewards attendance again (F-STATS-4's pattern); the rate is the honestly comparable number across players with very .different games-played totals	already in <code>communityPairStats</code> ; per-player <code>sameTeam / cleanSheetsTogether cleanSheets / rounds</code> already computable from <code>communityPlayerStats</code> . Currently only the raw COUNT is shown (achievements ladder, championship column). A RATE (<code>cleanSheets/rounds</code> , e.g. "42% of your mini-games end with a clean sheet") is .free	Clean-sheet involvement rate, not just count	B9
Shows whether a player's offensive involvement is trending up/down over a season –	already written per player <code>gamePlayerStats.teamGoalsFor/teamGoalsAgainst</code> per game (<code>functions/src/index.ts ~13320</code> , feeds evening-summary "contribution%"	Team-goal-share	B10

Why a player cares		Formula / source	Metric	#
currently a one-night snapshot that's thrown away after the .summary screen closes	per the discovery notes) but only surfaced for a single evening. Plotting it over the last N .evenings is a read of existing per-game rows, no new write		(contribution %) trend over time	
A genuine, high-drama football stat ("ice in your veins") built entirely from data already logged .in kick order	already exist with Wilson-ranked penTaken/penScored/penFaced/penSaved leaders (penaltyStats.ts). Nothing currently answers "how do you perform in a shootout SPECIFICALLY when your team needs a save/goal to survive" (sudden-death kick), because kick ORDER within the shootout is already stored (.roundHistory.penalties[] array order) but never read for this		Shootout clutch rating	B11
Rewards current form rather than lifetime totals, directly countering the tenure bias flagged in F-STATS-4 – same underlying data, .different lens	rollups already give lifetime cumulative goals/wins/rounds; a communityPlayerStats trailing-window delta (last 5 evenings' per-round rate vs the 5 before that) needs only .reading roundHistory / gamePlayerStats docs already written, no new field		Most" improved" (trailing-N-evenings delta)	B12

Part C – new data worth collecting (max 2, must survive the “no minute-tracking, no per-substitution prompts, no tackles/saves” constraint)

.The three roles disagree here, so both views are shown rather than merged
:Football-stats expert + data scientist (converge on one proposal)

Verdict	What it unlocks	UX cost	New input	#
Reject as new collection – already .B3	–	Zero extra prompts if it's just reading the existing per-game format field harder (see B3) – this is a data-USE gap, not a data-COLLECTION gap. Recommend against building this as new collection; fold into .B3 instead	Per-mini-game format/pitch-size tag when it's NOT already implied by the game's registered format (only relevant for evenings that mix formats mid-session, e.g. a big roster splitting into different sub-formats across the night – check whether this already exists before building)	C1
Accept – cheapest possible lever, evening-level only, matches the “no per-event/per-substitution” .constraint exactly	Unlocks filtering every metric in Part A/B by “does this evening count for form/records purposes” – directly strengthens F-STATS-1's small-sample problem (a scrappy trial evening currently pollutes the same evening-score/records pipeline as a full competitive night) and gives roundSummary.ts 's MIN_RECORD_BASIS a .cleaner denominator	One extra tap for the ADMIN only, once per evening, not per player and not mid-game – the cheapest possible new input by .construction	A single optional post-evening admin tag: “was tonight competitive / casual / trial-run?” (one tap, evening-level, admin-only, no per-player or per-event prompt)	C2

Sceptical statistician's dissent: Even C2 should be scrutinized before building – it is a NEW field that requires the admin to remember to set it every single evening, and an unset/mis-set flag is worse than no flag (a null this-was-competitive field silently defaults to “counts,” so nothing is actually gained until admin compliance is high, which is unverifiable without runtime data). Given ~596 total users and games this small, the sceptic's position is: **the existing data volume is already the binding constraint on every metric in Part A (see Part D) – no new INPUT fixes a sample-size problem that is fundamentally about total games played, not about a missing tag.** If forced to rank, the sceptic would spend the “one new admin tap” budget instead on **nothing** and put the effort into Part .A fixes (shrinkage/floors), which cost less UX and address the same root cause more directly

Consensus: All three roles agree nothing beyond C2 clears the bar, and even C2 is optional. No proposal here needs minute-tracking, per-substitution prompts, or event collection (tackles/saves) – none was seriously considered because the product constraint rules them .out before any statistical merit is even evaluated

Part D – the sceptic’s section: which numbers are noise dressed as signal

D1. Evening score – quantified above (F-STATS-1). At `gp=2` (very common – many evenings a given player only plays 2 mini-games), the win axis has 3 possible values total. The score is displayed to one decimal place across a 4-point range; a third of that range moves on a single coin-flip result. This is the single most .over-precise number in the app relative to its underlying sample

D2. Club-table position at the individual-club scale (~596 users total across the app; the “biggest club in the app” is cited in-code as 136 goals / 81 mini-games after four months – `roundSummary.ts` comment). At that :scale

A club table sorted by raw wins (F-STATS-4) is dominated by who has attended the most rounds, not who is best – with ~81 mini-games total in the biggest club, the person who’s played 60 of them will out-count almost anyone on wins alone even at a mediocre win rate, simply from volume. Quantified: at a flat 50% win rate, $E[\text{wins} \mid \text{rounds}=60] = 30$ vs $E[\text{wins} \mid \text{rounds}=20] = 10$ – a .3x gap in the DISPLAYED ranking metric from ATTENDANCE alone, zero skill difference assumed

nemesis/victim (F-STATS-3) at this club size will very often be resolving from $n=1$ or $n=2$ meetings for anyone `playerStatsService` who isn’t a long-tenured regular – the exact “noise wearing a trophy” failure `clubChemistry.ts`’s own comments warn about, just .in the sibling feature that didn’t get the guard

D3. Club level (F-STATS-2) – quantified above: a fully dormant club (zero game-nights, ever) reaches the same tier as an active one purely from members×age. The football-stats-expert and data-scientist roles agree this is a genuine design bug (activity should dominate a “how established is this club” score); the sceptic goes further: any single blended score trying to represent “size + age + activity” in one number is inherently going to have a dormant-club/active-club collision somewhere on the curve – the fix in F-STATS-2 (reweight) only moves where the collision happens, it can’t remove it. The sceptic’s actual recommendation is to split it into two displayed numbers (age/size vs activity) rather than reweight a single .blend

D4. Trust score at minimum sample (F-STATS-5) – quantified above: 4 possible values at `n=3`, and a single no-show costs a full tier. The sceptic notes this one is LOWER risk than D1/D3 because it’s an internal admin signal, not a badge shown to the scored player as an achievement – the harm mode (unfairly judged) .is real but contained to admin-facing UI, not amplified by a public leaderboard the way D2 is

D5. Where the three roles genuinely disagree

Football-stats expert: The win-rate-based evening score (F-STATS-1) and raw-wins club table (F-STATS-4) are both *fine* – football fans are used to small-sample “form” narratives (a hot week, a cold week) and over-engineering shrinkage into a fun, ephemeral evening-recap card would make it feel clinical instead of exciting. The expert’s priority list starts at F-STATS-3 (nemesis/victim, because it names a SPECIFIC other person with almost no evidence) and F-STATS-2 (club level, because it’s a durable status .symbol, not a one-night mood)

Data scientist: Disagrees on priority – the data scientist ranks F-STATS-1 (evening score granularity) highest, precisely because it’s shown with false precision (one decimal, framed as a “score”) rather than as a narrative; a coarse discrete quantity dressed as a .continuous one is the textbook statistical-validity problem, regardless of how “fun” the framing is

Sceptical statistician: Disagrees with both – argues the entire premise of ranking these findings by severity is secondary to the observation in D2: **at current app scale (~596 users, biggest club ~81 mini-games in 4 months), almost EVERY leaderboard/superlative in this app is running on samples too small for the precision it displays**, and no single formula fix addresses that; only either (a) explicit floors/“new” states (which `clubChemistry.ts` and `trustService.ts` already do, correctly) rolled out consistently everywhere records/superlatives are shown, or (b) accepting these are meant as fun/social features rather .than statistically rigorous ones and labeling them that way, resolves it structurally

- **Best-designed files in the codebase, by a clear margin:** `roundSummary.ts`, `clubChemistry.ts`, `penaltyStats.ts` (Wilson score). Read these before building anything new – they already show the pattern (explicit sample floors, `tied` vs `broke` distinctions, refusal to blend incommensurable numbers into one score) that the weaker files should be brought up to
- **Files that repeat a mistake the codebase already knows how to avoid:** `playerStatsService.ts` (F-STATS-3, no floor – `clubChemistry.ts` sits right next to it with the fix already written), `championship.ts` 's community sort (F-STATS-4, raw count where a sibling function in the same file already knows to normalize)
- **Highest-leverage, lowest-cost next step:** Part B is pure upside – every proposal reads data already being written today. B1 (clutch/opener), B3 (format splits), and B6 (rate-based head-to-head) are each a single new query/derivation over existing collections, no schema change, no new UX prompt

PRODUCT

Teamder – Product Audit (Senior PM pass)

Static analysis only, per BRIEF.md. All runtime/behavioural claims are marked Evidence cites D1-D4 discovery docs and `.SUSPECTED` – not verified at runtime `./Users/matan/Projects/soccer` source files under

The core loop .1

The loop that has to fire every week for Teamder to matter is: **a club's regular evening gets scheduled → the right N players show up → something gets recorded about it → the record makes people want to come back** :Traced through real screens/services

- .1 **Schedule** – either a coach manually creates a game or – the actually-scalable path – a `(gameService.ts:2373 createGameV2` → steps 3, `GameWizardForm` → `GameCreateScreen`) `GameSeries` doc `, seriesService.ts`) auto-clones the same weekly slot D2 §3). This second path is the one that `, runCreateSeriesOccurrences` matters for “every week”; the wizard-per-week path doesn't scale to a real club's cadence
- .2 **MatchDetailsScreen** → (פתוחים/שלי) `GamesListScreen` – **Fill the roster** `join (gameService.ts:4995 joinGameV2)`, fair-tap-order queueing → + so a fast phone can't steal a spot, waitlist (`joinFairness.ts`) spot-offer confirm/pass flow. This is genuinely well-built (dedicated (pure module + tests, D3 §3
- .3 **Play** – `LiveMatchScreen` (plain timer, phone+watch synced) or, if the `,organiser` opted in at creation, `AdvancedLiveMatchScreen` (teams → `finalizeRoundAndRotate` (rotation, goals/assists, shootout is the single choke (server callable, `index.ts:12753) commitRoundStats` .point that turns “we played” into numbers
- .4 **+ Record & payoff** – `RoundSummaryScreen` (club-wide “what happened”) + (personal, shareable PNG, evening score) `EveningSummaryScreen`

history. This is the emotional `MatchRoundsScreen` / `CommunityStatsScreen` .close of the loop – the one moment designed to be shared outside the app

Repeat – the `GameSeries` clones next week's occurrence; nothing else .5 .has to happen for the loop to fire again, which is the right design

,The loop is coherent and the hard parts (fairness, idempotent stat commits recurring series) are genuinely solved with unit-tested pure modules. The weak link is step 3→4: `commitRoundStats` is gated on the END-STATE roster and ,(per BRIEF ground truth – a player subbed out mid-round gets nothing) , `functions/src/index.ts`) the entire server side that computes the payoff 14K lines, 64 exported functions) has **zero tests** (D4 §8). The loop's most emotionally important artifact (the evening summary) is produced by the .least-verified code in the codebase

Why would a club come back next week – and what breaks it .2

Why they'd come back: the recurring `GameSeries` removes the single biggest weekly failure mode of a WhatsApp group (“did anyone actually schedule this week?”), fair-join queueing removes the second biggest and the evening summary/round summary give ,(arguing about who got a spot) .a reason to open the app the *morning after*, not just before kickoff

:What currently breaks it

- A sub-out player getting zero credit (BRIEF ground truth) is exactly the kind of bug a club notices instantly – “I played the whole second half and I have 0 rounds” is a screenshot-to-the-group-chat bug, and it directly .contradicts the app's own flagship promise (an accurate record)
- has no in-app CTA (D1 §6, D3 “Personal clubs”) – a `PromoteOrphanScreen` group that started as a one-off quick game and wants to become a real club has to be reached by a single 30-minute-delayed push notification or never converts. This is the exact moment (right after a good first game) .when conversion should be easiest, and it's the weakest link in the funnel
- No crash reporting (D4 §4/§6, confirmed) and no offline-detection layer for a screen that's operated pitch-side on a field – (D4 §4, confirmed) with patchy signal (the live-match timer / goal entry), silent failures .are invisible to the team that ships the app, not just to the user
- The backend aggregate pipeline that produces the “why come back” artifact has documented drift risk in three places (D2 §4: `communityPairStats` 's non-transactional pre-check/create latch race; `communityShowcase` 's incremental-vs-recompute status unconfirmed; three different functions writing different field subsets of `communityPlayerStats` at different times) and zero tests. None of these are proven live bugs, but they're ,exactly the shape of bug that erodes trust in “the numbers” quietly .which is fatal for a stats-differentiated product (see §5)

The wow moment .3

SUSPECTED – not verified at runtime. The wow moment is the personal a – (“סיכום הערב” , `EveningSummaryScreen`) **evening summary card** ,shareable, PNG-exportable stat card with an evening score (0-6.0-10.0 a comparison vs. the player's previous evening, and ,(`eveningScore.ts` who you just passed” standings. It's the one artifact explicitly built to – leave the app (OS share sheet, `expo-sharing`) and be seen by non-users .the only genuine viral surface in the product besides invite links

:Taps/time from install (SUSPECTED, static estimate)

- Cold start → sign-in → profile setup → post-sign-in onboarding → land on .roughly 6-10 taps, under 2 minutes (D1 §4) : MainTabs
- From there to the wow moment requires: join or create a club (a few more taps), register for a game (1 tap if open), **wait for the real-world kickoff time**, play the evening, and either the organiser ends it or the server-side finish trigger fires – then tap . MatchDetailsScreen
- So the tap count to the wow moment is small (~10-15 taps total) but the **wall-clock time is not minutes, it's however many days until the next scheduled game** – for a brand-new solo user with no club, that could be a week or more. This is a fundamentally slow-to-value product for the persona the app most wants to convert (a new user with no club yet), and no amount of UI polish changes that; the fix has to be in what happens *before* the first evening (see §4, “no club” path), not in the summary .screen itself

Time-to-value per entry point .4

→ **A. Someone starting a club (organiser).** CommunitiesFeed FAB InviteMembersSheet → (steps 2) GroupWizardForm / CreateGroupScreen nudge → GameCreateScreen (3-step wizard, GameWizardForm.tsx , 1760 lines) → share invite link. **Utility value** (a working invite + schedule exists) is reachable in minutes. **Product value** (the payoff in §3) is gated on the first real evening completing. The 3-step, format-heavy game wizard is a real tax on this persona weekly, mitigated only by the recurrence – meaning the wizard's cost is front-loaded once GameSeries and amortised, which is the right trade *if* organisers actually find and use recurring games (SUSPECTED – not verified; no telemetry on this reviewed .(here

B. Someone joining an existing club. Fastest sub-path: has an invite code/link → CommunityDetailsScreen (member view) directly, or lands on if discovering. Open (non-member preview) CommunityDetailsPublicScreen → requestJoinById) clubs auto-approve; closed clubs queue on admin D3 §2) – **time-to-value here depends entirely on a , pendingPlayerIds human admin's responsiveness**, which the app doesn't control and doesn't seem to escalate (no visible “admin hasn't responded in N days” nudge in the reviewed inventory). Once approved, joining a game is 1 tap; value is .then gated on the same real-evening wait as (A)

C. Someone with no club at all. This is actually the **fastest** path discovery feed, or create a “quick “פתוחים” GamesListScreen :to playing game” (isOrphanContext , hidden personal group, ensurePersonalGroupId) and invite friends directly – no club creation, no admin approval, no -waiting. This is the right on-ramp for a cold, club-less user. But it dead ends: the promotion path from “we played once” to “now we're a club” is the gap from §2 – the fastest entry point has the weakest PromoteOrphanScreen exit into retention. That's backwards for a product whose stated purpose “includes “fill shortage weeks by reaching strangers in-app .(project_app_purpose.md)

?Would a club abandon WhatsApp + a spreadsheet for this .5

Be honest: **partially, and the parts it wins on are real, but the bar isn't cleared yet**

:Wins that are genuine, not cosmetic

Fair join ordering (`joinFairness.ts`) solves a real WhatsApp failure - .mode (a group message thread has no atomic "who got the spot" semantics)

Auto/manual team balancing (`teamBalanceCore.ts` , draft flow) beats - someone eyeballs two teams in the group chat" for any club that cares" .about competitive balance

Automated stats/history (evening score, round summary, club - leaderboards, chemistry) is exactly the kind of thing a spreadsheet owner does manually today and would gladly give up - *if the numbers are trustworthy*

:Where it doesn't clear the bar

,Chat (`ChatsListScreen` / `ChatView` , ToS gate, profanity filter - block/report, typing indicators, read receipts - a fully-built second messaging product, D3 §7) is fighting a battle the app cannot win: the club's WhatsApp thread already has years of history, all members, and zero setup cost. Nothing in the inventory suggests in-app chat gives a club a reason to leave WhatsApp; it's the app trying to become the .default surface, which is the wrong fight to pick

Trust in the numbers is the whole pitch, and it's the least-verified - code in the app. `functions/src/index.ts` (14K lines, the code that actually produces every stat) has no tests. The one documented live stat bug (sub-out crediting) is exactly the kind of thing that, if a club organiser (the "spreadsheet person," someone who by definition trusts manual accuracy) catches it once, permanently downgrades the app back to the WhatsApp poll is more reliable than the app's numbers." A stats" .product's credibility is binary in a way a chat app's isn't

Setup cost asymmetry: a WhatsApp group is zero-config. Teamder's - organiser-facing surface (55 screens, admin roles, cards/discipline config, cover photo, rating toggles, format pickers) is a real ongoing administrative burden for a volunteer coach who was previously just into a group chat. The app needs to be winning "מי בא היום?" typing clearly on the loop (§1) to be worth that; today it wins on scheduling .fairness and stats, but only if the stats hold up

What is over-built .6

.Using core / supporting / noise, judged against the loop in §1

F-PM-1 – Wear OS companion app: noise relative to value delivered

Severity: P2 (meaningful – engineering cost, not user-facing harm) •

Confidence: Confirmed (read in code) •

, `plugins/wear-src/`) Feature / Screen / Flow: Wear OS companion •

(`watchSyncService.ts` , `withWearApp.js`)

,Evidence: `plugins/withWearApp.js` is the largest config plugin (12.4KB •

D4 §5) – copies a full Gradle module into the generated Android project every prebuild, wires 4 native manifest receivers/services, a widget provider, and a native `WatchBridge` bridge module (mirrored again on iOS under `modules/watch-bridge/`). Entire feature exists to mirror a

- start/pause/reset stopwatch (`useSyncedTimer.ts` , 105 lines) onto a .second screen
- Current behaviour: phone-only `LiveMatchScreen` already does the actual .job (start/pause/reset, all participants watch) with zero native code
- +Problem: at 596 users, the number of people who (a) own a Galaxy Watch 4 and (b) use it specifically to control a football match timer is almost certainly a small fraction of a small user base, yet this is the single most native-code-heavy subsystem in the app (a full second Gradle module + 2 platforms' worth of native bridge code) for a feature that .duplicates something the phone already does
- User impact: near-zero for the vast majority; SUSPECTED small positive for .the sliver of Watch owners
- Technical impact: a documented iOS build blocker already came directly ,from the watch scaffold (`project_ios_watch_build_blocker.md` - memory) costing failed release builds; every `app.json`/prebuild change now has to .reason about this plugin
- Recommendation: Noise. Freeze further watch investment; do not build the dormant `targets/watch` complication (already correctly shelved per memory). Consider whether the maintenance cost of the plugin justifies .keeping it live at all versus feature-flagging it fully off
- Expected benefit: removes a recurring build-fragility source, frees .attention for the core loop
- Effort: N/A (assessment; no action taken, read-only audit)

F-PM-2 – Full in-app chat product: noise, competes with a fight it can't win

- Severity: P2
- Confidence: Confirmed
- , `GameChatScreen` , `ChatsListScreen`) Feature / Screen / Flow: Chat tab (`chatService.ts` , `ChatView` , `DirectChatScreen` , `CommunityChatScreen`)
- Evidence: `chatService.ts` (420 lines) + `chatPush.ts` (separate push pipeline, D4 §1C) + `ChatTermsModal.tsx` (mandatory ToS gate) + a /hand-rolled Hebrew/English profanity filter (`profanity.ts`) + block report/mute/typing-indicators/read-receipts – a fully-featured second .messaging product, its own 4th bottom tab
- Current behaviour: every club already has a WhatsApp thread with full .history and 100% membership before it ever tries Teamder
- Problem: this is the most WhatsApp-shaped feature in the app, competing directly with the app it's positioned to unseat, on WhatsApp's own home .turf, with none of WhatsApp's network effects
- User impact: SUSPECTED low – a second, emptier chat thread membership .has to actively choose to use instead of the one they already have
- Technical impact: real moderation/compliance surface (ToS gate, report pipeline resolving real sender identity, App Review UGC policy requirements per D1 §3) – this is not a cheap feature to have built or .to keep maintaining
- Recommendation: Noise relative to the core loop. If kept, it should be positioned as a *game-day utility* (e.g., “who's running late,” tied to a live game) rather than a general-purpose messenger competing with .WhatsApp on breadth
- Expected benefit: n/a (audit only)
- Effort: N/A

vanity noise, cheap enough to keep: ("תארים ורמת מועדון") F-PM-3 – Club levels/titles

- Severity: P3 •
- Confidence: Confirmed •
- , CommunityStatsScreen / CommunityDetailsScreen :Feature / Screen / Flow •
src/utils/clubLevel.ts , src/data/clubAchievements.ts
- Evidence: D3 §2 – "Entirely derived client-side from existing aggregates
.no new collection/trigger." Zero incremental backend cost ...
- Current behaviour: a 1-10 club level + bronze/silver/gold badges computed
.purely from data the app already has
- Problem: no evidence this drives the weekly comeback (§2); it's the kind
of gamification layer that looks good in a screenshot and does little for
.retention on its own
- User impact: SUSPECTED mild positive (community pride surface), not
.differentiating
- .Technical impact: negligible – pure derivation, no new writes •
- Recommendation: Noise, but *cheap* noise (no dedicated collection, no
trigger, no test debt beyond what already exists) – not worth ripping
.out, not worth investing further in either
- Expected benefit: n/a •
- Effort: N/A •

F-PM-4 – Ads (banner + app-open) at 596 users: premature for the positioning

- Severity: P2 •
- Confidence: Confirmed •
- , BannerAd ,Feature / Screen / Flow: adsService.ts (661 lines) •
app-open interstitial
- ,Evidence: D3 §8 – layered guards (cooldown, daily cap, 2-day honeymoon
intentful-open suppression) show real design care went into *not*
.annoying users, but the feature exists at all
- Current behaviour: monetization surface live in a product still trying
.to prove it's worth switching to over WhatsApp+spreadsheet (§5)
- Problem: an ad-supported product reads as a consumer app, which cuts
against the "serious club-management tool" positioning the admin surface
is otherwise going for – an admin who just (roles, discipline, ratings)
spent 10 minutes configuring card-validity windows for their club is a
bad audience for a banner ad. Premature monetization before the core
.loop (§1) is proven to retain clubs
- .User impact: SUSPECTED mild negative on organiser trust/perception •
- .Technical impact: none beyond what's built; well-guarded already •
- Recommendation: Supporting-at-best, arguably noise until retention (the
actual product bet) is validated. Not a build-complexity problem – a
.positioning-priority problem
- Expected benefit: n/a •
- Effort: N/A •

F-PM-5 – Widgets (home screen): same critique as Wear, lower cost

- Severity: P3 •
- Confidence: Confirmed •
- TeamderPlayersWidgetProvider , TeamderWidgetProvider :Feature / Screen / Flow •

- Evidence: D4 §5 – native Kotlin widget providers mutate Firestore directly from the widget's play/pause/reset buttons
- Problem: same duplication-of-the-phone-screen critique as Wear (F-PM-1) but at a fraction of the native-code cost (Android-only, no second platform's worth of bridge code) – the cost/value ratio is much less bad here
- Recommendation: Supporting, not noise – cheap enough relative to Wear that it doesn't need to be singled out for cuts
- Effort: N/A

Quick calls on the rest of the named list

- Friends** (`friendsService.ts` , `FriendsScreen`) – Supporting. Real “server-side-trusted mutual-friend model, feeds “friends in this club discovery hints (`clubFriendsService.ts`) which *does* touch the core loop (helps a club-less user find a club via people they know). Keep
- Availability heatmaps** (`AvailabilityEditScreen` , 1004 lines radius maps) – Supporting, core-adjacent. This , `AvailabilityWeekScreen` is the data source for cross-community filler matching, which is a stated product-purpose feature (“fill shortage weeks by reaching strangers”). The size of the surface (1000+ line screen, multiple map modals) is disproportionate to how central it is to the loop, but it isn't noise – it's infrastructure for shortage-week fills. If filler matching isn't actually converting shortage weeks in practice this becomes the single most over-engineered (unverifiable here) *supporting* feature in the app
- `AchievementsScreen` , **Achievements** (personal titles ;Supporting. Standard engagement candy – (`AchievementCelebration` .cheap relative to its footprint (derived, celebratory modal). Fine as-is
- Chemistry / pair stats** – Supporting, and a good ROI example: built entirely on `pairStats` / `communityPairStats` that already exist for other reasons (D3 §6), reduced to “six cards” via a single query. Cheap reinforces the stats differentiator from §5. Keep, don't expand further without evidence it's used
- Mini-games history** (`MatchRoundsScreen`) – Supporting. Pure read-only display over data `commitRoundStats` already writes; near-zero incremental cost for legitimate “settle an argument” utility. Fine

What is missing that the core loop actually needs .7

F-PM-6 – No offline handling for the on-field, signal-poor moment that matters most

- Severity: P1
- Confidence: Confirmed (absence)
- `AdvancedLiveMatchScreen` / `LiveMatchScreen` :Feature / Screen / Flow
- Evidence: D4 §4 – “No dedicated offline-detection layer... no Offline’ is handled reactively per-call-site by catching’ ... `NetInfo` Firestore’s `unavailable` errors... there is no app-level ‘you’re “offline’ UI state
- Current behaviour: goal/assist entry, shootout kicks, and timer control all happen live, on a pitch, where signal is exactly the kind of intermittent that Firestore's offline cache handles for reads but not “necessarily for the UX of “did my tap register

- Problem: this is the single moment in the whole loop with the worst connectivity and the highest cost of a silent failure (a goal that didn't save is a stat the club will notice is wrong, undermining §5's whole .pitch
- User impact: SUSPECTED – admin taps “goal,” nothing visibly confirms it `commitRoundStats` ,synced, the app moves on; if the write never lands reconstructs from whatever `roundHistory` /state exists rather than what .the admin actually saw
- Technical impact: no offline banner, no queued-write indicator, no retry .affordance beyond ad hoc per-screen try/catch
- Recommendation: this is exactly the kind of missing capability a bug-hunting pass would find but a product audit should also flag as a gap against the stated positioning – it's the loop's weakest technical .point precisely where the app's differentiation (§5) is most at stake
- Expected benefit: protects the trust-in-numbers pitch at its most fragile .moment
- Effort: L (would need a real connectivity-aware write queue, not a (cosmetic banner

F-PM-7 – Orphan→club promotion has no in-app on-ramp (repeats D1 finding, framed as a funnel gap)

- Severity: P1
- Confidence: Confirmed
- `MatchDetailsScreen` , `PromoteOrphanScreen` :Feature / Screen / Flow
- `PromoteOrphan` Evidence: D1 §1/§6 – zero `navigate()` call sites to anywhere in `src/` ; only reachable via the `promotePrompt` push, ~30 min .after the orphan game finishes
- Current behaviour: the fastest, lowest-friction entry point in the whole app (§4C, “quick game,” no club needed) has exactly one, easily-missed .door into retention (a single push notification)
- Problem: this is the highest-leverage missing surface in the product ,it sits at the exact intersection of “the loop just worked” (§1 step 4 “people just had a good evening) and “the fastest on-ramp dead-ends Every other retention lever in the app (recurring series, evening .(4C§) summary, stats) only matters *after* a club exists; this is the one .screen that turns a one-off into a club in the first place
- User impact: SUSPECTED meaningful lost conversion – anyone who missed or .dismissed the push has no other way to find this screen
- Technical impact: none – the screen and callable already exist and work; this is purely a missing (`promoteOrphanToGroup`) CTA wire-up, per the screen's own header comment describing the intended .second entry point (never-built)
- Recommendation: add the CTA on `MatchDetailsScreen` for finished orphan games, as the screen's own comment already specifies – and remember it /needs registering consistently across `GameStack/CommunitiesStack` `ProfileStack` per the app's own established pattern (D1 §1) since .MatchDetails is hosted in all three
- Expected benefit: directly targets the weakest point in the fastest .entry-point funnel
- Effort: S (one CTA + one more stack registration)

F-PM-8 – Backend that produces the loop's payoff has no test coverage

- Severity: P1

- Confidence: Confirmed •
- Feature / Screen / Flow: `functions/src/index.ts` (the entire Cloud Functions surface, 14K lines, 64 exports) •
- Evidence: D4 §8 – “the entire Cloud Functions surface... has zero unit/integration tests. Only the Firestore rules are tested.” Confirmed •
- `, onGameRosterChanged , commitRoundStats` no test file for the exact functions that produce – `rollUpClubPairs , sealRoundSummary` •
- `.s` wow moment and §5’s stats pitch •
- Current behaviour: extracted pure sub-algorithms (rotation, evening ; `src/` score, chemistry, round summary math) ARE well unit-tested in the stateful Firestore-writing glue around them (the actual server code `.that` runs in production) is not •
- Problem: this is a coverage gap specifically at the layer with the most documented drift risk (D2 §4: three functions writing different field subsets of the same doc at different times; a non-transactional pre-check+create latch race in `rollUpClubPairs`) – the parts of the system most likely to silently misbehave are also the least protected against regressions •
- User impact: SUSPECTED – any regression here surfaces as exactly the kind of “the app’s numbers are wrong” moment that’s fatal to the stats pitch `.(5§)` •
- Technical impact: every future change to `commitRoundStats /roster` triggers ships on faith + manual mock QA (per project memory, mock QA `.(can’t` catch REAL-only issues •
- Recommendation: this is the single highest-leverage testing investment in the codebase given what’s actually at stake product-wise, not just /code-wise – prioritize test coverage for the finish-transition commit-stats/rollup chain specifically, ahead of general functions `.coverage` •
- Expected benefit: protects the one thing the product is trying to be `.known` for (§8) •
- Effort: L •

What should be the flagship feature .8

The fair, accurate record of the evening – the combination of fair-join queueing (nobody argues about who got a spot), automatic team balance (nobody argues about fairness of teams), and the evening/round summary (an accurate, shareable “here’s what actually happened”) – should be the flagship, marketed as *“the club that never needs a spreadsheet or .an argument again*

This is the one place the app is unambiguously better than WhatsApp + a spreadsheet (§5), it’s already the most heavily tested part of the codebase (round summary has more dedicated test files than any other feature per)

D3 §6 – “the single most heavily-tested feature in the app”), and it’s the ,artifact designed to leave the app and be seen by non-users (§3). Chat, ads Wear OS, and club levels are not flagship material – none of them are things a club would tell another club about. “We never argue about who .plays and the app remembers everything” is

The code is trying to be a club-management tool, a pickup-game finder, and a stats product simultaneously, and it shows in the architecture, not just the feature list:

- **Club-management tool:** roles (founder/coach), discipline/cards, internal ratings, equipment tracking, admin approval queues, cover-photo/branding – a genuinely deep admin surface
(IssueCardSheet , AdminApprovalScreen , CommunityPlayersScreen)
(AdminRatingSheet , ManageEquipmentSheet , GamesListScreen)
- **Pickup-game finder:** the discovery feed, guest browsing, quick/orphan games, cross-community filler, פתוחים/שלי, .matching, nearby-clubs teaser – built for someone with no club at all
- **Stats product:** evening score, round summary, club stats dashboard, chemistry, trust score, player compare – a full analytics layer on top of both of the above

:Where the split costs it, concretely

- **Navigation itself pays the tax.** MatchDetails and its entire drill / RoundSummary / EveningSummary / DraftBoard / DraftSetup) chain / GameEdit / AddMembers / AvailablePlayers / MatchPlayers / MatchRounds , History , AdminApproval , plus CommunityDetails ' chain , (LiveMatch are **registered identically** PlayerCompare , PlayerCard , GameCreate – **in three separate stacks** (GameStack, CommunitiesStack, ProfileStack D1 §1) purely so “back” returns to whichever of the three organizing metaphors (games / clubs / profile) the user actually came from. That's not a bug – it's documented, deliberate, and it works – but it's a direct, measurable engineering cost of not having committed to one primary axis.

• The admin surface and the finder surface serve different maturity

levels of user and don't obviously coexist well. A cold, club-less user (§4C, the fastest on-ramp) lands in the same tab structure as an established club's coach managing cards and equipment. Nothing in the inventory suggests progressive disclosure between these two very different jobs-to-be-done (SUSPECTED – not verified at runtime)

• The stats layer's credibility depends on the other two axes behaving

A stats product's core promise (accurate numbers) is only as good as the club-management data entry (who's actually on the roster, who subbed) and the finder-layer's guest/filler handling feeding it – and §7/F-PM-8 shows that exact join is the least-tested part of the system

If forced to pick one axis: the evidence points toward **pickup-game finder**

,with clubs as a side effect, not club-management tool. The fastest lowest-friction path through the app is a club-less quick game (§4C); the personal/orphan-group hack exists specifically so the rest of the data model “needs no special-casing” for that case (D3 §2); and the project's own stated purpose foregrounds “regulars-first, but ALSO fill shortage weeks by reaching strangers in-app.” The deep club-admin tooling (cards ratings, equipment) reads as built for the regulars-first half of that sentence, and is real, necessary depth once a club exists – but it shouldn't be competing for first-impression real estate with the pickup-finder path that's actually the product's fastest, most differentiated on-ramp

DELETE IT – subtraction audit (Teamder)

`documents:runQuery` Method: every collection count below is a live, read-only
`games` against project `soccer-app-52b6b` on 2026-09-02 (614 `users` docs, 79
docs, 174 `groups` docs – the entire history of the app to date). Counts over
would be capped; none were. Screen/feature inventory pulled from 1000
. `D3_features.md` and `../discovery/D1_screens.md`

Verdict	Cost	Evidence of use	Feature
DELETE	live CF triggers (<code>onVoteWritten</code> , 2 <code>onVoteWrittenLegacy</code> , ~103 lines), unused <code>firestore.ts</code> collection helpers, <code>rateVote</code> rate-limit op	coll.: 12 docs, but zero client ratings call sites for <code>col.ratingVote</code> / <code>col.globalRatingVote</code> outside <code>firestore.ts</code> itself – UI was deleted 2026-06-24 per memory	Legacy peer rating (1-5 stars)
DELETE (fold the entry point into community chat)	wired into 3 , <code>GameChatScreen.tsx</code> nav stacks, shares <code>ChatView</code> /ToS/profanity/report machinery	collection-group <code>messages</code> : 140 total ever, split groups=71 / <code>dmConversations</code> =60 / games=9 – 9 messages across 79 games (43 advanced-mode)	Game-scoped chat
KEEP BUT HIDE – real but tiny; demote from a 4th tab to an entry point off Profile/Community, see reasoning	LOC across 2,439 service+screens+components, mandatory ToS modal, profanity filter, report/block pipeline	msgs total, 34 131 = 60 + 71 <code>dmConversations</code> , <code>blocked</code> coll.=0 for the whole app	Community + DM chat (rest of the chat system)
DELETE	Full pipeline: <code>startFillerPulse</code> , push dispatch, <code>FillerInterestsSection</code> , <code>FillerPickerModal</code> , admin approve UI, <code>fillerMinTrust</code> gating	4 docs ever , :(cg) <code>fillerInterests</code> against 26/79 games with <code>acceptsFillers:true</code> – admins opt in, almost nobody applies	Cross-community filler matching
KEEP BUT HIDE – thin but real, low cost to leave as-is	LOC (service+screen), server- 885 trusted accept callable	docs; sample of 43 : <code>friendRequests</code> 30 users → 2 (6.7%) have any friend	Friends
INVESTIGATE – see reasoning, don't delete outright	<code>, seriesService.ts</code> dedicated , <code>seriesSchedule.ts</code> collection, weekly-clone cron, a full memory-tracked migration	docs ever (both active, 2 : <code>gameSeries</code> one created same week as this audit)	Recurring games / series
INVESTIGATE (architecture cost, not a feature to cut)	Every club-scoped read/write (rules, feeds, chemistry rollups) has to special-case/filter these	<code>isPersonal=true</code> : 121 of 174 groups (70%) – real communities are only 53	Personal/orphan quick-game groups
INVESTIGATE – can't verify push-driven usage from Firestore; flag for a push-open funnel check	Full wizard screen + <code>promoteOrphanToGroup</code> callable	has zero <code>PromoteOrphanScreen</code> <code>navigate()</code> call sites (confirmed ground truth); reachable only via <code>promotePrompt</code> push	Promote-orphan-to-community
DELETE	Dedicated <code>disciplineService.ts</code> dual-write path, D3 confirms it's "superseded... though still written for "backward compat	sampled users have any value set 0/30	Discipline lifetime) counters (<code>User.discipline</code>
MERGE into Joryio's <code>InAppMessageHost</code> (duplicate concept, see reasoning)	line service,-294 <code>CampaignGate.tsx</code> , segment-eval logic duplicated from Pulse, presence ping, <code>feature_campaigns</code> kill-switch	doc ever; <code>segments</code> : 1 1 : campaigns doc ever	In-app popup) campaigns (<code>campaignService</code>
KEEP	LOC, two pipelines (live + rollup) 372	docs, 594 : <code>communityPairStats</code> <code>communityPairRollups</code> : 19 (~1/active club)	Club chemistry / pair stats

Verdict	Cost	Evidence of use	Feature
KEEP BUT HIDE – thin evidence, but cost is near-zero (no collection of its own, no CF); not worth the effort to remove	LOC, pure client derivation 665	backs the clubRecords 5 docs for 174 : (records/milestones feed groups	Club-level titles & achievements (“תארים ורמת מועדון”)
KEEP – cheap, and D3 confirms AchievementCelebration fires from 3 surfaces	Screen + celebration modal + push type	sampled users have any 10/30 achievements value set	Personal achievements
KEEP BUT HIDE – real minority usage, but its main downstream consumer (filler pushes) is a delete candidate above; re-scope after that cut	LOC across 2 screens + feed 1,203 service; also feeds filler matching (which is being cut)	sampled users (27%) have 8/30 availability set	Availability (“פנויים לידר”)
KEEP – cheap for what it is, no evidence against it	LOC, one screen not duplicated 868 logic	No dedicated collection to count; single shared component, mode param	Map screen (games/communities)
KEEP – out of scope for usage-based deletion (monetization, not engagement)	LOC, layered guard logic 661	Revenue infra, not a Firestore-measurable feature	Ads (banner + app-open)
KEEP – cheap, low-risk	LOC, low 125	Curated per-version by Pulse; no collection count possible (lives in appConfig)	What’s new” modal”
KEEP	LOC 128	Same – infra, not measurable via collection count	Store-review prompt
INVESTIGATE – cost is disproportionate to any plausible reach; can’t prove near-zero usage without device telemetry, but the hardware-niche math argues for it	LOC service + ~4,423 LOC native 408 scaffolding under plugins/wear-src	No Firestore signal possible (native/local); memory shows 6+ separate incident/fix entries (targetSdk deadline scramble, timer-sync bugs, build blockers) for a Galaxy-Watch-4+-only audience inside a 614-user app	Wear OS companion + widgets
KEEP – small, cheap, no clutter cost	LOC 213	Not independently measurable	Weather forecast
MERGE – see duplicate-concepts reasoning	Two services (disciplineService + trustService) computing overlapping “is this player reliable” answers	docs 74 : communityPlayerEvents (real signal) vs User.discipline : 0/30 (dead)	Legacy discipline UI (per-club cards) vs Trust score
KEEP	LOC, cheap 93	docs – the single 351 : feedback highest-signal collection in this audit relative to a 614-user base	Feedback / bug report
KEEP	infra	docs 221	groupJoinRequests

Reasoning – every DELETE and MERGE

DELETE: Legacy peer rating (1~5 star voting)

, globalRatingVotes , ratingVotes still exports src/firebase/firestore.ts:1870–1927 but a – globalRatingVote , ratingVote , globalRatingSummary , ratingSummary repo-wide grep for callers of globalRatingVote / ratingVote outside that file returns **nothing**. functions/src/index.ts:6119 (onVoteWritten) and are two live Cloud Functions still listening on (onVoteWrittenLegacy) :6163 and the legacy ratings/{ratedUserId}/votes/{raterUserId} path respectively, transactionally maintaining groups/{groupId}/ratings/...

count/sum/average summary docs for writes that can never happen anymore – the UI that wrote them was deleted 2026-06-24 per project memory peer/crowd rating DELETED; internal admin rating” : `project_rating_1to10.md`) only”). The `ratings` collection’s 12 documents are fossils from before that deletion. `rateLimitService.ts:31,44` still reserves a `rateVote` op nothing calls. Nobody reads `onVoteWrittenLegacy`’s comment (hour/60) – and actually removed it (“Remove once the global build is widely adopted”) :the global build has been out since June, this is September. Pure subtraction delete both triggers, the 6 `firestore.ts` helpers, the `rateVote` rate-limit .entry, and archive/delete the 12 stale `ratings` docs

DELETE: Game-scoped chat

Chat is architected as one shape (`messages` subcollection) fanned across three scopes deliberately (D3: “One `/.../messages` subcollection shape serves all three chat scopes”). The collection-group count splits by parent as `groups=71`, `dmConversations=60`, **`games=9`**. Nine messages, total, ever, across games (43 of them advanced-mode games with active rosters and admins who’d 79 have reason to coordinate). `GameChatScreen.tsx` is registered identically in `,GameStack/CommunitiesStack/ProfileStack` (three copies of the route wiring reachable from `ChatsList` , `MatchDetails` , and a dedicated push type The entry surface costs more than the feature .(`scope=game` `chatMessage`) returns: a mostly-empty scope sits inside a chat list that’s supposed to be every chat the user can access,” diluting a feed that’s already thin (140” messages total for 614 users). Cut the game-chat entry points from and the chat list; players who want to coordinate about a `MatchDetailsScreen` specific evening already have the community chat one tap away – the club .context doesn’t disappear, just the redundant per-game bucket

InAppMessageHost MERGE: In-app popup campaigns (`campaignService`) into Joryio’s

Two independent systems both show the signed-in user a marketing/announcement campaigns) `src/services/campaignService.ts` :popup while they’re in the app `SegmentFilters` Firestore collection, 1 doc ever created, client-side (`CampaignGate.tsx` ,evaluation mirroring Pulse’s own preview, a presence ping and `src/components/joryio/InAppMessageHost.tsx` (renders the Joryio SDK’s own in-app campaign messages – SDK 1.2.0, actively used per ,Both are “an admin authors a targeted popup .(`project_joryio_journeys.md` the client evaluates eligibility, shows one winner, tracks impression.” The homegrown one has essentially never been used (1 campaign, 1 segment, ever) while the paid third-party SDK doing the identical job is the one getting `CampaignGate / campaignService` active journey-authoring attention. Sunset and route all future in-app popups through Joryio – one less bespoke .segment-matching implementation to keep in parity with Pulse’s copy

MERGE: Discipline lifetime counters into the per-club card timeline / trust score

D3 itself names this cluster: `User.discipline` (global lifetime yellow/red counters), `communityPlayerEvents` (per-club card timeline, 74 real docs), and s 0-100 reliability score are three surfaces all answering “is’ `trustService` this player reliable/well-behaved,” with D3 explicitly noting the global counters are “superseded on the player-card UI by the trust meter... though the underlying counters are still written for backward compat.” The 0/30 sample confirms the backward-compat write path is now producing nothing worth keeping in sync. Drop the `User.discipline` dual-write and the global-counter

read path; keep the per-club card timeline (it has real data and a real admin-facing screen, `PlayerTimelineScreen`) and the trust meter (the one – surface players actually see) as the two remaining, non-overlapping answers were they carded at this specific club” vs. “should I count on them” “.showing up

DELETE: Cross-community filler matching

This is the single clearest cost/benefit mismatch in the audit. The full pipeline exists end-to-end – opt-in toggle in the game wizard a manual pulse trigger `(fillerMinTrust + acceptsFillers)` `(fillerOpportunity)` a push type `(gameService.ts:3261 startFillerPulse)` `fillerInterests` → `submitFillerInterest`) a candidate-facing apply flow subcollection), an admin review section on `MatchDetailsScreen`, and a dedicated modal (`FillerPickerModal.tsx`) – and 26 of the app’s 79 games have actually opted in, so admins clearly know the toggle exists. And (33%), yet the `fillerInterests` collection-group query returns **4 documents total, ever**. Admins are opting in to a feature that essentially never produces a candidate. Either the discovery-surfacing extension described in project memory (`project_fillers_in_feed_plan.md` – “surface the dormant feed”) is the actual fix needed before this ‘פתוחים’ filler engine INTO the can be judged fairly, or the feature should be cut: today it’s pure maintenance surface (a push type, an admin section, a modal, a Firestore subcollection, `fillerMinTrust` gating logic in `trustService`) for 4 total uses. Given the memory entry says that extension was only “Approved and D3 finds no evidence it shipped, recommend killing the current “2026-07-06 dormant version rather than investing further – re-introduce only if/when the .feed-surfacing version is actually built and re-measured

One-liners on areas that are genuinely fine

- **Club chemistry / pair stats** – 594 `communityPairStats` docs, two pipelines kept in parity by tests (`clubChemistryParity.test.ts`). Real .usage, keep as-is
- **Feedback** – 351 docs for 614 users is the strongest usage signal in this .entire audit. Don’t touch it
- **Achievements (personal)** – cheap, 3 trigger surfaces, 10/30 sample .adoption. Fine
- **,Map, weather, what’s-new modal, store-review prompt** – all small .self-contained, no clutter cost. Leave alone

COMPETITORS

External research – pickup/amateur sports product landscape 2025–2026, for Teamder

Method: WebSearch + WebFetch only, no runtime access to Teamder itself. All claims below are from public web sources (search results, vendor sites, review aggregators), accessed

Where a source carries its own publish date I cite it; where it doesn't, I mark "2026-09 accessed 2026-09" and flag anything that is my inference rather than a verified fact as **INFERRED**. Visual/UI design is deliberately not described – only mechanics and product decisions.

The landscape, by category .1

1a. Pickup football (Teamder's closest genre)

Footy Addicts (UK) – footyaddicts.com / [App Store](#)

.What: find and join 5/6/7-a-side pickup games across Great Britain, no team commitment -

.Core loop: browse games by location → join a spot → play → profile tracks games played -

Monetization: not disclosed in search results; UK "go-to social football platform" – **INFERRED** venue/booking-fee take, same pattern -
as Plei/GoodRec below

Weakness (from users): "some players can be disrespectful towards others" – i.e. an open-join model has no social-graph filter, so -
game quality varies. Rated 4.5/5, 159K+ downloads. ([Footy Addicts App Store](#), [Trustpilot](#), accessed 2026-09)

.**Plei** (US) – largest pickup soccer organizer in the US, founded 2017 by the Duque brothers

What: on-demand soccer, "as easy as ordering an Uber" – Plei runs the logistics, partners with facility operators to fill unused time -
.slots

.Core loop: pick a game → pay per-seat (from \$6/game) → show up -

Monetization: **confirmed transactional** – per-seat payment plus a facility-partner revenue share. \$4.5M revenue in 2023, 180K+ users, -
targeting \$12M in 2025. ([Republic](#), [RefreshMiami](#), accessed 2026-09)

Weakness (user reviews): same-day cancellations, last-minute time changes, "unexpected facility fees." The company's own investor -
material admits **the concept is not IP-protected – anyone can replicate it**. That is a structural weakness of the whole "on-demand
pickup marketplace" category, not just Plei

GoodRec (US/Canada/Europe, ex-JustPlay) – goodrec.com

.What: pickup soccer, basketball, pickleball, volleyball leagues and drop-in games, 50+ cities -

Core loop: pick sport + city → join a game → a **host** (not an algorithm) makes teams, explains rules, sets tone → post-game rating of -
the game AND of the host

Trust mechanism worth noting: the *host* is rated, not just the venue or other players – this puts reputation on the organizer, which is -
the same person Teamder already elevates as game creator/admin

..Monetization: pay-per-game, same shape as Plei – **INFERRED** from "paying to show up -

Weakness (user reviews): "often bugged, player numbers exceeding limits, game info constantly incorrect, resulting in players paying -
to show up and being turned away" and a 24-hour no-refund cancellation window that users find harsh next to Plei's more forgiving 5-6
hour window. ([App Store](#), [PlayNow vs GoodRec comparison](#), accessed 2026-09)

Pickup (zero-login web app) – a smaller, philosophically opposite product worth reading closely: [manifesto](#) (accessed 2026-09, fetched
directly)

Three rules: one person commits to a specific court+time, a **shareable link** (not a public feed) coordinates RSVPs, and the venue is a -
verified real Google Maps location

Deliberately **rejects** ratings/vetting of other players ("no swiping 1-5 stars on people"), rejects a public discovery feed, rejects push -
notifications, and rejects "community" as a goal. Its thesis: *"your existing group chat is more valuable than any pickup social network*

This is close to a direct articulation of why WhatsApp+spreadsheet survives as Teamder's real competitor – and a useful foil to the -
open-discovery apps above

1b. Adjacent racket/court sports (worth stealing mechanics from)

Playtomic (padel/pickleball booking, Madrid-based, global)

Core mechanic worth studying: the **Playtomic Level**, an algorithmic 0.0-7.0 skill rating (0.25 increments) computed from match results, -
weighted by opponent level and score margin – wins vs. stronger opponents move the rating up more than wins vs. weaker ones.

Drives **open-match matchmaking**: strangers filter/join matches by level band so a 3.5 doesn't get steamrolled by a 5.5. ([Playtomic level algorithm](#), [padel level explainers](#), accessed 2026-09)

Some players dispute the algorithm's accuracy/gameability ([Proper Padel, Sep 2025](#)) – a caution that any auto-rating system needs a -
way to resist manipulation and stale data

Monetization: **confirmed hybrid** – court-booking commission (5-15% per transaction reported in secondary sources) + club SaaS -

subscription + optional player premium tier for deeper stats. Booking itself is free for players; they only pay court time. ([UK Padel Guide 2026 comparison](#), accessed 2026-09 – commission % is a secondary-source estimate, not Playtomic's own disclosure, so treat as **INFERRED/approximate**)

1c. Basketball pickup (smaller/newer, same “find a run” problem as football)

Several 2025-2026 entrants – **Fullcourt, HoopFind, Pickup: Basketball Runs, ATH (Are They Hooping), HoopRun** – converging on the same feature set: filter runs by location/cost/skill/competition level, and – Fullcourt specifically – **live/real-time court occupancy** (“how many players are at a court right now” + weekly heatmaps of best times to play) across 60,000+ tracked locations. That real-time-occupancy idea has no equivalent in the football apps surveyed and is a genuinely distinct discovery mechanic (see §3). HoopRun's stated pitch – eliminating “too-small teams, paying with cash, not knowing if it's even happening” – is a near-verbatim description of the WhatsApp pain points below. (App Store/Play listings, accessed 2026-09)

1d. Team/club management (not matchmaking – the “run my club” layer)

Spond (Norway, global reach in youth sports)

Core loop: availability polls, calendar, group chat, **payment collection** (dues, event fees) – all free to the club; Spond earns only a ~2.5% + **fixed-fee transaction cut when money moves through the app** (“2.5% + £0.20” UK-quoted). No subscription, no per-member fee. ([Spond payments](#), accessed 2026-09)

Weakness: “very little football-specific match-day depth (no line-ups, no live match recording, no playing-time tracking)” per third-party – comparison – Spond is communication/logistics, not a stats or live-match product. This is exactly the gap Teamder's live-match/rotation/stats layer fills that Spond doesn't

TeamSnap (US) – tiered subscription pricing, scheduling + comms + roster + stats + payments in one suite; aimed at more formal leagues than casual pickup groups

Heja (Nordics) – communication-first, freemium (Pro from ~£8/mo), standout feature is a **per-player fundraising webshop** – not relevant to Teamder given the no-payments constraint, but notable as a monetization pattern that leans on parents/sponsors rather than players. ([Spond vs Heja](#), [TeamStats comparison](#), accessed 2026-09)

GameChanger (gc.com) (US, baseball/softball-first, expanding to other youth sports)

Everything a coach/team needs – scorekeeping, stats, pitch-count tracking, video highlights, career stats, spray charts – is **free for coaches and team staff**; fans/parents get free live-stream viewing (5 free baseball/softball streams, unlimited for other sports). Monetization is not disclosed in these results but the free-for-organizer model plus a metered/premium fan layer is the closest verified analog to “the club/organizer never pays, someone downstream might.” ([gc.com/app-features](#), accessed 2026-09)

This is a strong model for Teamder's stats layer specifically: deep per-game stats (pitch count ≈ Teamder's per-round/per-mini-game – stats) given away free to drive adoption, monetized elsewhere (fan streaming, in GameChanger's case – irrelevant to Teamder, but the *shape* – stats free, something adjacent metered – is portable)

1e. Camera/AI stats hardware (a different investment thesis entirely)

Veo (Denmark) – AI-tracking camera + software for amateur clubs, from \$67/mo (Cam 3) or \$40/mo (Veo Go), one subscription covers the whole team, auto-highlights and analytics add-on. This is a **hardware-anchored** stats business, structurally unlike Teamder (software-only, self-reported/admin-reported stats). Relevant only as a reminder that “real” stats products in this space usually monetize hardware or a B2B/club subscription – not a p2p payment flow – reinforcing that Teamder's actual monetizable asset, if any, is data/insight, not the stat itself. ([veo.com/pricing](#), accessed 2026-09)

1f. Israeli-specific landscape

Targeted Hebrew search turned up essentially **no direct local competitor** to Teamder's “organize my regular pickup evening” use (for Hapoel Tel Aviv “החולצה האדומה” club-fan apps like “ישראל ספורט”, 365Scores) case. Results were dominated by fan/news apps which serve *spectators* of pro football, not organizers of amateur games. One tangential hit, “**Gamepool**” (ynet/mako coverage, older Hebrew tech-press pieces), described organizing games with friends and finding an open court – but it does not appear to be an active, prominent product today; no recent (2025-2026) coverage surfaced. **This is consistent with the codebase's own framing that**

Teamder's real competitor is WhatsApp + a spreadsheet, not another app – the global category (Plei/GoodRec/Footy Addicts) exists but has apparently not localized into Hebrew/Israel. This is an opportunity (no incumbent to dislodge) and a risk (no local proof the category converts at scale in this market). **INFERRED conclusion from an absence of search results – a true negative in web search is weak evidence, not proof no such app exists**

1g. Strava, as the retention reference model (not a competitor)

:Strava's mechanics, independent of sport, are the best-documented "why do people open this every day" pattern available

Kudos (lightweight social validation – over 14B interactions in 2025, per StriveCloud's write-up, +20% YoY) –

Segments/leaderboards with separate tracks for raw speed (KOM/QOM) vs. **Local Legend** – a rolling-90-day "most times through – this segment" badge that rewards *consistency*, not one-off performance

Strava's own framing: "every 2 minutes of app use converts to 1 hour of physical activity" – the app is a *record and share* layer on top –
of an activity that would happen anyway, not the activity itself

([StriveCloud Strava case study](#), [Trophy.so Strava gamification 2026](#), accessed 2026-09)

Ideas worth adopting .2

.Consistency badge over peak-performance badge (Strava's "Local Legend") – 2.1

Who: Strava. Why it works: it rewards showing up regularly, which is exactly the retention behavior a habit product wants, and it's achievable by an average player, not just the best one – broadening who can "win" something

) Teamder fit: strong. Teamder already has `roundHistory`, per-club attendance, and a club-level/tier system
attended in the club, last N weeks" badge is a מחזורים A rolling-window "most תארים ורמת מועדון). – `project_club_achievements.md`
) small addition on data Teamder already writes, and directly reinforces the regulars-first product purpose already stated in memory

(. `project_app_purpose.md`
90 days is Strava's default; a weekly-cadence club probably wants something like "last 10) Change needed: define the rolling window
and make sure it resets cleanly per season/club rather (rather than a fixed day count, since some clubs play more/less often מחזורים
.than being a lifetime counter that never changes hands

.Rate the host, not just the game (GoodRec) – 2.2

Who: GoodRec. Why it works: in an open-join context, the single biggest variable in game quality is the organizer, so putting reputation on them (not an anonymous "the game") gives strangers a real trust signal and gives good organizers a reason to keep organizing well
Teamder fit: good, but Teamder is club-first not stranger-first, so the stakes are lower – this matters most for the public/quick-game and surfaces where a filler/stranger is joining a game run by someone they've never met. It complements the already-planned פתוחים

(. `project_fillers_in_feed_plan.md`) Fillers-in-feed work

Change needed: needs to be opt-in/soft (a 1-5 or thumbs signal after a game the rater didn't already know the organizer from), not exposed as a public score inside existing club relationships where it would read as socially awkward to rate a friend

.Real-time occupancy / "who's there right now" (Fullcourt, basketball) – 2.3

Who: Fullcourt (basketball pickup). Why it works: for drop-in/open games, the biggest uncertainty a stranger has isn't "does this game exist" but "is it worth going right now" – live headcount collapses that uncertainty

Teamder fit: **partial fit, and worth flagging as a genuine gap**. Teamder's public games feed is date/time-based (register ahead), not doesn't map cleanly (with rosters, not a permanent open court מחזור a scheduled) live-occupancy based, because Teamder's format onto "how many people are physically at the field right now." The closer analog that *would* fit is showing live roster fill state on

public/open games in the feed – "7/14 confirmed, guestsOpenAt in 20 min" – which several teammate memory notes suggest already
Confirm this is surfaced prominently in the (. `project_release_1099_followup.md` shipped per מקום פנוי" live) exists in some form
.discovery feed itself, not just inside a game's detail screen

.Skill-band matchmaking on public/open joins (Playtomic Level) – 2.4

Who: Playtomic. Why it works: it lets strangers self-select into games that won't be a mismatch, which is the single biggest reason a first-time stranger either has a good experience and returns, or has a bad one and churns

Teamder fit: strong, and Teamder is unusually well-positioned to do this *better* than Playtomic, because it already has an internal 1-10 admin rating (`project_rating_1to10.md`) and auto-teams-by-internal-rating (`project_auto_teams_feature.md`) – the hard part (a working rating already feeding team balance) is done. The missing piece is exposing a coarse skill band as a **discovery filter on the public games feed**, not full algorithmic matchmaking

Change needed: keep it coarse (3-4 bands, not Playtomic's 0.25-increment scale) since Teamder's rating is admin-set and low-volume per player, not derived from hundreds of match results the way Playtomic's is – precision the underlying data can't support would just make the number feel arbitrary. Never show the raw internal rating to the rated player or strangers (this is already how the feature is scoped per memory – "shown to adder+admin" only for guest ratings) – expose only the band

.Deep stats free for the organizer, monetize (or don't) somewhere else (GameChanger) – 2.5

Who: GameChanger. Why it works: it removes the biggest reason a coach/organizer would resist adopting a new tool (cost), by giving

.away exactly the thing that makes the tool sticky (stats), and finds revenue in a layer the organizer doesn't touch
Teamder fit: strong as a validation of the *existing* strategy – Teamder's admin/club stats (chemistry, pair stats, king of the evening, ownGoals, penalty stats) are already free and already the retention engine. The lesson to take isn't a new feature, it's a monetization
.sequencing lesson: see §4

.Private-link-first discovery as the default, public feed as the exception (Pickup manifesto) – 2.6

Who: playpickups.com. Why it works: it matches how pickup football groups actually form (a friend invites a friend) rather than assuming
.strangers want a public social network of players

Teamder fit: this validates a decision Teamder has apparently already made – club-first with `isOrphanContext` quick games as an escape hatch, rather than a public-feed-first model like Plei/GoodRec/Footy Addicts. Don't second-guess this default because a competitor did discovery differently; the manifesto product's own thesis is that the discovery-feed model is solving a problem most
.casual groups don't have

Ideas explicitly NOT worth adopting .3

.Cash/credit-based no-show penalties (Plei, GoodRec) – 3.1

Both charge per-seat and use credit/refund windows (Plei: 5-6hr grace with replacement found; GoodRec: hard 24hr no-refund cutoff) as their no-show deterrent. This only works because money is already changing hands for the seat – remove the payment and the mechanism has no teeth. Given the explicit, standing constraint that Teamder has ruled out **all** in-app payments (no dues, no split-bill, no merch – `feedback_no_payments_in_app.md`), any version of this – even a “credits” pseudo-currency with no real money – risks reintroducing payment-shaped UX (balances, refunds, disputes) the owner has explicitly rejected. If no-show reliability becomes a real
.problem, solve it with **social** cost (visible attendance/reliability stat, tied to §2.1's consistency idea) not economic cost

.Public player-rating/vetting of other players (“swipe 1-5 stars on people”) – 3.2

The playpickups.com manifesto explicitly rejects this, and it's the right call to inherit: rating strangers 1-5 stars invites exactly the kind of low-effort negativity and gaming that turns a football app into a moderation problem. It also directly conflicts with Teamder's existing decision to keep rating **internal-only, admin-set, 1-10** (`project_rating_1to10.md`) rather than peer/crowd-sourced – that decision
.was made and shipped for good reason (peer rating was deleted 2026-06-24) and should not be reopened by a competitor pattern

.Facility/court booking marketplace (Plei, Playtomic, GoodRec's core revenue engine) – 3.3

This is the actual monetization backbone of nearly every well-funded app surveyed (commission on court bookings). It doesn't fit Teamder for two independent reasons: (a) the no-payments constraint rules out taking a commission on anything, and (b) Teamder's model assumes the club/organizer already has a regular field, not that the app needs to solve “where do I even play” – building a booking marketplace would be building a different, much bigger product (two-sided venue marketplace with live inventory) far outside
.the current scope

.Per-player fundraising webshop (Heja) – 3.4

A real, working monetization pattern elsewhere, but it is money-adjacent (players/parents transacting through the app to fund the team) and squarely inside the payments the owner has ruled out. Also culturally mismatched: Heja's fundraising webshop is aimed at youth-
.club/parent contexts with a fundraising norm; Teamder's adult regulars-first pickup context has no equivalent norm to attach it to

.Full anonymous open-discovery feed as the primary surface (Footy Addicts/GoodRec/Plei model) – 3.5

These apps' core loop is the public feed – that's their whole product. Bolting an equally weighted public discovery feed onto a club-first app risks diluting the thing that already differentiates Teamder (regulars who know each other, low-friction because trust already exists) in exchange for chasing a stranger-acquisition loop that: (a) has no proven local demand (see §1f – no Hebrew-market precedent found), and (b) is exactly the surface where the reviewed competitors' users complain most (cancellations, no-shows, disrespectful players, inconsistent info) – i.e. adopting this model imports its worst-documented failure mode along with its growth mechanic. If Teamder
.invests in discovery, it should stay secondary to club membership, per §4's answer below, not become co-equal with it

.Skill rating with full decimal precision derived from an ELO-style algorithm (Playtomic's 0.25-increment level) – 3.6

Playtomic can support this because it has hundreds of thousands of matches feeding the algorithm and because padel score margins give a clean signal. Teamder's internal rating is admin-set, low-frequency, and explicitly 1-10 by design decision (not algorithmically derived from match results). Importing Playtomic's precision would manufacture false confidence in a number that isn't statistically
.supported at Teamder's scale (~596 users) and cadence (weekly, not match-by-match)

The match-finding question .4

What Teamder has today (per the discovery ground truth in the shared brief and memory): a games feed with public games, indicator already (open spot) "מקום פנוי" isOrphanContext quick games, and – per project_release_1099_followup.md – a live .shipped. It is **club-first**: the primary unit is a club/community, and public/orphan games are the escape hatch, not the front door

:What the surveyed products optimize for, and how

Filters that recur across every "find a game" competitor surveyed (Footy Addicts, Plei, GoodRec, the basketball cluster, Pickup: – Basketball Runs): location/distance, time, cost, and – increasingly – **skill/competition level**. Fullcourt adds **live occupancy** as a filter .dimension basketball apps have that football apps in this survey did not appear to emphasize

What actually gets a stranger to show up, synthesizing across sources: (1) a **real, verified venue** (Google Maps-backed, not user- - typed text) so there's no "wrong address" failure mode; (2) a **visible organizer/host with some reputation signal** (GoodRec's host rating) so the stranger isn't trusting an anonymous listing; (3) a **skill-level label** set correctly so the stranger knows what they're walking into (Playtomic's level bands, generic "casual/intermediate/competitive" labels used elsewhere); (4) for many groups, **no public discovery at all** – the playpickups.com manifesto's contrarian but well-evidenced position is that a shareable link into an existing social .graph converts better than a public feed, because the trust is inherited from the relationship, not manufactured by ratings Two patterns notably require **payment already in the loop as the reliability mechanism** (Plei's credit system, GoodRec's - .cancellation window) – see §3.1, not importable here

Recommendation for Teamder specifically, based on this research (not a code-level finding, a strategic one): Given (a) the no- payments constraint removes the single most common reliability lever this category uses, (b) no local precedent exists for a Hebrew/Israeli stranger-discovery pickup-football product converting at scale (§1f), and (c) Teamder's actual documented product purpose is "regulars-first, but also fill shortage weeks by reaching strangers" (project_app_purpose.md) – i.e. stranger discovery is explicitly a **secondary, shortage-filling** mechanism, not the primary growth engine – the research supports **investing in discovery only as a shortage-filling layer bolted onto the existing club-first feed** (consistent with the already-planned project_fillers_in_feed_plan.md), not building out a competing public-feed-first product surface. The two lowest-risk, highest-leverage moves the competitor research actually supports are §2.3 (surface live roster-fill state more prominently in the feed – cheap, data already exists) and §2.4 (coarse skill-band filter – cheap, rating data already exists). A full Plei/GoodRec-style public marketplace is .not supported by this research as a good use of effort for Teamder's current scope and constraints

Monetization .5

The constraint, stated precisely (per feedback_no_payments_in_app.md , 2026-07-25): zero money/payments scope – no dues, no split-bill, no merch, no payment collection of any kind between users or from users, and explicitly no money-based KPI on any manager dashboard. This is a **hard, explicit product decision**, not an oversight – treated here as fixed, not something this research argues .against

:How the surveyed products actually make money, mapped against that constraint

?"Compatible with "no in-app payments	Example	Model
No – this <i>is</i> the thing ruled out	Plei, GoodRec, Playtomic	Per-seat/per-booking commission
Grey area – this charges the <i>club/organizer</i> , not a player-to-player payment, but it is still a payment collected through/for the app. Only viable if billed outside the app (e.g. an external B2B invoice to a club, never a player-facing checkout)	Playtomic (club SaaS tier), TeamSnap	Club/team subscription (organizer pays, not players)
No, as an in-app purchase – same reasoning as above	Playtomic premium, Veo Analytics add-on	Player-facing premium tier (deeper stats/analytics)
No – requires the dues/payments flow Teamder has ruled out entirely, not just a commission on top of one	Spond (2.5%+fee on dues collected)	Transaction fee on money already moving through the app
Yes – no money changes hands between the app and the player; this is the cleanest fit for the constraint	Not directly confirmed for any single app surveyed, but standard across the sports-app monetization literature reviewed	Advertising (display/video, ad network)
Yes, if structured as the sponsor paying Teamder directly (a B2B deal, invoiced outside the app) rather than any club/player-facing transaction – e.g. a sponsored club badge, a "supported by X" line on a club page, or a sponsor slot in the release/whats-new surface	Referenced generically across sports-app monetization guides (e.g. "venues or sponsors like Nike pay to be featured")	Sponsorship placement (a local business/brand pays to be visible to the club's user base)
Yes, in principle – Teamder already aggregates chemistry/pair-stats/attendance data across ~596 users and dozens of clubs; a fully anonymized, club-level insight product (e.g. "engagement benchmarking for community organizers") sold B2B to something outside the player-facing app would not touch the no-payments constraint. This is speculative and would need real diligence on privacy/consent before going further than an idea	INFERRED , not directly confirmed for a named competitor in this search set, but explicitly discussed as a pattern in the general monetization literature reviewed (white-label licensing, B2B lead-gen)	B2B data/insight product
Partial fit – Teamder has no real "fan" audience distinct from players today, so this pattern doesn't map cleanly without inventing a new audience segment first	GameChanger (free for team, paid-ish for deep fan streaming)	Free-core, monetize a downstream/adjacent audience (fans, not players)

Bottom line: given the constraint as stated, the only two models from this survey that don't require reopening the payments question are **advertising** and **sponsorship-as-a-B2B-deal** (sponsor pays Teamder directly, never routed through a player-facing transaction) – both of which keep money entirely outside the player's view, which is consistent with the spirit of the existing rule, not just its letter. A club/organizer-facing B2B subscription is the next most plausible option but sits in a genuine grey zone against the stated constraint .and would need explicit owner sign-off before being treated as in-scope; it is flagged here, not recommended

Sources (all accessed 2026-09 unless a publish date is shown)

- [Footy Addicts App Store](#) · [Trustpilot](#) ·
- [Plei on Republic](#) · [RefreshMiami](#), [Plei \\$1.22M raise](#) ·
- [GoodRec App Store](#) · [PlayNow vs GoodRec Toronto comparison](#) ·
- [playpickups.com manifesto](#) (fetched directly) ·
- [Playtomic Levels & Algorithm, official help center](#) · [Playtomic padel levels blog](#) · [Proper Padel – is the rating flawed?, Sep 2025](#) · [UK Padel Guide 2026 booking-app comparison](#) ·
- Basketball pickup apps: [Fullcourt \(Google Play\)](#), [HoopFind](#), [Pickup: Basketball Runs](#), [HoopRun](#), [ATH](#) ·
- [Spond vs Heja](#) · [TeamStats vs Heja vs Spond](#) · [Spond payments help doc](#) ·
- [GameChanger app features](#) · [gc.com](#) ·
- [Veo pricing](#) ·
- [מגזין הספורט – מאיצטדיון למגרש השכונתי](#) · [mako](#) · [מגזין הספורט – מאיצטדיון למגרש השכונתי](#) · Hebrew search: [ynet](#) ·
- [StriveCloud – Strava gamification case study](#) · [Trophy.so – Strava gamification 2026](#) ·

General monetization pattern literature: [ideausher – sports facility booking app monetization](#), [choicely – sports app monetization](#), [ptolemay – free apps that make money without ads](#) •

WhatsApp-fatigue pain points: [dev.to – replaced WhatsApp sports groups](#), [happyroster – beyond the group chat](#), [klubraum – WhatsApp alternatives](#) •

נספח G – מפת המוצר (Discovery)

D1_screens

D1 – Screen & Surface Inventory (Teamder)

Read-only discovery pass. Scope: `src/navigation/*.tsx`, `src/screens/**`, `src/components/**` (for `src/navigation/tabLeaveGuard.ts`, `src/navigation/navigationRef.ts`, `App.tsx`, `(modals/sheets`, `.app.json`, `src/services/deepLinkService.ts`

.All file paths below are relative to `/Users/matan/Projects/soccer` unless given in full

Navigator tree .1

RootNavigator (`src/navigation/RootNavigator.tsx`) – picks ONE of these, no shared history:

```
|
├ !userHydrated          → SplashScreen (src/screens/SplashScreen.tsx)
├ !onboardingDone        → OnboardingScreen (pre-signin 3-slide pitch)
├ !currentUser           → AuthStack(initialRoute="SignIn")
├ !isGuest && !hasCompletedOnboarding → PostSignInOnboardingScreen
├ !isGuest && !profileComplete → AuthStack(initialRoute="ProfileSetup")
├ !groupHydrated         → SplashVisual (fallback splash)
└ else                   → MainTabs
```

MainTabs (`src/navigation/MainTabs.tsx`) – bottom tab bar, 4 tabs, RTL so array order right→left: ProfileTab (initial/"home") is first in code but visually the LEADING/rightmost tab per the RTL comment.

```
|
├ ProfileTab      → ProfileStack      (title "בית" / tabHome)
├ CommunitiesTab → CommunitiesStack  (title "מועדונים")
├ GameTab        → GameStack         (title "מחזורים")
└ ChatTab        → ChatStack         (title "צ'אטים", badge = unread count)
```

Every tab press resets its nested stack to a hard-coded root

`ChatsList` / `CommunitiesFeed` / `GamesList` – (map in `MainTabs.tsx` `TAB_ROOT`)
.self-healing any deep-linked or drilled-down stack state – `Profile` /

AuthStack (`src/navigation/AuthStack.tsx`)

`ProfileSetup` → `EmailAuth` → `SignIn`

tab "צ'אטים" – ChatStack (`src/navigation/ChatStack.tsx`)

`PlayerCompare`, `PlayerCard`, `DirectChat`, `CommunityChat`, `GameChat`, `ChatsList`

tab “מחזורים” – GameStack (src/navigation/GameStack.tsx)

```
, MatchDetails , GameEdit , GameCreate , GamesMap , Requests , GamesList  
, AvailablePlayers , LiveMatch , MatchRounds , RoundSummary , EveningSummary  
, PlayerCard , DraftBoard , DraftSetup , MatchPlayers , AddMembers  
, CommunityDetailsPublic , CommunityDetails , PlayerTimeline , PlayerCompare  
, CommunityStats , CommunityPlayers , CommunityEdit , AvailabilityEdit  
PromoteOrphan , History , AdminApproval , CommunityHistory
```

tab “מועדונים” – CommunitiesStack (src/navigation/CommunitiesStack.tsx)

```
, CommunitiesCreate , CommunitiesMap , Requests , CommunitiesFeed  
, CommunityPlayers , CommunityEdit , CommunityDetailsPublic , CommunityDetails  
, PlayerCompare , PlayerCard , CommunityHistory , CommunityStats  
, EveningSummary , DraftBoard , DraftSetup , MatchDetails , PlayerTimeline  
, AvailablePlayers , MatchPlayers , AddMembers , MatchRounds , RoundSummary  
GameCreate , History , AdminApproval , LiveMatch , GameEdit
```

tab “בית” – ProfileStack (src/navigation/ProfileStack.tsx)

```
, BlockedUsers , ProfileEdit , Requests , AvailabilityWeek , Profile  
, PlayerCompare , PlayerCard , NotificationsSettings , AvailabilityEdit  
, Statistics , Achievements , History , AdminApproval , PlayerTimeline  
, DraftBoard , DraftSetup , MatchDetails , Feedback , Referrals , Friends  
, MatchPlayers , AddMembers , MatchRounds , RoundSummary , EveningSummary  
, CommunityEdit , CommunityDetails , LiveMatch , GameEdit , AvailablePlayers  
GameCreate , CommunityHistory , CommunityStats , CommunityPlayers
```

Deliberate duplication (documented, not a bug)

The code comments are explicit about this: MatchDetails (and its whole / RoundSummary / EveningSummary / DraftBoard / DraftSetup – “drill chain” / GameEdit / AddMembers / AvailablePlayers / MatchPlayers / MatchRounds / CommunityEdit – plus CommunityDetails (and ITS chain (LiveMatch , AdminApproval , CommunityHistory / CommunityStats / CommunityPlayers are registered PlayerCompare , PlayerCard , GameCreate , History identically in GameStack, CommunitiesStack and ProfileStack so that ,back” always returns to whichever tab the user actually drilled in from” instead of jumping to a different tab's root. PlayerTimeline is in all .three of those but NOT in ChatStack

Screens registered in only ONE stack (by design, confirmed no cross-stack callers found)

- .MapScreen with mode param, one per stack, no drill-in needed – GamesMap / CommunitiesMap •
- .only in CommunitiesStack – CommunitiesCreate •
- :GameStack + ProfileStack only (NOT CommunitiesStack). Verified via grep – AvailabilityEdit •
- , ProfileScreen.tsx , GamesListScreen.tsx) every navigate('AvailabilityEdit') call site lives inside GameStack or ProfileStack screens – no screen hosted (AvailabilityWeekScreen.tsx .by CommunitiesStack calls it, so this asymmetry is currently safe
- , ProfileEdit , Feedback , Referrals , Friends , Statistics , Achievements •
- ProfileStack only. All – AvailabilityWeek , NotificationsSettings , BlockedUsers , AvailabilityWeekScreen.tsx / ProfileScreen.tsx call sites for these are inside navigate() .themselves only reachable via ProfileStack – consistent, no gap found
- .GameStack only. See §6 (orphan candidate) – this one IS suspicious – PromoteOrphan •

Confirmed instance of the app’s known “screen missing from a stack” bug class ⚠️

is registered only in (src/screens/games/PromoteOrphanScreen.tsx) PromoteOrphan
:Its own file header comment says it’s reachable two ways . GameStack .tsx

Tap on the promotePrompt push ... 2. (Future) Inline CTA on the finished orphan .1
.game’s details screen

Grep for navigate.*PromoteOrphan across src/ finds zero call sites anywhere in
or src/components - the “(Future)” CTA on MatchDetailsScreen.tsx was never src/screens
built (grep for PromoteOrphan / orphan inside MatchDetailsScreen.tsx returns nothing). So
today PromoteOrphan is reachable only via the promotePrompt push notification
If that in-app CTA is ever .(GameTab in navigationRef.ts ,routed through navigateForPush)
- added to MatchDetailsScreen , it would need to fire only when the game was opened via GameTab
opening the same finished orphan game from CommunitiesStack or ProfileStack (both of which
also host MatchDetails) would silently no-op today, exactly the bug class this app has hit
ProfileStack.tsx / GameStack.tsx / CommunitiesStack.tsx before (per in-code comments in
describing the exact same failure mode for other screens, which is why those stacks duplicate the
match/community chains). SUSPECTED currently-dormant risk, not yet a live bug because no CTA
.exists yet

Screen table .2

.Format: file path · Hebrew title (from he.ts key or inline) · purpose · who reaches it · how

Reached via	Who	Purpose	Title	File
app cold start	anyone	Boot splash (animated pitch + ball + wordmark) shown while user/group state hydrates	-	src/screens/SplashScreen.tsx
Navigator when onboardingDone	anyone, not yet onboarded	Pre-signin 3-slide pitch carousel over a blue gradient, uses OnboardingPreviews screenshots	-	src/screens/onboarding/OnboardingScreen.tsx
supported only by onboardingScreen	n/a	NOT a route - a sub-component rendering framed screenshot images inside OnboardingScreen	-	src/screens/onboarding/OnboardingPreviews.tsx
Navigator when onboardingCompleted	signed-in user w/o completed onboarding	Single post-signin step: name + profile picture, sets onboardingCompleted	-	src/screens/onboarding/PostSignInOnboardingScreen.tsx
back initial route	anyone without currentUser	Sign-in landing (Apple/Google/guest/email entry)	-	src/screens/auth/SignInScreen.tsx
"המשך עם מייל"	anyone	Email+password sign-in/sign-up + password reset	"התחברות עם מייל"/"הרשמה עם מייל"	src/screens/auth/EmailAuthScreen.tsx
authStack when onboardingCompleted	signed-in, profile incomplete	First-run name capture (post OAuth sign-in with no name)	-	src/screens/auth/ProfileSetupScreen.tsx

Reached via	Who	Purpose	Title	File
ProfileTab root	any signed-in user	Player-card-as-dashboard: identity, stats grid, referral card, discipline row, next-game card, quick actions; opens <code>HamburgerMenu</code> for settings/support/sign-out	“בית” (tab)	<code>src/screens/tabs/ProfileScreen.tsx</code>
Profile → hamburger/avatar tap	self	Edit name + profile picture (upload or built-in avatar)	–	<code>src/screens/tabs/ProfileEditScreen.tsx</code>
Profile, GamesList “ביקוש באזור שלך” card, AvailabilityWeek	any user	Set weekday/time/radius availability so the matcher can surface shortage games	“מצא לי מחזורים”	<code>src/screens/profile/AvailabilityEditScreen.tsx</code>
Profile “הצג שבוע מלא” link	any user	Full 7×3 availability grid (who’s free nearby)	“פנויים לידך”	<code>src/screens/home/AvailabilityWeekScreen.tsx</code>
Profile hamburger	self	Per-notification-type push toggles + OS-permission gate	“הגדרות התראות”	<code>src/screens/profile/NotificationsSettingsScreen.tsx</code>
Profile hamburger	self	List + unblock chat-blocked users	“משתמשים חסומים”	<code>src/screens/profile/BlockedUsersScreen.tsx</code>
Profile hamburger;) also push <code>growthMilestone</code>	self	Achievements grid + detail popover, plays <code>AchievementCelebration</code>	“ההישגים שלי”	<code>src/screens/profile/AchievementsScreen.tsx</code>
Profile hamburger	self	Player numbers + relational superlatives (most-played-with, nemesis, etc.)	“סטטיסטיקה”	<code>src/screens/profile/StatisticsScreen.tsx</code>
Profile hamburger;) push <code>(friendRequest*</code>	self	Manage mutual friendships, incoming requests	“חברים”	<code>src/screens/profile/FriendsScreen.tsx</code>
Profile “שחקנים שהצטרפו דרכי” tile	self	Everyone the user referred (invitedBy)	“שחקנים שהצטרפו דרכי”	<code>src/screens/profile/ReferralsListScreen.tsx</code>
Profile hamburger (Support)	any user	Report bug / suggest feature form	–	<code>src/screens/FeedbackScreen.tsx</code>
header bell icon, in every tab stack	any user (sections vary by role)	Unified inbox: friend requests + community-join + game-join requests, each with “approve” “all	“בקשות”	<code>src/screens/RequestsScreen.tsx</code>

Reached via	Who	Purpose	Title	File
GameTab root	any user	Matches feed: פתוחים/שלי segmented list, filters, map link, FAB create	"מחזורים"	src/screens/games/GamesListScreen.tsx
communityDetails "צור מחזור"	game organiser	Thin shell over GameWizardForm ; picks community, calls createGameV2	"יצירת מחזור חדש"	src/screens/games/GameCreateScreen.tsx
imported by GameEditScreen	n/a	NOT a route - shared 3-step form used by Create+Edit	-	src/screens/games/GameWizardForm.tsx
GamesList admin actions	organiser only	Same wizard, bootstrapped from an existing game, updateGameV2	"עריכת מחזור"	src/screens/games/GameEditScreen.tsx
GamesList row tap, Details, History, push, deep link	any user (roles gate admin actions)	Read-mostly single-match view: info grid, roster preview, admin actions, sticky join/cancel CTA	"פרטי המחזור"	src/screens/games/MatchDetailsScreen.tsx
MatchDetails "ערו"	any user	Full roster: registered / waitlist / pending / guests	"שחקני המחזור"	src/screens/games/MatchPlayersScreen.tsx
GamesList admin section	organiser/admin	Coach-only: find invitable players matching weekday/city/hour	"שחקנים פנויים"	src/screens/games/AvailablePlayersScreen.tsx
GamesList admin section	admin	Admin bulk-registers community members straight into a game	"הוספת שחקנים מהמועדון" / "שריון מקומות מראש"	src/screens/games/AddMembersScreen.tsx
MatchDetails "קב"	admin/organiser	Step 1 of team draft: pick captains + order (snake/regular)	"חלוקת כוחות"	src/screens/games/DraftSetupScreen.tsx
from DraftSetup	admin/organiser (readOnly for others)	Step 2: live turn-based captain draft board → summary	"חלוקת כוחות"	src/screens/games/DraftBoardScreen.tsx
MatchDetails "ליי" started	any participant; controls admin-only	Pure match-timer surface (start/pause/resume/reset/end); renders PlainLiveMatchScreen or, when advanced flag on, AdvancedLiveMatchScreen	-	src/screens/LiveMatchScreen.tsx
AdvancedLiveMatchScreen (conditionally inside LiveMatchScreen)	n/a	NOT its own route - internal variant rendered by LiveMatchScreen when advanced mode is on: teams-on-pitch rotation, filler picker, shootout, equipment handoff, retro goals	-	src/screens/AdvancedLiveMatchScreen.tsx
Completed-game CTA	any participant	Club-wide, non-personal "what happened that evening" recap	"סיכום המחזור"	src/screens/games/RoundSummaryScreen.tsx
MatchDetails "ערו" EveningSummary	any participant	Personal shareable card for the viewer's evening, PNG share	"סיכום המחזור" (ה) key summaryTitle ; feature is "סיכום" branded	src/screens/games/EveningSummaryScreen.tsx

Reached via	Who	Purpose	Title	File
			per project "הערב (memory)	
היי MatchDetails	any participant	Per-committed-mini-game history: teams, score, goals, shootout	"היסטוריית המשחקים"	src/screens/games/MatchRoundsScreen.tsx
romotePrompt gap)	game creator	Post-orphan-game "turn who played into a community" wizard; calls promoteOrphanToGroup	"צור מועדון מהמחזור"	src/screens/games/PromoteOrphanScreen.tsx

(מועדונים "tab) Communities

Reached via	Who	Purpose	Title	File
unitiesTab root	any user	Communities feed: my clubs / pending / open-to-discover, FAB create	"מועדונים"	src/screens/communities/PublicGroupsFeedScreen.tsx
mmunities FAB	any user	Thin shell over GroupWizardForm , createGroup calls	-	src/screens/groups/CreateGroupScreen.tsx
imported by nityEditScreen	n/a	NOT a route - 2-step form shared by Create/Edit community	-	src/screens/groups/GroupWizardForm.tsx
push, deep link (member)	member/admin	Full "stadium" club page: hero, stats grid, next game, active players, share CTA; ⋮/≡ opens admin menu	dynamic (group name)	src/screens/communities/CommunityDetailsScreen.tsx
l discovery row, ber), games list "מועדונים באזור"	any signed-in user, non-member	Non-member preview reading /groupsPublic/{id} only (no roster/admin data)	dynamic (group name)	src/screens/communities/CommunityDetailsPublicScreen.tsx
ails hamburger	admin	Shell over GroupWizardForm for existing club	"עריכת מועדון"	src/screens/communities/CommunityEditScreen.tsx
rsPreview tap / hamburger	any member (admin sees extra actions)	Full member list + per-member stats, admin rating sheet, card/equipment sheets	"הסגל"	src/screens/communities/CommunityPlayersScreen.tsx
mmunityDetails	any member	Club-wide stat dashboard (leaderboards, top scorers, superlatives)	"סטטיסטיקת המועדון"	src/screens/communities/CommunityStatsScreen.tsx
ails hamburger	any member	Full list of the club's finished games	"היסטוריית מחזורים"	src/screens/communities/CommunityHistoryScreen.tsx
Screen / push (st no gameld)	admin	Pending join requests across every community the viewer admins	"בקשות לסגל"	src/screens/groups/AdminApprovalScreen.tsx

(tab "צ'אטים") Chat

Reached via	Who	Purpose	Title	File
ChatTab root	any user	List of every chat the user can access (communities + games + DMs), sorted by recency	"צ'אטים"	src/screens/chat/ChatsListScreen.tsx
ChatsList row, MatchDetails, push (chatMessage scope=game)	game participant	Game-scoped chat, shared ChatView	game / "...טוען" title	src/screens/chat/GameChatScreen.tsx
ChatsList row, CommunityDetails, push	member	Community-scoped chat	community title	src/screens/chat/CommunityChatScreen.tsx
"שלח הודעה", ChatsList row on PlayerCard, push	any user	on-1 DM chat-1	other users "הודעה" / name "ישירה"	src/screens/chat/DirectChatScreen.tsx

Players (shared, registered in Chat/Communities/Game/Profile stacks)

Reached via	Who	Purpose	Title	File
tap any player row/avatar app-wide	any user	Read-only public profile of any user (avatar, 3 stats, invite CTA stub)	user's name	src/screens/players/PlayerCardScreen.tsx
player card / community "השוואה" players	any user (within a shared community)	Head-to-head viewer-vs-other player card, PNG share	"השוואה"	src/screens/players/PlayerCompareScreen.tsx
"ציר" CommunityPlayers "זמן"	community admin	Admin-only per-community timeline of cards + equipment handoffs	"ציר הזמן" של "	src/screens/players/PlayerTimelineScreen.tsx

Map / misc

Reached via	Who	Purpose	Title	File
GamesList/CommunitiesFeed map icon	any user	Single component, two modes (games/communities), full-screen map with legend + detail card	dynamic	src/screens/map/MapScreen.tsx
hidden trigger in ProfileScreen (showLab state)	dev only	NOT a navigator route – DEV-ONLY overlay <Modal> mounted directly by ProfileScreen for previewing product animations; __DEV__ -gated, never shipped	-	src/screens/dev/AnimationLab.tsx

Modals and bottom sheets .3

:Global (mounted once, not screen-specific – likely in App.tsx)

| Component | Purpose | Host |

|---|---|---|

| Pulse-authored popup campaigns, routed via navigateCampaign | App.tsx root | src/components/CampaignGate.tsx |

sheet after a version "מה חדש באפליקציה" One-time | src/components/WhatsNewModal.tsx → src/components/WhatsNewGate.tsx |

| bump | App.tsx root

| Force/optional store-update prompt | App.tsx root | src/components/UpdateModal.tsx |

Renders Joryio SDK in-app campaign messages || (HtmlMessageView.tsx +) src/components/joryio/InAppMessageHost.tsx |

| App.tsx root

Detects a device screenshot → slides up a bug- | (ScreenshotAnnotator.tsx +) src/components/ScreenshotReportSheet.tsx |

| report sheet with the capture pre-attached, freehand annotate | App.tsx / navigator-level

Global themed replacement for `Alert.alert`, used from ~30 screens/components | (`appAlert`) `src/components/AppDialog.tsx` | anywhere

Standard styled confirm/notice popup | GameWizardForm, GameCreateScreen, | `src/components/ConfirmDialog.tsx` | CommunityDetailsScreen

Destructive-action confirm w/ mandatory ack checkbox | LiveMatchScreen, | `src/components/ConfirmDestructiveModal.tsx` | CommunityDetailsScreen, GamesListScreen, MatchDetailsScreen

:Screen-scoped
Component	Purpose	Opened from

| Nudge to set availability days | GamesListScreen | `src/components/AvailabilityNudgeModal.tsx` |

Filter the Communities feed (auto-join/regular games/nearby) | | `src/components/CommunityFilterSheet.tsx` | PublicGroupsFeedScreen

| Filter the matches list (day/location) | GamesListScreen | `src/components/GameFilterSheet.tsx` |

You're already registered for an overlapping game" | GamesListScreen" | `src/components/games/RegistrationConflictModal.tsx` |

| Admin "add/edit a guest" form | MatchDetailsScreen, MatchPlayersScreen | `src/components/GuestModal.tsx` |

Full-screen "you earned a title" animation | ProfileScreen, PlayerCardScreen, | `src/components/AchievementCelebration.tsx` | AchievementsScreen

| Admin edits a player's internal 1-10 rating | CommunityPlayersScreen | `src/components/AdminRatingSheet.tsx` |

| Big interactive search-radius map | AvailabilityEditScreen | `src/components/availability/AvailabilityRadiusMapModal.tsx` |

Big read-only radius preview map | AvailabilityEditScreen, CommunityFilterSheet, | `src/components/games/RadiusMapModal.tsx` | GameFilterSheet

| Full-screen location picker (search + map, real coords) | GameWizardForm | `src/components/games/LocationSearchSheet.tsx` |

| Mandatory chat ToS accept gate (App Review 1.2 / UGC policy) | ChatView | `src/components/chat/ChatTermsModal.tsx` |

| Pick club cover (curated gallery or device upload) | CommunityDetailsScreen | `src/components/community/CoverImagePicker.tsx` |

| Post-club-creation "invite people now" sheet | CommunityDetailsScreen | `src/components/community/InviteMembersSheet.tsx` |

| Admin issues a yellow/red card to a player | CommunityPlayersScreen | `src/components/community/IssueCardSheet.tsx` |

| Admin toggles who holds club ball/jerseys | CommunityPlayersScreen | `src/components/community/ManageEquipmentSheet.tsx` |

| Admin credits/undoes a missed goal post-match | MatchDetailsScreen | `src/components/match/RetroGoalsSheet.tsx` |

| "סיים ערב" Records ball/jersey holders right after | `src/components/match/EquipmentHandoffModal.tsx` | AdvancedLiveMatchScreen

| Who completes the team?" filler picker | AdvancedLiveMatchScreen" | `src/components/match/FillerPickerModal.tsx` |

| Tap-to-swap editor for an existing team draft | DraftSetupScreen | `src/components/match/TeamsEditModal.tsx` |

| Bottom-of-round winner picker | AdvancedLiveMatchScreen | `src/components/match/WinnerPickerModal.tsx` |

| Penalty-shootout tiebreaker flow | AdvancedLiveMatchScreen | `src/components/match/Shootout.tsx` |

Live "winner stays" rotation surface | | (renders WinnerPickerModal) `src/components/match/RotationPanel.tsx` | AdvancedLiveMatchScreen

Anchored popover on a tapped player avatar (live match) | `TeamScore` (used | `src/components/match/PlayerActionMenu.tsx` | within AdvancedLiveMatchScreen)

| Typed-confirmation account deletion | ProfileScreen (hamburger) | `src/components/profile/DeleteAccountSheet.tsx` |

Bottom sheet: settings/nav/support/sign-out/delete | ProfileScreen, | `src/components/profile/HamburgerMenu.tsx` | CommunityDetailsScreen, MatchDetailsScreen

| Anchored "❗ what is this" popover | used inline across many forms | `src/components/InfoTip.tsx` |

NOT itself a feature modal – shared spring-animated sheet primitive underlying most of | `src/components/anim/SpringSheet.tsx` | the above (ConfirmDialog, filters, IssueCardSheet, InviteMembersSheet, DeleteAccountSheet, ManageEquipmentSheet, RetroGoalsSheet, etc.) | n/a (primitive)

Onboarding / auth flow – cold start to usable app .4

.while `userStore` hydrates (animated) `SplashScreen` .1

First-ever launch, no account: `OnboardingScreen` – 3-slide pitch over blue gradient, using .2
real cropped screenshots (`OnboardingPreviews.tsx`) of the games feed / club detail / game
.detail

.3 `SignInScreen` → `AuthStack` / Apple / Google / guest-browse – “המשך עם מייל”.

.4 Guest path: `currentUser.isGuest === true` → skips every remaining gate, straight to
`MainTabs` (App Store 5.1.1(v) browse-without-account requirement)

.5 Email path: `EmailAuthScreen` (sign-in or sign-up toggle, forgot-password)

.6 If the OAuth/email provider gave no name → `AuthStack` → `ProfileSetupScreen` (name capture)

.7 single combined step: name + profile picture (upload or – `PostSignInOnboardingScreen`
built-in avatar). Sets `/users/{uid}.onboardingCompleted = true`. Note: this check
is evaluated **before** the `!profileComplete` check in (`!hasCompletedOnboarding`)
so a returning user who never finished onboarding sees this screen even if , `RootNavigator.tsx`
their name/avatar are otherwise fine

.8 Falls through to group hydration, then `MainTabs` , landing on `ProfileTab` (home / player-card
;dashboard) – no more dedicated “pending request” or “no community yet” full-screen states
`RootNavigator.tsx` those surface inline (toasts + a “pending” tag on the communities feed) per
.comments

Entry points .5

Deep links (`src/services/deepLinkService.ts` , `app.json` scheme)

Custom schemes: `teamder://session/<id>` , `teamder://team/<id>` (also legacy `footy://` kept -
(for compat per project memory

`/team/<id>` Universal/App Links hosting URLs: `https://teamderfc.web.app/session/<id>` and -
,also accepts the legacy `teamder.web.app / .firebaseapp.com` domains for old shared links)
(though those domains aren't actually served

Short links: `/i/<code>` via `inviteLinks` + `serveInviteCode` Cloud Function (per project -
memory `project_share_attribution_shortlinks.md`), plus a `?invitedBy=<uid>` attribution query
param and a base64url `b=` source token decoded client-side (`decodeSourceToken` in
(`deepLinkService.ts`)

The app deliberately does **not** use React Navigation's declarative `linking` prop – URLs are -
parsed and **stashed** (`storage.setPendingInvite`), and only `RootNavigator.tsx` 's single
,consumer effect calls `navigateInvite()` once the user is fully signed-in/onboarded/hydrated
to avoid racing auth. `App.tsx` wires `Linking.getInitialURL()` (cold start) + a warm-start
. `deepLinkService` listener into `Linking.addEventListener('url', ...)`

session → `GameTab/MatchDetails` ; team → member sees → (`navigationRef.ts`) `navigateInvite()` -
. `CommunitiesTab/CommunityDetailsPublic` non-member sees , `CommunitiesTab/CommunityDetails`

Push notification taps (`navigateForPush` , `src/navigation/navigationRef.ts` , called from
one big switch over `NotificationType` , routing ~20 push types to – (`App.tsx:794–795`
/ `MatchDetails` / `CommunityDetails` / `AdminApproval` / `Achievements` / `Friends` / `EveningSummary`
`PromoteOrphan` / chat screens / the campaign router. Full mapping is documented inline in
and is quite thorough – every `NotificationType` in (lines ~140-419) `navigationRef.ts`
is claimed to be covered `src/types/index.ts`

In-app popup-campaign / admin-broadcast taps – `navigateCampaign()` (same file) is a small
/ `openScreen` / `openProfile` / `openCommunity` / `openGame` / `openUrl`) allowlisted router

used both by `CampaignGate` (in-app popups authored in Pulse) and by the `(dismiss .navigateForPush push-type branch of adminBroadcast`

:Share links generated BY the app (outbound, for completeness)

Club invite share (`CommunityShareInviteCta`, `InviteMembersSheet`) – builds a `.invite` URL with attribution (`https://teamderfc.web.app`) `HOSTING_ORIGIN`

PNG capture handed to the OS share sheet – `PlayerCompareScreen` / `EveningSummaryScreen` – `.not a deep link ,(expo-sharing)`

Screens that appear orphaned – SUSPECTED .6

- registered, and reachable – (`src/screens/games/PromoteOrphanScreen.tsx`) **PromoteOrphanScreen** •
via the `promotePrompt` push, so not fully dead, but the in-app CTA its own header comment promises (“Inline CTA on the finished orphan game’s details screen”) does not exist in today (confirmed via `grep` – zero hits for `PromoteOrphan` / `orphan` in `MatchDetailsScreen.tsx` that file). SUSPECTED: either never built, or removed and the comment went stale. Also only registered in `GameStack`, not `CommunitiesStack` / `ProfileStack` (see §1) – if the missing CTA is ever added, it will need to guard against being invoked from those stacks
 - lives in `src/screens/` alongside real routes, and its file – **AdvancedLiveMatchScreen.tsx** •
header is a byte-for-byte copy of `LiveMatchScreen.tsx`’s “pure match timer, NO teams” comment ,which directly contradicts what the file actually renders (teams, rotation, filler picker shootout, retro goals, equipment handoff). SUSPECTED stale/copy-pasted header comment – not a navigation orphan (it’s a real conditionally-rendered sub-component of `LiveMatchScreen`, not a registered route), but worth flagging as a documentation trap for the next person who reads it “expecting a “pure timer
 - lives under `screens/` but is not a – (`src/screens/dev/AnimationLab.tsx`) **AnimationLab.tsx** •
navigator route at all; it’s a `__DEV__`-gated `<Modal>` mounted directly inside `ProfileScreen.tsx`. Not orphaned (has a live call site), just miscategorized by directory .location – flagging so it isn’t mistaken for a missing/dead screen during further audit passes
- No other screens in the 47-file `src/screens/**` tree were found registered-but-uncalled: every remaining screen has at least one confirmed `navigate()` call site or push/deep-link route into it .(traced via the navigator files + `navigationRef.ts` + targeted greps for each route name)
-

Notes on components directory (context, not exhaustive) .7

,holds ~180 files: generic UI primitives (`Button`, `Card`, `InputField`, `Avatar` `src/components/Toast`, `EmptyState`, `ScreenHeader/ScreenContainer...`), the `anim/` animation library (documented in ,project memory as the house standard: `BallSwitch`, `RollAwayCta`, `RollInView`, `ArcPopIn` , `community/` , `chemistry/` , `chat/`) `MatchClockLoader`, etc.), and domain-grouped subfolders , `profile/` , `players/` , `match/` , `joryio/` , `home/` , `games/` , `draft/` , `compare/`

None of these are navigator routes; they’re composed into the screens listed in `(availability/`

.The modal/sheet subset is inventoried in §3 .2§

D2 – Backend & Data Layer Discovery (Teamder)

.Read-only inventory. No quality judgements – drift/staleness noted only as observed fact

```
,Scope read: functions/src/index.ts (13,974 lines, 64 export const + a 3-function re-export)
Push,adminUserPush,chatPush,clubChemistry,holidays,notificationDedup,reviewAlerts,roundSummary,teamBalanceCore}.ts
,(427 lines) firestore.indexes.json ,(1,988 lines) firestore.rules ,(lines 1,950) src/firebase/firestore.ts
,(lines 2,191) src/types/index.ts
,Setup: admin.initializeApp() at functions/src/index.ts:80, const db = admin.firestore() :81
matches memory: App Check) 103: const ENFORCE_APP_CHECK = false ,88: const messaging = admin.messaging()
.(enforcement is off on all callables
```

Three modules are **generated backend twins** of client logic, kept byte-parity-tested against originals: clubChemistry.ts (from genClubChemistry.mjs), roundSummary.ts (from src/Utils/*.ts), and (per header comments elsewhere) teamBalanceCore.ts. These are pure, (genRoundSummary.mjs).import-free functions – no direct Firestore I/O – called by index.ts handlers

Firestore collection inventory .1

,Legend for “grows”: U=per-user, C=per-club(community/group), G=per-game, R=per-round/mini-game
 .effectively unbounded (needs a TTL sweep to stay bounded)=∞

Core entities

Holds	Doc id	Collection
, stats.* (goals/assists/wins/pen*/ownGoals/cleanSheets – server-maintained by commitRoundStats, see c/firebase/firestore.ts:186–209), availability, ievements, discipline state, fcmTokens (legacy root copy), subscriptions, friends[], notifiedMilestones n/a (that's on groups)	uid (Firebase Auth uid)	users
om the public user – notificationPrefs, fcmTokens[] for read-restriction (see docs.userPrivatePush comment, firestore.ts:1929–1946)	push fixed id	users/{uid}/private/push
Per-user popup-impression ledger (frequency capping)	campaign id	users/{uid}/campaignSeen/{cid}
count, lastMessageAt/lastText/lastSenderId/lastSenderId, scope/parentId/title	`\${scope}__\${parentId}`	users/{uid}/chatUnread/{chatKey}
{muted}	`\${scope}__\${parentId}`	users/{uid}/chatSettings/{chatKey}
block marker	blocked uid	users/{uid}/blocked/{blockedUid}
ity: roster (playerIds / adminIds / pendingPlayerIds), torId, notifiedMilestones[], ball/jersey holder arrays adminRatings	groupId (auto)	groups
olic discovery projection: name/city/geo/cover/ memberCount, t30 / gamesLast60 / activityAt (server-only, written by a daily/hourly activity sweep – see §4)	groupId (mirror) =	groupsPublic
age, status, groupId, userId :Community join request	rid (auto)	groupJoinRequests
Friendship request state machine	`\${fromUserId}__\${toUserId}`	friendRequests
night: roster (players/waitlist/pending/participantIds), status, lity, liveMatch, draftTeams, teams, guests, latches :capacityNoticeSent, rsvpNudgeSent, reminderSent, publicOpenedAt, holidayNotifiedAt recurringNextCreatedAt, openedNotificationSent autoTeamsAt	gameId (auto or {sourceId}_w{startsAt} for legacy recurring clones)	games
r-registration ticket: tappedAt, requestedAt (=server time)	uid	games/{id}/joinRequests/{uid}

Holds	Doc id	Collection
➤filler interest: <code>status</code> pending/cancelled/approved/declined	uid	<code>games/{id}/fillerInterests/{uid}</code>
Game chat message	auto	<code>games/{id}/messages/{msgId}</code>
Chat read-receipt / typing ephemeral	uid	<code>, games/{id}/reads/{uid}</code> <code>games/{id}/typing/{uid}</code>
Post-hoc goal correction record	auto	<code>games/{id}/retroGoals/{retroGoalId}</code>
game roster + result (teams, goals[], winnerSide, shootout) – the “advanced match” ledger	roundId	<code>games/{id}/roundHistory/{roundId}</code>
by latch – 1 doc per committed round; its count is the evening’s mini-game count (works even for pre- <code>roundHistory</code> games)	roundId	<code>games/{id}/committedRounds/{roundId}</code>
Per-player wearable/heatmap metrics for the evening	uid	<code>games/{id}/physical/{uid}</code>
Idempotency latch – evening-finish crediting fires exactly once	once fixed id	<code>games/{id}/finishCredited/once</code>
latch – <code>achievements.gamesJoined</code> incremented exactly once per (game,uid), survives cancel→rejoin	uid	<code>games/{id}/joinCredited/{uid}</code>
collision note: this exact subcollection name also exists under <code>groups/{gid}</code> (below) – different purpose, same pattern	uid	<code>games/{id}/memberCredited/{uid}</code>
/parallel per-round doc (client-hydrated <code>Game.matches</code> “in Firebase mode”) – teamA/B, goals, scores. Distinct from <code>.d}/roundHistory</code> (the server-only advanced-match ledger)	auto	<code>rounds</code>
Legacy per-user <code>ed,wins,losses,ties,attendancePct,cancelRate}</code> (<code>src/firebase/firestore.ts:1684–1707</code>)	uid	<code>playerStats</code>
weekly-fixture settings template (independent of any one match) <code>settings</code> , <code>lastOccurrenceAt</code> ,	auto	<code>gameSeries</code>
Audit: who deleted a game	gameId =	<code>gameDeletions/{gameId}</code>

Ratings

Grows	Written by	Holds	Doc id	Collection
U	Server only, via) onVoteWritten trigger (applyVoteDelta	Global (cross-community) rating summary: count/sum/average, appliedEvents[] (last-30 event-id latch, embedded on the doc itself – see §5)	uid (rated user)	ratings
U×raters	Client create/update (rater, own vote)	Individual vote	rater uid	ratings/{uid}/votes/{raterUid}
C×U	Same as above, scoped per-club	Legacy per-community rating summary, superseded by global ratings but kept live for app ≤1.0.11 via onVoteWrittenLegacy	uid	groups/{gid}/ratings/{uid}
C×U×raters	Client, fully writable during rollout (rules comment firestore.rules:1355– 1362)	Legacy per-club vote	rater uid	groups/{gid}/ratings/{uid}/votes/{raterUid}

Chat

Written by	Holds	Doc id	Collection
Client (members only)	Community chat message	auto	groups/{gid}/messages/{msgId}
Client, self-only	Receipts / typing	uid	, groups/{gid}/reads/{uid} groups/{gid}/typing/{uid}
Client create (2 participants, friends-only gate via otherAllowsMe)	on-1 conversation-1) meta (participants[]	sorted([uidA,uidB]).join('__')	dmConversations/{convId}
Client (participants only)	DM message	auto	dmConversations/{convId}/messages/{msgId}
Client, self-only	Receipts / typing	uid	, dmConversations/{convId}/reads/{uid} .../typing/{uid}
Client create (≤1.0.49, legacy direct-write path) and reportChatMessage callable (1.0.50+, real author/text – target end- state is client write:false per rules comment, not yet flipped)	Reported chat message (store- safety)	auto	chatReports/{id}

Community timeline / discipline

Grows	Written by		Holds	Doc id	Collection
C×events, ∞ (no observed TTL sweep in ranges read so far)	Server (discipline card issue/revoke helpers, called from onGameRosterChanged)	Per-club yellow/red cards + ball/jersey , userId , groupId , type) handoffs (revoked , expiresAt , at		auto	communityPlayerEvents

Stats & rollups (server-only writers; see §4 for mechanics)

Writers	Written by	Holds	Doc id	Collection
Players	<code>(commitRoundStats)</code> Server only	One evening's line for one player (goals/assists/wins/cleanSheets/rounds) – the per-game source <code>roundSummary.ts</code> reads <code>PlayerEvening</code> as	<code>{gameId}__{uid}</code> SUSPECTED, matches) (sibling id schemes)	<code>gamePlayerStats</code>
Players	Server only <code>(commitRoundStats</code> increments; <code>sealRoundSummary</code> merges <code>bestEvening.*</code> via <code>Math.max</code> , not increment)	Lifetime-in-this-club career line (goals/assists/rounds/wins/cleanSheets/games/ <code>bestEvening.*</code> high-water-marks)	<code>{groupId}__{uid}</code>	<code>communityPlayerStats</code>
Clubs	<code>, commitRoundStats)</code> Server only <code>, onGameRosterChanged</code> <code>(sealRoundSummary</code>	+ Club-wide cumulative counters <code>, kingGoalsSum/Count</code> <code>, kingAssistsSum/Count</code> <code>eveningsSealed</code> , <code>chemistrySince</code>	<code>groupId</code>	<code>communityStats</code>
Clubs	Server only, via <code>nextRecordBaseLine()</code> full-object recompute in <code>sealRoundSummary</code>	Best-ever single-evening holders per metric + (goals/assists/involvement/cleanSheets/wins) <code>firstEverSeen[]</code>	<code>groupId</code>	<code>clubRecords</code>
Players, sealed (TL expected)	Server only, fully recomputed (not incremental) each finish	Snapshot of one player's club-table rank/score/delta after one evening	<code>{gameId}__{uid}</code>	<code>eveningStandings</code>
Locations	<code>)</code> Server only, create-once <code>(sealRoundSummary</code>	The sealed, never-recomputed “story of the evening” (leaders, records fired, milestones, pair highlight) – <code>roundSummary.ts</code> 's <code>, RoundSummary</code> shape <code>SUMMARY_VERSION=1</code>	<code>gameId</code>	<code>roundSummaries</code>
Players	Server only: <code>commitRoundStats</code> increments <code>assistsAToB/BToA</code> directly (<code>index.ts:13417</code>); <code>rollUpClubPairs</code> increments the rest once/evening	Per-club pair totals (sameTeam/winsTogether/.../assistsAToB) – <code>clubChemistry.ts</code> 's <code>PairTotals</code> shape	<code>{groupId}__{pairKey}</code> (<code>a__b</code> pairKey = sorted)	<code>communityPairStats</code>
Locations	Server only, create-once latch for <code>rollUpClubPairs</code>	Marker-only: “this evening's pair stats are “counted	<code>{groupId}__{gameId}</code>	<code>communityPairRollups</code>
Players, sealed	Server only, via <code>commitRoundStats</code> (<code>index.ts:13417,13450,13478</code>). Rules comment (<code>firestore.rules:1949–1950</code>) says “written ONLY by <code>onGameRotationChanged</code> ” – stale : that trigger is now dead code (see §5), the real writer moved to <code>commitRoundStats</code>	Cross-club “played together” tally – deliberately NOT club-scoped (see <code>clubChemistry.ts:18–21</code> comment on why club chemistry can't reuse this)	global,) <code>pairKey(a,b)</code> (<code>a__b</code> sorted)	<code>pairStats</code>

Grows	Written by	Holds	Doc id	Collection
– ∞ ilyCleanup deletes where atedAtMs < now–30d	Client (whitelisted types only, clientNotifTypeAllowed) + server (everything else, Admin SDK bypass)	:Outbound FCM dispatch queue , recipientId , type , read , delivered , payload schemaVersion , dedupeKey	auto or deterministic) bucket id (dedupeIdFor	notifications
G, swept in ilyCleanup e target game is hed/cancelled	,Server only allow read,write: if false	Dedup window (60s) for game- update push spam + lastDispatchedAt per action category	gameId =	gameUpdateLatches/{gameId}
Pulse triage ∞	Server (onFeedbackCreated trigger) + Pulse dashboard service account. No client access at all	Pulse work-list item	auto (Cloud Tasks-adjacent doc)	tasks
dmin closes ∞	Client create-only (self, type ∈ {bug,suggestion})	User bug report / suggestion	auto	feedback
TL sweep in , ∞ ws per distinct fingerprint	Client create/update (no auth gate – pre-auth failures must be capturable), size-capped fields	Aggregated crash/error signature	fp (fingerprint)	errors
			see Chat above	chatReports
ampaign count) ∞	Pulse service account only (bypasses rules) – “the doc IS the request” since Pulse can’t call a callable	Admin marketing campaign , status , type : (push/popup) , sendAt , segment attempts , metrics.*	auto	campaigns
doc 1	Server-only doc, read by pushToAdmins	Per-admin-alert-type push on/off toggles	fixed	adminConfig/prefs
doc 1	Server-only	Admin device tokens[] for founder alerts	fixed	adminConfig/push
doc 1	,Server-only adminUserPush.ts:24	24h rolling send timestamps for campaign anti-spam	) fixed (RATE_DOC	adminConfig/pushRate
doc 1	,Server-only reviewAlerts.ts:123	– (capped 300) seenIds[] dedupe for store-review polling	fixed	adminConfig/reviewState
ixed types 10~	adminPush.ts:40 ,Server-only	Flood-guard throttle (20s window) per admin-alert type	NotifType =	adminPushLatches/{type}
small	Server-only	Server-enforced rate counters (e.g. createGroupCallable) – client cannot read or reset	op-scoped	serverRateLimits/{rid}
U×ops	Client, self-scoped	Legacy client-writable rate-limit counters – self-resettable, being phased out in favor of serverRateLimits	{uid}_{op}	rateLimits/{uid}_{op}
istinct cities#~	Server-only	Cached lat/lng for the cross- community filler matcher	normalized city name	cityGeocode/{normalizedName}
doc 1	index.ts:12525 ,Server-only	Last-run marker for runDailyCleanup	fixed	cronMeta/dailyCleanup

Grows	Written by	Holds	Doc id	Collection
C	Server-only (<code>updateShowcaseOnGroupChange</code> / <code>updateShowcaseOnGameChange</code> triggers)	Public denormalised snapshot powering <code>/c/{groupId}</code> share page (finished-game count, top attenders, recent games, roster)	<code>groupId =</code>	<code>communityShowcase/{groupId}</code>
r generated) ∞ (link	Client create (self-attributed)	<code>{type,targetId,invitedBy}</code> for <code>/i/<code></code> short-link resolution	short code	<code>inviteLinks/{code}</code>
per campaign link	Server-managed (read at <code>index.ts:11973</code> , aggregate bump at <code>:12376</code>)	Ad/marketing link metadata + click counters	<code>linkId</code>	<code>adLinks/{linkId}</code>
unclear TTL , ∞	Server (<code>trackLinkClick</code> onRequest)	Per-click event beacon	auto/short id	<code>linkClicks/{id}</code>
U(inviters)	(<code>trackLinkClick</code>) Server	Per-inviter click counter	inviter uid	<code>inviteClicks/{inviter}</code>
but internal 1 d maps grow nbounded by tinct-link-key forever - no thing observed	,Server-only) <code>bumpLinkClickAggregate</code> (<code>index.ts:12441-12480</code>)	:Cross-source click aggregate , <code>days.<YYYY-MM-DD></code> , <code>total</code> , <code>byDayLinks.<day>.<key></code> <code>byDayMeta.<day>.<key></code>	fixed doc in a metrics collection with one known doc	<code>metrics/linkClicks</code>
ked, ~2-3 docs	Server-only (<code>updateAppConfig</code> callable, single hard-coded admin uid gate)	, <code>latestVersion</code> , <code>minimumSupportedVersion</code> (force/soft update gates)	<code>ios</code> ... <code>android</code>	<code>appConfig/{platform}</code>
ent - 1 doc per ss filler pulse, self-cleaning	Server-only (<code>fillerPulseTask</code> / <code>startGameFillerPulse</code> , <code>index.ts:10623-10946</code>) - deleted on completion ("create-once marker so <code>fillerPulseChains</code> doesn't grow unbounded", comment at <code>:10814</code>)	Cross-community filler-matching chain state (which candidates pulsed, in what order)	<code>gameId =</code>	<code>fillerPulseChains</code>

Cloud Function inventory .2

:All 64 `export const` in `functions/src/index.ts` , plus 3 re-exported from `chatPush.ts` . Trigger legend , `onDocumentCreated` =CALL= `onCall` , SCHED= `onSchedule` , TASK= `onTaskDispatched` (Cloud Tasks), FS-C . `onRequest` =FS-W= `onDocumentWritten` , FS-U= `onDocumentUpdated` , HTTP

One-line purpose	Trigger	file:line	Function
ush queue – builds Hebrew FCM message, sends, marks delivered	notifications/{id} FS-C	index.ts:1773	onNotificationCreated
ffer into one consolidated admin push	TASK	index.ts:2146	flushPendingJoinerNotifsTask
atch of a game's future moment (public-open/auto-teams/reminder)	TASK	index.ts:2260	scheduledGameMomentTask
o one reconcileJoinsTask (dedup via deterministic task id)	games/{gameId}/joinRequests/{uid} FS-C	index.ts:2637	onJoinRequestCreated
ed join requests by tap-time into respecting caps + red-card blocks	TASK	index.ts:2663	reconcileJoinsTask
ner: public-mirror sync, join/leave ng, growth-milestone push, new-pending-request fan-out	groups/{groupId} FS-W	index.ts:3992	onGroupPendingChanged
oid participants on live-timer field changes (feeds widget/Wear tile)	games/{id} FS-W	index.ts:4307	onGameTimerChanged
op return ; logic migrated into ot deployed only to not shrink the function set	games/{id} FS-W	index.ts:4432	onGameRotationChanged
nish-crediting, evening standings, triggers, round-summary sealing, st promotion, capacity/notification fan-out, game-delete audit	games/{gameId} FS-W	index.ts:4818	onGameRosterChanged
ta into the global rating summary	ratings/{uid}/votes/{raterUid} FS-W	index.ts:6117	onVoteWritten
y per-club summary (app ≤1.0.11)	FS-W groups/{gid}/ratings/{uid}/votes/{raterUid}	index.ts:6167	onVoteWrittenLegacy
ded admin uid sets version gates	CALL	index.ts:7079	updateAppConfig
ts estimatedRating (0-5, 0.1 side games/{id}.guests[] . -write on the game doc; enforces edBy===caller (adder-owned)	CALL	index.ts:7148	setGuestRating
n cascade, confirmed: (1) leaves nip array, dissolves a group if the ver (cascade-deletes its games , tyShowcase docs), else hands admin/oldest player; (2) strips the roster (participantIds array- omputes participantIds ; (3)	CALL	index.ts:7225	deleteMyAccount

One-line purpose	Trigger	file:line	Function
removal via <code>arrayRemove</code> ; (4) <code>derName / senderAvatarId /</code> every chat message the caller ever <code>.actionGroup('messages')</code> <code>__name__</code> , <code>450/batch, capped</code> <code>ok msgs</code>) (index.ts:7351-3370); (5) ymises the <code>users/{uid}</code> doc <code>Url / availability / ,"</code> שחוסר <code>letedAt fcmTokens</code> , stamps <code>:/push</code> ; (6) deletes the Firebase I (confirmed by absence from the ly): <code>communityPlayerStats</code> , <code>rStats</code> , <code>gamePlayerStats</code> , <code>andings</code> , <code>roundSummaries</code> , <code>ratings</code> , <code>notifications</code> , , <code>communityPlayerEvents</code> , - these keep the deleted uid as a ver (a fact, not evaluated here for GDPR-adequacy)			
at-report with verified author/text	CALL	index.ts:7419	<code>reportChatMessage</code>
a community-only game (adds to es old client-spoofable notif write	CALL	index.ts:7479	<code>sendGameInvite</code>
adds member(s) to a game roster	CALL	index.ts:7697	<code>adminAddPlayers</code>
Admin reorders players/waitlist	CALL	index.ts:7850	<code>adminReorderRoster</code>
trigger of the cancellation notice	CALL	index.ts:7961	<code>notifyPlayerCancelled</code>
trigger of "teams are ready" push	CALL	index.ts:8065	<code>notifyTeamsReady</code>
if a user's personal/default group	CALL	index.ts:8121	<code>ensurePersonalGroup</code>
er clock (client time-sync anchor)	CALL	index.ts:8199	<code>getServerTime</code>
s hidden personal/one-off group (o a real, discoverable community. : for §4/§5 : every one-off game a ares ONE personal group, so its of them; this callable does a hard of <code>communityPlayerStats /</code> pped to just the promoting game (, derived from the group's most- ent didn't pass one), then purges <code>Stats</code> , <code>eveningStandings</code> , <code>communityPairRollups</code> , and oup entirely (pair stats are wiped, see comment index.ts:8454-8459)	CALL	index.ts:8217	<code>promoteOrphanToGroup</code>
up cover image upload plumbing	CALL	index.ts:8725	<code>uploadGroupCover</code>
ed group creation (rate-limited via <code>serverRateLimits</code>)	CALL	index.ts:8789	<code>createGroupCallable</code>
-time migration: backfills missing <code>creatorId</code>	CALL	index.ts:8984	<code>backfillGroupCreatorIdsOnce</code>

One-line purpose	Trigger	file:line	Function
ase in sync with group changes	groups/{id} FS-W	index.ts:9469	updateShowcaseOnGroupChange
ase in sync with game changes	games/{id} FS-W	index.ts:9499	updateShowcaseOnGameChange
lic /c/{groupId} share page	HTTP	index.ts:9740	serveCommunityPage
links, redirects, beacons the click	HTTP	index.ts:9808	serveInviteCode
shing pulse – dispatches the next candidate in a game's filler chain	TASK	index.ts:10789	fillerPulseTask
ne filler pulse for one game (finds number candidates, builds the chain)	CALL	index.ts:10881	startGameFillerPulse
admin a filler expressed interest	games/{id}/fillerInterests/{uid} FS-C	index.ts:11062	onFillerInterestCreated
availability-heatmap counts (Pulse feature)	CALL	index.ts:11177	availabilityCounts
submission – validates game still not started, candidate not already a participant on resubmit (checks existing status: 'pending' before writing)	CALL	index.ts:11309	submitFillerInterest
approves a filler into the game roster	CALL	index.ts:11424	approveFiller
Admin declines a filler	CALL	index.ts:11602	declineFiller
recipient of a new friend request	friendRequests/{rid} FS-C	index.ts:11684	onFriendRequestCreated
s [] (client can't do this directly)	CALL	index.ts:11714	acceptFriendRequest
Removes a friendship	CALL	index.ts:11778	removeFriendship
invites friends into a community	CALL	index.ts:11825	inviteFriendsToGroup
New-signup admin alert	users/{uid} FS-C	index.ts:11935	onNewUserJoined
Admin alert on game creation	games/{id} FS-C	index.ts:12016	onGameCreatedAlert
Admin alert on join	games/{id} FS-U	index.ts:12036	onGameJoinedAlert
Admin alert on community creation	groups/{id} FS-C	index.ts:12063	onCommunityCreatedAlert
Admin alert on community join	groups/{id} FS-U	index.ts:12073	onCommunityJoinedAlert
inSince maps as membership changes	groups/{id} FS-U	index.ts:12095	stampMembershipDates
admin alert on availability change	users/{uid} FS-U	index.ts:12137	onAvailabilityUpdated
min alert on new error fingerprint	errors/{fp} FS-C	index.ts:12178	onErrorLogged
Filler Pulse creates a push campaign	campaigns/{id} FS-C	index.ts:12194	onCampaignCreated
tracks campaign engagement metric	CALL	index.ts:12205	trackCampaignEvent
review metadata for an invite link	HTTP	index.ts:12230	getInvitePreview

One-line purpose	Trigger	file:line	Function
Click beacon for ad/invite links	HTTP	index.ts:12362	trackLinkClick
Admin alert on new feedback	feedback/{id} FS-C	index.ts:12484	onFeedbackSubmitted
3 transitions – 7 sub-sweeps, see §3	SCHED (every 5 minutes , Asia/Jerusalem)	index.ts:12535	cronEvery5Min
stage/filler-matching/store-review polling – 6 sub-sweeps, see §3	SCHED (every 15 minutes , Asia/Jerusalem, us-central1 , 540s timeout, 512MiB, secrets ASC_P8 / PLAY_SA)	index.ts:12552	cronEvery15Min
notifications, holiday notices, gated daily sweep – 5 sub-sweeps, see §3	SCHED (every 60 minutes , Asia/Jerusalem)	index.ts:12647	cronEvery60Min
friends-only: friends who share a club	CALL	index.ts:12670	getFriendsInClubs
Writing callable, fully confirmed. Used against game.createdBy / adminIds); binds every credited game's actual roster (anti-forgery game a non-participant); excludes 11 players because the write cost O(n) (per-player) and the 500-op cap is hit around n≈13 (comment was previously 20, “that allowed v”). One atomic batch, latched by rounds/{roundId}.create() in the SAME batch	CALL	index.ts:12753	commitRoundStats
player wearable/physical metrics	CALL	index.ts:13541	saveGamePhysical
pitch calibration for physical tracking	CALL	index.ts:13668	savePitchCalibration
3rd fact – same roster-binding anti-commitRoundStats (per comment index.ts:12816)	CALL	index.ts:13761	addRetroGoal
ts a retry as idempotent success (per notificationDedup - adjacent comment at index.ts:13835)	CALL	index.ts:13843	removeRetroGoal
Writes a Pulse work-list item	feedback/{id} FS-C	index.ts:13918	onFeedbackCreated
1 bump + “one push until opened	games/{gameId}/messages/{msgId} FS-C	chatPush.ts:381 (re-exported index.ts:78)	onGameChatMessage
Same, community scope	groups/{groupId}/messages/{msgId} FS-C	chatPush.ts:391	onCommunityChatMessage
DM scope + friends-only re-check	FS-C dmConversations/{convId}/messages/{msgId}	chatPush.ts:405	onDmChatMessage

:Non-exported but load-bearing internal functions (called by the above, defined in `index.ts`)

```
,(the single writer of /notifications , dedup+aggregate logic – §5) createNotificationOnce
, loadUsers , canonicaliseNotificationPayload , deliverBatch , resolveRecipients
, reconcileGameJoins , enqueueGameMoments , dispatchGrowthMilestoneIfNeeded
, runCloneRecurringGames , runCreateSeriesOccurrences , runFlipScheduledGames / flipScheduledGameOnce
, runHolidayGameNotices , runExpireStaleOffers , runFlipPublicGames / flipPublicGameOnce
, runSendRsvpNudges , runSendGameReminders , runDailyCleanup , runCleanupStaleGames
, applyVoteDelta ,(index.ts:4613) sealRoundSummary ,(index.ts:4515) rollUpClubPairs
, fanOutTeamsReadyPush / generateDraftTeamsForGame / generateForGame / runScheduledAutoGenerateTeams
. linkAggKey , bumpLinkClickAggregate
```

Scheduled jobs .3

Only 3 `onSchedule` exports exist; everything else fires via Cloud Tasks (precise one-shot enqueued by `enqueueGameMoments` / `onJoinRequestCreated` . All three bodies read and (`onTaskDispatched` `runSweep(label, fn)` confirmed directly (index.ts:12535-12654); each sub-sweep runs inside a shared wrapper (index.ts:12513-12519) that **catches and logs but never rethrows** – one failing sweep can never block the others in the same tick

Confirmed sub-sweeps, in call order	Cadence	Job
<pre>, runFlipPublicGames , runFlipScheduledGames (series-driven fixtures first) runCreateSeriesOccurrences legacy clone, only for recurring games with no) runCloneRecurringGames , runScheduledAutoGenerateTeams ,(series yet sweepDueCampaigns(Date.now()) , runExpireStaleOffers</pre>	every 5 min, Asia/Jerusalem	<code>cronEvery5Min</code>
<pre>, runSendRsvpNudges , runSendGameReminders confirmed) runSendRateReminders , runSendShortageWarnings disabled no-op, per fork B – dead cron entry kept from a 2026-06-14 product , runFindFillerCandidates ,(decision runReviewAlerts(ASC_P8.value(), PLAY_SA.value())</pre>	every 15 min, Asia/Jerusalem, region us-central1 , timeoutSeconds:540 , memory:512MiB , secrets ASC_P8 + PLAY_SA (heavier budget explicitly to carry findFillerCandidates)	<code>cronEvery15Min</code>
<pre>personal-) runSendPromotePrompts , runCleanupStaleGames group→community promote nudge, 30min-6h post-finish window, latched by , runHolidayGameNotices ,(games/{id}.promotePromptSent ,(Firestore-latched, see below) runDailyCleanupIfDue (in-memory-latched, see below) runClubActivityIfDue</pre>	every 60 min, Asia/Jerusalem	<code>cronEvery60Min</code>

Two different “run at most once a day” gates living side by side in the same file, with different

:durability (a concrete, verified inconsistency – index.ts:12521-12645)
reads/writes `cronMeta/dailyCleanup.lastRunAt` in **Firestore** – durable across `runDailyCleanupIfDue` –
.cold starts and multiple concurrent function instances, gates at 23h
gates on `let lastActivitySweep = 0` , a **plain JS module-level variable** `runClubActivityIfDue` –
gates at 20h. This resets to 0 on every cold start, and each concurrent instance of the ,(index.ts:12639)
hourly function has its own independent copy – so under normal Cloud Functions scaling behaviour this is
not a reliable once-per-day guarantee the way its Firestore-backed sibling is. The sweep it guards
does a full `games` collection range-query (last 60 days) plus a full (`runClubActivitySweep`)
scan + batched write to every doc – so an unintended extra run is a real (if bounded) cost `groupsPublic`
.event, not just a no-op

Cloud-Tasks-driven (not cron, but effectively “scheduled” – fired at a precise future timestamp by
/ registration-open / public-open / auto-teams) `scheduledGameMomentTask` :(`enqueueGameMoments`

```
, flushPendingJoinerNotifsTask ,1h-reminder per game), reconcileJoinsTask (2s-debounced join-burst settle)  
    . fillerPulseTask
```

Design pattern noted repeatedly in comments: **“fire-but-verify”** – the precise Cloud Task fires the moment exactly on time, and the cron sweep re-scans every 5 min as a safety net for missed/failed task .delivery. No task cancellation on reschedule; stale tasks just no-op via their guard reads

```
polls Apple App Store Connect + Google Play ( reviewAlerts.ts:100 ) runReviewAlerts(ascP8, playSa)  
cronEvery15Min Developer API for new reviews and FCM-pushes the founder – confirmed called from  
    .which is why that job alone carries the heavier timeout/memory/secrets config ,(index.ts:12568-12570)
```

Drift risk	Recompute vs increment	Source of truth	Rollup
idStats call n-history path exists	Increment (FieldValue.increment , per client comment firestore.ts:191–208 : “so a full setDoc of a User object doesn’t clobber the tallies”)	call inputs (one commitRoundStats mini-game’s result)	users/{uid}.stats. {goals,assists,wins,pen*,ownGoals,cleanSheets}
Id} (see §5)	Written once per commit, per player	doc id scheme) commitRoundStats (confirmed, e.g. index.ts:13417	gamePlayerStats/{gameId}__{uid}
sed/duplicate vening self- eliberate one- ited, gated on ut means this ite semantics ne doc shape	Mixed, now 3-way: most fields incremental in the normal flow, bestEvening.* is a recomputed-max, and the promotion path is a full destructive overwrite	commitRoundStats goals/assists/rounds/wins/cleanSheets,) + (increment) onGameRosterChanged + (on finish games: increment(1)) sealRoundSummary Math.max high- , bestEvening.* + (water-mark, NOT increment confirmed) promoteOrphanToGroup 3rd writer – deletes every row for the group not belonging to the promoting game, then non-merge set() overwrites the kept rows with exactly ,that game’s tally (index.ts:8404–8417	communityPlayerStats/{groupId}__{uid}
ngsSealed rement ; the :true) means the promotion pped – by the intentional for it also means ssistsSum / nt are wiped ery promotion	Mostly incremental; eveningsSealed is a plain read-then-write (non- atomic outside the surrounding transaction); the promotion path is a full overwrite	, commitRoundStats onGameRosterChanged ,(kingGoals/kingAssists sums+counts)) sealRoundSummary computed as , eveningsSealed cur+1 in JS, not ,(FieldValue.increment confirmed) promoteOrphanToGroup – non-merge set() full overwrite of {groupId, rounds, goals, tiedRounds, updatedAt} , dropping kingGoalsSum/Count ,etc. entirely since it’s a non-merge set (index.ts:8423–8429	communityStats/{groupId}
baseline + this / construction ary firing at anteed by the se latch, since ne same code ath right after)	Full recompute of the baseline object each call (not incremental)	→ sealRoundSummary nextRecordBaseline()	clubRecords/{groupId}
d from current game is never deleted/expired	Full recompute per finish, one merge-set per player	+ communityPlayerStats + gamePlayerStats read at finish time , communityStats	eveningStandings/{gameId}__{uid}

Drift risk	Recompute vs increment	Source of truth	Rollup
il does NOT aled summary	Sealed once, create-only, never recomputed – explicit design choice (“a goal added by an admin next week must not rewrite last week’s story”, <code>roundSummary.ts:23–24</code>)	Everything above, snapshotted	<code>roundSummaries/{gameId}</code>
Guarded by _{gameId} ning part; the ndStats are ounds latch	Incremental (<code>FieldValue.increment</code> on 9 fields)	assists, direct) <code>commitRoundStats</code> <code>rollUpClubPairs</code> + (increment (everything else, once/evening)	<code>communityPairStats/{groupId}___{pairKey}</code>
ays written by gger is dead om the actual writer	Incremental	<code>commitRoundStats</code>	(global) <code>pairStats/{pairKey}</code>
PLICITLY forbids se two fields (badge-faking	Full recompute per sweep (<code>row.d30 / row.d60</code> computed fresh)	An hourly/daily activity sweep (<code>index.ts:12626</code> , inside <code>cronEvery60Min</code> ’s neighborhood)	<code>groupsPublic.</code> <code>{gamesLast30,gamesLast60,activityAt}</code>
g observed; a / u: keys × without bound	Purely incremental (<code>FieldValue.increment</code> on <code>total</code> , nested <code>days.<day></code> , <code>byDayLinks.<day>.</code> <code><key></code>)	(HTTP beacon) <code>trackLinkClick</code>	<code>metrics/linkClicks</code>
the main drift hantom docs)	Incremental, guarded (<code>update()</code> not <code>set-merge</code> , so a bogus <code>campaignId</code> fails instead of littering the collection – <code>adminUserPush.ts:450–</code> <code>461</code>)	callable <code>trackCampaignEvent</code>	<code>campaigns/{id}.metrics.*</code>
nts[] array tries – see §5	) Transactional delta (<code>applyVoteDelta</code>	<code>onVoteWritten</code>	(global rating summary) <code>ratings/{uid}</code>
/ construction	Recomputed on each trigger fire (denormalised projection)	changes <code>games / groups</code>	<code>communityShowcase/{groupId}</code>

`/ clubAssists` **Notable non- `FieldValue.increment` computation:** `sealRoundSummary`’s club-wide are summed by **iterating every community member’s `communityPlayerStats` row on every** `clubCleanSheets` **call** (O(n) over club size) rather than maintained as a running total – explicitly justified in a code .comment as “exact, just not O(1)” because `communityStats` doesn’t track those two fields (per fork B)

Batch-size ceiling on `commitRoundStats`, confirmed (`index.ts:12843–12860`): the per-round write is one atomic batch containing against-pairs ($|A| \times |B|$) + same-team pairs ($C(|A|, 2) + C(|B|, 2)$) + O(n) per-player tallies + the `committedRounds` latch. Worst case $\approx 2n^2 + 15n$ operations, which crosses Firestore’s 500-op batch cap around $n \approx 13$ per side. `MAX_SIDE` is hardcoded to **11** (real football’s max) specifically so ,the worst case (~400 ops) stays under the cap with margin; a code comment notes the cap used to be 20 that allowed the overflow” – i.e. this was a real production incident (a batch that throws with the” latch inside it means **every retry re-fails identically**, permanently losing that `committedRounds` .round’s stats) that was fixed by lowering the side-size cap, not by splitting the batch

confirmed as a second, destructive writer to `communityPlayerStats` and `, promoteOrphanToGroup` worth calling out separately here because it is the one place – (see the two rows above) `communityStats` in the whole backend where an aggregate collection normally maintained by `FieldValue.increment` is instead **deleted and rebuilt from scratch** by an entirely different code path, gated only by `group.isPersonal === true` and caller-is-admin. `, eveningStandings` , `communityPairStats` It also unconditionally deletes every and `communityPairRollups` doc for the group, and the single `clubRecords/{groupId}` doc , `roundSummaries` i.e. promoting a personal group is the only operation in the codebase that can make – (index.ts:8460-8496) ,docs disappear after being sealed (their general invariant elsewhere is create-once `roundSummaries` .(never-deleted

s	Mechanism	Latch
ly e	in the same batch as the guarded increment; <code>batch.create()</code> ALREADY_EXISTS (code 6) caught and skipped	<code>games/{id}/finishCredited/once</code>
ut g	create-once, same batch as <code>achievements.gamesJoined</code> increment	<code>games/{id}/joinCredited/{uid}</code>
ly al d	create-once	<code>games/{id}/committedRounds/{roundId}</code>
f n	ALREADY_EXISTS caught, <code>batch.create()</code>	<code>groups/{id}/memberCredited/{uid}</code>
P it 's p e)	check, then create in first chunk of a chunked (450-op) <code>.get().exists</code> batch	<code>communityPairRollups/{groupId}__{gameId}</code>
er	<code>summaryRef.create()</code> check, then <code>.get().exists</code>	<code>roundSummaries/{gameId}</code>
e	boolean flag, non-transactional get-then-set	<code>games/{id}.reminderSent</code>
e	boolean flag, transactional re-check-then-set	<code>games/{id}.rsvpNudgeSent</code>
e	transactional re-read-then-set	<code>games/{id}.capacityNoticeSent</code>
s e	presence check (idempotency comment index.ts:3415-3434)	<code>games/{id}.publicOpenedAt</code>
n e	boolean, set together with status change	registration-open flip / <code>games/{id}.openedNotificationSent</code>
er e	boolean-ish timestamp latch, PLUS a deterministic clone doc id (<code>{sourceId}_w{startsAt}</code> via <code>.create()</code>) as a second, independent guard	<code>games/{id}.recurringNextCreatedAt</code>
e	merge-set regardless of push outcome	<code>games/{id}.holidayNotifiedAt</code>
s n w	transactional claim-and-clear	<code>pendingJoinFlushAt</code> / <code>games/{id}.pendingJoinerIds</code>
s e n	deterministic Cloud Task id collapses concurrent enqueues (ALREADY_EXISTS swallowed)	task-id + <code>games/{id}/joinRequests/{uid}</code> <code>rj-{gameId}-{windowBucket}</code>
e n	<code>dedupeIdFor(type, recipient, entity, reason, cooldownBucket)</code> deterministic id, <code>.create()</code> semantics; AGGREGATE_ON_DUPLICATE - types merge into the existing doc instead of failing	<code>notifications/{deterministic-bucket-id}</code>
r d)	deterministic id; overwrite-safe because <code>onNotificationCreated</code> only fires on CREATE, so a repeat write never re-pushes	<code>notifications/{gameId}__teamsReady__{uid}</code>
s e	transactional <code>arrayUnion</code> claim	<code>groups/{id}.notifiedMilestones[]</code>
/ is	array of the last-30 applied Firestore trigger <code>event.id</code> s, embedded on the aggregate doc itself (not a separate marker doc) - unusual pattern vs.	<code>ratings/{uid}.appliedEvents[]</code>

s	Mechanism	Latch
nt	everything else in this list	
ə)	transactional 20s throttle window	adminPushLatches/{type}
→)	24h rolling timestamp array, filtered on read	adminConfig/pushRate
w e nt		
n of d	per-user timestamp stamped after send, filtered on next campaign	users/{uid}.lastBroadcastAt
n er e : y)	counter, parks the campaign in <code>status: 'error'</code> past the cap	(cap 6) campaigns/{id}.attempts
n	plain get-then-set, 60s window – explicitly not transactional (comment accepts the race)	__cancel / gameUpdateLatches/{gameId}__update
e e g ur	Durable, Firestore-backed: plain get-then-set, 23h gate, confirmed	cronMeta/dailyCleanup.lastRunAt
y ill n ur	In-memory JS variable, 20h gate	(module-level <code>let</code> , not a Firestore doc) lastActivitySweep
y” h w	boolean flag, set after dispatch	games/{id}.promotePromptSent
e e is	swallow <code>try{}catch{} / db.create()</code>	gameDeletions/{gameId}
e ə; a a r- o y it)	same batch commit , <code>batch.create()</code>	create, in the <code>games/{id}/committedRounds/{roundId}</code> SAME batch as every stat increment
: h g :	plain get-then-set (not transactional)	games/{id}/fillerInterests/{uid}.status=== 'pending' re-check

Stale-documentation flag: `firestore.rules:1949–1950` comments that `pairStats` is “written ONLY , `onGameRotationChanged`) server-side by `onGameRotationChanged` ” – but that trigger’s body is now a no-op .`index.ts:4432`, confirmed dead by fork B). The actual writer is `commitRoundStats` (`index.ts:13417/13450/13478`) .The rules comment was not updated when the write path moved

Converters .6

rebuilds every document **field-by-field** on both read (`fromFirestore`) and `src/firebase/firestore.ts` write (`toFirestore`) – per the standing memory note, a field added to a type but not to both converter :functions does nothing silently (broke 1.0.85 draft mode previously). Nine converters

	Backs	Lines	Converter
	<code>users</code>	150-432	<code>userConverter</code>
	<code>groups</code>	432-817	<code>groupConverter</code>
	<code>groupsPublic</code>	817-899	<code>groupPublicConverter</code>
	<code>groupJoinRequests</code>	899-1119	<code>joinRequestConverter</code>
	<code>games</code>	1119-1634	<code>gameDocConverter</code>
	<code>rounds</code>	1634-1683	<code>roundConverter</code>
(the SUSPECTED-dead legacy collection, §1)	<code>playerStats</code>	1684-1707	<code>playerStatsConverter</code>
<code>dmConversations/{id}/messages</code> , <code>groups/{id}/messages</code> , <code>games/{id}/messages</code>		1712-1739	<code>chatMessageConverter</code>
	<code>friendRequests</code>	1739-1760	<code>friendRequestConverter</code>

:Confirmed concrete instance of the field-strip trap, found during this audit (not previously logged)
deliberately omits `gamesLast30` / `gamesLast60` from (line 836-838) `groupPublicConverter.toFirestore`
client writes (“owned by the server sweep... a client write would let anyone claim their club is the most
,active”), while `fromFirestore` (lines 870-877) **does** read them back – asymmetric by design, not a bug
but it means these two fields only round-trip in one direction and a future refactor that “simplifies” the
.converter by making both directions symmetric would silently break the anti-spoofing guarantee
and its converter are still wired up client-side even though (`firebase.ts:1783-1785`) `col.playerStats()`
– no `index.ts` writer was found for the top-level `playerStats` collection and rules deny all writes
.SUSPECTED dead code path, not confirmed

, `pairStats()` :Several top-level accessors return **unconverted** raw references (no `.withConverter`)
/ `gameTyping` / `groupReads` / `gameReads` , `userBlocked` / `userChatSettings` / `userChatUnread` , `notifications()`
/ `globalRatings` / `ratingVotes` / `ratings` , `dmTyping` / `dmReads` , `chatReports()` , `groupTyping`
these collections have no field-strip risk on the client (raw `DocumentData`), but – `globalRatingVotes`
.by the same token get no type safety either

Indexes .7

, `messages.createdAt`) composite indexes + 2 field-override blocks 25 – `firebase.indexes.json`
both applied at COLLECTION and COLLECTION_GROUP scope – needed because chat messages , `messages.senderId`
live under 3 different parent collections and some queries, per comments elsewhere, may run as
.collection-group reads
:Composite indexes present, by collection

, (`status,registrationOpensAt`) , (`visibility,publicOpenAt`) : (Of the 25 10) **games** •
, (`participantIds CONTAINS,startsAt DESC`) , (`participantIds CONTAINS,startsAt ASC`)
, (`status,autoTeamsAt`) , (`visibility,status,startsAt`) , (`groupId,startsAt`) , (`createdBy,startsAt`)
, (`participantIds CONTAINS,status,startsAt`) , (`groupId,status,startsAt DESC`) , (`recurring,startsAt DESC`)

not 10, games is by ,13 – (seriesId,startsAt) , (status,startsAt) , (isPublic,status,startsAt)
 .far the most-indexed collection, consistent with it being the busiest query target
 .powers a “top assist pairs” query – (groupId,assists DESC) : **communityPairStats** •
 .(adminUserPush.ts:424–430) matches sweepDueCampaigns ‘s exact query shape – (type,status,sendAt) : **campaigns** •
 . (userId,status) , (groupId,status,userId) : **groupJoinRequests** •
 . (pendingPlayerIds CONTAINS,createdAt DESC) , (playerIds CONTAINS,createdAt DESC) : **groups** •
 . (gameId,index) : **rounds** •
 . (dedupeKey,read) : **notifications** •
 . (groupId,userId,type) , (groupId,type,at DESC) , (groupId,userId,at DESC) : **communityPlayerEvents** •

Cross-referenced against every query found across the full file (all 6 ranges) – all confirmed
 :satisfied by an existing index

where status==scheduled AND registrationOpensAt<=now (flip-registration sweep) → covered by games
 where visibility==community AND publicOpenAt<=now → covered by games ; (status,registrationOpensAt)
 where status==open AND autoTeamsAt<=now → covered by games ; (visibility,publicOpenAt)
 → recurring==true AND startsAt<=cutoff orderBy startsAt desc where games ; (status,autoTeamsAt)
 covered by (recurring,startsAt DESC) ; games where groupId==X AND status==finished orderBy startsAt desc (auto-
 . (groupId,status,startsAt DESC) balance history lookup, loadRecentSplits) → covered by
 where pendingPromotion.offeredAt < cutoff (waitlist-offer TTL sweep) is a single inequality on games
 .one field – Firestore auto-indexes this, no composite needed

Two additional candidate queries were flagged for manual verification and **both checked out as already
 :covered, not gaps**

where status=='finished' AND startsAt>=lower AND startsAt<=upper – games :(index.ts:8561-8566) runSendPromotePrompts -
 one equality + a range on the SAME field (startsAt) as the range – fully satisfied
 .by the existing (status,startsAt) index (item in the games list above)
 mirroring query (index.ts:9231-9237): games where- getCommunityStats -
 groupId==X AND status in [finished,cancelled] orderBy startsAt desc, limit 200 – an in clause is index-equivalent to
 equality
 for Firestore’s composite-index matching, so this is fully satisfied by the existing
 .index (groupId,status,startsAt DESC)

No missing composite index was found anywhere in functions/src/index.ts across the full read of all
 / approveFiller / submitFillerInterest) lines. commitRoundStats , the filler system 14,000~
 and cronEvery15Min / cronEvery60Min ‘s sub-sweeps were all confirmed to either query by ,(fillerPulseTask
 direct document id/field-equality (no composite needed) or to reuse one of the 25 indexes already listed
 .above

Open items / SUSPECTED – remaining after full coverage

,This document was assembled from 6 parallel range-reads of index.ts (all 6 completed and cross-checked)
 plus direct reading of firestore.rules , firestore.indexes.json , src/firebase/firestore.ts , and
 , promoteOrphanToGroup , deleteMyAccount plus targeted direct re-reads of , src/types/index.ts
 the three cronEveryNMin bodies, and the filler-interest system to confirm/replace the , commitRoundStats
 sub-agent summaries for those specific areas (the sub-agents’ own file-write attempts raced on the shared
 ,output path and were superseded – their prose summaries were used as a pointer to what to verify directly
 .(not taken as final
 :Remaining genuinely open items, not resolved by direct reading (kept as SUSPECTED rather than guessed)

votes **The entire per-community/global rating-vote system** (ratings , groups/{gid}/ratings , their •
 subcollections, onVoteWritten / onVoteWrittenLegacy) may be **fully dead**, not just “legacy kept for
 old app versions” as the firestore.rules comments (firestore.rules:1355–1362) state. Memory

records that peer/crowd rating was deleted 2026-06-24 in favor of an (`project_rating_1to10`)
`ratings/{uid}/votes/{rater}` internal-admin-only 1-10 rating – if the current client no longer writes to
.at all, both trigger functions and both collections are pure dead weight, not active legacy-compatible code
`col.globalRatingVotes` / `col.ratingVotes` **Not confirmed either way** – would need a client-side grep of
.call sites (out of this document's scope, which is backend + data layer) to settle
s exact: `removeRetroGoal` / `addRetroGoal` , `savePitchCalibration` , `getInvitePreview` , `setGuestRating` •
field-level deltas, and `serveCommunityPage` 's write behavior (if any) were read at the grep/signature
.level, not line-by-line – flagged SUSPECTED in §2 rather than asserted
no TTL/expiry sweep was found anywhere in the file despite – (discipline cards) `communityPlayerEvents` •
cards carrying an `expiresAt` snapshot field; expiry appears to be enforced only at READ time (client-side
is this card still active" check against `expiresAt`), not by any backend deletion. Worth a second pass"
.if store/read-cost pressure on this collection ever comes up

D3_features

D3 – Feature Inventory (Teamder)

= מועדון (`MatchRound`) one mini-game inside it = משחקון (`Game`) an evening/game session = מחזור .Read-only product inventory
club/community (`Group`). Line refs are approximate anchors, not exact ranges. "Tests exist?" reports what was actually found under
. `tests/` , not what *should* exist

A structural pattern worth stating once instead of per-feature: the two mega-services (`gameService.ts` , 8192 lines; `groupService.ts` ,
2194 lines) have **no dedicated test file of their own** – nothing named `gameService.test.ts` / `groupService.test.ts` exists. Their
pure/stateless sub-algorithms were deliberately extracted into `src/utils/*` (join fairness, rotation engine, evening score, penalty stats,
club chemistry, round summary, series schedule, played-games) specifically so they *could* be unit-tested outside Firebase – and those
extracted modules are well covered. The stateful CRUD/transaction bodies that remain in the two mega-services are exercised only via
mock-mode manual QA. `achievementsService` , `trustService` , `friendsService` , `chatService` , `notificationsService` , and
. `disciplineService` also have no direct test file

Auth & Profile .1

Guest ("browse as guest") sign-in

What: Anonymous Firebase Auth session so a user can browse public communities/games without registering (App Store guideline •
.5.1.1(v) compliance)

Files: `src/services/userService.ts:61` `signInAsGuest` ; `User.isGuest` flag (`src/types/index.ts:~45` , runtime-only, never
.persisted)

Who: Any first-time visitor •

`.isGuest:true` **Inputs:** none. **Outputs:** `User` with •

Touches: Firebase Anonymous Auth •

Tests: none found •

Google sign-in

`userService.ts:167` `signInWithGoogle` :**Files** •

`./users/{uid}` **Who:** Any user. **Inputs:** OAuth flow. **Outputs:** `User` doc created/hydrated on •

Tests: none found (auth race handling is covered indirectly by `tests/logic/authRaceRetry.test.ts`) •

Apple sign-in

.userService.ts:214 signInWithApple :Files •

.Who: iOS users. Apple gives neither name nor avatar, so PostSignInOnboardingScreen always runs afterward •

.Tests: none found •

Email/password auth

.What: Sign-in, sign-up, and password reset by email •

Files: userService.ts:263 signInWithEmail, :306 signUpWithEmail, :346 sendPasswordReset ;
.src/screens/auth/EmailAuthScreen.tsx (one screen toggles sign-in/sign-up)

.User :Who: Any user. Inputs: email, password. Outputs •

.Tests: none found •

Onboarding (pre- and post-sign-in)

What: 3-slide pre-sign-in pitch (OnboardingScreen.tsx , real in-app screenshots via OnboardingPreviews.tsx) + a single post-sign-in profile-customisation step (name + avatar/photo) (PostSignInOnboardingScreen.tsx). completeOnboarding patches the .User.onboardingCompleted profile and flips

.src/screens/onboarding/*.tsx ; userService.ts:355 completeOnboarding :Files •

.Who: New users only (gated by onboardingCompleted) •

.Tests: none found for the onboarding screens; completeOnboarding untested directly •

Profile edit (name, avatar/photo, position)

What: Edit display name and picture (gallery photo upload OR one of the built-in avatar illustrations – no more jersey-picker). •

.Preferred pitch position (gk|def|mid|att) settable from Profile

, src/services/photoService.ts:96 pickAndUploadAvatar ; userService.ts:717 updateProfile :Files •
.src/screens/tabs/ProfileEditScreen.tsx ; :271 deleteUserPhoto

.Who: Any signed-in user, own profile only •

./{uid}/avatar.jpg Touches: /users/{uid} , Firebase Storage •

.Tests: none found •

DM privacy toggle

.What: “Friends-only DMs” switch – when on, only friends can open a new 1:1 chat with the user •

.User.dmFriendsOnly ; userService.ts:422 setDmFriendsOnly :Files •

.Who: Any user, own account. Tests: none found •

Delete account

What: Full self-service account deletion – leaves every live/upcoming game first (•

.gameService.leaveAllGamesForAccountDeletion), then deletes the /users/{uid} doc and the Auth user

.gameService.ts:6498 leaveAllGamesForAccountDeletion ; userService.ts:650 deleteOwnAccount :Files •

.Who: Any user, own account only. Tests: none found •

(״שחקנים שהצטרפו דרכי״) Referrals list

.What: Lists every user whose invitedBy points at the viewer, newest first, with join timestamp •

; :807 listInvitedUsers , userService.ts:780 getInvitedUsersCount :Files •

.src/screens/profile/ReferralsListScreen.tsx

.invitedBy Who: Any user, own referrals. Touches: /users query on •

.Tests: none found •

Find-invitable-players (organiser tool)

- What:** Finds users whose saved availability (weekday + city + hour) matches a specific game and who aren't already in it, so a coach can push them a direct invite to game
- Files:** `userService.ts:513 findAvailablePlayers`; `src/screens/games/AvailablePlayersScreen.tsx` (thin presentation layer) – filters live entirely in the service
- Who:** Coach/admin of the game's club only
- Tests:** none found directly; `tests/logic/discovery.test.ts` covers adjacent discovery logic

Physical / Health Connect linkage

- What:** Reads a player's own session metrics (distance, steps, calories, HR zones, sprints, effort score) from the phone's wearable data hub – Android via Health Connect only; iOS (HealthKit) is a stubbed no-op (`nativeHealth()` returns null, panel just doesn't render). The read window is scoped to only the minutes the live-match timer was actually running (`liveMatch.activeIntervals`), not the whole evening
- Files:** `src/services/healthService.ts` (496 lines); `src/services/physicalSyncService.ts` (124 lines) (`computeEffort`, `computeHrZones`) `src/Utils/physical.ts`
- Who:** Any user, own device only (per memory: "Physical = phone Health Connect ONLY"; the earlier Wear OS Option-B path was removed)
- Tests:** `tests/logic/physical.test.ts`

Clubs .2

Create / edit community

- What:** Two-step wizard (details → advanced: open/private, max members, internal-rating toggle + hide-rating, cards master toggle + yellow/red validity). Community no longer owns field/format/schedule – those moved to per-Game
- Files:** `groupService.ts:283 createGroup`, `:1106 updateGroupMetadata`; `src/screens/groups/GroupWizardForm.tsx` (shared) `src/screens/communities/CommunityEditScreen.tsx`, `CreateGroupScreen.tsx`, by create+edit
- Who:** Any user creates; only coaches/admins edit
- Touches:** `/groups/{id}`, `/groupsPublic/{id}` mirror
- Tests:** `tests/logic/format.test.ts` / `formatPicker.test.ts` cover format-picker logic used by both game and group wizards; no group-wizard-specific test

Personal ("orphan") clubs + promotion

- in game "משחק חד-פעמי" – ללא קבוצה **What:** Every user gets a lazily-created hidden `isPersonal:true` group the first time they use creation, so the rest of the data model (rules, notifications, fillers, balance) needs no special-casing. Filtered out of feeds/search. After the orphan game finishes, the creator can promote it into a real, named community (invite specific participants)
- Files:** `groupService.ts:489 ensurePersonalGroupId`, `:519 promoteOrphanGroup`; `Group.isPersonal / hidden` (`promoteOrphanToGroup` `src/types/index.ts:~855`); `src/screens/games/PromoteOrphanScreen.tsx`; server callable
- Who:** The orphan game's creator
- `game.promotePromptSent` **Triggers:** `promotePrompt` push ~30 min after the orphan game finishes, latched by
- Tests:** none found directly

Join a community (code, browse, request/approve)

- What:** Join by invite code, or discover via public browse (`listPublicGroups` / `searchPublicGroups` reading `/groupsPublic`, never the private `/groups` doc for non-members). Open clubs auto-approve; closed clubs queue a `pendingPlayerIds` request an admin must approve/reject
- Files:** `groupService.ts:722 requestJoinByCode`, `:756 requestJoinById`, `:784 cancelJoinById`, `:828 approveMember`, `:979 getMemberApprover`, `:1000 rejectMember`, `:620 getPublic`, `:634 listPublicGroups`, `:649 searchPublicGroups`;

`src/screens/communities/CommunityDetailsPublicScreen.tsx` (non-member preview),
.`src/screens/groups/AdminApprovalScreen.tsx` (cross-club combined admin queue)

- .**Who:** Any user requests; only coaches approve/reject
- .(`rateLimitService.ts`) **Rate limit:** `joinRequest` op capped at 20/hour
- .**Tests:** none found

Roles: founder / coaches / members

What: `creatorId` (founder) can promote/demote coaches; a coach has full management rights (approve members, create/cancel .games, remove players). Founder can never be demoted

- Files:** `groupService.ts:1263 promoteToCoach`, `:1309 demoteCoach`; `Group.adminIds / creatorId` (.`src/types/index.ts:~765`); `getTeamCreatorId` helper
- .**Who:** Founder only for role changes
 - .**Tests:** none found

Leave / remove member / delete club

- .`:1650 deleteGroup`, `:1518 removeMember`, `groupService.ts:1356 leaveGroup` **Files**
- .**Who:** `leaveGroup` – self; `removeMember / deleteGroup` – coach/founder
- .**Side effects:** deleting a group fans out `groupDeleted` pushes to every member+admin
- .**Tests:** none found

Internal (admin-set) player ratings

- What:** Opt-in per-club mode where coaches set a 1-10 skill rating per member directly (`adminRatings`), instead of any peer-voting. .Can be hidden from non-admins (`hideInternalRating`). Drives team-balancing
-) `hideInternalRating / adminRatings / Group.internalRating`; `groupService.ts:1060 setAdminRating` **Files**
 - .(`src/types/index.ts:~900`)
 - .**Who:** Coaches set; visible to members unless hidden
 - .`tests/ratingAndColors.test.ts` **Tests**

Yellow/red cards (“discipline”) – per-club

- What:** Master switch (`cardsEnabled`) + configurable validity windows (`yellowCardValidityDays / redCardValidityDays`). A coach issues/revokes a card manually or it auto-issues from “I’m late” (>5min → yellow, >60min → red). An **active** red card blocks .the player from registering to that club’s games. Distinct from the global lifetime `User.discipline` counters
- Files:** `src/services/disciplineService.ts` (443 lines); `src/services/communityEventsService.ts` (342 lines, `communityPlayerEvents` collection, also backs the “last took ball/jerseys” hints); `src/utils/cardState.ts`; .`src/screens/players/PlayerTimelineScreen.tsx` (admin-only per-player timeline, long-press to revoke)
- .**Who:** Coaches issue/revoke; every member sees badges
 - .`tests/cardState.test.ts` **Tests**

Equipment holder tracking (ball / jerseys)

- What:** End-of-evening handoff popup records who’s taking the ball/jerseys home; `Group.ballHolderIds / jerseysHolderIds` .show everyone who should bring it next time. Timeline also logs each handoff event
- .(`LastTakenMap`) `communityEventsService.ts`; `groupService.ts:807 setEquipmentHolders` **Files**
 - .**Who:** Any member
 - .**Tests:** none found directly

(“תארים ורמת מועדון”) Club-level titles & achievements

What: Community-level parallel to personal achievements – club badges (bronze→silver→gold) plus one overall “club level” (1-10, etc. Entirely “מועדון על” → “מועדון חדש” weighted score from game-nights, club goals, member count, and club age), tier-named

derived client-side from existing aggregates (`getCommunityStats` , `getCommunityChampionship` , `Group.createdAt`) – no new
collection/trigger
/ `CommunityDetailsScreen.tsx` **Files:** `src/data/clubAchievements.ts` , `src/utils/clubLevel.ts` ; surfaced via
. `CommunityStatsScreen.tsx`
. **Who:** Visible to any member/visitor of the club
. `ClubMetrics` **Tests:** none found by name (`clubLevel` / `clubAchievements`); pure derivation from

Club card (feed row)

What: The community-feed row component that renders a club's relation to the viewer (member/admin/pending/discover), activity
.bar, floating badges, full-bleed cover – subject of the 5 most recent commits on this branch
Files: `src/components/community/ClubCard.tsx` ; `src/utils/clubCard.ts:resolveClubCard` ; used from
. `src/screens/communities/PublicGroupsFeedScreen.tsx`
. `tests/logic/clubCard.test.ts` : **Tests**

Cover photo

What: Admin-uploaded full-bleed hero photo (Storage `/groups/{id}/cover.jpg`) or a built-in gallery image (`coverImageId` ,
.random on creation). Falls back to a bundled stadium image
. `coverImageId` / `Group.coverPhotoUrl` ; `photoService.ts:186 pickAndUploadGroupCover` : **Files**
. **Who:** Coaches only (enforced via `storage.rules` admin lookup)
. **Tests:** none found

Nearby clubs discovery

. **What:** Surfaces geographically-near clubs (used in the games-feed discovery filler and the map). Cached/invalidatable
. **Files:** `src/services/nearbyClubsService.ts` (167 lines, `invalidateNearbyClubs`)
. **Who:** Any user. **Tests:** none found

Friends-within-club cross-reference

What: For a given set of clubs, resolves which of the viewer's friends are also members – used to show “3 friends are in this club”
.style hints
. **Files:** `src/services/clubFriendsService.ts:27 fetchFriendsInClubs` (60 lines)
. **Tests:** none found

Club invite links / codes

What: Short `inviteCode` for code-based join (built into `Group`), plus short attributable links `/i/<code>` (see Social domain for the
.shared invite-attribution mechanics)
; `groupService.ts:570 inviteFriendsToGroup` ; `Group.inviteCode` : **Files**
. `src/services/inviteLinkService.ts:35 createShortInviteUrl`
. `inviteLinkService` **Tests:** none found for

Community history / players / stats surfaces

What: `CommunityHistoryScreen` (all finished evenings), `CommunityPlayersScreen` (full roster with per-member stats, admins-
.first), `CommunityStatsScreen` (club dashboard – detailed under Statistics domain)
. `CommunityStatsScreen.tsx` , `CommunityPlayersScreen.tsx` , `src/screens/communities/CommunityHistoryScreen.tsx` : **Files**
. **Who:** Members (history/players); public preview is member-gated

(מחזור) Create / edit a game

- What:** 3-step wizard – (1) when/where (date, field name, city/address, field type), (2) format (format, #teams → computed max players, duration, extra time, half/penalties/referee toggles, advanced-mode opt-in), (3) management (visibility, requires-approval, recurring toggle, cancellation deadline, notes, bring-ball/shirts, cross-community filler opt-in)
- Files:** `src/screens/games/GameWizardForm.tsx` (1760 lines, shared by create/edit)
- `:2846 updateGameV2 , gameService.ts:2373 createGameV2 ; GameEditScreen.tsx`
- Who:** Any club member creates (subject to `createGame` rate limit: 10/hour); only creator/coach edits
- `/games/{id}` **Touches**
- `formatPicker.test.ts , tests/logic/formatWideStructures.test.ts` **Tests**

Game lifecycle / state machine

- What:** Pure status-transition + permission helpers: `scheduled → open → locked → active → finished/cancelled`. Gates every action screen-side (`canJoinGame`, `canCancelRegistration`, `canAddGuest / canRemoveGuest`, `canLockRegistration`, `canStartEvening`, `canEnterLive`, `canEndEvening`, `canCancelGame`, `canDeleteGame`) plus visibility filters for the 3 feeds (My Games / Open Games / History)
- Files:** `src/services/gameLifecycle.ts` (300 lines, fully pure – no Firestore)
- `registrationQa.test.ts` **Tests:** none found by that exact name; exercised indirectly through

Fair join ordering (registration burst fairness)

- What:** When registration opens, everyone gets the push simultaneously; ordering by server-write-arrival let a fast connection “steal” a spot from someone who tapped first. Fixed by carrying a server-synced `tappedAt` on every join and assigning spots strictly by tap time after a short settle window, with a 15s anti-backdate grace clamp
- Files:** `src/services/joinFairness.ts` (114 lines, pure, mirrored 1:1 server-side)
- Who:** Affects every player joining an open game
- `tests/logic/joinFairness.test.ts` **Tests**

Join / cancel / waitlist / approval

- What:** Join a game (auto-seat, waitlist if full, or `pending` if the game `requiresApproval`); cancel a registration; waitlist promotion via either instant auto-seat or a confirm-or-pass “spot offer” the head-of-waitlist must act on within a window; admin approve/reject “for gated games; bulk “approve all
- Files:** `gameService.ts:4828 requestJoinGame , :4995 joinGameV2 , :5420 approveGameJoin , :5591 rejectGameJoin , :5697 cancelGameV2 , :6223 confirmSpotOffer , :6328 passSpotOffer , :6417 adminAdvanceOffer ;`
- `src/screens/RequestsScreen.tsx ; src/services/requestsService.ts` (unified inbox + `approveAllForGame`)
- Who:** Any member joins; creator/coach approves
- Touches:** `/games/{id}` `player/waitlist/pending` arrays, `joinRequest` rate limit (20/hr)
- `registrationQa.test.ts , tests/logic/joinCtaCapacity.test.ts` **Tests**

Registration-conflict detection

- What:** Warns/blocks a user from double-booking overlapping games in the same club/time window
- `:4688 findRegistrationConflict` **Files:** `gameService.ts:170 RegistrationConflict` interface
- Tests:** none found directly

Recurring games / series

- What:** A `GameSeries` doc (separate top-level collection, `gameSeries`) is the template a weekly cron clones into a new occurrence; deleting one occurrence never kills the series or spawns a surprise match (memory: this was a real bug class before the refactor). Offsets (registration/public/guests-open) are stored relative-to-kickoff so every clone keeps the same relative schedule across DST

Files: `src/services/seriesService.ts` (172 lines); `src/utils/seriesSchedule.ts` (`settingsFromGame`, `pure`); `GameSeries` / `(src/types/index.ts:1388–1443)` `GameSeriesSettings` types •
. **Who:** Creator/coach toggles “recurring game” in the wizard •
. `tests/logic/seriesSchedule.test.ts` : **Tests** •

Registration windows (public-open / guests-open)

What: `registrationOpensAt`, plus separate `publicOpenAt` / `guestsOpenAt` timestamps that widen visibility/guest-adding beyond the initial community-only window

Files: `Game` fields (`src/types/index.ts:~1443+`); enforced in `gameLifecycle.ts` and mirrored in `firestore.rules` (memory •
. flags a past `.get(k,default)` null-trap bug here, fixed)
. `registrationQa.test.ts` **Tests:** covered transitively by •

Guests

What: A non-account “guest” player added by name (+ optional adder-set o-s rating). Guests are full cycle players: they occupy roster capacity (waitlisted if full), earn per-game scorer rows, own-goal attribution, and count in `communityPlayerStats`. Only the adder may set/clear the guest's rating (enforced server-side via the `setGuestRating` callable – element-level ownership can't be expressed in Firestore rules). Admin can rename/remove but never touch the rating

Files: `gameService.ts:7635` `addGuest`, `:7748` `updateGuest`, `:7834` `setGuestRating`, `:7900` `removeGuest`, •
. (`src/types/index.ts:1936`) `:7976` `adminReorderGuests`; `GameGuest` type

. **Who:** Any registered player adds a guest; only the adder rates them; admin manages roster placement •

Tests: none found directly; guest-goal handling covered by `tests/logic/assistCredit.test.ts` (per the recent fix commit “an assist to a guest was thrown away”)

Cross-community filler matching

What: A game can opt in (`acceptsFillers` + `fillerMinTrust` threshold) to receive push invitations to trust-qualified strangers from OTHER clubs when short on players. Server sends `fillerOpportunity` pushes to nearby opted-in available players (`AvailabilityEditScreen`'s `acceptsFillerPush`), 30 min before kickoff, one push per player (deduped). Candidate expresses interest (`submitFillerInterest` callable → `/games/{id}/fillerInterests/{uid}` `status:'pending'`); admin reviews/approves/declines from a dedicated section on MatchDetails. Fully wired end-to-end (opt-in toggle → push → apply → admin approve → roster)

Files: `gameService.ts:3261` `startFillerPulse` (manual pulse trigger from `AvailablePlayersScreen`), `fillerInterests` subcollection; `src/components/match/FillerInterestsSection.tsx`, `FillerPickerModal.tsx`; `src/services/notificationActionService.ts:174` `handleFillerOpportunityAction`; `src/screens/games/MatchDetailsScreen.tsx` (apply banner + admin section)

. **Who:** Admin opts a game in; any non-member candidate can apply; admin approves •

. **Tests:** none found directly for the filler flow itself •

discovery feed “פתוחים” surfacing filler-accepting games INTO the public) **Note:** the memory system's “Fillers-in-feed plan appears to describe a *discovery-surfacing* extension on top of an already-built filler system – the (specifically, flipping default opt-in underlying opt-in/push/apply/approve mechanics are live, not dormant. Worth re-confirming against Pulse/product notes whether that extension shipped

Admin bulk roster tools

What: `AddMembersScreen` – admin registers several community members straight into the roster in one action (overflowing to waitlist), each gets a push. `adminReorderRoster` / `adminReorderGuests` – manual roster reordering

. `src/screens/games/AddMembersScreen.tsx`; `:3380` `adminReorderRoster`, `gameService.ts:3291` `adminAddMembers` : **Files** •

. **Who:** Coach/admin. **Tests:** none found •

Cancellation / deletion

What: Player self-cancel (`cancelGameV2`); admin removes a player (`removePlayer`); admin cancels the whole evening (`cancelGameByAdmin`); hard delete (`deleteGame`) – only via the admin delete button or the o-roster stale-cleanup cron; every .deletion is audited

Files: `gameService.ts:5697 cancelGameV2` , `:5975 removePlayer` , `:6717 cancelGameByAdmin` , `:4544 deleteGame` ; `.gameDeletions/{gameId}` audit doc (per memory)

Tests: none found directly

Visibility / registration lock

`:6850 lockRegistration` , `gameService.ts:6784 setVisibility` **Files**

Who: Creator/coach. **Tests:** none found

(״פתוחים״/״שלי״) Games discovery feed

What: Segmented Matches tab (open/mine) that adapts its below-list discovery content (nearby-clubs teaser, “still didn’t find a , `games_feed_rich_min`) match?” filler card) to how full the visible list is – server-tunable via Remote Config thresholds `.( games_feed_clubs_max/radius_km/min_members , games_feed_demand_min`

Files: `src/screens/games/GamesListScreen.tsx` (1584 lines); `src/config/gamesFeedDiscovery.ts`; `remoteConfigService.ts` `.RC_DEFAULTS`

Tests: `tests/logic/discovery.test.ts` , `homeHero.test.ts` , `heroAtmosphere.test.ts` (related home/discovery UI logic)

Teams .4

(״חלוקת כוחות״) Manual drafting

, קבוצה א, → 1st tap) **What:** Two-step flow. Step 1: manager taps players to designate captains – selection ORDER sets team order and picks snake/regular draft order. Step 2: captains pick players in turns (auto-advances on tap, no confirm), – (... ב, → 2nd .horizontal team cards, flips to a summary at completion. Dynamic for 2~4 teams

Files: `src/screens/games/DraftSetupScreen.tsx` (840 lines), `DraftBoardScreen.tsx` (652 lines) `, :3216 setDraftTeamFeedback` , `:3187 publishDraftTeams` , `gameService.ts:3138 saveDraftTeams` `. :3244 notifyTeamsReady`

`.( src/types/index.ts:1103–1151 ) DraftTeamsResult / DraftTeam` **Who:** Admin/coach. **Touches**

Tests: `tests/logic/draft.test.ts`

Auto-balance by internal rating

What: Server-scheduled (`autoTeamsAt`) or on-demand auto-generated balanced teams from each player’s internal rating; fires a `. teamsGenerated` push per player listing their teammates; players can like/dislike the generated split

Files: `gameService.ts` (grep hits for `balance` / `autoTeam` inside the create/update-game paths) `. NotificationType.teamsGenerated`

Who: Admin triggers or schedules; all players notified

Tests: `balanceVariety.test.ts` , `balanceParity.test.ts` , `tests/logic/balanceTeams.test.ts` `. balanceMetaPersistence.test.ts`

Rotation engine (winner-stays queueing)

What: Pure engine for “winner stays on” rotation across 2~5 teams: who plays next, borrowed-filler completion when a team is short (`temporary` returns filler home next cycle vs. `permanent` keeps them), recording a winner or a tie (bothOut/veteranOut modes), .and reordering the waiting queue

Files: `src/services/rotationEngine.ts` (514 lines – `pickRandom` , `rosterOf` , `canStart` , `nextFillNeeded` , `applyChosenFill` `., startRotation(Skeleton)` , `recordWinner(Skeleton)` , `recordTie(Skeleton)` , `acceptsReorder` , `resolveRoundInstance`)

, rotationFill.test.ts , rotationCounts.test.ts , rotationDeep.test.ts , tests/rotationEngine.test.ts :Tests •
. tieConfirmParity.test.ts , tests/logic/rotationQueueReorder.test.ts

Manual live-roster editing

What: Swap two players between teams, move a player to a specific team, reorder the waiting queue, mark a player “went home” •
. (removes from rotation) / restore them, nudge rotation after a cancelled fill

, :4335 markPlayerWentHome , :4307 reorderWaiting , :4249 movePlayerToTeam , gameService.ts:4199 swapPlayers :Files •
. :4488 nudgeRotationAfterFillCancel , :4387 restorePlayer

.Who: Admin (advanced live-match screen) •

. tests/wentHomeRestore.test.ts :Tests •

Live Match .5

Plain timer screen (non-advanced games)

What: Deliberately minimal shared stopwatch – start/pause/resume/reset + “end game”. No teams, no formations, no scores. Clock •
derived from 3 synced Firestore primitives (timerRunning , timerLastStartedAt , timerAccumulatedMs) via useSyncedTimer ,
. so phone AND paired Wear OS watch stay in lockstep with zero per-tick pushes

Files: src/screens/LiveMatchScreen.tsx (1174 lines; routes to the advanced screen if game.advancedMode===true); •
; src/services/useSyncedTimer.ts (105 lines); src/services/serverClock.ts (130 lines, serverNow / syncServerClock)
. :7185 resetTimer , :7107 pauseTimer , gameService.ts:7034 startTimer

.Who: Admin controls; all participants watch •

.Tests: none found by that name for the timer itself; serverClock untested directly •

Advanced live match (rotation + scorer entry)

What: Full admin-only match manager: team assignments per zone (up to 5 teams A-E, bench, keeper zones), per-round •
goal/assist/own-goal entry, penalty shootout, formation slot placement, per-round win tallying, stoppage-time history, and ending a
round/evening. ADVANCED_MODE_ENABLED = true is a hardcoded const in GameWizardForm.tsx:261 (not a remote flag) –
. organiser opts each game in at creation/edit via the advancedMode toggle

) **Files:** src/screens/AdvancedLiveMatchScreen.tsx (2664 lines); LiveMatchState / LiveMatchPhase / LiveMatchZone types •
. (src/types/index.ts:1981–2177

.Who: Admin/coach only •

. tests/advancedMatchStats.test.ts :Tests •

Goals / assists / own goals

What: recordGoal logs a goal with collision-proof id, credits an assist only to a real attributed non-scorer, and separately tracks an •
evening-long per-player goalTally (drives the ball-icon badge) independent of the per-round log. Own goals carry the scorer (the
stat, but credit no striker “שערים עצמיים” player who put it in their own net, on the conceding team, via a mandatory picker) for the
. tally. Guests are full participants in all of this

) **Files:** gameService.ts:3489 recordGoal , :3563 removeGoal , :3622 undoLastGoal ; RoundGoal type •
. (src/types/index.ts:1223

, cleanSheetsPersistence.test.ts / tests/logic/cleanSheets.test.ts , tests/logic/assistCredit.test.ts :Tests •
. tests/cleanSheetsPersistence.test.ts , tests/assistPersistence.test.ts

Penalty shootout (“שובר שוויון”)

What: Tie-break flow for a drawn mini-game – pick which team kicks first, set each side’s (sticky) keeper, log each kick •
(scored/missed) in order, undo the last kick, or clear the shootout entirely. Feeds kicker/keeper penalty stats (penTaken/penScored/penMissed/penFaced/penSaved/penConceded on UserStats) and factors into the evening-score formula

(5% weight, taken from the wins axis) – the recent fix commit “the evening score ignored the shootout” patched exactly this integration

, :3736 recordShootoutKick , :3717 setShootoutKeeper , gameService.ts:3698 startShootout :Files •
;(src/types/index.ts:~2050) LiveMatchState.shootout ; :3802 clearShootout , :3778 undoLastShootoutKick
. src/utls/penaltyStats.ts
, tests/shootoutPersistence.test.ts ,Tests: tests/logic/penaltyStats.test.ts (51 tests per memory) •
. tests/logic/eveningScorePen.test.ts , tests/logic/championshipPenalty.test.ts

commit / rotation (מחזור) Round

What: finalizeRoundAndRotate commits a finished mini-game into games/{id}/roundHistory , advances the rotation queue, •
and is the single write path that fans out into commitRoundStats (goals/assists/wins/losses/rounds/cleanSheets/ownGoals/per*
.stats server-side)

.(src/types/index.ts:1247) Files: gameService.ts:3655 finalizeRoundAndRotate ; MatchRound type •
. tests/roundIdempotency.test.ts :Tests •

Retro goals (post-match stat correction)

What: Admin, after a game is FINISHED, credits a missed goal (+ optional assist) directly to a player's totals – a pure stat correction, •
detached from any mini-game score/winner, via a dedicated retroGoals subcollection and the addRetroGoal / removeRetroGoal
.callables

) Files: gameService.ts:1358 addRetroGoal , :1376 removeRetroGoal , :1288 getRetroGoals ; RetroGoal type •
. (src/types/index.ts:1239

.Who: Community admin, finished games only •

.Tests: none found by name; covered indirectly via championshipPenalty.test.ts /stat-rollup tests •

Stoppage / timer-control history

What: Chronological log of every start/resume/pause press (server-time-stamped) so players can settle “the clock kept running” •
disputes; separately, activeIntervals accumulates every window the timer actually ran across the WHOLE evening (survives
.round resets) – this is what scopes the Health Connect physical read

. activeIntervals / LiveMatchState.timerEvents ;(src/types/index.ts:2163) Files: TimerEvent type •
.Tests: none found directly •

Substitution ≥30s stint rule

Status: PLANNED, NOT BUILT. No per-player stint-timing exists in commitRoundStats or elsewhere – confirmed absent by grep •
(only the memory record + the rotation/filler “swap” mechanics exist, which move a player between teams but don't track a
.continuous on-field duration for stat-crediting purposes)

Statistics .6

(“סטטיסטיקה”) Player statistics screen

What: A dedicated page combining numeric stats (attended games, attendance %, distinct teammates, goals, goals-per-evening) •
with relational “people” superlatives – most-played-with, most-wins-with, biggest victim (beat them most), nemesis (they beat you
.most), most-assisted-to, most-assisted-by. Computed in ≤3 reads (one games scan + two pairStats halves)

. src/screens/profile/StatisticsScreen.tsx ;Files: src/services/playerStatsService.ts (248 lines, compute) •
.Who: Any user, own stats •

Tests: none found by name (playerStatsService); indirectly via tests/logic/playedGames.test.ts (isAttendedGame shared •
.helper)

Pair / chemistry stats ("who plays well together")

- What:** Club-wide directional pair rollup (assists-between-two-players, same-team record, head-to-head record) reduced to "six chemistry cards" (best duo, best win-rate together, etc.) in a single query per club rather than scanning every mini-game
- Files:** `gameService.ts:745 getPairStats` (per-pair, used by player cards); `src/services/clubChemistryService.ts:39 get`
`.src/utils/clubChemistry.ts` ;(80 lines, club-wide `communityPairStats` rollup + `pickChemistry`)
- Who:** Any member of the club
- `.clubChemistryReal.test.ts` , `clubChemistryParity.test.ts` , `tests/logic/clubChemistry.test.ts` :**Tests**

Player comparison ("head-to-head")

- What:** Shareable head-to-head card between the viewer and another player in a shared community; captures to PNG via the OS
`.share` sheet (same pattern as the evening summary)
- Files:** `src/services/playerCompareService.ts:118 getComparison` (201 lines)
`.src/screens/players/PlayerCompareScreen.tsx`
- Tests:** none found by name

Club stats dashboard

- What:** Aggregates everything the community has accumulated – total goals/assists/mini-games/evenings, leaderboards (top scorer, assister, winner, most loyal – via `longestStreak` / `currentStreakByUser`), a top-10 scorers table, and superlatives (deadliest goals-per-mini-game ratio, organisation rate = finished/(finished+cancelled), average attendance). Two backend rollups feed it with zero extra reads: `getCommunityChampionship` (cumulative per-player goals/assists/rounds/wins from `communityPlayerStats`) and `getCommunityStats` (evenings held, streaks, active-member counts, scanned over the most-recent 200 terminal games – not strictly all-time beyond that)
- Files:** `gameService.ts:891 getCommunityStats` , `:1067 getCommunityChampionship` , `:1195 getCommunityDeadlyDuo` (top assist pair via `orderBy+limit(1)` , not a full scan), `:1237 getGameChampionship` (per-game version, ranks by goals×2+assists);
`.src/screens/communities/CommunityStatsScreen.tsx` (892 lines)
- Tests:** none found by name; `championshipPenalty.test.ts` covers a slice of the penalty-stat ranking logic that feeds these
`.tables`

Records / leaderboards / club insights (assistant lines)

- What:** `assistantInsightsService` computes the facts behind the in-app coach/assistant's personalized lines – goals/assists THROUGH this specific club, 1-based scorer/assister rank, "X goals to the crown", and who the "rival" one place above is. Re-sorts the club table by goals and separately by assists (distinct from the table's own wins-first ordering) specifically so a claim like "two goals and you pass X" is never made against an ordering where it'd be false
- Files:** `src/services/assistantInsightsService.ts` (226 lines, `ClubInsight`) ; `src/utils/assistant.ts` (likely narrative-picking logic, not directly opened)
`.tests/logic/assistant.test.ts` :**Tests**

related fun facts & (top scorer) "מלך השערים"

- What:** Not a separate feature – the top-scorer/top-assister crowns are a derived read of `getCommunityChampionship`'s ranked `players[]` , plus club-wide fun facts also returned there: tied-round rate, shootout-decided rate, scoreless (0:0) rate, goals scored by (per memory "מלך השערים העצמיים" guests, and own goals across the club
- Files:** same as club stats dashboard above

self-based scoring model – ("איזה ערב היה לך") Evening score

- What:** Pure, unit-tested scoring formula (0 Firebase deps) producing a 6.0-10.0 score from category weights: no-shootout = 50% wins · 30% goals · 20% assists; in-a-shootout = 45% wins · 30% goals · 20% assists · 5% penalties (the shootout axis is carved OUT of the wins weight, not renormalised). Measured against the PLAYER'S OWN history, not "share of team's goals" (deliberately, to avoid punishing a player for a teammate scoring). Goals/assists "perfect 10" targets are the community-historical average of the evening's top scorer/assister, so 10 is always reachable and specific to that club

Files: `src/utils/eveningScore.ts` (re-exported at `src/services/eveningSummaryService.ts:35`); `SCORE_WEIGHTS_NO_PEN` / `SCORE_WEIGHTS_PEN` consts
• `, eveningProgress.test.ts , eveningSummary.test.ts , tests/logic/eveningScorePen.test.ts :Tests`
• `. eveningStats.test.ts`

personal, shareable – (“סיכום הערב”) Evening summary card

What: Per-player shareable summary for a finished game: rounds played vs. total, win rate, goals/assists, the evening score + title/emoji, situational “insight” strips, a comparison vs. the player’s PREVIOUS evening, and per-metric (goals/assists/wins) standing in the club table – including WHO was passed and who’s still ahead, all computed server-side at end-of-evening (`onGameRosterChanged`) after the ranking is final and stored at `eveningStandings/{gameId__uid}` so the card never re-ranks
• `.client-side`

Files: `src/services/eveningSummaryService.ts:240` `getEveningSummary` (411 lines);
• `. src/screens/games/EveningSummaryScreen.tsx` (PNG capture + OS share sheet)

Who: Any participant, own card. Feature marked SHIPPED per memory (with the ⚠️ noted stale-REST-list read trap already worked
• `.around via runQuery`)
• `. roundSummaryRealEvening.test.ts , tests/logic/eveningSummary.test.ts :Tests`

club-wide, non-personal – (“סיכום המחזור”) Round summary

What: The SAME document for everyone who was there – what happened at the club that evening: leaders (top scorers/assisters/clean-sheets/goal-involvement/winners), best/worst team highlights, the standout pair, and a chronological events feed (records broken, milestones hit). Sealed once at evening-end and never recomputed, because “which records stood” stops
• `.being answerable once another evening is played`

Files: `src/services/roundSummaryService.ts:23` `get` (37 lines, thin reader); `src/utils/roundSummary.ts` (`RoundSummary` , `RoundSummaryInput` , record/milestone metric types – the actual computation, ~200+ lines);
• `. src/screens/games/RoundSummaryScreen.tsx` (363 lines)

Tests: `tests/logic/roundSummary.test.ts` , `roundSummaryBackfill.test.ts` , `roundSummaryLines.test.ts` ,
• `. roundSummaryParity.test.ts , roundSummaryRealEvening.test.ts` – the single most heavily-tested feature in the app

(“היסטוריית המשחקים”) Match rounds history

What: Read-only per-mini-game history for a finished evening – teams, score, winner, full goal log (scorer/assister/own-goal), and shootout kicks when decided on penalties. Pure display over data already persisted by `commitRoundStats` ; a denied read
• `.degrades to empty state, never a crash`

`. games/{id}/roundHistory` **Files:** `src/screens/games/MatchRoundsScreen.tsx` (693 lines), reading
• **Tests:** covered by the round-summary/rotation test suites above (shared data model)

Trust score (“reliability meter”)

What: Single 0-100 reliability score per user, replacing the old two-counter yellow/red discipline UI (that UI’s underlying events still get logged server-side). Formula: `round(attendanceRate*100) - softCancels*3 - hardCancels*10` , clamped 0-100, over a rolling 90-day window of terminal games; returns `null` (renders “new”) below a minimum-games floor. Soft cancel = before the
• `.admin deadline; hard cancel = after`

Files: `src/services/trustService.ts:78` `computeTrustFromGames` , `:171` `tierForScore` (220 lines)
• `. src/components/TrustMeter.tsx`

`. ( fillerMinTrust )` **Who:** Shown on any player’s card; also gates filler-candidate eligibility
• `. ( trustService )` **Tests:** none found by name

Discipline lifetime counters

What: `User.discipline` – global, lifetime yellow/red counters + recent events, distinct from the per-club `communityPlayerEvents` timeline (Clubs domain) and superseded on the player-card UI by the trust meter above, though the underlying counters are still
• `.written for backward compat`

.src/services/disciplineService.ts :Files •
(Cross-referenced from Clubs domain – same service backs both surfaces) •

Social .7

Chat – community / game / DM

What: One /.../messages subcollection shape serves all three chat scopes (community, game, DM); membership is enforced entirely by firestore.rules (the service does not re-check). Features: send (≤1000 chars), realtime subscribe (windowed to the most recent 100), typing indicators, read receipts (ChatReader), unread badges, delete-own-message, mute a chat, block/unblock a user (client-side list at /users/{uid}/blocked/{id}), and report a message (server-side callable resolves the real message + sender so a reporter can't frame someone with fabricated content). First-line client-side Hebrew+English profanity filter blocks send before it reaches the server (ChatMessageBlockedProfanity analytics event); real moderation is report+block+delete, not the filter

;src/components/chat/ChatView.tsx ;src/data/profanity.ts ;Files: src/services/chatService.ts (420 lines)
;src/screens/chat/{ChatsListScreen,CommunityChatScreen,GameChatScreen,DirectChatScreen}.tsx
.src/screens/profile/BlockedUsersScreen.tsx

Who: Members-only per scope; DMs additionally respect the sender's dmFriendsOnly toggle •

Tests: tests/profanity.test.ts . No test file for chatService itself •

Friends

What: Send/accept/decline/cancel a friend request, remove a friend, list friends/incoming/outgoing requests, resolve the relationship between two users. Friend acceptance is a TRUSTED server-side callable (Admin SDK) – a client can't fake a mutual friendship by .writing both User.friends arrays directly

) Files: src/services/friendsService.ts (385 lines); FriendRequestDoc / FriendRequestStatus types •
.src/screens/profile/FriendsScreen.tsx ;(src/types/index.ts:428–451

Who: Any user •

.(friendsService) Tests: none found by name •

Notifications (push) – dispatch, prefs, dedup

What: 20+ notification types (join/approve/reject, game reminders, spot-opened/offered, guest-promoted, batched “players joined”, growth milestone, invite-to-game, rate-reminder, filling-up, RSVP nudge, player-cancelled, group-deleted, promote-prompt, group-invitation, shortage-warning, friend-request(-accepted), teams-generated, evening-summary, filler-*). Per-type user toggle (NotificationPrefs , 17 switches) gates delivery at the Cloud Function consumer. Dedup layer rotates deterministic Firestore doc IDs through cooldown “buckets” per notification kind so the same logical event firing many times in quick succession (e.g. an admin editing a game 10×) produces exactly one push, while a genuinely later event still gets through

Files: src/services/notificationsService.ts (784 lines – dispatch , loadPreferences / savePreferences , registerDeviceToken / pruneApnsTokens / unregisterThisDevice , getPushPermissionStatus / requestAndRegisterPushToken , inviteToGame); src/services/notificationDedup.ts (375 lines, cooldownMsFor , dedupeKeyFor); src/services/notificationActionService.ts (223 lines – tap-action handlers for game-reminder/spot-offer/filler-opportunity); src/screens/profile/NotificationsSettingsScreen.tsx (also gates on OS-level permission, surfacing an “enable notifications” prompt before the per-type toggles even matter)

.notificationsService Tests: tests/logic/apnsTokenShape.test.ts , inAppActionVocabulary.test.ts . No direct test for •

Requests inbox (unified)

What: One “Requests” screen (header bell) aggregating 3 actionable queues – incoming friend requests, community-join requests for clubs the viewer admins, game-join requests for games the viewer created – each with a bulk “approve all”. Reuses existing tight queries (no full-collection scans); the badge-count path skips name resolution for cheapness

.src/screens/RequestsScreen.tsx ;Files: src/services/requestsService.ts (154 lines) •

Tests: none found by name •

Sharing (WhatsApp + generic OS share)

What: Israeli-phone-number validation/normalization, “open WhatsApp chat with this number”, and “share this text via WhatsApp” (falls back gracefully if WhatsApp isn’t installed). Evening-summary/player-compare cards use the generic OS share sheet (`.expo-sharing`) instead, not this service

Files: `src/services/whatsappService.ts` (80 lines: `isValidIsraeliPhone` , `normalizeIsraeliPhone` , `openWhatsApp` , `.shareToWhatsApp`)

Tests: none found

Invite attribution & deep links

What: Every share/invite path credits the inviter via `User.invitedBy` (set once, never overwritten). Mechanisms: (1) short attributable links `/i/<code>` (`inviteLinkService`); (2) full deep-link parse/build (`deepLinkService.parseInviteUrl / buildInviteUrl / buildAppInviteUrl` , `?invitedBy=<uid>`); (3) iOS clipboard-deferred deep link (since iOS can’t pass a deep link through the App Store install) – consumed once on first launch if fresh (`clipboardInviteService`); (4) Android install-referrer attribution, same idea via the Play install broker (`installReferrerService`). All four converge on the same `invitedBy / .invitedByType / invitedByTargetId / invitedAt` fields, written transactionally

Files: `src/services/deepLinkService.ts` (253 lines), `src/services/clipboardInviteService.ts` (98 lines), `) User.invitedBy*` ; `src/services/installReferrerService.ts` (223 lines), `src/services/inviteLinkService.ts` (66 lines) `.( src/types/index.ts:~150`

Tests: none found for these 4 services directly; `tests/logic/copyDirectionality.test.ts` is unrelated (RTL copy, not clipboard `.invite`)

Other .8

Ads (banner + app-open)

What: AdMob banner (bottom, kill-switchable) + app-open interstitial with layered guards: master Remote Config switch, cooldown (4h default), max-3/day, a 2-day new-account honeymoon, and a suppression window right after the user opened the app via a `.push/deep-link` (“intentful open” shouldn’t be interrupted by an ad). `AdDebugOverlay` is a dev/QA-only diagnostic

Files: `src/services/adsService.ts` (661 lines: `initializeAds` , `showAppOpenAdIfAvailable` , `noteIntentfulOpen` , `BannerAd` , `.component` , `AdDebugOverlay`)

Tests: none found by name

Widgets & Wear OS companion

What: One adaptive payload fans out to 3 surfaces – Android home-screen widget (`TeamderWidgetProvider` , `play/pause/reset` buttons mutate Firestore directly from native Kotlin), a Wear OS Tile/app (Galaxy Watch 4+, native module `WatchBridge` over the Data Layer), and the phone live-match screen itself – all reconstructing the identical synced clock locally from the same 3 timer `.primitives`

Files: `src/services/watchSyncService.ts` (408 lines: `computeWatchPayload` , `publishWatchState` , `useWatchSync`); native scaffolding under `plugins/wear-src` , `plugins/withWearApp.js` ; per memory the Wear OS “physical” Option-B path was `.removed` (phone Health Connect only now)

Tests: none found by name

Weather forecast

What: Per-game weather forecast (temperature/condition icon) for the field’s location/kickoff time

Files: `src/services/weatherService.ts` (213 lines: `getForecastFor` , `weatherIcon`)

Tests: none found

Map (games / communities)

What: One full-screen map component, two modes (toggle between games and communities), with mode-specific filter chips, a colour legend, and a “show the other layer” overlay toggle. Content arrives pre-loaded as serializable `MapItem[]` from whichever .list screen launched it – the map never re-fetches, so it works identically in mock mode

Files: `src/screens/map/MapScreen.tsx` (868 lines)

Tests: `tests/logic/geo.test.ts` (adjacent geo-math, not the screen itself)

Location services (geocode / Israeli places)

What: Forward/reverse geocoding, Israeli-city autocomplete (≥ 2 chars), and a GovMap places search (`govmapService`) used for .precise field-location picking in the game wizard

Files: `src/services/geocodeService.ts` (236 lines), `src/services/govmapService.ts` (78 lines),
 `src/services/israelLocationService.ts` (139 lines)

Tests: `tests/logic/geo.test.ts` , `holidays.test.ts` (unrelated grouping, Hebrew calendar)

“(פנויים לידך” / “מצא לי משחקים”) Availability

What: User marks preferred weekdays $\times$ 3 time-of-day windows $\times$ home city $\times$ travel radius, so the server-side matcher can offer/route them into open games and cross-community filler opportunities. A dedicated full-week grid screen (`AvailabilityWeekScreen`) opened from a home podium teaser; a separate Pulse-side (admin) heatmap consumes the same . `availabilityFeedService` day/window counts

Files: `src/screens/profile/AvailabilityEditScreen.tsx` (1004 lines), `src/screens/home/AvailabilityWeekScreen.tsx` ;
 `src/services/availabilityFeedService.ts` (129 lines: `TIME_WINDOWS` , `AvailabilityDayCounts`); `UserAvailability` type
 (`src/types/index.ts:617`

Tests: none found by name

Feedback / bug report

What: In-app “report a problem / suggest a feature” form (bug vs. suggestion segmented toggle, ≤ 2000 chars), reached from the .profile hamburger’s Support section. Writes to `feedback` ; a separate admin panel (Pulse) triages

`src/screens/FeedbackScreen.tsx` ; **Files:** `src/services/feedbackService.ts:42` `submitFeedback` (93 lines)

Tests: none found by name

What’s new” modal”

What: One-time post-update highlights modal, curated per-version in `appConfig/whatsNew` (Pulse-authored). Shows only items whose version falls in `(seenVersion, currentVersion]` – a user who skipped several versions sees ALL of them flattened, newest-first, in one modal, not broken out per-version. If no items are curated for a version, nothing shows and the baseline .advances silently

Files: `src/services/whatsNewService.ts` (125 lines: `resolveWhatsNew` , `markWhatsNewSeen`)

Tests: none found

Store-review prompt

What: Thin wrapper over `expo-store-review` , triggered at two “emotional peak” moments (admin’s game fills to capacity; a game the user was in just finished), layered with 3 guards: in-session dedup, 90-day cooldown, and an `isAvailableAsync()` platform .check. The OS never reports what the user actually did with the prompt

Files: `src/services/storeReviewService.ts:80` `maybeRequestStoreReview` (128 lines)

Tests: none found

Soft/force update popup

What: Compares the running app version against a server-controlled `latestVersion` / `minimumSupportedVersion` doc to decide .none / optional / force , deep-linking to the correct store. Per memory the soft-popup gate has repeatedly gone stale relative to .actual store availability (watch.py cron exists specifically to keep it honest)

.Files: `src/services/updateService.ts` (118 lines: `getCurrentVersion`, `compareVersions`, `checkForUpdate`, `openStore`) •
.Tests: none found by name •

In-app popup campaigns

What: Client-side consumer of marketing popups authored in Pulse (`campaigns/{id}`). On home mount, fetches active popups, evaluates each campaign's audience segment against the current user CLIENT-SIDE (shared `SegmentFilters` shape with Pulse's own preview), applies a per-user frequency cap, and shows the single best match. Push-type campaigns are sent server-side and never touch this path. Master kill-switch (`feature_campaigns`) can disable the entire system – popup + eligibility query + presence .ping + engagement events – without a rebuild

.Files: `src/services/campaignService.ts` (294 lines: `getEligiblePopup`, `markPopupSeen`, `trackCampaignEvent`) •
.Tests: none found by name •

Remote Config feature flags

What: Central server-tunable knob set: ad cadence/kill-switches, per-feature kill-switches (`feature_quick_games`, `feature_referrals`, `feature_friends`, `feature_feedback`, `feature_ios_clipboard_invite`, `feature_campaigns`), a full-screen maintenance-mode gate, store-review cadence, editable support-email/store-URL content, games-feed discovery density .thresholds, and an announcement banner

.Files: `src/services/remoteConfigService.ts` (`RC_DEFAULTS`, 220 lines) •
.Tests: none found by name •

Rate limiting

What: Generic per-user, per-operation spam guard (`createGroup` 5/day, `createGame` 10/hr, `inviteToGame` 30/hr, `joinRequest` 20/hr, `rateVote` 60/hr) anchored to one Firestore doc per (uid, op) to avoid a single hot-key document. Fails open on a transient Firestore error (never locks a user out for a network blip); a malicious client could reset their own window early, but the floor .throughput stays the same as a clean reset – server-side enforcement is a stated future hardening, not yet layered in

.Files: `src/services/rateLimitService.ts:61` `consume`, `:110` `enforceRateLimit` (120 lines) •
.Tests: none found •

Marketing SDK integration (Joryio)

What: Wrapper around the Joryio push-automation/journey SDK – identify, track events, push-token registration, in-app-message .display/impression tracking, opt-in/out of marketing tracking, marketing-push subscription mirror , `project_joryio_sdk_parity_gaps`) **.Files:** `src/services/joryio.ts` (385 lines). Fully detailed in existing project memory .(`project_joryio_journeys` . `joryioPushRegistration.test.ts` , `joryioPushClick.test.ts` , `tests/logic/joryioConfig.test.ts` :**Tests** •

Analytics event catalog

What: Central `AnalyticsEvent` map + `logEvent` – infrastructure, not a standalone user-facing feature, but every feature above .logs through it

.Files: `src/services/analyticsService.ts` (589 lines) •
.`tests/logic/analyticsWiring.test.ts` :**Tests** •

Error logging infra

What: Central `logError` / `logUnexpected` / `logRenderError`, a global handler installer, and an `isExpectedDenial` helper that distinguishes “user isn't allowed to read this (fine)” from a real bug – used pervasively (visible in nearly every service snippet above) .to avoid polluting the error dashboard with expected permission denials

.Files: `src/services/errorLog.ts` (428 lines) •
.Tests: none found by name •

- **What:** `__DEV__` -gated screen that mounts the app's real product animations (BallSwitch, RollAwayCta, RollInView, ArcPopIn, MatchClockLoader, etc. – see `src/components/anim`) standalone for preview/screen-recording without navigating through live app state. Never shipped to production
- **Files:** `src/screens/dev/AnimationLab.tsx` (323 lines)

Built but not reachable from the UI (SUSPECTED)

- exported React component, confirmed by grep to have zero – (`src/services/adsService.ts:605`) `AdDebugOverlay()`
- import/render call sites anywhere under `src/`. Dead UI, presumably an ad-debugging tool that was written but never wired into any screen (not even a `__DEV__` -gated one, unlike `AnimationLab.tsx`)
- the type comment explicitly states this lifetime counter “is not currently written – (`src/types/index.ts:~695`) `UserStats.goals` by any path”; kept only so the profile UI can render a stable `0`. Distinct from the per-club `communityPlayerStats` goals total, which IS live
- **DM-related helper exports** in `friendsService.ts` are exercised entirely through the service object, not/ `friendRequestId` independently reachable – not a gap, just noting the module has no standalone test entry point
- **Substitution ≥30s stint stat rule** – confirmed genuinely absent (see Live Match domain), matching the memory record; not merely hard-to-find UI
- defaults to `false` – the in-app announcement banner exists in code but ships permanently `RC_DEFAULTS.announcement_enabled` off unless a Remote Config publish flips it; worth flagging since it's easy to assume it's dead code rather than an intentionally-off lever
- a single-purpose remote toggle for whether the home screen's – (lines 25) `homeConfigService.getAvailabilityCardEnabled` availability card renders at all; small enough that its existence is easy to miss when reasoning about the home screen

Flag-gated features (confirmed, not suspected)

- a hardcoded (not Remote Config) constant – (`src/screens/games/GameWizardForm.tsx:261`) `ADVANCED_MODE_ENABLED = true` already flipped on in prod per project memory (shipped 1.0.28); each game individually opts in via its own `advancedMode` field, so this is a per-game toggle, not a global kill-switch waiting to be flipped
- full-screen blocking gate, off by default, remotely flippable without a rebuild – `RC_DEFAULTS.maintenance_mode`
- six independent kill- – (quick_games, referrals, friends, feedback, ios_clipboard_invite, campaigns) `RC_DEFAULTS.feature_*` switches that hide their respective entry points; all default `true` except none are off by default currently, i.e. all six ship on
- **App Check enforcement** – per existing project memory, disabled on all callables (`ENFORCE_APP_CHECK=false`) to unblock iOS, pending App Attest verification; a security posture flag, not a product feature, but affects every callable listed throughout this document

D4_infra

D4 – Cross-Cutting Infrastructure Map (Teamder)

`./Users/matan/Projects/soccer` Read-only inventory. All paths relative to

Architecture

:Two independent push pipelines exist

A. End-user push (/notifications/{id} docs)

Client writes a doc via `notificationsService.dispatch()` (`src/services/notificationsService.ts:175`) – deterministic doc ID – `.( dedupeIdFor )` so retries/races collapse into one write instead of re-firing the trigger

is the actual (`notifications/{id}` Firestore `onDocumentCreated` on) `onNotificationCreated` `functions/src/index.ts:1773` – sender: canonicalises payload server-side (`canonicaliseNotificationPayload` , so a client can't spoof `gameTitle` / `groupName`), builds `{title, body}` via `buildMessage(type, payload)` (`functions/src/index.ts:621`), resolves recipients (`.( deliverBatch )` `admin.messaging().sendEachForMulticast` `resolveRecipients`), and sends via

Some flows skip the client write and go straight through a **callable** that writes server-side (`sendGameInvite` , - `. notifyPlayerCancelled`) so the sender identity/rate-limit can't be forged

Scheduled/derived pushes are created server-side via `createNotificationOnce` from cron/task handlers (reminders, nudges, - `.shortage warnings, evening summary, etc.`) – same `notifications/{id}` pipeline, same `onNotificationCreated` sender

B. Chat push – separate pipeline, `functions/src/chatPush.ts` , triggered directly on message-create (`onGameChatMessage` , `onCommunityChatMessage` , `onDmChatMessage`), NOT via `/notifications` . “One push until opened” rule: pushes only on the unread- `.( chatPush.ts:167–203 )` count 0→1 transition

C. Founder/admin alerts – `functions/src/adminPush.ts` `pushToAdmins()` , pushes to `adminConfig/push.tokens` (the Pulse :dashboard's own device), gated by `adminConfig/prefs` per-type toggles and a 20s flood-guard latch per type. Types `newUser` | `review` | `error` | `bug` | `suggestion` | `gameJoin` | `gameCreate` | `communityCreate` | `communityJoin` | `availabilityUpdate`

D. Admin→user campaigns (“adminBroadcast”) – `functions/src/adminUserPush.ts` . Pulse writes a `campaigns/{id}` doc (Pulse authenticates as a service account, not a Firebase user, so it can't call a callable – the doc itself is the request). Segment-filtered (see `SegmentDef` / `FieldKey` in that file), hard per-user rate limit of 1 broadcast/24h (`PER_USER_DAY_MS`), `MAX_RECIPIENTS=20000` , capped `.( CAMPAIGN_MAX_ATTEMPTS=6 )` re-claim attempts

Fires from	Recipient	Trigger	Type
gameService.ts / groupService.ts	admin	join request to game or community	joinRequest
, groupService.ts:852,958 gameService.ts:5455,5562	requester	join request approved	approved
, groupService.ts:1006,1045 gameService.ts:5609,5678	requester	join request rejected	rejected
gameService.ts:2800	community members subscribed via newGameSubscriptions	new game opened	newGameInCommunity
recise Cloud Task (T-60) + cronEvery15Min safety net (index.ts:1899) sendGameReminderForGame	game players	T-60min before kickoff	gameReminder
- gameService.ts:3075,4627,6048,6199,6753 type (see below) STRICT_UNREAD_DEDUP	affected players	game edited/cancelled/deleted/player removed	gameCanceledOrUpdated
gameService.ts:6187,6562,6682	promoted player	a waitlist slot opened and was auto-filled	spot0opened
neService.ts:5760,6042,6175,6305,6398,6474	waitlist head	slot opened, confirmation required (head of waitlist)	spot0ffered
(onGameRosterChanged via) gameService.ts	the player who added the guest	a waitlisted guest promoted into roster	guestPromoted
index.ts:2146) flushPendingJoinerNotifsTask consolidates a 3-min window	community admins	“batched “N players joined	gamePlayersJoined
server-side on group growth	community admins	community crosses a member- count milestone	growthMilestone
callable (index.ts:7479), via sendGameInvite notificationsService.inviteToGame	invitee	admin/player invites a specific user	inviteToGame
(index.ts:7697) callable adminAddPlayers	added player	admin directly adds a player	addedToGame
post-game trigger	players	game finished	rateReminder
game roster trigger	interested/subscribed users	roster crosses a fullness threshold	gameFillingUp
in cron-15 ,(index.ts:1991) runSendRsvpNudges	community members not yet responded	T-5h before kickoff, still on the fence	gameRsvpNudge
llable (index.ts:7961), notifyPlayerCancelled via notificationsService.notifyPlayerCancelled	(createdBy) game admin	registered player cancels	playerCancelled
groupService.ts:1668,1818	every member+admin	admin deletes a community	groupDeleted
promotePromptCron	game creator	orphan (personal-group) game just finished	promotePrompt
index.ts:8217) callable promoteOrphanToGroup	invited participants	orphan game promoted to a real community	groupInvitation
shortage-check cron	game admin	roster under min/80%-of-max near kickoff	gameShortageWarning

Fires from	Recipient	Trigger	Type
(index.ts:11684) onFriendRequestCreated	recipient	friend request sent	friendRequest
llable (index.ts:11714) - acceptFriendRequest never sent on decline (by design)	original requester	friend request accepted	friendRequestAccepted
) auto-teams pipeline / notifyTeamsReady callable (index.ts:8065	every player	auto/manual team balance published	teamsGenerated
round-close pipeline	every player	evening/round finishes	eveningSummary

Server-only types (used in the push data.type payload / buildMessage but not in the client NotificationType union – a real union/reality gap)

These .gameOnHoliday , adminBroadcast , fillerNoCandidates , fillerInterestReceived , fillerOpportunity , chatMessage are handled by navigateForPush() (which takes a bare string , not the union) and by server buildMessage() , but a client-side .exhaustiveness check against NotificationType would silently miss them

Tap → deep-link routing

is the single tap handler (Notifications.addNotificationResponseReceivedListener + handleResponse() App.tsx:676–800 : getLastNotificationResponseAsync for cold starts). It

- .Reports Joryio campaign delivery/click if data.trackingId present (marketing pushes, before the type bail-out) .1
- .(adsService.noteIntentfulOpen()) Suppresses the next app-open ad .2
- Runs action-button side effects for JOIN_GAME / CANCEL_GAME (→ notificationActionService.handleGameReminderAction) and .3
- CONFIRM_SPOT / PASS_SPOT (→ handleSpotOfferAction), deduped per (notifId, action, gameId) via an in-memory .handledActionTaps set
- Polls up to 30s for navigationRef.isReady() (cold start can take that long through splash/ad/auth-restore), then calls .4
- .navigateForPush(type, data)

:is the full type→screen map navigateForPush() src/navigation/navigationRef.ts:168–420

- .data.scope depending on GameChat | DirectChat | CommunityChat / ChatTab → chatMessage -
- if gameId , else AdminApproval (join request) or CommunityDetails / MatchDetails → rejected / approved / joinRequest -
- .CommunityDetailsPublic (approved/rejected – rejected users go to the *Public* variant since they aren't members)
- game-scoped types (gameReminder , gameCanceledOrUpdated , spotOpened , spotOffered , guestPromoted , inviteToGame , 12 -
- addedToGame , rateReminder , gameFillingUp , gameRsvpNudge , gamePlayersJoined , playerCancelled , teamsGenerated ,
- fillerNoCandidates , fillerInterestReceived , fillerOpportunity , gameShortageWarning) all route to MatchDetails if
- .CommunitiesFeed gameId present, else
- .GameTab/EveningSummary → eveningSummary -
- .(no destination doc exists anymore) CommunitiesFeed → groupDeleted -
- .ProfileTab/Achievements → growthMilestone -
- .GameTab/PromoteOrphan → promotePrompt -
- .CommunitiesTab/CommunityDetails → groupInvitation -
- .ProfileTab/Friends → friendRequestAccepted / friendRequest -
- / data.value / data.action reuses navigateCampaign() (same router the in-app popup uses) – reads → adminBroadcast -
- .groupId / gameId / data.url

Dedup / cooldown (src/services/notificationDedup.ts , mirrored server-side)

per-type minimum spacing bucket used to build the deterministic – (line 81) COOLDOWN_MS: Record<NotificationKind, number> •
.doc ID

currently only gameCanceledOrUpdated : suppresses a new dispatch – (notificationsService.ts:75) STRICT_UNREAD_DEDUP •
.(STALE_UNREAD_TTL_MS) entirely while ANY unread doc with the same dedupeKey exists (not just the same bucket), TTL 7 days

Server-side `onNotificationCreated` additionally latches `gameCanceledOrUpdated` by `(gameId, category)` in `(index.ts:1790-1829)` `gameUpdateLatches/{gameId}__{update|cancel}` to stop a rapid edit/cancel storm from double-pushing

FCM token plumbing

the whole file is built around a **fixed historical bug**: the app used to store – `src/services/notificationsService.ts:104-162` the raw APNs device token (64 hex chars) on iOS and hand it to `sendEachForMulticast`, which only accepts FCM tokens → every iOS push silently failed (see `project_ios_push_never_worked.md` in memory; documented in code at line 106-126). Fix: `@react-native-firebase/messaging`'s `getToken()` does the APNs→FCM exchange natively; `looksLikeApnsToken()` (regex `^[0-9a-f]{64}$`) guards against ever storing/re-storing a dead APNs-shaped token, and `pruneApnsTokens()` (line 514) proactively strips them once a real token lands

Tokens live at `/users/{uid}/private/push.fcmTokens` (self-only rules) – moved off the public `/users/{uid}.fcmTokens` for privacy (Security Audit Finding #1, per comments)

`DEAD_TOKEN_CODES = messaging/registration-token-not-registered`, `chatPush.ts` `.messaging/invalid-registration-token` – auto-pruned on send. `adminPush.ts` only prunes `not-registered` (narrower)

Deep links & share links .2

Schemes / hosts

Custom schemes: `teamder://`, `footy://` (legacy compat), `com.studiogameslime.soccerapp://` – declared in `app.json:8-12` `(expo.scheme array)`

Hosting domains treated as ours: `teamderfc.web.app`, `teamderfc.firebaseio.com`, `teamder.web.app`, `teamder.firebaseio.com` (`src/services/deepLinkService.ts:36-41`) – the last two are dead (Firebase project doesn't own `.teamder.web.app`; kept only so a pre-existing shared link still parses)

iOS Universal Links: `applinks:teamderfc.web.app` (`app.json:27-29`) – per memory notes, **broken** (Associated Domains entitlement issue on the local profile)

Android App Links: `intentFilters` in `app.json:57-83`, `autoVerify: true`, covering `https://teamderfc.web.app/session/*`, `/team/*`, `/app/*`. **Gap found**: the `/i/` short-link path is NOT in this intent-filter list, so a short link opens Android's browser/chooser instead of auto-verifying straight into the app (long-form `/session` and `/team` links do verify)

`src/services/deepLinkService.ts` – Parsing

handles both scheme and hosting forms, plus a generic `/app` or `/go` acquisition link (optionally (line 103) `parseInviteUrl(url)` carrying `?g=<gameId>` to deep-link into a game while still recording attribution

.Attribution: `?invitedBy=<uid>` on any link form

Acquisition/UTM: `?b=<base64url-token>` (Pulse's short encoded source) decoded via a hand-rolled `base64url→UTF-8` decoder (`decodeSourceToken`, no Buffer/atob available in Hermes), with `?s= / ?utm_source=` fallback; `?c= / ?utm_campaign=`; `?l=` = per-link attribution id

writes to `storage` (AsyncStorage-backed) rather than wiring into React Navigation's `linking` prop – `stashPendingInvite()` deliberate, because the navigator tree depends on auth/onboarding state and auto-linking would race an unmounted target screen

.RootNavigator consumes the stash once ready

and `buildAppInviteUrl(invitedBy)` build the LONG share `buildInviteUrl({type:'session'|'team', id, invitedBy})` `(https://teamderfc.web.app/session/<id>?invitedBy=<uid>)` URL

`src/services/inviteLinkService.ts` – Short links

writes `{type, targetId, invitedBy, createdAt}` to `inviteLinks/{code}` (7-char base62 random `createShortInviteUrl()` `.https://teamderfc.web.app/i/<code>?invitedBy=<uid>` code, collision-ignored as astronomically unlikely) and returns

.Fail-safe: on write failure returns `fallbackLong` (the long URL) verbatim so a share is never broken

is duplicated in BOTH the Firestore doc and the URL query string – belt-and-suspenders because `serveInviteCode` `invitedBy` (below) needs the doc to inject full context, but if that CF cold-starts/misses, `invite.html` still recovers `invitedBy` client-side from the query string alone

`functions/src/index.ts` – Server-side resolution

SSR two route families: `/c/{groupId}` (community showcase) and – (`onRequest` ,line 9740) `serveCommunityPage` `/team/{groupId}` (invite landing), injects Open Graph meta from `loadShowcaseSummary(groupId)` , 5min browser / 10min CDN .cache. Falls back to the static template on any render error

resolves `/i/{code}` → `inviteLinks/{code}` doc, serves the SAME invite template – (`onRequest` ,line ~9808) `serveInviteCode` with `window.__INVITE__` injected (pure alias, no redirect, so the short URL stays in the address bar and OG/WhatsApp preview + .install-referrer attribution survive)

JSON endpoint `invite.html` 's client JS calls (`/invite-preview?code=`) to render the live – (line 12230) `getInvitePreview` .context card (game time/spots, community name/city/member count) before the user leaves the landing page

.beacon endpoint hit by `invite.html` 's `sendBeacon('/track-click?...')` for click attribution – (line 12362) `trackLinkClick`

`public/invite.html` – Landing page

Single static file serving ALL invite variants (`personal_invite` / `game` / `community` / `generic` / `campaign`), content swapped . `ctxType` client-side via the `V` lookup table (line 233) based on

iOS-specific behavior (line 231, `primary()`): Safari throws a visible error for an unhandled custom scheme when the app isn't installed, so on iOS it **never** attempts `footy://` – it copies the HTTPS invite URL to the clipboard (for `installReferrerService` - style deferred attribution recovery) and goes straight to the App Store. Android attempts the deep link, falls back to Play Store (with . `&referrer=` encoding type/id/invitedBy/UTM) after a 1.5s timeout if `visibilitychange` never fires

Emits GA-style `dataLayer` events (`landing_view` , `landing_primary_cta_click` , `landing_scroll_depth` , . `landing_context_loaded` / `landing_context_failed`) plus loads `/js/joryio.js` for the web SDK

Feature flags & remote config .3

(`RC_DEFAULTS`) `src/services/remoteConfigService.ts:22-75` – Firebase Remote Config

All keys, with in-code (shipped) defaults – Remote Config values only take effect if published in the console AND the fetch (: `fetchAndActivate` , 1h min interval in prod, 0 in dev) succeeds; falls back to these defaults on any failure/absence

Purpose	Default	Key
master kill-switch, app-open ad format	true	app_open_ad_enabled
min gap between app-open ad shows	4h	app_open_cooldown_ms
cap	3	app_open_max_per_day
no ads for fresh accounts	days 2	app_open_new_user_grace_ms
suppress after a push/link-driven open	20s	app_open_intentful_suppress_ms
bottom banner ad kill-switch	true	banner_enabled
entry-point flag	true	feature_quick_games
entry-point flag	true	feature_referrals
entry-point flag	true	feature_friends
entry-point flag	true	feature_feedback
entry-point flag	true	feature_ios_clipboard_invite
master kill-switch for the whole in-app campaign system (popup, eligibility query, presence ping, engagement events)	true	feature_campaigns
blocking full-screen gate	false	maintenance_mode
gate copy (falls back to a default string)	' '	maintenance_message
store-review prompt	true	review_prompt_enabled
	90	review_prompt_cooldown_days
	studiogameslime@gmail.com	support_email
	live store URLs	store_url_android / store_url_ios
discovery-density threshold	5	games_feed_rich_min
	3	games_feed_demand_min
	3	games_feed_clubs_max
	30	games_feed_clubs_radius_km
	10	games_feed_clubs_min_members
in-app banner	false	announcement_enabled
	' '	/ announcement_text announcement_url

is a re-render hook keyed on a module-level `activatedTick` , so UI-reactive flags (maintenance gate, `useRemoteConfig()` .announcement banner, hidden buttons) update once the boot fetch lands

Hardcoded flags/constants (not server-tunable – require a redeploy/rebuild to flip)

Note	File	Value	Constant
admin-only advanced e-match + orer goal-etry; gates UI section ill marked nfinished” in omments	src/screens/games/GameWizardForm.tsx:261	true	ADVANCED_MODE_ENABLED
pp Check sabled on ALL allables to block iOS (per memory: ending re-enable once App Attest verified)	functions/src/index.ts:103	false	ENFORCE_APP_CHECK
Health Connect (Android) physical-tivity read	src/services/healthService.ts:149	true	HEALTH_ENABLED
env-driven	src/services/adsService.ts:123	process.env.EXPO_PUBLIC_ADMOB_ENABLED === '1'	ADS_ENABLED
on-demand filler-push pulse engine; deliberately OFF so deploying the code doesn't start ushing on prod – 15-min sweep mains the live path	functions/src/index.ts:10538	false	PULSE_ENGINE_ENABLED
hides ads/mock anners for App/Play store screenshot capture; also implicitly hides the ommunity card per	, src/services/adsService.ts , CommunityDetailsScreen.tsx MockModeBanner.tsx	EXPO_PUBLIC_SCREENSHOT_MODE === '1'	SCREENSHOT_MODE

Note	File	Value	Constant
memory note			
the mock-ata / real-Firebase switch every service checks	src/firebase/config.ts:72	!FIREBASE_CONFIGURED	USE MOCK_DATA

Force-update / soft-update gates (Firestore, not Remote Config)

doc holds `latestVersion` (soft popup) and `minimumSupportedVersion` (hard force-update), read by `appConfig/{platform}` `src/services/updateService.ts:52-61`. Per memory, `latestVersion` watcher/popup mechanism has had multiple stale periods – `.verify` live before relying on it

toggles `.env.example` / `.env`

Firebase config (7 keys), Google OAuth (3 client IDs), Israel-streets resource ID, AdMob (`EXP0_PUBLIC_ADMOB_ENABLED` , `app/banner/app-open` unit IDs, `EXP0_PUBLIC_ADMOB_USE_TEST_IDS`), `EXP0_PUBLIC_SCREENSHOT_MODE` , `EXP0_PUBLIC_APP_CHECK_DEBUG_TOKEN` . Joryio SDK keys are NOT env-only – they have hardcoded literal fallbacks in `src/services/joryio.ts:39-49` (deliberately, per an in-code postmortem: an env-only key resolved to `''` in a store build once `.because .env` is gitignored and `eas build --local` copies the project via git – shipped 1.0.94 with analytics silently off)

Error / loading / empty / offline states .4

`src/services/errorLog.ts` – Error logging

- Client errors funnel into ONE aggregated Firestore collection, `errors/{fingerprint}` – fingerprint = `djb2(operation + normalized(message))` , so the same bug across many users/occurrences becomes one doc with a running `.count` (via `increment()`), not N docs
- In-memory coalescing: writes buffered per fingerprint, flushed at most once per 12s (`FLUSH_MS`), plus a hard per-fingerprint session `.cap` of 30 writes (`SESSION_WRITE_CAP`) so a stuck device can't hammer one doc all day
- Three categories stamped on the doc: `crash` (`uncaught` / `uncaughtRender` / `unhandledRejection`), `silent` (post-condition violations – the action didn't throw but the expected outcome didn't happen, via `logUnexpected()`), `action` (everything else, a `.caught` throw)
- maps ~35 known operation labels to Hebrew “what failed” strings for the admin/Pulse dev-inbox; unknown (line 145) `OP_TITLES` `.ops` get a generic `<prefix>` `.<operation>` fallback
- filters out network/offline/timeout blips (`unavailable` , (line 98) `TRANSIENT_CODES` / `isTransientEnvError()`) `deadline-exceeded` , `cancelled` , `auth/network-request-failed` , `auth/timeout` , plus message-text matches for “client is `.offline`”/“network request failed”) – these are dropped entirely, never logged, so the dev inbox isn't drowned by dead-zone noise
- a SEPARATE helper call sites use to skip `logError` on best-effort writes that fail for expected reasons – `isExpectedDenial(err)` (`permission-denied` , `unauthenticated` , `unavailable` , `deadline-exceeded` , `cancelled` , `resource-exhausted` , any `.app-check` -related code)
- Global handlers installed once via `installGlobalErrorHandlers()` : `ErrorUtils.setGlobalHandler` for uncaught JS errors (with a synchronous-ish `flush()` before the fatal tears down the JS context) and `promise/setimmediate/rejection-tracking` for `.unhandled` promise rejections
- No Crashlytics/Sentry wired up (confirmed by absence in `package.json` and by `ErrorBoundary.tsx` 's own comment: “Production `.logs` are minimal until Crashlytics/Sentry is wired up”)

src/components/ErrorHandler.tsx – Render crashes

button that (try again) “נסה שוב” Class component, `getDerivedStateFromError` + `componentDidCatch`. Fallback UI is a Hebrew •
resets boundary state – deliberately NO auto-retry (avoids a crash-loop). Wraps the navigator from the outside so a navigator-
internal crash is still caught, but does not touch `navigationRef` (may be undefined mid-crash). `onError` prop plugs into
`.errorLog.ts` → `logRenderError`

.Toasts – src/components/Toast.tsx (201 lines) – app-wide transient feedback component

Empty states

- `.generic` empty-state component – (lines 109) `src/components/EmptyState.tsx`
- `.football-pitch-themed` empty illustration (likely games/roster lists) – (lines 48) `src/components/EmptyPitch.tsx`
- `.match-specific` empty hint – `src/components/match/MatchEmptyHintCard.tsx`

Loading / skeletons

- the only dedicated skeleton component found; most loading states fall – (lines 80) `src/components/anim/MatchCardSkeleton.tsx`
- `.back` to plain `ActivityIndicator` (29 files reference it directly rather than a skeleton)

Offline handling

- **No dedicated offline-detection layer** – no `@react-native-community/netinfo` (or equivalent) dependency in `package.json`, no `NetInfo` import anywhere in `src/`. “Offline” is handled reactively per-call-site by catching Firestore’s `unavailable / client is offline` errors (the same codes `errorLog.ts` treats as transient/non-actionable) rather than proactively via a connectivity listener or an offline banner. Firestore’s own offline persistence/cache presumably covers reads while offline; there is no `.app-level` “you’re offline” UI state

Retry paths

- is the only generic app-wide retry UI found. Individual screens implement their own retry inline (grep for “נסה שוב” `ErrorBoundary` “retry” patterns beyond the boundary turned up nothing else app-wide – retries appear to be ad hoc per-screen `try/catch` + `.fetch` rather than a shared retry component/hook)

Platform-specific code .5

branches – 28 files Platform.OS

`firebase/appCheck.ts`, `PinnedAdminMessageCard.tsx`, `IssueCardSheet.tsx`, `ChatView.tsx`, `src/components/GuestModal.tsx`,
`SignInScreen.tsx`, `ProfileSetupScreen.tsx`, `EmailAuthScreen.tsx`, `screens/FeedbackScreen.tsx`, `firebase/auth.ts`,
and, `ProfileScreen.tsx`, `GroupWizardForm.tsx`, `CreateGroupScreen.tsx`, `PromoteOrphanScreen.tsx`, `GameWizardForm.tsx`,
`errorLog.ts`, `clipboardInviteService.ts`, `campaignService.ts`, `analyticsService.ts`, `adsService.ts` :services
`, notificationsService.ts`, `joryio.ts`, `installReferrerService.ts`, `healthService.ts`, `feedbackService.ts`
`.watchSyncService.ts`, `userService.ts`, `updateService.ts`

Purpose	Plugin
registered FIRST in <code>app.json</code> plugins list (memory: mods run in REVERSE registration order, so first = runs last) – merges notification icon color config	<code>withNotificationColorMerge.js</code>
Android Health Connect wiring	<code>withHealth.js</code>
bundles Ionicons font asset	<code>withIoniconsAsset.js</code>
wires the vendored <code>@joryio/react-native-sdk</code> native Android module into Gradle	<code>withJoryioSdk.js</code>
strips an “always” location permission the base config would otherwise request	<code>withRemoveAlwaysLocation.js</code>
wiring <code>DETECT_SCREEN_CAPTURE</code> / <code>expo-screen-capture</code>	<code>withScreenCapture.js</code>
native-build C++ toolchain workaround (fmt library consteval fix)	<code>withFmtConstevalFix.js</code>
copies <code>plugins/wear-src/{wear,watch,widget}</code> into the generated <code>android/</code> project every prebuild (since <code>android/</code> is gitignored), wires <code>settings.gradle</code> , <code>app/build.gradle</code> (adds <code>play-services-wearable:18.2.0</code>), registers <code>WatchBridgePackage</code> in <code>MainApplication</code> , and registers 4 manifest receivers/services: <code>TeamderWidgetProvider</code> , <code>TeamderPlayersWidgetProvider</code> , <code>TimerActionReceiver</code> , <code>PlayersRemoteViewsService</code> , <code>WearTimerCommandService</code> plus	(12.4KB, largest) <code>withWearApp.js</code>

Not currently registered in `app.json` 's plugin list: `@bacons/apple-targets` is still a `package.json` dependency (`^4.0.7`) but per memory was removed from `app.json` on 2026-07-22 because it broke `production-ios-local` builds – the `targets/watch/` and `.targets/complication/` Swift sources exist on disk but are dormant/unwired

plugins/wear-src/ – Wear OS companion

- `(src/main, build.gradle)` the actual Wear OS Gradle module – `wear/`
- the phone-side Kotlin bridge (`WatchBridgePackage.kt`, `WatchBridgeModule.kt`, `WearTimerCommandService.kt`) – `watch/`
- exposed to JS as `NativeModules.WatchBridge` (Android) – mirrored on iOS by `modules/watch-bridge/` (a proper Expo module, `.requireNativeModule('WatchBridge')`, `modules/watch-bridge/ios/WatchBridgeModule.swift`)
- Android home-screen widget Kotlin (`TeamderWidgetProvider.kt`, `TeamderPlayersWidgetProvider.kt`, – `widget/`
- `.TimerActionReceiver.kt`, `PlayersRemoteViewsService.kt`, `TeamderMessagingService.kt`) + `res/` (layout/drawable/xml)
- Client bridge consumed via `src/services/watchSyncService.ts` (line ~250: “Android → classic `NativeModules.WatchBridge`; iOS → the local Expo module”)

iOS widget/watch targets (dormant) – `targets/watch/*.swift`, `targets/complication/*.swift` – Swift sources for .a native watchOS app + complication, gated behind the currently-unregistered `@bacons/apple-targets` plugin

(package.json) Dependencies .6

Runtime – Firebase/backend: `firebase` (`^12.13.0`, web SDK – used for Firestore/Auth/Functions client calls), `@firebase/app` (`^0.14.12`), `@react-native-firebase/{app,analytics,app-check,auth,messaging,remote-config}` (all `^24.0.0` except `auth` pinned exactly to `24.0.0`) – the native-bridge Firebase SDKs used specifically for FCM (the whole point per `notificationsService.ts` comments – `firebase/messaging` web SDK can't do the APNs↔FCM exchange), Analytics, App Check, Remote Config. Running both the JS `firebase` package and the native `@react-native-firebase/*` suite side by side is a real duplication-of-purpose (SUSPECTED: likely intentional – web SDK for Firestore/Functions/Auth-general, native SDK specifically for the modules that need .native bridging – but worth confirming there's no drift between two Auth instances)

Ads: `react-native-google-mobile-ads` (`^14.7.2`)

Analytics/marketing: `@react-native-firebase/analytics`, `@joryio/react-native-sdk` (vendored, `file:./vendor/joryio/react-native-sdk` - not on npm)

`.expo-auth-session`, `expo-apple-authentication`, `@react-native-google-signin/google-signin` :**Auth**

Navigation: `@react-navigation/{native,native-stack,bottom-tabs}` (all ^6.x)

Animation: `react-native-reanimated` (~3.17.4) only - no Lottie, no `react-native-animatable`, no Moti; the custom “football animation set” (per memory) is built directly on Reanimated

`react-native-view-shot`, `expo-sharing`, `expo-file-system`, `expo-image-manipulator`, `expo-image-picker` :**Media / capture**
`.expo-asset`, (evening-summary card export / “סיכום הערב” 4.0.3 - likely powers the shareable)

Other native/Expo: `expo-location`, `expo-clipboard`, `expo-haptics`, `expo-store-review`, `expo-web-browser`, `expo-crypto`, `expo-build-properties`, `expo-localization`, `expo-notifications` (~0.31.5), `react-native-health-connect` (^3.5.3, Android), `react-native-webview` (physical-activity), `react-native-play-install-referrer` (^1.1.9, Android deferred-deep-link attribution), `react-native-safe-area-context`, `react-native-screens`, `react-native-gesture-handler`, `react-native-svg`

State: `zustand` (^4.5.4) - the only state-management library; no React Query/SWR/Redux - data fetching is hand-rolled per service

Core: `expo` (^53), `react` 19.0.0, `react-dom` 19.0.0, `react-native` 0.79.6

Dev: `typescript` ~5.8.3, `jest` ^29.7.0 + `ts-jest`, `@firebase/rules-unit-testing` (^5.0.1, powers the separate `tests/rules/*.test.mjs` Firestore-rules suite), `patch-package` (^8.0.1 - `postinstall` runs it, meaning local patches are carried against `node_modules`, consistent with memory’s “2 local patches carried” note for the Joryio SDK bridge), `babel-plugin-module-resolver`

:**SUSPECTED findings**

No lint/format tooling configured: no `.eslintrc*` / `.prettierrc*` file anywhere in the repo, and `eslint` is not a direct-dependency (only present transitively in `package-lock.json`). Yet the source is full of `// eslint-disable-next-line` comments (seen repeatedly in `notificationsService.ts`, `remoteConfigService.ts`, etc.) - these directives currently do nothing; either lint used to run and its config was dropped, or it never ran in CI and the disables are cargo-culted

No crash-reporting SDK (no Sentry/Crashlytics/Bugsnag dependency) - confirms the in-code comment in `ErrorBoundary.tsx` that -remote crash reporting isn’t wired up; the custom `errorLog.ts` Firestore pipeline is the only error-visibility mechanism

deletes `@react-native-firebase/*/dist/module/package.json` files (`package.json:17`) before running `postinstall` - `patch-package` - a workaround for a known RNfirebase/Metro module-resolution bug; fragile if the upstream package layout changes

is a live dependency for a plugin that is NOT registered in `app.json` - dead weight until the iOS (4.0.7^⁺) `@bacons/apple-targets` - `.watch-build` blocker (memory) is resolved

Analytics .7

src/services/analyticsService.ts

is a plain `const` object (not a TS `enum`) of `snake_case` string literals, organized under ~65 comment section `AnalyticsEvent` • headers (Navigation, Auth, Profile, Groups, Games, Discipline, Live match, Ratings, Achievements, Friends, Quick games, Approval flow, Wear OS, Home widget, Discovery, Cover photo, Notifications, Player card, Sharing, App lifecycle, Onboarding, Geo, Filler matching, Errors, Public community page, then a large second “full-funnel coverage” addendum covering Auth attempts, Activation, Support, Availability, Communities join/lifecycle/discovery/engagement, Discipline, Live-match timer/entry/rounds, Penalty shootout, Goals/assists, Retro goals, Roster, End-of-evening, Recap, Chat surface/messaging/terms/moderation/engagement/health, Ratings, .Guests, Teams/draft, Auto-teams, Games create/edit/recurring/roster-admin)

.distinct event constants (grep count of `key: 'value'` lines) **266~** •

is DELIBERATELY typed to the union, not `string`, specifically to stop (line 546) `logEvent(name: AnalyticsEventName, params)` • a typo’d literal (`'grop_created'`) from compiling - enforced at every one of the “300+ `logEvent` call sites” (per the file’s own .comment)

Dual-sink: every event goes to BOTH `@react-native-firebase/analytics ( analytics().logEvent(...) )` AND `Joryio ( . joryio.track(name, cleaned) , fire-and-forget, queued/batched client-side so it can't slow the caller)` strips `undefined / null` and non-primitives (Firebase Analytics requires string/number/boolean params) and `cleanParams()` .stamps a platform tag .Failures logged via `errorLog.ts ( logError('analyticsLogEvent', ...) )` rather than thrown

vendor/joryio/ + Joryio – src/services/joryio.ts (385 lines)

Backed by the REAL vendored SDK `@joryio/react-native-sdk` (not on npm; native Android module wired via `plugins/withJoryioSdk.js`) – this file is the app's single integration point so the SDK can be swapped without touching 300+ call .sites .Degrades to warn+no-op if the native module isn't linked (Jest / Expo Go) – never crashes . Per-platform SDK keys (`SDK_KEYS.ios/android/web` , line 39) with hardcoded literal fallbacks (see §3 for the “1.0.94 shipped with .analytics off” postmortem baked into the comments) remaps ~14 snake_case event names to the Title Case names already used by 3,861 back-filled historical (line 71) `NAME_OVERRIDES` .events in the Joryio workspace (`game_joined` → `Game Joined` , etc.) so funnels don't split Also handles push-token registration (`joryio.registerPushToken` , called from `notificationsService.registerDeviceToken`) , campaign delivery/click reporting (`reportPushDelivered` , `trackPushClick` , called from `App.tsx` 's notification-tap handler), and .presumably user identify/attributes (only 5 user attributes sent per memory – “build from events” is the workaround)

Tests .8

project test files under `tests/` (Jest, `jest.config.js: roots: ['<rootDir>/tests']` , `ts-jest` , node environment, `@/` → **68** `src/`) – confirmed count (72 `.test.ts*` files total minus 4 belonging to `functions/node_modules/wonka` third-party package). Plus a **separate, non-Jest suite:** `tests/rules/*.test.mjs` (`firestore.test.mjs` , `antiHijack.test.mjs` , `oldClientCompat.test.mjs` , `publishTeams.test.mjs`) using `@firebase/rules-unit-testing` against the Firestore emulator – tests `firestore.rules` directly, `(bigGame.fixture.json)` has its own `package.json / node_modules / fixture`

Coverage by area (grouped from `tests/*.test.ts` + `tests/logic/*.test.ts`)

Live match / rotation engine: `rotationEngine` , `rotationCounts` , `rotationDeep` , `rotationFill` , `rotationQueueReorder` , • `, overlayTouchRegion` , `shootoutPersistence` , `roundIdempotency` , `wentHomeRestore` , `cardState` (discipline) `, logic/teamSlots` , `logic/joinCtaCapacity` , `logic/joinFairness` , `motionTokens` , `animationTriggers` `. logic/rotationQueueReorder` `, logic/eveningStats` , `logic/eveningScorePen` , `logic/eveningProgress` :**Round/evening summary & stats** • `, logic/roundSummaryLines` , `logic/roundSummaryBackfill` , `logic/roundSummary` , `logic/eveningSummary` `, logic/cleanSheets` , `cleanSheetsPersistence` , `logic/roundSummaryRealEvening` , `logic/roundSummaryParity` `, logic/championshipPenalty` , `logic/penaltyStats` , `advancedMatchStats` , `logic/assistCredit` , `assistPersistence` `, logic/balanceVariety` , `logic/balanceTeams` , `logic/balanceParity` , `balanceMetaPersistence` , `logic/tieConfirmParity` `. logic/playedGames` , `logic/formatWideStructures` , `logic/formatPicker` , `logic/format` , `logic/draft` `, logic/clubChemistryReal` , `logic/clubChemistryParity` , `logic/clubChemistry` , `logic/clubCard` :**Club system** • `. logic/officialAccount` `, logic/homeHero` , `logic/heroAtmosphere` , `logic/geo` , `logic/gameFilters` , `logic/discovery` :**Discovery/home/games list** • `. logic/scoreBand` , `logic/registrationQa` `, logic/joryioPushClick` , `logic/joryioConfig` , `logic/apnsTokenShape` , `logic/analyticsWiring` :**Analytics/Joryio** • `. logic/joryioPushRegistration` `. logic/authRaceRetry` :**Auth** • `. ratingAndColors` , `profanity` , `logic/inAppActionVocabulary` ,**Copy/i18n/moderation:** `logic/copyDirectionality` (RTL) •

- Misc:** `logic/holidays` , `logic/types` (type-level sanity), `logic/assistant` (assistant-insights logic), `logic/physical` (Health • `.logic/geo` ,Connect physical sync), `logic/seriesSchedule` (recurring game-series)
- Rules (separate suite):** general Firestore rules coverage, anti-hijack scenarios, old-client backward-compat, team-publish • `.permission` edge cases

Large areas with NO test coverage found

- the entire Cloud Functions surface (60+ exported functions: every `onCall` , - (lines 13,974) `functions/src/index.ts` • `onDocumentCreated/Updated/Written` , `onSchedule` , `onTaskDispatched` , `onRequest`) has **zero unit/integration tests**. Only the Firestore *rules* are tested (`tests/rules/`), not the function logic itself (`buildMessage` , `resolveRecipients` , `deliverBatch` , `.canonicaliseNotificationPayload` , the cron eligibility windows, `adminUserPush` 's segment evaluator, etc.)
- `, reviewAlerts.ts` , `holidays.ts` , `clubChemistry.ts` , `adminUserPush.ts` , `adminPush.ts` , `functions/src/chatPush.ts` • no dedicated test files (some logic is indirectly - (server copy) `notificationDedup.ts` , `teamBalanceCore.ts` , `roundSummary.ts` covered client-side via the `logic/` parity tests, e.g. `clubChemistryParity` / `roundSummaryParity` , but the server module itself isn't imported/exercised)
- `.Chat` (`src/services/chatService.ts` , `activeChat.ts`) - no test file
- **Deep links / invite links** (`deepLinkService.ts` , `inviteLinkService.ts` , `clipboardInviteService.ts` , `installReferrerService.ts`) - no test file (the URL-parsing logic with its `base64url` decoder, multi-scheme/multi-host matrix, and `.acquisition-token` handling is untested)
- itself (dispatch/dedupe/token lifecycle client-side) - no direct test file (only downstream Joryio push- `notificationsService.ts` .registration/click tests exist)
- the two largest services (gameService alone hosts ~6,800+ lines with dozens of - `groupService.ts` , `gameService.ts` `notificationsService.dispatch` call sites) have no direct unit test; covered only incidentally through the `logic/` behavioral tests that consume their outputs (rotation, balance, roster)
- `, availabilityFeedService.ts` , `campaignService.ts` , `requestsService.ts` , `clubFriendsService.ts` , `friendsService.ts` , `, weatherService.ts` , `israelLocationService.ts` , `govmapService.ts` , `geocodeService.ts` , `nearbyClubsService.ts` , `, trustService.ts` , `storeReviewService.ts` , `photoService.ts` , `whatsNewService.ts` , `whatsappService.ts` , `, playerCompareService.ts` , `communityEventsService.ts` , `homeConfigService.ts` , `rateLimitService.ts` , `, useGameEvents.ts` , `watchSyncService.ts` , `remoteConfigService.ts` , `userService.ts` , `playerStatsService.ts` • no dedicated test files found for any of these ~24 services - `serverClock.ts` , `useSyncedTimer.ts`
- **UI layer** - no component/screen tests at all (no `@testing-library/react-native` , no snapshot tests); everything under `tests/` is pure-logic/unit testing of extracted functions, not rendered components. `errorLog.ts` , `Toast.tsx` , `ErrorBoundary.tsx` , `. EmptyState.tsx` have no tests
- **Config plugins** (`plugins/*.js`) and native Wear/widget code (`plugins/wear-src/` , `modules/watch-bridge/`) - untestable by • `.this` suite by nature (native/build-time), and indeed nothing attempts to

.End of D4 infrastructure map